

High Performance with MongoDB

Best practices for performance
tuning, scaling, and architecture

Asya Kamsky | Ger Hartnett | Alex Bevilacqua

Alto rendimiento con MongoDB

Mejores prácticas para el ajuste del rendimiento, el escalado y la arquitectura

Asya Kamsky, Ger Hartnett, Alex Bevilacqua



Alto rendimiento con MongoDB

Copyright © 2025 Packt Publishing. Todos

los derechos reservados . Ninguna parte de este libro podrá reproducirse, almacenarse en un sistema de recuperación de datos ni transmitirse en ninguna forma ni por ningún medio sin la autorización previa por escrito del editor, excepto en el caso de citas breves incluidas en artículos críticos o reseñas.

En la preparación de este libro se ha hecho todo lo posible para garantizar la precisión de la información presentada. Sin embargo, la información contenida en este libro se vende sin garantía, ni expresa ni implícita. Ni los autores, ni Packt Publishing ni sus distribuidores serán responsables de ningún daño causado o presuntamente causado, directa o indirectamente, por este libro.

Packt Publishing se ha esforzado por proporcionar información sobre las marcas comerciales de todas las empresas y productos mencionados en este libro mediante el uso adecuado de mayúsculas. Sin embargo, Packt Publishing no puede garantizar la exactitud de esta información.

Directora de cartera : Sunith Shetty

Líder de relaciones : Sathya Mohan

Gerentes de proyecto : Aniket Shetty y Sathya Mohan

Ingeniero de contenido : Siddhant Jain

Editor técnico : Aniket Shetty

Editor de copias : Safis Editing

Indexador : Hemangini Bari

Corrector : Siddhant Jain

Diseñador de producción : Deepak Chavan

Primera publicación: septiembre de 2025

Referencia de producción: 1220825

Publicado por Packt Publishing Ltd.

Casa Grosvenor

11 Plaza de San Pablo

Birmingham

B3 1RB, Reino Unido.

ISBN 978-1-83702-263-2

www.packtpub.com

A Ben y Mark por su apoyo inquebrantable.

– Asia

A Luci, Clíodhna, Hannah y Jane. Gracias por todo su apoyo.

– Ger

Gracias a todos mis increíbles amigos y colegas de MongoDB por ayudarme a difundir
la idea de lo maravilloso que puede ser MongoDB.

- Alex

Expresiones de gratitud

Agradecemos a todos los que desarrollaron, documentaron, administraron y dieron soporte a los productos MongoDB a lo largo de los años. En particular, agradecemos a los colegas que hicieron sugerencias o crearon contenido interno que contribuyó a este libro, incluyendo a Cesar Albuquerque de Godoy, Pierre Depretz, Dave Walker, Steve Wotring, Chris Harris y Xiaochen Wu.

Colaboradores

Acerca de los autores

Asya Kamsky es una experta principal en MongoDB, donde trabaja desde 2012. Antes de descubrir MongoDB, dedicó casi dos décadas a persuadir a las bases de datos relacionales para que hicieran lo que nunca fueron diseñadas para hacer en una serie de startups, la mayoría de las cuales nadie recuerda. Asya tiene una amplia experiencia en desarrollo de software, bases de datos y en explicar a la gente lo que hacen.

equivocado.

Ger Hartnett es ingeniero jefe del equipo de rendimiento de producto de MongoDB, donde le apasiona trabajar en fascinantes retos de optimización y escalabilidad. Antes de MongoDB, fundó una startup que creó una plataforma de comunicación para proyectos (que presentaba problemas de escalabilidad). Previamente, diseñó y ajustó software embebido en Intel, donde fue coautor de un libro. Incluso antes, desarrolló sistemas en empresas como Tellabs, Digital y Motorola.

Alex Bevilacqua es el Gerente Principal de Producto de Experiencia del Desarrollador en MongoDB. Antes de unirse a MongoDB en 2018, trabajó como ingeniero de software y arquitecto de sistemas, implementando soluciones en diversos lenguajes, tecnologías y frameworks.

Además de su pasión por la programación, que consume más que sus horas de trabajo, a Alex normalmente se le puede encontrar en un estadio con uno de sus hijos, quienes juegan hockey de representación.

Acerca de los revisores

Keith Smith es un experto en sistemas y almacenamiento con décadas de experiencia en sistemas operativos e infraestructura de datos. Residente en EE. UU., ha contribuido a la innovación en empresas como Sun Microsystems, NetApp y MongoDB. Keith comenzó su carrera diseñando sistemas UNIX en tiempo real y posteriormente desarrolló sistemas operativos extensibles en Harvard, donde obtuvo su doctorado. Su trabajo abarca el almacenamiento basado en objetos, los catálogos de metadatos y las arquitecturas de sistemas de archivos. Desde 2020, lidera el desarrollo de motores de almacenamiento en MongoDB. A Keith le apasionan los sistemas escalables y continúa expandiendo los límites de la tecnología de almacenamiento de datos.

John Page es un ingeniero full-stack que se unió a MongoDB en 2013. Desde entonces, ha ocupado puestos técnicos de alto nivel centrados en el éxito del cliente en todas las fases del ciclo de desarrollo. Antes de trabajar en MongoDB, dedicó 18 años a desarrollar sistemas de bases de datos documentales para las fuerzas del orden y el ejército estadounidense. Geólogo de formación, John puede encontrar silicio en la naturaleza, construir una computadora a partir de componentes básicos y desarrollar todo tipo de sistemas, desde sistemas operativos y bases de datos hasta interfaces y documentación.

James Kovacs es director de ingeniería en Experiencia de Bases de Datos en MongoDB. Se especializa en bibliotecas de cliente, mapeadores de objetos y documentos, integraciones de frameworks e IA/ML. Cuenta con más de 25 años de experiencia profesional en ingeniería de software, habiendo trabajado en diversas empresas e industrias durante ese tiempo. Cuando no está frente al teclado, se le puede encontrar jugando a las mesas.

juegos/TTRPG, cocinar deliciosas comidas veganas en la cocina o enseñar karate en el dojo.

Stephanie Eristoff es ingeniera de software del equipo de Ejecución de almacenamiento de MongoDB, al que se unió en 2024. Aporta experiencia en pruebas de rendimiento de colecciones e índices de series temporales y también ha contribuido a los equipos de Rendimiento y Ejecución de consultas. Fuera del trabajo, a Stephanie le gusta hornear, correr y andar en bicicleta.

Katya Kamenieva es gerente de producto en MongoDB, donde ha impulsado mejoras en el marco de agregación, los flujos de cambios y otras funciones desde 2019. Su trabajo se centra en optimizar la interacción de los usuarios con los datos en MongoDB. Fuera del trabajo, es madre de dos hijas pequeñas y encuentra la atención plena en momentos de tranquilidad, una buena taza de café, yoga y acuarelas.

Jawwad Asghar es ingeniero de software en MongoDB, donde ha sido un miembro clave del equipo de rendimiento desde 2022. Ayuda a impulsar cambios en la base de datos optimizando su rendimiento para casos de uso diversos y exigentes de los clientes. Antes de unirse a MongoDB, trabajó 10 años como ingeniero de software de control en la industria ferroviaria. Cuando no está sentado en su escritorio, disfruta paseando en bicicleta por la ciudad de Nueva York, acampando y haciendo senderismo en el norte del estado.

Alice Doherty es ingeniera de software en MongoDB y se unió al equipo de Rendimiento de Producto en 2023. Aporta una valiosa experiencia en la detección de problemas de rendimiento únicos, tanto para clientes como para equipos de ingeniería internos, además de impulsar mejoras de rendimiento en los productos de MongoDB. Fuera del trabajo, a Alice le gusta mantenerse activa, cocinar y disfrutar de la naturaleza.

Linda Qin es ingeniera de servicios técnicos de MongoDB. Se incorporó a MongoDB en 2013 y desde entonces ha trabajado en el equipo de soporte técnico. Linda se especializa en el diagnóstico de problemas y el ajuste del rendimiento, con amplia experiencia en fragmentación. También participa en el desarrollo de herramientas para optimizar el diagnóstico y mejorar la eficiencia. Fuera del trabajo, a Linda le gusta cocinar y hacer senderismo.

Matt Panton es gerente de producto en MongoDB. Desde su incorporación en 2022, Matt se ha centrado en mejorar las funciones clave de los sistemas distribuidos, como la fragmentación y la replicación. Su trabajo se centra en lograr que MongoDB sea escalable y resiliente.

Fuera del trabajo, a Matt le gusta jugar al fútbol, correr y cocinar.

Sébastien Méndez (pero llámenlo Seb) es ingeniero jefe del equipo de Ejecución de Consultas de MongoDB. En su época, participó activamente en la escena de las demostraciones como "rakiz", creando demostraciones en ensamblador de Z80, creando diskmag en su Amstrad CPC 6128 y organizando fiestas de programación. Tras explorar diversas industrias, Seb descubrió su pasión por el procesamiento de datos: creó una capa de persistencia, un motor de consultas y una canalización de datos en streaming altamente configurable. Actualmente, se especializa en los flujos de cambio de MongoDB, ayudando a definir su hoja de ruta en torno a la observabilidad, el rendimiento y las nuevas funcionalidades. Fuera del trabajo, a Seb le gustan los videojuegos, leer ciencia ficción y fantasía, experimentar con Arduinos y ampliar su colección de camisetas de cultura pop.

Kevin Arhelger es ingeniero del equipo de Servicios Técnicos de MongoDB, donde se centra en mejorar el rendimiento de los clientes en diversos entornos.

Administra sistemas Linux profesionalmente desde 2006 y se especializa en rendimiento de software y bases de datos desde 2008. Con una amplia experiencia...

En administración de servidores, automatización de la nube y ajuste de rendimiento de pila completa, Kevin aporta un enfoque holístico para resolver desafíos técnicos complejos.

Manuel Fontan es tecnólogo sénior del equipo de Currículo de MongoDB. Anteriormente, fue ingeniero sénior de servicios técnicos en el equipo principal de MongoDB. Entretanto, trabajó como ingeniero de fiabilidad de bases de datos en Slack durante poco más de dos años y luego en Cognite hasta su reincorporación a MongoDB. Con más de 15 años de experiencia en desarrollo de software y sistemas distribuidos, es un hombre curioso por naturaleza y posee un Máster en Ingeniería de Telecomunicaciones por la Universidad de Vigo (España) y un Máster en Software Libre y de Código Abierto por la Universidad Rey Juan Carlos (España).

Parker Faucher es un ingeniero de software autodidacta con más de seis años de experiencia en educación técnica. Ha creado más de 100 vídeos educativos para MongoDB, lo que lo ha consolidado como un experto en tecnologías de bases de datos. Actualmente, Parker se centra en la IA y las tecnologías de búsqueda, explorando soluciones innovadoras en estos campos en rápida evolución. Cuando no está desarrollando sus conocimientos técnicos, Parker disfruta de pasar tiempo de calidad con su familia y mantiene un gran interés por coleccionar cómics.

Sarah Evans es ingeniera curricular en MongoDB, donde se especializa en hacer accesibles conceptos técnicos complejos para todos los estudiantes. Con más de diez años de experiencia en ingeniería de software, educación y desarrollo de currículos para bootcamps técnicos, su objetivo es hacer que aprender MongoDB sea menos intimidante y más emocionante. Cuando no está ayudando a desarrolladores a mejorar sus conocimientos de bases de datos, Sarah se encuentra bailando espontáneamente con...

su familia, explorando el aire libre con su perro o cuidando su jardín.

Contenido

[Agradecimientos Prefacio](#)

[Cómo le](#)

[ayudará este libro A quién va](#) ____

[dirigido este libro Qué](#) ____

[cubre este libro Para](#) ____

[aprovechar al máximo este libro](#) ____

[Póngase en](#) ____

[contacto con nosotros 1. Sistemas y arquitectura de MongoDB](#)

[¿Qué son los sistemas?](#)

[Características de los sistemas](#) ____

[Cambiar los sistemas es un negocio arriesgado Un](#) ____

[sistema sin retrasos es simple Un sistema con](#) ____

[retrasos puede comportarse de maneras inesperadas Intentando arreglar](#) ____

[las oscilaciones Los sistemas](#) ____

[nos sorprenden Un sistema](#) ____

[de software típico Eficiencia](#) ____

[algorítmica \(complejidad\)](#) ____

[Evite la optimización prematura Ley de](#) ____

[Amdahl \(límite de aceleración paralela\)](#) ____

[Localidad y almacenamiento](#) ____

[en caché Ley de Little \(rendimiento versus latencia\)](#) ____

[Comprensión de la arquitectura de MongoDB](#) ____

[El modelo de documento: la base de MongoDB](#) ____

[Componentes arquitectónicos clave de MongoDB](#) ____

[El sistema de servicios de datos](#) ____

[Motor de consultas](#) ____

[Motor de almacenamiento/WiredTiger](#) ____

[Bibliotecas](#) ____

[Otros componentes del sistema que utiliza mongod](#) ____

[Gestión de la complejidad en las plataformas de datos modernas](#)

[Modelo de datos flexible con capacidades rigurosas](#) ____

[Redundancia y resiliencia integradas](#) ____

Escalamiento horizontal con distribución inteligente

Herramientas de rendimiento

Encontrar cuellos de botella

Un proceso incremental para la optimización

Resumen

Referencias

2. Diseño de esquemas para el rendimiento

Comprender los principios básicos del diseño de esquemas

No existe una única manera correcta

Colocación de datos

Leer y escribir compensaciones

Documentos pequeños versus grandes

Mitos comunes

Puntos fuertes del diseño del esquema de MongoDB

Relaciones de uno a muchos

Incorporación de entidades débiles

Atributos dinámicos

Cachés e instantáneas

Optimización para casos de uso comunes

Evolución del esquema

Validación de esquemas

Errores comunes en el diseño de esquemas

Sobrenormalización

Sobreincrustación

Otros antipatrones comunes

Patrones de diseño de esquemas por beneficio

Patrones para optimizar el rendimiento de lectura

Patrones para optimizar el rendimiento de escritura

Patrones para la optimización de consultas y análisis

Patrón de archivo para la optimización del almacenamiento

Aplicación en el mundo real: La aplicación Socialite

Escenario 1: Perfil de usuario y feed de actividad

Escenario 2: Sistema de chat

Resumen

Índices

Introducción a

los índices

¿Qué es un índice?

Eficiencia de recursos y compensaciones Uso de recursos Conceptos

erróneos comunes sobre los índices en MongoDB Tipos de índices en

MongoDB Índices de campo único Índices

compuestos Índices

multiclave Índices dispersos

Índices comodín Índices

parciales

Diseño de índices eficientes

Cardinalidad y selectividad

Construcción de índices compuestos Consultas de igualdad

Ordenamientos y consultas de

rango La directriz ESR

Maximización de recursos con índices parciales Consultas

cubiertas: el santo grial del rendimiento Orden de índice

ascendente versus descendente Tuberías de indexación y

agregación Resumen 4. Agregaciones Marco de

agregación

de MongoDB Conceptos

básicos de la tubería de agregación Consideraciones

de rendimiento

Flujo de la tubería de agregación

Optimización de las canalizaciones de agregación

Técnicas de optimización

Filtrar datos de forma anticipada

Evite los \$unwind y \$group innecesarios

Diseñar operaciones grupales eficientes

Evite problemas comunes de rendimiento de \$lookup

Uso eficiente de \$project y \$addFields

Trabajar con grandes conjuntos de datos

Límites de la canalización de agregación

Administración de restricciones de memoria con allowDiskUse Agregación en entornos distribuidos Optimización de la agregación para colecciones fragmentadas Comprensión de las operaciones locales de fragmentos frente a las fusionadas Supervisión y creación de perfiles del rendimiento de la agregación Utilización de vistas materializadas Resumen 5. Replicación Comprensión de los conjuntos de réplicas de MongoDB .

Componentes de un

conjunto de réplicas Replicación y alta disponibilidad Comprensión del proceso de elección de MongoDB Configuración del conjunto de réplicas . Replicación encadenada Etiquetas del conjunto de réplicas y nodos de análisis _____

Internos y rendimiento de la replicación _____

Control de flujo

Replicación y el oplog _____
Gestión del retraso de replicación _____

Estrategias de lectura y escritura _____

Preferencia de lectura _____
Preocupación y durabilidad de la escritura .

Resumen 6. .

Fragmentación

Comprensión de la arquitectura central de fragmentación _____
Componentes arquitectónicos de un clúster fragmentado _____
Fragmentación de una colección y selección de una clave de fragmentación .
Por qué es malo dispersar y reunir _____
Selección estratégica de claves de fragmentos _____
Clave de fragmento para operaciones de segmentación _____
Clave de fragmento con buena granularidad .
Evite aumentar o disminuir los valores de clave de fragmento _____

Tipos de fragmentación _____

Fragmentación basada en rangos _____
Fragmentación hash _____
Fragmentación basada en zonas _____

Administración avanzada de fragmentación

Refragmentación: si, cuándo y cómo Consideraciones sobre el balanceador División previa:

si, cuándo y cómo Mover colecciones no fragmentadas

Colocar fragmentos de colecciones

fragmentadas juntos Resumen 7. Motores de almacenamiento

Exploración

de los motores de

almacenamiento

Descripción general de WiredTiger

Una operación de búsqueda

Una operación de actualización

Una operación de inserción

Desalojo, puestos de control y recuperación

Compresión y cifrado

Configuración para mejorar el rendimiento

Cambiar el tamaño de la caché de WiredTiger Cambiar

syncdelay Cambiar

minSnapshotHistoryWindowInSeconds Cambiar el funcionamiento del

desalojo Cambiar al motor de almacenamiento

en memoria Cambiar el tamaño máximo de la página de hoja

Resumen 8. Flujos de cambio Comprensión de

la arquitectura

de los flujos de cambio

Cómo funcionan los flujos de cambio: desde operaciones de escritura hasta eventos

Estructura y ciclo de vida del evento

Implementar flujos de cambio de manera efectiva

Cómo elegir el alcance y la estrategia de filtrado adecuados

Filtrado del lado del servidor con canales de agregación

Estrategias y rendimiento de la búsqueda de documentos

Creación de un servicio de seguimiento de precios

Gestión del rendimiento y la durabilidad

Estrategias de optimización de recursos

Manejo de flujos de eventos de gran volumen

Consideraciones especiales para implementaciones fragmentadas	
Patrones avanzados y preparación para la producción	
Visibilidad de transacciones y agrupación de eventos	
Limitaciones del tamaño de los documentos y ciclo de vida de la	
recopilación Supervisión y	
comprobaciones del estado	
Consideraciones sobre el conjunto	
de réplicas	
Resumen del ajuste	
del rendimiento Resumen 9. Transacciones Descripción de las transacciones ACID de varios docum	
Historia y evolución de las transacciones en MongoDB	
Introducción a las propiedades ACID en MongoDB	
Atomicidad a nivel de documento versus transacciones multidocumento	
Atomicidad a nivel de documento	
Atomicidad de transacciones multidocumentales	
Cuándo utilizar transacciones multidocumento en	
MongoDB	
API de transacciones y gestión de sesiones	
API principal	
API de devolución de llamada	
Preocupaciones de lectura/escritura y comportamiento de las transacciones	
Consideraciones sobre el rendimiento en las transacciones	
Conjunto de réplicas frente a transacciones de clúster fragmentado	
Consideraciones sobre la caché de WiredTiger	
Gestión de errores y límites de tiempo de ejecución de transacciones	
Adquisición de bloqueos y gestión de contenciones	
Optimización del tamaño y la duración de las transacciones	
Antipatrones de transacciones comunes y sus costos de desempeño	
Transacciones de larga duración y su impacto en el rendimiento del sistema	
Uso innecesario de transacciones en las que la atomicidad de un	
solo documento sería suficiente	
Transacciones de un solo documento	
Transacciones para operaciones de solo lectura	

Malentendido del alcance y la atomicidad de las transacciones .

Pequeñas transacciones frecuentes en documentos/colecciones

activos Manejo de errores y lógica de reintento __
inadecuados Monitoreo insuficiente de las métricas de
las .

transacciones Resumen 10. Bibliotecas de cliente

¿Qué son los controladores?

Cómo funcionan los controladores

MongoDB Características principales de los

controladores MongoDB Consistencia y confiabilidad a
través de

especificaciones compartidas

Experiencia idiomática Optimización

del rendimiento ¿Qué son los mapeadores de documentos de objetos (ODM)?

Descripción de los ODM

Características principales

de los ODM Aplicación de esquemas y validación

de datos API de _____

consulta intuitivas Gestión de

relaciones Middleware y ganchos de

ciclo de vida Seguridad de tipos e

integración de IDE Impacto en la .

productividad del desarrollador Cuándo usar ODM

¿Qué son los marcos de aplicación?

El valor de los marcos de aplicación

Aprovechamiento de los ODM y ORM en los marcos

Marcos populares compatibles con MongoDB

Mejores prácticas al usar frameworks con MongoDB _____

Más allá de lo básico

Patrones asíncronos y no bloqueantes _____

Condiciones de falla en la superficie y manejo

Gestión de conexiones _____

Preocupaciones de lectura/escritura y preferencias de lectura

Compresión y rendimiento de la red _____

Resumen .

11. Gestión de conexiones y rendimiento de la red

Comprender los fundamentos de la conexión

Estado latente

Rotación de conexiones

Saturación de la red

Comprender el ciclo de vida de la conexión

Establecimiento de conexión

Utilización y agrupación de conexiones

Terminación de la conexión

Arquitectura de conexión de MongoDB

TCP/IP y el protocolo de cable MongoDB

Agrupación de conexiones de controladores

Manejo de la conexión al servidor MongoDB

Monitoreo y resolución de problemas de conexiones

Mejores prácticas para la monitorización de conexiones

Optimización de la administración de la conexión

Optimización del grupo de conexiones

Configuración del tiempo de espera de la

conexión Optimización del lado del

servidor Configuración del sistema operativo

Optimización del rendimiento aprovechando la compresión de red Beneficios y

desventajas de la compresión de red Algoritmos de compresión

disponibles Implementación de la compresión de

red Estrategias de conexión para entornos sin

servidor Resumen 12. Conceptos avanzados de consulta e indexación

Comprensión

de la ejecución de consultas Etapas del plan o "cómo se pueden
usar los índices"

Usando el comando explicar

La sección queryPlanner

La sección de estadísticas de ejecución

Análisis de mensajes de registro

Identificación de patrones problemáticos

Relación de segmentación de consultas

Esperando disco u otros recursos

Cómo influir en la ejecución de consultas

Usando la pista

Uso de la configuración de consultas

Otras opciones

Semántica e índices de MQL

Desafíos con matrices e índices multiclave La igualdad no es solo
igualdad \$elemMatch Desduplicación .

Desafíos con null

y \$exists Mejores

prácticas adicionales Actualizaciones _____

findAndModify Agregación y consulta

\$sample _____

\$facet \$where _____ .

\$function, \$accumulator y _____ .

mapReduce

\$text _____

\$regex e índices Agregación versus expresiones de coincidencia \$expr e

índices

Cláusula \$or e índices _____

Colecciones especiales, tipos de índices y características

Colecciones de series _____

temporales Índices geoespaciales

Búsqueda en Atlas Búsqueda vectorial en Atlas Intercalaciones _____

Resumen 13. Sistemas operativos

y recursos del sistema _____

_____ .

Requisitos técnicos

Gestión de recursos para un rendimiento óptimo _____

Utilización de la CPU

Gestión de la memoria _____

Almacenamiento

Red _____

Configuración de sistemas para el rendimiento de MongoDB

Comprenda la relación ideal de operaciones simultáneas por núcleo de CPU

Busque otros procesos que consumen mucha CPU durante las caídas de rendimiento

Seleccione el sistema de archivos adecuado para su aplicación

Sistema de archivos de extensión (XFS)

Cuarto sistema de archivos extendido (ext4)

Otros sistemas de archivos

Configuración del sistema de archivos

Evite RAID con paridad

Ajustar la configuración de lectura anticipada

Cambiar la configuración de caché del sistema de archivos

Comprobar el estado del SSD

Evite el doble cifrado y la compresión

Asegúrese de que el uso de la memoria residente se mantenga por debajo del 80 %

Mejores prácticas de redes Uso del escalado

automático para el rendimiento de Atlas Resumen 14. Monitoreo y

observabilidad

Diferencias clave entre el monitoreo y la observabilidad

Métricas y señales principales de MongoDB Métricas operativas Métricas específicas de

rendimiento

Monitoreo con MongoDB Atlas

Características de la interfaz de usuario de Atlas

Panel de rendimiento en tiempo real (RTPP)

Asesor de rendimiento

Información de consultas

Alertas

Problemas con la búsqueda en Atlas

Problemas de conexión

Problemas de consulta

Problemas de oplog

Herramientas de monitorización autogestionadas

mongostat: instantánea de actividad en tiempo real

[mongotop: Tiempos de lectura y escritura a nivel de colección](#) [—](#)

[serverStatus: Métricas completas a través del shell](#)

[Perfilador de bases de datos: análisis profundo de las operaciones lentas](#)

[Integración con sistemas de monitorización externos](#) [—](#) [—](#)

[Prometeo + Grafana](#)

[Integración de Atlas para Prometheus](#)

[Visualizando con Grafana](#)

[Perro de datos](#)

[Plataformas de monitorización del rendimiento de aplicaciones](#)

[OpenTelemetry](#) [.](#)

[Consideraciones para herramientas externas](#)

[Patrones de rendimiento comunes y qué monitorear](#)

[Cuellos de botella de E/S de](#)

[disco Presión de caché \(utilización de caché de WiredTiger\)](#) [.](#)

[Retraso de replicación y deriva de la ventana del](#)

[registro de](#) [.](#)

[operaciones Resumen 15. Depuración de](#)

[problemas de rendimiento Identificación de](#)

[operaciones inherentemente lentas Caso práctico: Solución de](#)
[problemas](#)

[de rendimiento del clúster](#)

[con Atlas Diagnóstico de la](#)

[causa Identificación de la causa](#)

[raíz Implementación de la](#) [—](#)

[solución Resultados y aprendizajes Caso práctico: Consulta](#)
[de administrador](#)

[inesperada que provoca un](#)

[análisis de recopilación Diagnóstico](#)

[de la causa Implementación](#)

[de la solución Resultados y aprendizajes](#)

[Gestión de operaciones bloqueadas Caso](#) [—](#)

[práctico: Investigación de](#)

[fugas del cursor Diagnóstico de](#)

[la causa Identificación de la causa](#)

[raíz Implementación de la](#) [—](#)

[solución Resultados y aprendizajes Caso práctico: Explosión de consultas mal optimizadas](#)

[Diagnosticar la causa _____](#)

[Implementando la solución _____](#)

[Resultados y aprendizajes _](#)

[Abordar las limitaciones de recursos de hardware _____](#)

[Estudio de caso: Optimización de recursos del clúster Atlas _____](#)

[Diagnosticar la causa _____](#)

[Implementando la solución _____](#)

[Resultados y aprendizajes _](#)

[Caso de uso: Diagnóstico de IOP insuficientes en _____](#)

[MongoDB autogestionado _____](#)

[Diagnosticar la causa _____](#)

[Implementando la solución _____](#)

[Resultados y aprendizajes _](#)

[Enfoque sistemático para la resolución de problemas de rendimiento _____](#)

[Resumen _ .](#)

[Epílogo _____](#)

[Conclusiones clave _____](#)

[La mentalidad de rendimiento _____](#)

[Próximos pasos prácticos _____](#)

[Consideraciones futuras _____](#)

[Reflexiones finales _____](#)

[Índice _____](#)

[Otros libros que te pueden gustar _ .](#)

Prefacio

En el mundo actual, impulsado por los datos, el rendimiento no es solo una característica; es una necesidad. A medida que los volúmenes de datos crecen exponencialmente y las expectativas de los usuarios en cuanto a velocidad y fiabilidad aumentan, la capacidad de crear y mantener sistemas de bases de datos de alto rendimiento se ha vuelto crucial para las aplicaciones competitivas. MongoDB es una base de datos popular y potente, diseñada para las necesidades de las aplicaciones modernas. Almacena datos en documentos que se alinean naturalmente con las estructuras de datos utilizadas en los lenguajes de programación. Su arquitectura distribuida proporciona escalabilidad integrada, alta disponibilidad y rendimiento para cargas de trabajo críticas. Al comprender la arquitectura, los patrones operativos y las técnicas de optimización de MongoDB, podrá aprovechar al máximo su potencial de rendimiento para crear aplicaciones rápidas, e

Este libro ofrece una guía práctica para comprender y mejorar el rendimiento de MongoDB. Explica cómo identificar los problemas de rendimiento más comunes, analizar el comportamiento de las consultas y del sistema mediante herramientas integradas y aplicar técnicas de optimización probadas. Los temas abarcan desde el diseño de esquemas y las estrategias de indexación hasta el ajuste de la configuración de almacenamiento y la gestión de los recursos del sistema. Tanto si ejecuta un único conjunto de réplicas como si opera un gran clúster fragmentado, el libro le ayudará a tomar decisiones informadas que mejoren la estabilidad y la eficiencia en implementaciones reales.

Cómo le ayudará este libro

Este libro profundiza en conceptos fundamentales y técnicas de optimización avanzadas, brindándole el conocimiento y las herramientas prácticas para diseñar, construir y operar implementaciones de MongoDB de alto rendimiento a cualquier escala.

Ya sea que esté solucionando problemas con una consulta lenta, diseñando esquemas para una nueva aplicación o escalando un clúster grande para gestionar cargas de trabajo crecientes, este libro es su recurso de confianza. El objetivo es desmitificar la optimización del rendimiento de MongoDB y brindarle estrategias que funcionen en entornos reales.

Los capítulos están diseñados para ser modulares, lo que permite centrarse en áreas específicas de interés, ya sea el ajuste de índices, la resolución de consultas lentas o la configuración de recursos del sistema. Sin embargo, quienes se inician en el ajuste del rendimiento pueden beneficiarse de una lectura lineal para construir una base conceptual sólida. Si bien algunos temas se basan naturalmente en capítulos anteriores, la mayoría de las secciones están diseñadas para ser comprensibles por sí solas. Esto hace que el libro sea flexible como lectura continua y una guía de referencia para resolver problemas de rendimiento específicos a medida que surjan. A lo largo del libro, encontrará ejemplos prácticos, técnicas de diagnóstico y fragmentos de código que puede adaptar a su entorno para un aprendizaje práctico.

Para quién es este libro

Este libro está dirigido a desarrolladores, arquitectos de sistemas, ingenieros de DevOps y administradores de bases de datos que se toman en serio la optimización del rendimiento de MongoDB. Tanto si crea aplicaciones con uso intensivo de datos, como si es responsable del escalado y la gestión de MongoDB en producción, del diseño de sistemas distribuidos para el crecimiento futuro o de la integración de MongoDB en pipelines de CI/CD, encontrará una guía práctica adaptada a sus necesidades. Se asume un conocimiento básico de MongoDB, y si bien es útil la experiencia en diseño de esquemas u operaciones de bases de datos, cada capítulo se desarrolla progresivamente para apoyar a estudiantes de todos los niveles.

Qué cubre este libro

[Capítulo 1](#) Sistemas y arquitectura MongoDB , introduce el núcleo Conceptos de pensamiento sistémico y muestra cómo se aplican a MongoDB. Arquitectura, ajuste del rendimiento e identificación de cuellos de botella.

[Capítulo 2](#) , Diseño de esquemas para el rendimiento , cubre cómo documentar Las opciones de modelado afectan directamente la eficiencia de lectura y escritura, una de las Aspectos más críticos del rendimiento de MongoDB.

[Capítulo 3](#) , Índices , profundiza en las estrategias de indexación, la herramienta principal para optimizar el rendimiento de las consultas.

[Capítulo 4](#) Agregaciones , Le muestra cómo utilizar la agregación marco para realizar análisis de datos complejos de manera eficiente.

[Capítulo 5](#) , Replicación , analiza la replicación de MongoDB mecanismo y cómo aprovecharlo eficazmente para lograr una alta disponibilidad.

[Capítulo 6](#) , Sharding , explica técnicas de escalamiento horizontal para Distribuir datos y cargas de trabajo entre múltiples servidores.

[Capítulo 7](#) , Motores de Se centra en WiredTiger, MongoDB almacenamiento , motor de almacenamiento y explica sus opciones de configuración y Características de rendimiento.

[Capítulo 8](#) , Cambio de flujos , muestra cómo construir sistemas reactivos con notificaciones de cambios de datos en tiempo real.

[Capítulo 9](#) , Actas , Cubre ACID multidocumento transacciones, sus casos de uso y sus implicaciones en el rendimiento.

[Capítulo 10](#), Bibliotecas de clientes , examina cómo configurar y utilizar Controladores MongoDB para un rendimiento óptimo de la aplicación.

[Capítulo 11](#), Gestión de conexiones y rendimiento de la red , detalla estrategias de agrupación de conexiones y optimización de la red técnicas.

[Capítulo 12](#), Conceptos avanzados de consulta e indexación , explora Temas de indexación avanzada y lenguaje de consulta MongoDB (MQL) Mejores prácticas.

[Capítulo 13](#), Sistema operativo y recursos del sistema , proporciona Orientación sobre la configuración de su sistema operativo y hardware para Cargas de trabajo de MongoDB.

[Capítulo 14](#), Monitoreo y Observabilidad , proporciona una Guía completa para supervisar su implementación de MongoDB y establecer una gestión proactiva del rendimiento.

[Capítulo 15](#), Metodologías de depuración de , te equipa con problemas de rendimiento para identificar, diagnosticar y resolver problemas de rendimiento. cuellos de botella en la producción.

Para aprovechar al máximo este libro

Necesitará el siguiente software:

Software cubierto en el libro	Requisitos del sistema operativo
MongoDB versión 8.0	Windows, macOS o Linux
Búsqueda en MongoDB Atlas	Windows, macOS o Linux
Shell de MongoDB	Windows, macOS o Linux

Después de leer este libro, le recomendamos que consulte algunos de los otros recursos disponibles en

<https://www.mongodb.com/developer> y <https://learn.mongodb.com/>

Descargue los archivos de código de ejemplo

El paquete de código del libro está alojado en GitHub en

<https://github.com/PacktPublishing/Alto-Rendimiento-con-MongoDB/>

También tenemos otros paquetes de códigos de nuestro rico catálogo de libros y videos disponibles en <https://github.com/PacktPublishing>
¡Échales un vistazo!

Descarga las imágenes a color. También proporcionamos un archivo PDF con imágenes a color de las capturas de pantalla/diagramas utilizados en este libro. Puedes descargarlo aquí: <https://packt.link/gbp/9781837022632> .

Convenciones utilizadas

A lo largo de este libro se utilizan varias convenciones textuales.

`CodeInText` : Indica palabras clave en el texto, nombres de tablas de bases de datos, nombres de carpetas, nombres de archivos, extensiones de archivo, rutas de acceso, URL ficticias, entradas del usuario y manejadores X (Twitter). Por ejemplo: « \$project y \$addField tienen propósitos diferentes y tienen diferentes implicaciones de rendimiento ».

Un bloque de código se establece de la siguiente manera:

```
A documento simple que representa # a usuario
usuario =
{ "nombre": "Jane Doe",
  "edad": 30,
  "correo electrónico": "janedoe@example.com",
  "dirección": { "calle":
    "123 Main St", "ciudad": "Cualquier
    ciudad", "estado": "CA",
    "código postal":
    "12345"
  },
  "intereses": ["senderismo", "fotografía", "viajes"]
}
```

Cualquier entrada o salida de la línea de comandos se escribe de la siguiente manera:

```
db.adminCommand({ estado del servidor: 1 })
```

Negrita : Indica un término nuevo, una palabra importante o palabras que aparecen en pantalla. Por ejemplo, las palabras en menús o cuadros de diálogo aparecen en el texto así: "En Grafana, vaya a la pantalla Importar (haga clic en el icono + y seleccione Importar)".



Las advertencias o notas importantes aparecen así.



Los consejos y trucos aparecen así:

Ponte en contacto con nosotros

Los comentarios de nuestros lectores siempre son bienvenidos.

Comentarios generales : si tiene preguntas sobre cualquier aspecto de este libro o tiene algún comentario general, envíenos un correo electrónico a customercare@packt.com y mencione el título del libro en el asunto de su mensaje.

Errata : Aunque hemos tomado todas las precauciones para garantizar la precisión de nuestro contenido, pueden producirse errores. Si encuentra algún error en este libro, le agradeceríamos que nos lo comunicara. Visite <http://www.packt.com/submit-errata> , haga clic en Enviar erratas , y rellena el formulario.

Piratería : Si encuentra copias ilegales de nuestras obras en cualquier formato en Internet, le agradeceríamos que nos las proporcionara.

Con la dirección de la ubicación o el nombre del sitio web. Por favor, contáctenos en copyright@packt.com con un enlace al material.

Si está interesado en convertirse en autor : si hay un tema en el que es experto y está interesado en escribir o contribuir a un libro, visite <http://authors.packt.com/> _____

.

Comparte tus pensamientos

Una vez que hayas leído " Alto Rendimiento con MongoDB" ,¡nos encantaría conocer tu opinión! Haz [clic aquí para ir directamente a la página de reseñas de Amazon](#) y compartir tus comentarios.

Su reseña es importante para nosotros y para la comunidad tecnológica y nos ayudará a garantizar que ofrecemos contenido de excelente calidad.

Descargue una copia gratuita en PDF de este libro

¡Gracias por comprar este libro!

¿Te gusta leer mientras viajas pero no puedes llevar tus libros impresos a todas partes?

¿Tu compra de libro electrónico no es compatible con el dispositivo que eliges?

No te preocupes, ahora con cada libro de Packt obtendrás una versión PDF de ese libro sin DRM sin costo.

Lee en cualquier lugar, en cualquier dispositivo. Busca, copia y pega código de tus libros técnicos favoritos directamente en tu aplicación.

Los beneficios no terminan ahí, puedes obtener acceso exclusivo a descuentos, boletines informativos y excelente contenido gratuito en tu bandeja de entrada todos los días.

Siga estos sencillos pasos para obtener los beneficios:

1. Escanea el código QR o visita el siguiente enlace:



<https://packt.link/libro-e-gratuito/9781837022632>

2. Envíe su comprobante de compra.
3. ¡Listo! Te enviaremos tu PDF gratuito y otros beneficios directamente a tu correo electrónico.

1

Sistemas y MongoDB Arquitectura

Toda implementación de base de datos opera dentro de un sistema más amplio que incluye redes, aplicaciones, hardware y usuarios. Para escalar u optimizar MongoDB eficazmente, debemos abordarlo con la mentalidad de un diseñador de sistemas, en lugar de pensar únicamente como ingenieros de bases de datos. Este capítulo presenta ese cambio de mentalidad. Le ayudará a comprender cómo pequeños cambios en un área pueden tener efectos significativos, y a veces inesperados, en todo el sistema. Resolver problemas de rendimiento no se trata solo de ajustar la configuración técnica; también requiere una comprensión profunda del comportamiento de los sistemas.

Comenzaremos analizando los sistemas de forma más amplia, comenzando con ejemplos de la naturaleza y del mundo real. Se presentarán conceptos clave como retrasos, bucles de retroalimentación y cuellos de botella. Estas ideas ayudarán a explicar por qué los sistemas a veces se comportan de forma contraintuitiva o sorprendente cuando intentamos modificarlos. A medida que avance el capítulo, iremos reduciendo gradualmente nuestro enfoque. Pasaremos de los sistemas generales a los sistemas de software, luego a los servicios de datos y, finalmente, a MongoDB. A lo largo del camino, ofreceremos una visión general de los cuellos de botella comunes y examinaremos dos ejemplos específicos con más detalle. También presentaremos una guía paso a paso.

proceso que puede utilizar para diagnosticar y resolver problemas de rendimiento y escalabilidad.

Todo sistema contiene un factor limitante o cuello de botella. Cuando un sistema tiene múltiples entradas y componentes, el verdadero factor limitante no siempre es evidente. Mejorar otras partes del sistema no aumentará el rendimiento general a menos que se aborde el cuello de botella en sí. En mi experiencia, muchas personas dedican tiempo a optimizar la parte equivocada del sistema porque hacen suposiciones incorrectas sobre dónde se encuentra el cuello de botella. Un enfoque estructurado, basado en la medición y la experimentación, facilita la identificación y la solución del verdadero problema.

asunto.

Este capítulo cubrirá los siguientes temas:

- Examinar las características fundamentales de los sistemas
- Exploración de un sistema de software típico e identificación de posibles cuellos de botella en el rendimiento
- Introducción a la arquitectura de MongoDB y sus fundamentos
- Comprender la conexión entre los componentes principales de MongoDB y su papel en el rendimiento de la aplicación
- Debate sobre estrategias para gestionar la complejidad en las plataformas de datos modernas
- Revisión de herramientas de monitoreo del rendimiento y técnicas de observabilidad
- Encontrar cuellos de botella y el proceso iterativo para su ajuste

¿Qué son los sistemas?

Un sistema es un conjunto interconectado de elementos organizados para cumplir un propósito. Donella Meadows define los sistemas de la siguiente manera:



Un conjunto de cosas —personas, células, moléculas o lo que sea— interconectadas de tal manera que producen su propio patrón de comportamiento a lo largo del tiempo. El sistema puede verse afectado, restringido, desencadenado o impulsado por fuerzas externas. Pero la respuesta del sistema a estas fuerzas es característica de sí misma, y esa respuesta rara vez es simple en el mundo real. [1]

Vemos sistemas a muchas escalas. Nuestro sistema solar es un sistema de planetas. La Tierra tiene sistemas meteorológicos y climáticos, así como sistemas políticos y sociales como países y ciudades. Una ciudad contiene sistemas institucionales como las universidades, que, a su vez, tienen sistemas académicos como facultades, profesores y estudiantes. La facultad de ingeniería cuenta con un sistema de laboratorio con múltiples computadoras conectadas mediante una red de puentes y enrutadores. Cada computadora es un sistema de componentes de hardware y software, que incluye procesadores, memoria y aplicaciones.

Una tienda en la ciudad es un sistema que gestiona productos como computadoras, con procesos para la reposición de inventario de proveedores y fábricas. Más allá de la ciudad, un bosque es un sistema ecológico que contiene árboles, cada uno con sus raíces, ramas y hojas.

Un jabalí en el bosque posee sistemas biológicos, como el esquelético, el nervioso, el circulatorio y el digestivo. Incluso una sola célula dentro de su oreja es un sistema, con sus propios subsistemas internos, como las mitocondrias, el núcleo y los ribosomas.

El hilo común que unifica a todos estos diferentes sistemas es que sus elementos interactúan entre sí dentro del sistema y con otros sistemas que los rodean.

El rendimiento de un sistema no suele estar determinado por el elemento más fuerte o rápido, sino por el eslabón más débil. Esto a veces se denomina cuello de botella o factor limitante . Este es el elemento dentro de un sistema que restringe significativamente su rendimiento o crecimiento general. Los cuellos de botella pueden cambiar con el tiempo. Si se soluciona un cuello de botella, otro elemento se convertirá en el cuello de botella. Recuerde que todo lo que es válido para los sistemas generales también lo es para los sistemas de software. En secciones posteriores, analizaremos los cuellos de botella de software típicos y algunas de las tácticas que puede utilizar para solucionarlos.

Características de los sistemas

En esta sección, analizaremos los posibles problemas que podemos encontrar al cambiar de sistema. Veremos cómo los buffers y los retrasos pueden dificultar la comprensión del comportamiento de un sistema y cómo los cambios pueden generar resultados inesperados.

Cambiar los sistemas es un asunto arriesgado . Los elementos de un sistema pueden interactuar de forma no lineal. Por ejemplo, una escuela es un sistema con elementos como estudiantes, aulas, patios de recreo y canastas de baloncesto. El comportamiento del sistema no siempre es predecible, como se explica aquí:

- Pequeños cambios pueden tener grandes efectos (efecto mariposa) : Una acción insignificante puede tener consecuencias enormes e impredecibles. Por ejemplo, un estudiante empieza a correr en el patio. Otro estudiante corre tras el primero.
Pronto, más y más estudiantes ven la diversión y se unen. De repente, la mitad del patio está jugando a la mancha.
- Ciclos de retroalimentación (reacción en cadena) : Este es un proceso directo, paso a paso, donde un evento lleva a otro. En la cafetería, un estudiante cuenta un chiste. Si es gracioso, algunos otros se ríen. Su risa despierta la curiosidad de los demás en la cafetería, y pronto, todos ríen, incluso quienes no lo oyeron. La risa se propaga y se fortalece a sí misma. Esto se llama ciclo de retroalimentación positiva .
- Más que la suma de sus partes (sinergia) : Un sistema puede producir resultados mayores que la simple suma de sus componentes.
Durante los ensayos de banda, un solo golpe de batería suena simple. Una sola nota de trombón es genial. Pero cuando toda la banda toca junta, la música suena mucho mejor que simplemente sumando sonidos individuales.
- Resultados inesperados (emergencia) : Los sistemas pueden generar resultados que no son evidentes a partir de sus componentes individuales. Por ejemplo, en la clase de arte, un estudiante mezcla pintura roja y amarilla para obtener naranja, lo cual le sorprende.

Las interacciones no lineales implican que pequeñas acciones pueden desencadenar grandes cambios, los ciclos de retroalimentación pueden amplificar sus efectos, los elementos pueden interactuar inesperadamente y pueden surgir nuevos resultados.

En general, a los humanos nos cuesta comprender las interacciones no lineales entre diferentes...

elementos del sistema y tienden a ver el comportamiento como una serie de eventos discretos, lo que hace fácil pasar por alto los patrones subyacentes.

Podemos construir modelos de sistemas y observar su comportamiento. Por supuesto, el mundo real siempre será más complejo que cualquier modelo que podamos construir. Un modelo de sistema tiene entradas, existencias, variables y bucles de retroalimentación. La Figura 1.1 muestra un ejemplo de un sistema de almacenamiento/inventario .

Usaremos este ejemplo para explicar los conceptos de existencias, variables y ciclos de retroalimentación. Leyendo desde la esquina superior izquierda en sentido antihorario, el sistema tiene una entrada de entregas y una salida de ventas. Los productos de la tienda están en stock, lo que se denomina inventario.

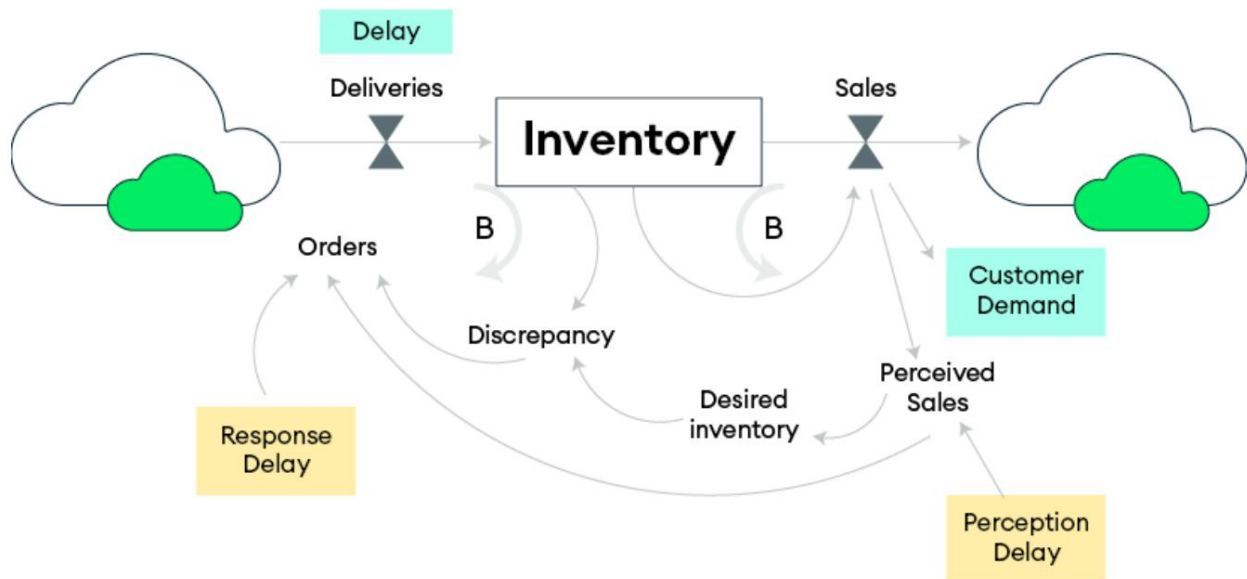


Figura 1.1: Un modelo de sistema de ejemplo que muestra la gestión del inventario de la tienda

El retraso en la entrega y la demanda del cliente son factores que regulan el flujo de productos. B significa un ciclo de retroalimentación equilibrado. La demanda del cliente hace que los productos salgan del sistema como ventas, lo que reduce el inventario. El gerente no quiere seguir revisando...

Existencias diarias, por lo que existe una variable de retraso de percepción. Tras el retraso, el gerente compara las ventas percibidas con el inventario deseado. Para evitar una reacción exagerada ante un cambio en la demanda del cliente, cuenta con una variable de retraso de respuesta que limita el tamaño del pedido al proveedor, quien entrega más producto tras un retraso.

Este modelo solo utiliza bucles de retroalimentación de equilibrio. El otro tipo de bucle de retroalimentación es el bucle de refuerzo, que suele aumentar el tamaño de las acciones. Por ejemplo, en una cuenta de ahorros, cuanto más dinero se tenga en ella, más intereses se generan, lo que, a su vez, aumenta la cantidad de dinero en la cuenta.

Un sistema sin retrasos es sencillo. Supongamos que el gerente desea mantener un inventario de 10 días, con una demanda de 200 artículos diarios. Esto significa que el inventario deseado es de 2000. El día 10, la demanda aumenta un 10 %. Los clientes compran ahora 220 artículos diarios. El gerente aumenta el tamaño del pedido y se entrega más inventario ese mismo día. El inventario deseado aumenta a 2200.

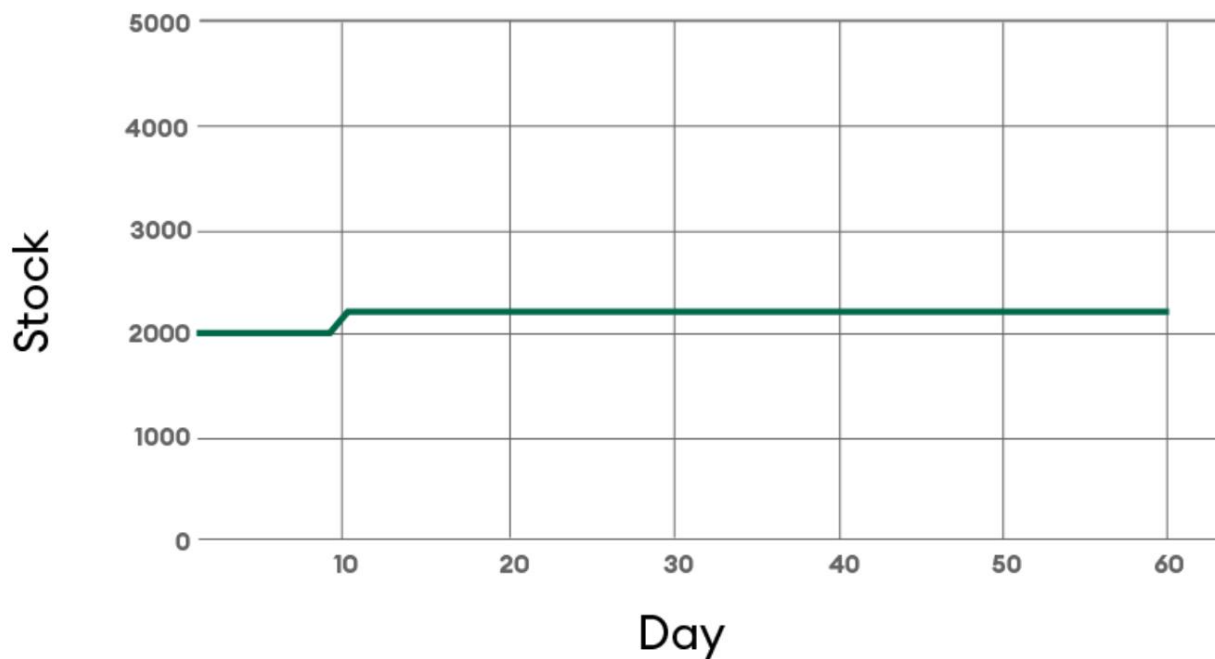


Figura 1.2: Comportamiento a lo largo del tiempo en un sistema simple sin retrasos

Cuando no hay retrasos en el sistema, se pueden realizar ajustes instantáneamente a los cambios en la demanda, lo que hace que el comportamiento del sistema sea sencillo y predecible.

Un sistema con retrasos puede comportarse de maneras inesperadas

En sistemas complejos, a menudo hay demoras entre la realización de una acción y la observación de su efecto, como describe Donella Meadows:



Debido a los retrasos en la retroalimentación dentro de los sistemas complejos, cuando un problema se hace evidente puede ser innecesariamente difícil de resolver. [1]

Por ejemplo, si los pedidos de inventario tardan varios días en llegar, un aumento repentino de la demanda podría no atenderse con la suficiente rapidez. Este retraso puede provocar sobrecorrecciones o escasez, generando inestabilidad.

Comprender y gestionar estos retrasos es fundamental para mantener el buen rendimiento del sistema.

Ahora, agreguemos algunos retrasos:

- Retraso de percepción : Antes de cambiar el pedido, el gerente promedia las ventas durante un período de 5 días
- Retraso en la respuesta : El gerente compensa el 40% del faltante de inventario al cambiar el pedido
- Retraso de entrega : La fábrica tarda 5 días en entregar el pedido.

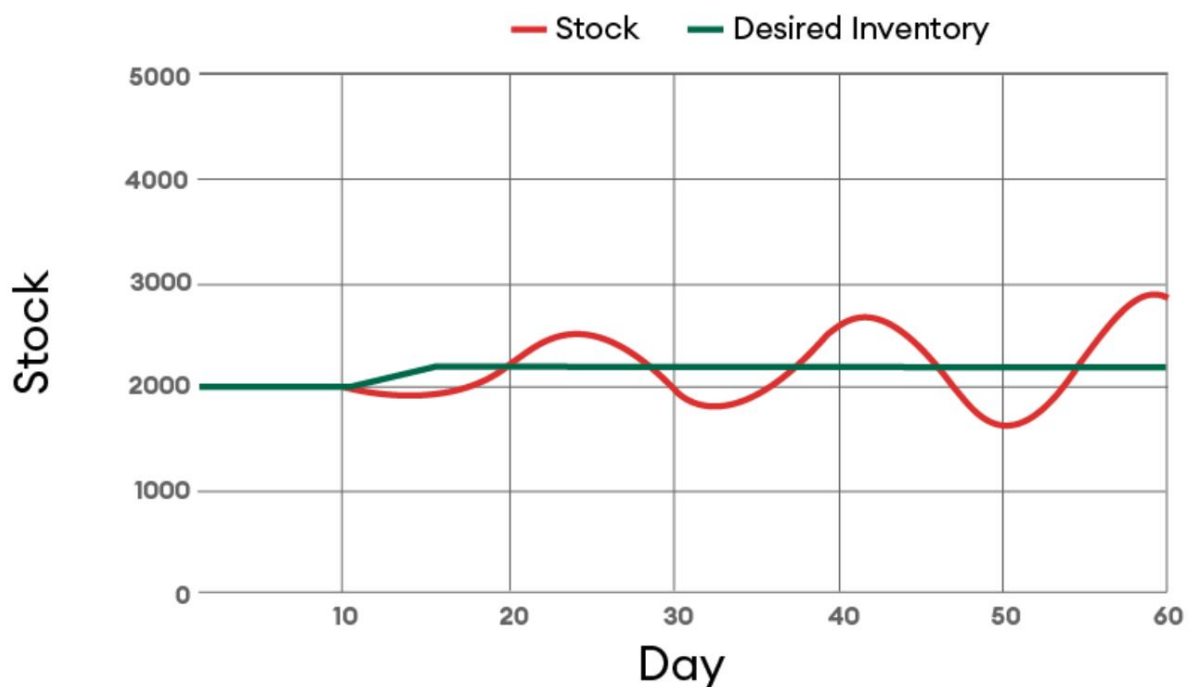


Figura 1.3: Los niveles de inventario oscilan después de que aumenta la demanda debido a ajustes de pedidos retrasados

Como puede ver en la Figura 1.3, , Después del día 10, cuando la demanda aumenta en primero con un 10%, vemos que los niveles de existencias comienzan a bajar. Luego, después de 5 días,

El gerente ajusta el pedido para compensar el 40% del faltante. El aumento de entregas comienza a llegar y, el día 20, el inventario alcanza el deseado. Sin embargo, el aumento de entregas continúa y, para el día 25, la tienda tiene exceso de existencias. El gerente reduce el pedido. El ciclo continúa y el inventario oscila entre exceso y defecto.

Intentando arreglar las oscilaciones

Aquí hay tres opciones que puedes probar para solucionar este comportamiento no deseado:

- Percepción más rápida (reducir el retraso)
- Respuesta más rápida (aumentar orden)
- Respuesta más lenta (orden decreciente)

Cierra los ojos un momento y decide cuál de estas opciones probarías primero. Analizaremos el resultado de cada una.

En primer lugar, reduciremos el retraso de percepción de cinco días a dos, como se muestra en la Figura 1.4 .

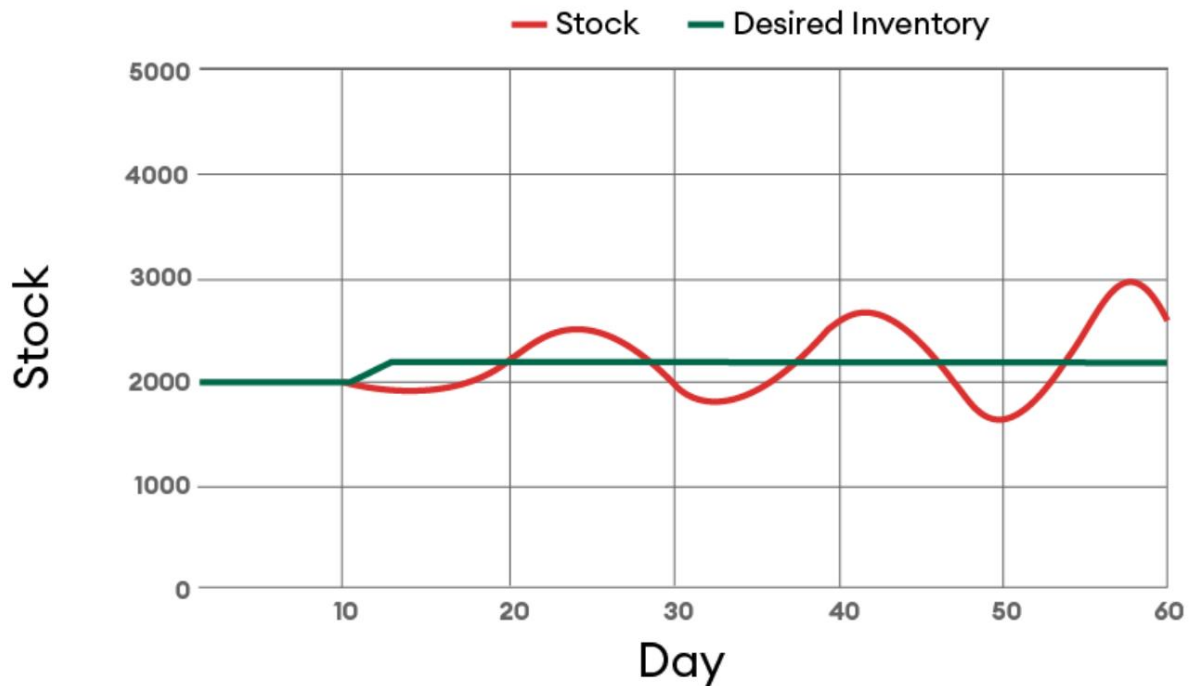


Figura 1.4: Reducción del retraso de percepción de cinco días a dos días para una respuesta más rápida

Así, el gerente reacciona con mayor rapidez y el inventario deseado sube a 2200 unidades más rápido, pero, en todo caso, las oscilaciones comienzan un poco antes y son ligeramente mayores. El inventario máximo justo antes de los 60 días es ahora de 2994 artículos. Era de 2981 antes de cambiar el retraso de percepción.

A continuación, intentaremos responder más rápido modificando el pedido para compensar el 60 % del déficit, que originalmente era del 40 %. El comportamiento empeora aún más. Las oscilaciones se vuelven mucho más... Como puede ver en la Figura 1.5, el inventario máximo aumenta a 3800 y baja a 1400 algunos días.

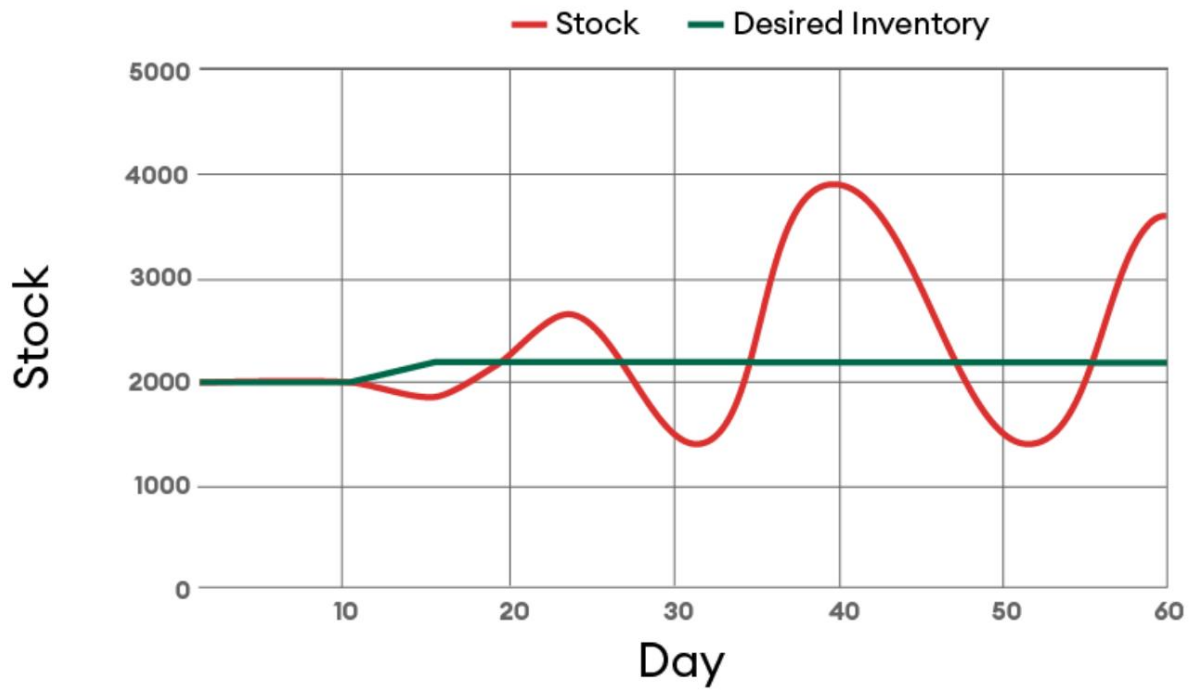


Figura 1.5: Reducción del retraso en la respuesta: las oscilaciones del inventario aumentan

Por último, probaremos una respuesta más lenta, reduciendo el retraso de respuesta (el cambio en el orden) al 25% en lugar del 60%, como vimos en el ejemplo anterior.

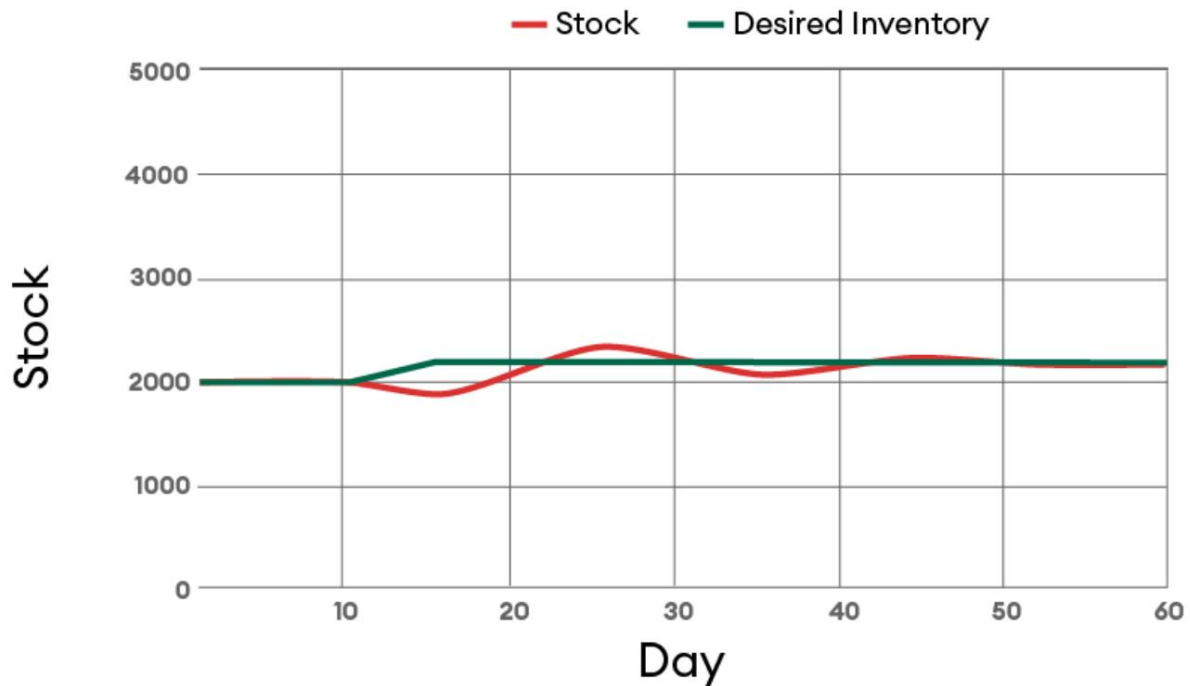


Figura 1.6: Los cambios de pedidos más lentos reducen las oscilaciones del inventario

Como se muestra en la , El comportamiento del sistema es mucho mejor. Figura 1.6 , el inventario aún oscila, pero comienza a estabilizarse cerca del inventario deseado unos 20 días después del cambio en la demanda.

Los sistemas nos sorprenden. Los

sistemas pueden reaccionar de forma contraintuitiva. En este sistema relativamente simple, el gerente de la tienda reaccionaba con demasiada rapidez. Nuestra tendencia natural era obtener percepciones y respuestas más rápidas, pero esto empeoraba aún más el comportamiento del sistema. Solo cuando probamos algo que parecía contraintuitivo (respuestas más lentas) el comportamiento del sistema mejoró. Esto también aplica a los sistemas de software, y veremos algunos ejemplos prácticos más adelante en este capítulo. Cambiar algunas variables puede tener un gran impacto en el comportamiento general, pero otras podrían tener poco o ningún efecto. En estos ejemplos,

Vimos que un retraso en la percepción tenía poco efecto, pero un retraso en la respuesta sí.

Los sistemas reales siempre son más complejos que los modelos y pueden tener variables ocultas o bucles de retroalimentación. Este sistema era relativamente simple. Un sistema más complejo puede tener múltiples acciones y más bucles de retroalimentación. Los sistemas son inherentemente oscilatorios. El comportamiento del sistema se revela como una serie de eventos a lo largo del tiempo. Nuestras mentes humanas no son...

Somos buenos en comprender el crecimiento no lineal y las interacciones entre elementos. Hacemos suposiciones sobre cuál es el cuello de botella actual e intentamos mejorar los elementos del sistema que no aumentarán el rendimiento a menos que se mejore el factor limitante.

Un sistema de software típico

Ahora, apliquemos lo que hemos discutido sobre sistemas en general a un sistema de software. Comenzaremos analizando un sistema de software típico a gran escala, como se muestra en la Figura 1.7, donde se muestran , y describir algunas de las los posibles cuellos de botella.

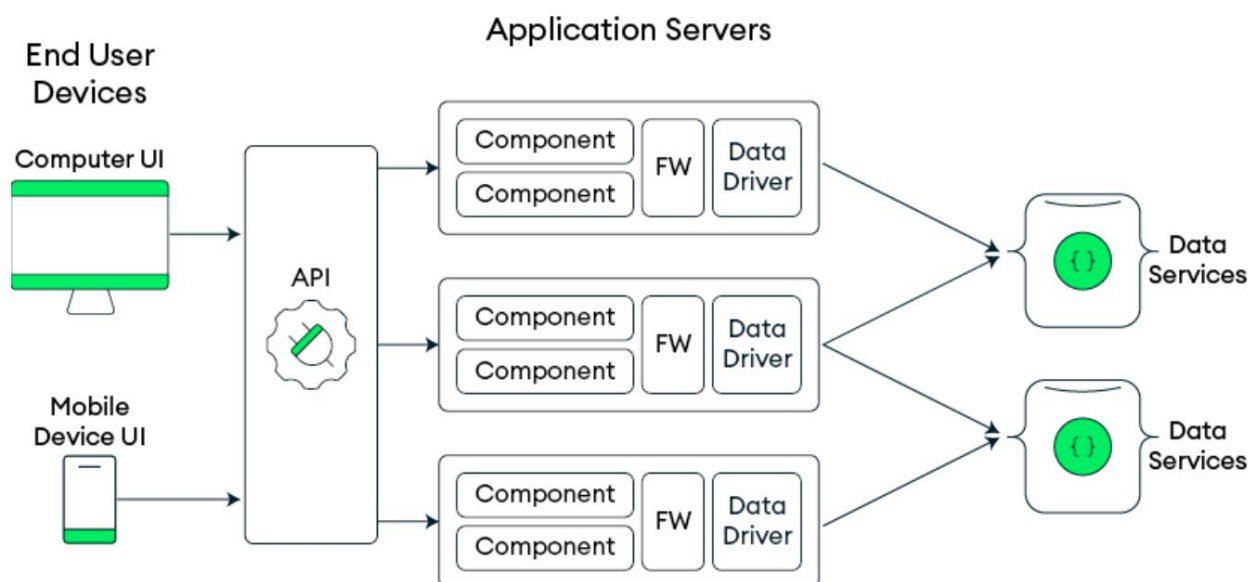


Figura 1.7: Vista de alto nivel de un sistema de software que muestra un flujo de solicitud de cliente a base de datos

De izquierda a derecha, vemos dos clientes. Estos podrían ser una computadora de escritorio y un dispositivo móvil. Estos clientes se conectan a través de internet a una puerta de enlace de interfaz de programación de aplicaciones (API), que envía solicitudes y recibe notificaciones. La puerta de enlace de API se conecta a múltiples servidores de aplicaciones. Estos modifican, enrutan y reenvían las solicitudes de los clientes a uno o más componentes de software que se ejecutan en dichos servidores. Cada servidor de aplicaciones puede tener múltiples componentes de software que utilizan uno o más frameworks (FW), como Spring o Rails. Estos suelen contener abstracciones de servicios de datos o componentes de modelado de objetos que utilizan un controlador específico de la base de datos (a veces llamado biblioteca de cliente) que se comunica con los servicios de datos.

Cualquiera de estos componentes (las computadoras en las que funcionan o las redes que los conectan) podría ser un cuello de botella para el rendimiento o la escalabilidad de la aplicación general.

A continuación se muestran dos ejemplos de cuellos de botella de rendimiento en un sistema de software:

- Un cuello de botella en la puerta de enlace API : A medida que el sistema gana popularidad y más usuarios se conectan a la puerta de enlace API, esta llega a un punto en el que gestiona tantas conexiones y solicitudes que la CPU se utiliza al máximo. Esto se puede identificar observando la CPU inactiva disponible en el equipo que ejecuta la puerta de enlace API. Esto se puede solucionar escalando verticalmente el equipo o la instancia en la nube que ejecuta la puerta de enlace API.
- Un cuello de botella en el marco : el software de aplicación puede enviar solicitudes aparentemente simples al marco, pero en realidad...

Estas solicitudes pueden desencadenar operaciones costosas en la base de datos, como un escaneo completo de una colección o una tabla. Esto es ineficiente.

Las consultas consumen CPU y ancho de banda de almacenamiento en la base de datos, a la vez que envían más datos de los necesarios a los servidores de aplicaciones. Como resultado, se consume un ancho de banda de red innecesario y se requieren recursos de CPU adicionales en el servidor de aplicaciones para procesar los datos.

Este último cuello de botella podría identificarse de varias maneras:

- Al perfilar la aplicación se podría descubrir que las solicitudes al marco toman una cantidad de tiempo sorprendente.
- Durante el desarrollo, se puede usar un seguimiento de extremo a extremo para rastrear una solicitud de usuario de un cliente a través del código de la aplicación, el marco y la abstracción de servicios de datos para ver qué tipo de consulta se ejecuta en los servicios de datos y las operaciones de entrada/salida (E/S) posteriores.
- Esto podría identificar ampliaciones de lectura o escritura, donde una pequeña solicitud de cliente resulta en una operación de base de datos desproporcionadamente grande. Por ejemplo, una consulta de cliente para un punto de datos de 32 bytes podría atravesar todo el sistema y provocar un escaneo de tabla en la base de datos que extrae 32 GB de datos mediante E/S y los almacena en caché sin motivo alguno. Esto podría solucionarse configurando el marco para que sea más eficiente.

Proporcionaremos ejemplos específicos de este tipo de cuellos de botella y soluciones a lo largo del libro, que culminarán en una serie de ejemplos específicos. estudios de caso en [Capítulo 15](#), Depuración de problemas de rendimiento .

Identificar posibles cuellos de botella es solo el primer paso para optimizar un sistema de software. Para eliminarlos, necesitamos un sistema estructurado.

Enfoque aplicando principios de ingeniería de rendimiento.

Los principios de la ingeniería de rendimiento abarcan desde la teoría algorítmica hasta la ley de Little. Al aplicar estos principios, los desarrolladores pueden minimizar los cuellos de botella, mejorar la escalabilidad y maximizar la eficiencia de los recursos. En las siguientes subsecciones, exploraremos algunos de estos principios y veremos cómo guían el diseño de sistemas de alto rendimiento, lo cual puede tener presente al leer este libro.

Eficiencia algorítmica (complejidad)

La elección del algoritmo y la estructura de datos suele ser el primer paso en la introducción al rendimiento en la mayoría de los cursos de informática. Un algoritmo que escala lineal o logarítmicamente con el tamaño de entrada tendrá un mejor rendimiento que uno que escala exponencialmente. Ninguna microoptimización compensará un algoritmo inherentemente lento. En los sistemas modernos, esto podría implicar el uso de bibliotecas integradas (a menudo optimizadas en C o ensamblador) o el aprovechamiento de algoritmos paralelos.

Evite la optimización prematura

Como dice el refrán, “ Haz que funcione, , Entonces hazlo rápido ”. En otras palabras, céntrate en construir un sistema correcto y fiable antes de preocuparte por el rendimiento”.

Esta idea es compartida por el científico informático Donald Knuth, quien famosamente advirtió que “ la optimización prematura es la raíz de todos los males ”. Los desarrolladores a menudo se equivocan sobre qué es lo que ralentiza sus sistemas; las herramientas de creación de perfiles descubren regularmente cuellos de botella en el rendimiento.

Lugares inesperados. Los procesadores modernos también pueden ejecutar código compilado de maneras que desafían la intuición humana.

Al resistir la necesidad de optimizar demasiado pronto, mantiene su código limpio y fácil de mantener. Además, garantiza que sus esfuerzos de rendimiento se basen en datos reales, no en suposiciones.

Ley de Amdahl (límite de aceleración paralela)

La ley de Amdahl explica por qué ciertas cargas de trabajo no se aceleran proporcionalmente con más núcleos. Si una fracción S de un programa debe ejecutarse secuencialmente, por ejemplo, para proteger una estructura de datos compartida, incluso con un número infinito de procesadores, la aceleración se limita a $1/S$.

Por ejemplo, si el 10 % de una tarea es serial, la mejor aceleración general posible se aproxima a 10 veces (suponiendo que el 90 % restante se ejecuta en muchos procesadores). Al utilizar procesadores multinúcleo y computación distribuida, es necesario identificar y reducir las porciones seriales de la carga de trabajo.

Localidad y almacenamiento en caché

Los programas suelen acceder a una porción relativamente pequeña de datos o código repetidamente en un corto período de tiempo. Las cachés se pueden utilizar para acelerar el acceso a datos de uso frecuente. Los sistemas utilizan cachés en varios niveles: cachés en memoria para los resultados de la base de datos y cachés de CPU para evitar recálculos o accesos lentos, almacenando los resultados de operaciones costosas y reutilizándolos. Sin embargo, el almacenamiento en caché introduce...

Complejidad, como datos obsoletos y sobrecarga de memoria. Por eso se menciona en el siguiente chiste:



Hay dos problemas difíciles en informática: la invalidación de caché, el nombramiento de cosas y los errores de 1 dígito.

León Bambrick

Irónicamente, ahora hay un contrapunto a ese chiste:



Hay dos problemas difíciles en la informática: sólo tenemos un chiste y no es gracioso.

Phillip Scott Bowden

El uso de almacenamiento en caché (por ejemplo, en operaciones de E/S) es crucial para obtener un buen rendimiento de dispositivos lentos, pero también es aplicable en otras capas de sistemas distribuidos.

Ley de Little (rendimiento versus latencia)

El rendimiento es multidimensional. A veces, el objetivo es gestionar un gran volumen de operaciones (rendimiento), mientras que otras veces, la prioridad es minimizar el retraso entre el inicio y la finalización de una sola operación (latencia). En muchos casos, ambos son igualmente importantes.

Puede aumentar la concurrencia para mejorar el rendimiento (por ejemplo, mediante subprocesos múltiples, E/S asíncronas o bucles de eventos sin bloqueo).

Sin embargo, estas técnicas pueden introducir retrasos en las colas que afectan la latencia. Demasiadas operaciones concurrentes pueden llevar a un sistema a un estado ineficiente. Es importante identificar primero el objetivo de rendimiento y luego optimizarlo según corresponda. Para un alto rendimiento, maximice el uso de recursos (CPU, red y disco) con concurrencia. Para una baja latencia, minimice la espera y la profundidad de procesamiento. Equilibrar estos factores es un arte, guiado por análisis y benchmarks.

En resumen, la ingeniería de rendimiento debe basarse en datos, utilizando perfiladores y monitores para identificar cuellos de botella. Debe basarse tanto en conocimientos clásicos (como la complejidad algorítmica y la ley de Amdahl) como en técnicas modernas (como la computación paralela, la vectorización y el balanceo de carga distribuido). Concentre los esfuerzos de optimización donde generen el mayor beneficio y verifique siempre el impacto con mediciones.

Comprensión de la arquitectura de MongoDB

Ahora que comprende los sistemas en general, es hora de analizar los sistemas MongoDB en particular. Antes de eso, analicemos la arquitectura de MongoDB a grandes rasgos. Profundizaremos en cada uno de estos componentes a lo largo del libro.

Primero, es importante comprender los fundamentos de MongoDB. MongoDB se creó para cerrar la brecha entre las bases de datos relacionales tradicionales y los almacenes de clave-valor simples. La idea central era combinar las ventajas de ambos mundos (flexibilidad,

Velocidad y un modelo de datos fácil de entender), a la vez que ofrece funciones como consultas potentes, indexación y agregaciones complejas. Además, desde sus inicios, MongoDB contaba con bibliotecas nativas para numerosos lenguajes, replicación integrada, escalabilidad horizontal mediante fragmentación y un modelo de datos de documentos intuitivo.

Los almacenes de clave-valor suelen ser elogiados por su velocidad y simplicidad. MongoDB, sin embargo, va más allá de este enfoque simple al permitir almacenar y consultar documentos completos (objetos) con relaciones y estructura enriquecidas. Las bases de datos relacionales, por otro lado, organizan los datos en tablas con columnas y relaciones predefinidas y aplican esquemas estrictos.

En la siguiente sección, analizaremos más de cerca el modelo de documento de MongoDB y cómo se diferencia de una base de datos relacional.

El modelo de documento: la base de MongoDB

En una base de datos relacional, los datos suelen normalizarse en tablas independientes para una amplia gama de casos de uso. Esto puede requerir uniones complejas para vistas específicas de la aplicación. Por el contrario, MongoDB fomenta el modelado de datos como documentos específicos de la aplicación, optimizados para el acceso y uso de esta. Esta flexibilidad permite representar relaciones complejas y datos jerárquicos en un solo documento.

La notación de objetos JavaScript (JSON) es un formato de datos liviano y legible para humanos que se utiliza para almacenar e intercambiar datos entre

aplicaciones. El modelo de documentos de MongoDB almacena datos en estructuras similares a JSON llamadas documentos Binary JSON (BSON).

He aquí un ejemplo de un documento:

```
A documento simple que representa # a usuario
usuario =
  { "nombre": "Jane Doe",
    "edad": 30,
    "correo electrónico": "janedoe@example.com",
    "dirección": { "calle":
      "123 Main St", "ciudad": "Cualquier
      ciudad", "estado": "CA",
      "código postal":
      "12345"
    },
    "intereses": ["senderismo", "fotografía", "viajes"]
  }
```

Esto contiene varias propiedades de un usuario. En una base de datos relacional, los intereses del usuario se almacenan en una tabla independiente, que debe consultarse por separado o combinarse cada vez que se necesiten los intereses del usuario junto con el resto de su información.

Desde una perspectiva de rendimiento, en MongoDB, la matriz de intereses se almacena con la información del usuario, por lo que toda la información del usuario puede leerse desde el almacenamiento en un solo acceso y recibirse por una aplicación en una sola consulta. Además, los intereses se pueden indexar para que sea fácil y rápido consultar a todos los usuarios interesados en senderismo, por ejemplo.

¿Alguna vez has intentado modelar datos jerárquicos en una base de datos relacional? Si lo has hecho, sabes que a menudo parece como intentar encajar una clavija cuadrada en un agujero redondo. En MongoDB, las estructuras anidadas son...

Ajuste natural, sin necesidad de uniones complejas ni diagramas entidad-relación.

Componentes arquitectónicos clave de MongoDB: El servidor MongoDB

utiliza un único

proceso mongod que puede ejecutarse en cualquier lugar. Puede implementar el proceso mongod en diversos tipos de computadoras, incluyendo portátiles, servidores autogestionados en centros de datos de su empresa, instancias en la nube administradas por su empresa e instancias en la nube administradas por MongoDB Atlas.

Muchos de los conceptos que analizaremos se aplican a este proceso de servidor; sin embargo, también es un componente fundamental del sistema distribuido MongoDB. Una implementación básica en producción normalmente consistiría, como mínimo, en un conjunto de réplicas con tres computadoras, cada una ejecutando un proceso mongod. Una de ellas sería la principal y las otras dos, las secundarias. Todos los miembros de un conjunto de réplicas tienen la misma copia de los datos. Aprenderá más sobre [Capítulo 5](#) proporciona esto en [enlace faltante], pero lo más importante es que la replicación alta disponibilidad para su aplicación al supervisar el estado de todos los miembros y elegir automáticamente una nueva principal en caso de fallo de la anterior.

Al escalar horizontalmente MongoDB, se considera el conjunto de réplicas como el primer fragmento y se añaden tantos fragmentos adicionales como se necesiten. Cada fragmento es un conjunto de réplicas. Cada fragmento contiene un subconjunto de los datos, y un proceso de enrutador inteligente dirige las operaciones al fragmento correcto, eliminando así la necesidad de que la aplicación sepa cómo...

Los datos se distribuyen. Aprenderá más sobre esto en [Capítulo 6](#),

Fragmentación .

Atlas es un conjunto integrado de bases de datos en la nube y servicios de datos para Acelere y simplifique la forma de construir con datos. Atlas amplía

La flexibilidad y facilidad de uso de MongoDB le permiten crear textos completos Búsqueda, análisis en tiempo real y funciones basadas en eventos.

Las bibliotecas de cliente (o controladores) de MongoDB permiten que el código de su aplicación comunicarse con la base de datos utilizando una biblioteca nativa y API que parecerse a cualquier otra API en el lenguaje que estás usando. Construyes una Solicítelo en su lenguaje de programación preferido y el controlador lo convierte en mensajes que MongoDB entiende.

MongoDB proporciona controladores oficiales para la mayoría de los populares lenguajes de programación, incluido [Py](#) [thon](#), [Java](#), [C#](#) y Node.js, entre otros. Puede encontrar la lista completa en MongoDB. documentación en <https://www.mongodb.com/docs/drivers/>

Aquí hay un ejemplo simple de cómo conectarse a MongoDB e insertar un documento que utiliza PyMongo, el controlador oficial de MongoDB para Python:

```
desde pymongo importar MongoClient
# aConectar MongoDB
cliente = MongoClient('mongodb://localhost:27017/');
db = cliente['base_de_datos_del_almacén'];
colección = db['productos'];
    Definir # a documento de producto con información anidada
producto = {
    "nombre": "Teclado ergonómico",
    "precio": 129,99,
    "categoría": "Accesorios de computadora",
    "detalles": {
```

```

        'Peso: 1,2;
        'Dimensiones: 18 x 6 x 1 pulgadas'
    },
};
# Insertar el documento
colección.insert_one(producto);

```

Lo que no se ve en el código anterior es todo el trabajo pesado que realiza la biblioteca del cliente: administrar las conexiones, comunicar el protocolo de cable MongoDB al servidor, manejar los tiempos de espera de la red, implementar la lógica de reintento y transformar los diccionarios de Python a BSON y viceversa.

El siguiente código consulta la colección de productos en busca de productos asequibles (menos de \$100):

```

# Encuentra productos asequibles
affordable_products = collection.find({'price': {'$lt': 100}})
for product in affordable_products:
    print(f"Producto asequible encontrado:
          {product['name']}")

```

Cuando MongoDB devuelve un documento, llega como una estructura de datos nativa en su lenguaje, que en este caso es un diccionario de Python.

No se requiere una conversión compleja. Esta correspondencia natural entre el código de la aplicación y la representación de la base de datos supone una ventaja significativa para la productividad del desarrollador.

El sistema de servicios de datos

Ahora acerquémonos a la parte de servicios de datos del sistema de software general que vimos anteriormente en la Figura 1.7 .

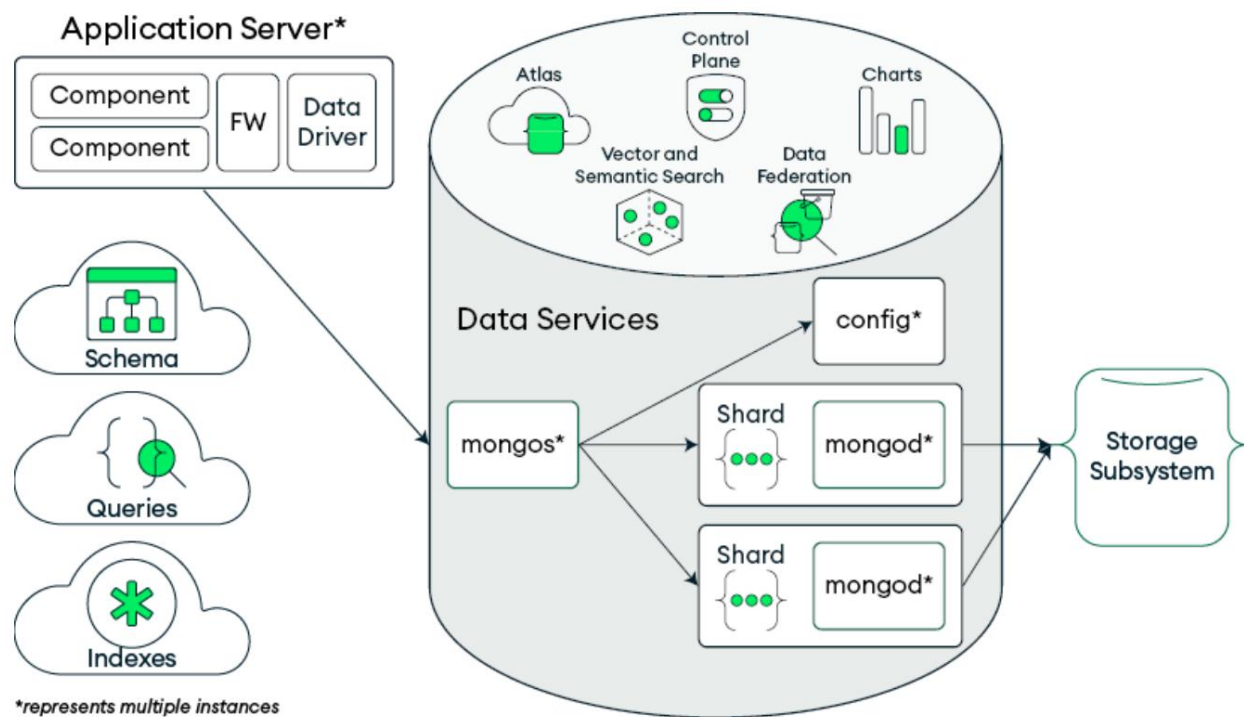


Figura 1.8: Arquitectura de MongoDB que muestra las conexiones entre los componentes principales de la base de datos

De izquierda a derecha en la Figura 1.8, los , vemos que la aplicación servidores se comunican con los servicios de datos a través de un controlador o una biblioteca cliente. En este caso, se muestra un clúster fragmentado. Para implementaciones más sencillas, el servidor de aplicaciones podría conectarse a un único conjunto de réplicas sin el servidor de Mongos ni el de configuración. Elementos conceptuales como el esquema, las consultas y los índices se comparten entre el servidor de aplicaciones y la capa de servicios de datos. Algunos problemas de rendimiento y escalabilidad pueden deberse a estos elementos conceptuales, y modificarlos podría generar mejoras.

En este caso, estamos viendo un diagrama de una implementación de Atlas que incluye elementos como la interfaz de usuario de Atlas (que incluye monitoreo)

y copia de seguridad), el plano de control Atlas, gráficos y funciones como búsqueda vectorial y semántica y federación de datos.

El resto de componentes se consideran la base de datos principal y son los siguientes:

- **mongod** : Un único proceso de servidor. Atlas puede implementar múltiples procesos mongod en conjuntos de réplicas (para alta disponibilidad) y fragmentos (para
- **escalamiento horizontal**). **mongos** : Un proceso de enrutador para un clúster fragmentado. Enruta las consultas a uno o más fragmentos y combina los resultados de
- **varios fragmentos en respuestas**. **config** : Una base de datos interna especial que almacena los metadatos de un clúster fragmentado. Los metadatos reflejan el estado y Organización de todos los datos dentro del clúster fragmentado. **Mongos** utiliza estos metadatos para dirigir las operaciones de lectura y escritura a los fragmentos correctos. También mantiene una caché de estos datos.

Finalmente, en la Figura 1.8, , Los procesos mongod guardarán y recuperarán el los datos se almacenan a través del subsistema de almacenamiento. Este podría ser un disco en un sistema MongoDB autogestionado o un almacenamiento en la nube, como el siguiente:

- Almacenamiento en bloque elástico (EBS) en AWS y Alibaba Cloud
- Discos administrados en Azure
- Discos persistentes (PD) en GCP
- SSD efímeros conectados localmente

Al usar Atlas, presentamos el sistema general de servicios de datos como una caja negra con una pequeña cantidad de configuraciones que debe considerar. Atlas se encarga de los detalles por usted. Las configuraciones desde la perspectiva de Atlas son las siguientes:

- El número de fragmentos y miembros del conjunto de réplicas.
- El tamaño de la instancia, que determina la cantidad de unidades centrales de procesamiento virtuales (vCPU), la cantidad de memoria y el rendimiento de la red.
- La configuración de almacenamiento, que incluye el tamaño y el rendimiento del dispositivo. El rendimiento suele definirse como operaciones de entrada/salida por segundo (IOPS).

Si gestiona MongoDB en su propia nube o servidores físicos, tendrá visibilidad de muchas de las configuraciones de los componentes dentro del servidor y del software en el que se ejecuta el proceso mongod . Esto es un arma de doble filo. Dispone de más opciones para ajustar y mejorar el rendimiento. Sin embargo, también tendrá más elementos que gestionar y posibles cuellos de botella que explorar.

Ahora, si profundizamos en mongod , observamos que es un sistema con múltiples elementos. El siguiente diagrama (Figura 1.9) muestra los elementos principales dentro del servidor MongoDB (mongod):

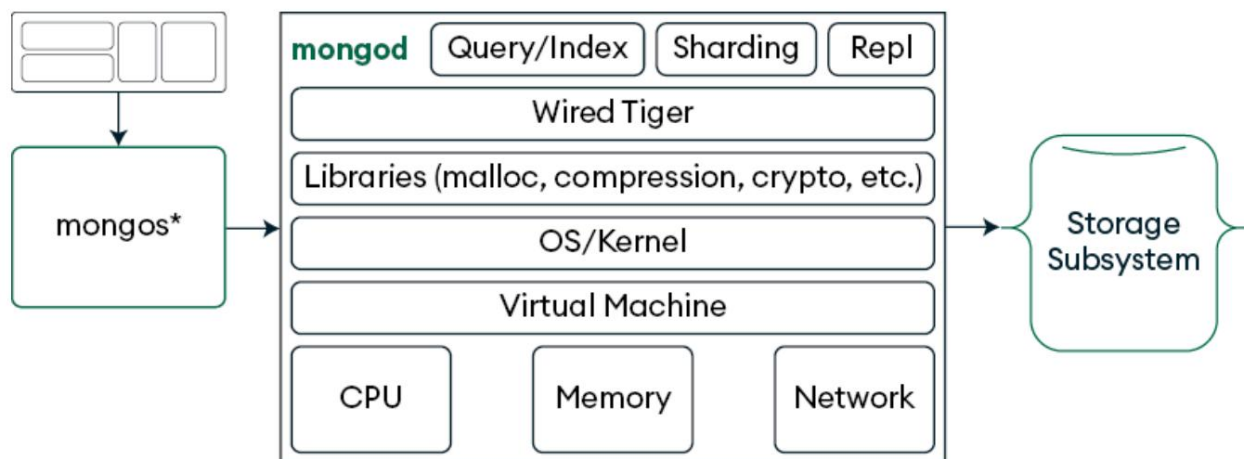


Figura 1.9: Elementos dentro del servidor MongoDB

El motor de consultas de

MongoDB toma su consulta, determina la forma más eficiente de encontrar los datos que necesita y devuelve los resultados. Utiliza índices para optimizar la ejecución de consultas y envía las solicitudes al motor de almacenamiento para ejecutar las operaciones.

Un índice es como el índice al final de un libro. En lugar de leer cada página para encontrar un tema, se puede usar para encontrar directamente qué páginas lo contienen. De igual forma, en una base de datos, un índice ayuda al motor de consultas a localizar datos de forma más eficiente.

MongoDB le permite hacer mucho más que un almacén de clave-valor.

Las capacidades de consulta e indexación de MongoDB rivalizan (y, en cierto modo, superan) a las bases de datos relacionales tradicionales:

- Puede realizar consultas sobre cualquier campo, no solo sobre la clave principal
- Puede realizar operaciones lógicas complejas
- Puede realizar consultas dentro de documentos y matrices anidados
- Puede agregar y analizar datos
- Puede crear índices en cualquier combinación de campos, incluidas matrices y subdocumentos
- Puede agregar varios índices a una colección de documentos

Los índices hacen que las consultas sean más eficientes. Al igual que muchos aspectos del rendimiento, los índices también tienen costos. Al crear o insertar nuevos documentos, cada índice requiere una operación de escritura adicional en el almacenamiento. De igual manera, al actualizar un campo indexado de un documento, se requiere una escritura adicional para actualizar el índice.

Puede configurar el sistema de consultas para que proporcione planes explicativos que detallen cómo se ejecutó una consulta. Esto puede ser valioso.

Información para encontrar posibles cuellos de botella y generar ideas para mejoras.

Entraremos en mucho más detalle sobre los índices y las consultas en el Capítulo 3. , Índices , y [Capítulo 12](#) , Conceptos avanzados de consulta e indexación .

Motor de almacenamiento/WiredTiger

Cada proceso de mongod necesita almacenar los datos en el disco. El almacenamiento

El motor es el sistema de archivos de MongoDB; es el componente responsable de Administrar cómo se almacenan, recuperan y mantienen los datos en el disco.

Desde MongoDB 3.2, WiredTiger ha sido el motor de almacenamiento predeterminado.

El motor de almacenamiento se encarga de conservar los datos y los índices en el disco. obtener los datos del disco, así como manejar la compresión y cifrado de los datos en disco. También gestiona el registro conocido como

El registro de escritura anticipada en muchas bases de datos. El motor de almacenamiento también

Contiene un caché en memoria del acceso más reciente.

documentos. Puede cambiar el tamaño de la caché de WiredTiger usando

La configuración `wiredTigerCacheSizeGB` . La memoria restante será...

utilizado por la caché del sistema de archivos (página) del sistema operativo. Aprenderás

Más información sobre los motores de almacenamiento en [Capítulo 7](#) , Motores de almacenamiento .

Bibliotecas

Al desarrollar MongoDB, utilizamos muchos otros bloques de construcción y bibliotecas. Estas también tienen configuraciones que se pueden cambiar para mejorar actuación.

A partir de MongoDB Server 8.0, se ha incorporado una versión más nueva de la memoria

El asignador (TCMalloc) admite cachés por CPU, en lugar de por subproceso.

Cachés. Esto puede reducir la fragmentación de la memoria y hacer que su base de datos sea más resistente a cargas de trabajo de alta tensión. Es posible que sea necesario modificar algunos ajustes de configuración del sistema para usar cachés por CPU. Encontrará más detalles sobre la [Capítulo 13](#), realmente conoce su sistema operativo y configuración del sistema en [Si recursos del sistema] , puede cambiar controles como el tamaño lógico de la página, pero esto podría requerir cambiar la configuración en el código fuente de MongoDB y reconstruir el servidor. Nuevamente, Atlas proporciona valores predeterminados e implementa las mejores prácticas.

Los algoritmos de compresión son más accesibles. Puede especificar el algoritmo de compresión de almacenamiento que se utilizará al crear una colección o mediante las opciones de configuración al iniciar el servidor. De forma predeterminada, Atlas utiliza un algoritmo llamado Snappy . Al ejecutar MongoDB autogestionado, dispone de cuatro opciones de compresión:

- Ninguno
- Snappy : más rápido, menor consumo de CPU; no comprime tanto los datos
- Zstd : Un punto intermedio entre Snappy y Zlib
- Zlib : más lento, mayor consumo de CPU; comprime mejor los datos

También puede usar la compresión de red entre el controlador y el servidor MongoDB. En este caso, puede seleccionar el algoritmo de compresión a través del controlador en el código de su aplicación, independientemente de si el servidor... Está en Atlas o no.

MongoDB utiliza OpenSSL para gestionar el cifrado TLS/SSL para lo siguiente:

- Conexiones cliente-servidor (por ejemplo, mongod MongoDB Shell)
- Replicación y comunicación en clústeres fragmentados

- Controladores MongoDB que se conectan a la base de datos

Las diferentes versiones de OpenSSL tienen diferentes características de rendimiento y escalabilidad en instancias/servidores de nivel superior.

Otros componentes del sistema que mongod USOS

MongoDB se ejecuta dentro de un sistema operativo, que a su vez es administrado por un núcleo. En sistemas operativos como Linux, parámetros como `vm.dirty_ratio` y `vm.dirty_writeback_centisecs` ajustan los mecanismos del núcleo para la gestión de la memoria virtual. Estos parámetros modifican la forma en que se gestionan los datos antes de escribirse de la memoria al disco.

MongoDB puede ejecutarse en máquinas virtuales (VM). Sin embargo, debe asegurarse de que esté configurado correctamente desde la perspectiva de la VM.

Por ejemplo, la sobreasignación de vCPU puede generar contención de CPU entre máquinas virtuales que se ejecutan en el mismo host, lo que puede reducir el rendimiento. Puede obtener más información sobre conceptos como el aumento de capacidad y cómo configurar MongoDB para máquinas virtuales en las notas de producción del servidor

MongoDB en <https://www.mongodb.com/docs/manual/administration/production-notes/> .

MongoDB se ejecuta en varios tipos de CPU en sistemas en red con diferentes cantidades de memoria. Esto suele depender del tamaño de la instancia en la nube o del servidor donde se implemente MongoDB. En general, las instancias o servidores más grandes tendrán más CPU, memoria y mayor ancho de banda de red.

Como vimos anteriormente, con Atlas, puedes seleccionar diferentes tipos y configuraciones de almacenamiento en la nube. Normalmente, tienes las mismas opciones al ejecutar tus propias instancias en la nube o almacenamiento autogestionado. Además, puedes elegir diferentes sistemas de archivos y configuraciones, como la lectura anticipada . Cuando un programa lee un archivo, Linux anticipa las lecturas futuras y carga datos adicionales en la memoria. Esto reduce el número de operaciones de acceso al disco, agilizando las lecturas secuenciales. Esto se discutirá con más detalle en , Operación [Capítulo 13](#) Sistema y recursos del sistema .

Gestión de la complejidad en las plataformas de datos modernas

Retrocedamos un poco del proceso mongod para observar de nuevo la capa de servicios de datos. Para su aplicación, el elemento de servicios de datos es un componente, pero en realidad puede ser un sistema distribuido complejo con múltiples servidores individuales.

El enfoque de MongoDB es proporcionar una API de servicios de datos que simplifique esta complejidad utilizando tres principios clave: un modelo de datos flexible, redundancia y resiliencia integradas y escalamiento horizontal.

Modelo de datos flexible con capacidades rigurosas

El modelo de documento permite representar relaciones de forma natural, a la vez que admite una rigurosa gobernanza de datos cuando es necesario. Un usuario de MongoDB lo explicó a la perfección: « Podemos enviar actualizaciones a nuestra aplicación sin tener que coordinar migraciones complejas de bases de datos » .

El esquema evoluciona con nuestro código, no en contra de él ". Puede obtener más información sobre Este modelo de datos flexible en [Capítulo 2](#) , Diseño de esquema para Actuación .

Redundancia y resiliencia integradas

En lugar de tratar la alta disponibilidad como una característica adicional, MongoDB Incorpora resiliencia en su arquitectura central a través de la replicación:

- Conmutación por error automática : si falla el nodo principal, se realiza una elección determina una nueva primaria en cuestión de segundos
- Autocuración : cuando los nodos fallidos se recuperan, se recuperan automáticamente. Ponte al día y vuelve a unirse al conjunto de réplicas.

Nuevamente, Atlas proporciona valores predeterminados, implementa las mejores prácticas y Simplifica la cantidad de configuraciones que debe considerar. Al autogestionar MongoDB, puede profundizar en la configuración de replicación. Para

Por ejemplo, si ejecuta MongoDB en sus propios servidores, puede cambiar

Parámetros del servidor para ajustar cómo se procesan los datos de replicación por lotes.

Los lotes reducen el tráfico de red y la cantidad de IOPS requeridas en

secundarias, pero pueden añadir latencia para las escrituras. También puede configurar control de flujo , que puede evitar el retraso en la replicación al ralentizarla

Escribe si las secundarias se están quedando atrás. Puedes obtener más información sobre replicación en , [Capítulo 5](#)

Escalamiento horizontal con inteligencia distribución

La capacidad de fragmentación de MongoDB distribuye datos entre múltiples conjuntos de réplicas manteniendo juntos los datos relacionados:

- Equilibrio automático : MongoDB reequilibra continuamente los datos entre fragmentos a medida que crecen sus datos
- Enrutamiento de consultas : El enrutador MongoDB (mongos) de forma inteligente Dirige las operaciones a los fragmentos apropiados
- Fragmentación de zonas : puede alinear la distribución de datos con su Infraestructura (manteniendo los datos de los clientes europeos en Europa) servidores, por ejemplo)

MongoDB Atlas simplifica la fragmentación al automatizar procesos clave, permitiendo las mejores prácticas y proporcionando funciones integradas para garantizar Rendimiento óptimo. Puedes leer más sobre esto en Fragmentación . [Capítulo 6](#),

Herramientas de rendimiento

Puede utilizar herramientas de supervisión del rendimiento para ayudar a encontrar potenciales cuellos de botella o factores limitantes. Estos podrían estar en cualquiera de los Componentes o elementos que mencionamos anteriormente. Se darán más detalles. proporcionado en [Capítulo 14](#) Monitoreo y Observabilidad .

Los procesos mongod y mongos escriben archivos de registro con rendimiento información. También admiten el comando `serverStatus` , que Devuelve información sobre el proceso en ejecución. El motor de consultas puede “Explica” cómo se ejecutará una consulta y admite una función de generación de perfiles. Herramientas de línea de comandos, como `mongostat` y `mongotop` , son proporcionadas para monitorear un proceso mongod en ejecución .

Para supervisar la utilización de todo tipo de recursos del sistema, puede utilizar Comandos del sistema operativo, monitoreo de MongoDB Atlas y otros

Herramientas GUI diseñadas específicamente para monitorear múltiples servidores.

Para los recursos del sistema específicamente, los comandos de Linux como `top`, `iostat`, `vmstat`, `df`, y `netstat` se puede utilizar para monitorear la CPU, memoria, almacenamiento y red en un momento determinado en un solo servidor. Se pueden utilizar herramientas como `dstat` y `sar` para combinar datos de herramientas más sencillas y visualizarlo a lo largo de un período de tiempo.

La monitorización de MongoDB Atlas proporciona datos históricos y en tiempo real de los recursos del sistema, integrados con métricas específicas de la base de datos. Incluye herramientas como el Panel de Rendimiento en Tiempo Real (RTPP), el Asesor de Rendimiento, la Información de Espacios de Nombres y el Generador de Perfiles de Consultas. También puede configurar alertas para eventos de rendimiento específicos.

Para implementaciones autogestionadas, puede usar herramientas gráficas como Netdata, Prometheus + Grafana, Zabbix, Munin u Observium. Puede usar marcos de observabilidad como OpenTelemetry. Finalmente, puede usar herramientas comerciales de monitorización del rendimiento de aplicaciones (APM) como AppDynamics, Datadog, Dynatrace, LogicMonitor, New Relic, SolarWinds y Splunk Observability.

Detección de cuellos de botella. Como se mencionó al principio de este capítulo, todo sistema tiene un factor limitante o cuello de botella. Con múltiples entradas en un sistema, no siempre está claro cuál es dicho factor. Necesitamos encontrar el elemento o componente que constituye el cuello de botella actual. Para ello, debemos utilizar las herramientas mencionadas anteriormente para analizar el rendimiento de todos los posibles cuellos de botella mencionados hasta ahora.

En resumen, la siguiente tabla enumera los posibles cuellos de botella que debe evitar. que puede encontrar al ajustar una aplicación de software típica. El primero La columna muestra los posibles cuellos de botella que podría encontrar en ambos Atlas y sistemas autogestionados. La segunda columna muestra la Cuellos de botella adicionales de los que eres responsable con la autogestión Implementaciones. Como puede ver, el uso de Atlas reduce significativamente el Número de cuellos de botella que debes tener en cuenta.

Atlas y autogestión (nube privada)	Autogestionado (nube privada)
Código de interfaz de usuario/aplicación	Red
Estructura	Replicación
Conductor	Motor de almacenamiento
Red	Bibliotecas (asignador, compresión, y cifrado)
Esquema/modelo	Sistema operativo/núcleo
Consultas	Máquina virtual
Índices	Memoria (detalles adicionales)
Fragmentación	CPU (detalles adicionales)

almacenamiento en la nube	Almacenamiento (detalles adicionales)
UPC	—
Memoria	—

Tabla 1.1: Atlas vs. autogestionado (nube privada)

En el resto de esta sección, analizaremos con más detalle dos ejemplos de cuellos de botella.

Primero analizaremos un cuello de botella de almacenamiento, incluyendo las mediciones e indicadores que nos ayudarán a identificarlo. En este ejemplo, tenemos una carga de trabajo que se dedica principalmente a la lectura de datos:

- El 95% de las operaciones de base de datos en estado estable son lecturas de documentos que utilizan el campo `_id` (clave principal)
- Los documentos se recuperan en orden aleatorio, lo que significa que se accede a los valores `_id` en un orden diferente de cuando se insertaron. El conjunto de datos es 10 veces más grande
- que la memoria del servidor de base de datos, por lo que la mayoría de las operaciones de lectura no pueden satisfacerse con una lectura de caché y necesitan acceder al dispositivo de almacenamiento.
- El dispositivo de almacenamiento está especificado para proporcionar 4000 IOPS

Observamos que el sistema ejecuta 5500 operaciones de base de datos por segundo con mongostat o la monitorización de Atlas. Intentamos aumentar el rendimiento escalando el tipo de instancia. El rendimiento de la base de datos se mantiene en 5500 operaciones por segundo. Tras realizar este experimento, sabemos que el cuello de botella no es la CPU ni el tamaño de la memoria.

de los sistemas que ejecutan el servidor MongoDB. Reducimos el tamaño de la instancia.

A continuación, observamos las mediciones de E/S:

- Si ejecuta instancias autoadministradas, puede consultar la cantidad de IOPS usando el comando `iostat` en Linux
- Si se ejecuta en Atlas, puede observar la cantidad de IOPS que se utilizan en la métrica IOPS de disco máximo cuando consulta el monitoreo

Para intentar mejorar el rendimiento, aumentamos el número de IOPS de almacenamiento de 4000 a 8000. El rendimiento de la base de datos se duplica a 11 000 operaciones por segundo. Observamos esta situación con bastante frecuencia con nuestros clientes. Suponen que necesitan una instancia de base de datos más potente cuando, en realidad, necesitan más IOPS del subsistema de almacenamiento.

En el segundo ejemplo, nos centraremos en un subconjunto de la Figura 1.8 . Aquí, tenemos un servidor de aplicaciones que interactúa con una capa de servicios de datos:



Figura 1.10: El servidor de aplicaciones se conecta al almacenamiento a través de servicios de datos

Inicialmente, asumimos que el cuello de botella se encuentra en la capa de servicios de datos o por debajo, pero al analizar la utilización de CPU y E/S, observamos tiempo de CPU inactivo y que no se alcanzan las IOPS de almacenamiento ni el ancho de banda. Por lo tanto, asumimos que el factor limitante se encuentra aguas arriba.

A través de mediciones en el servidor de aplicaciones, vemos que no tiene

Tiempo de inactividad de la CPU, por lo que es el cuello de botella. No genera suficientes solicitudes para cargar completamente los servicios de datos o las capas de almacenamiento en la nube.

Como modelo general de un servicio en un sistema, las solicitudes llegan y son ejecutadas por varios subprocesos dentro del servicio. Algunos subprocesos pueden interactuar mediante recursos compartidos; por ejemplo, en un servidor MongoDB, una consulta ejecutada por un subproceso podría ocupar espacio en la caché del motor de almacenamiento, ralentizando así otros subprocesos.

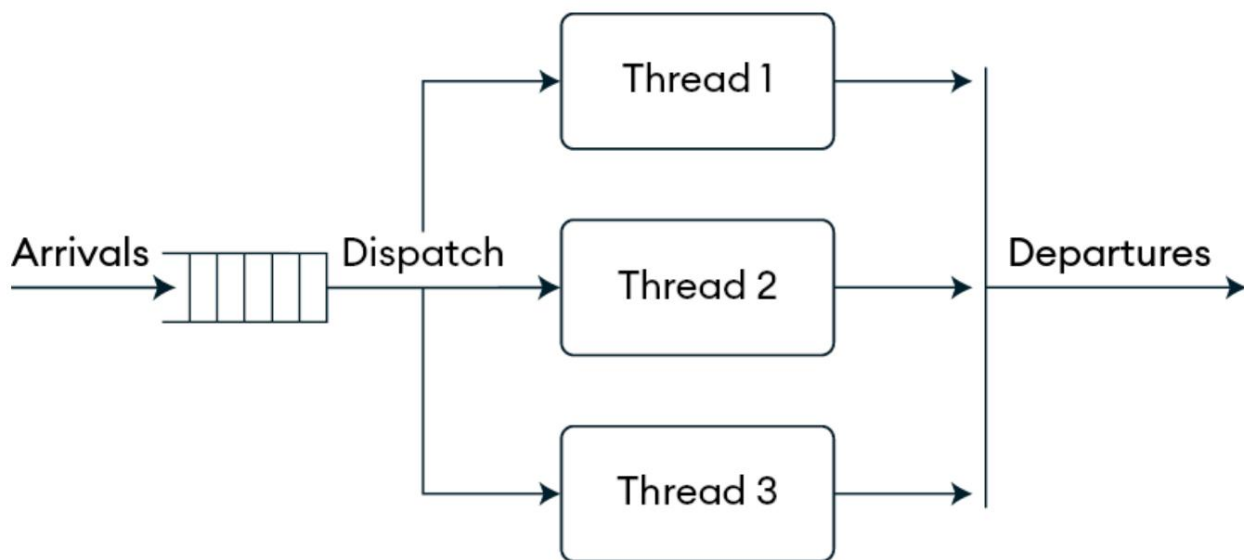


Figura 1.11: Los subprocesos de servicio manejan solicitudes

En este ejemplo, el servidor de aplicaciones no envía suficientes solicitudes que puedan ejecutarse en múltiples subprocesos dentro de la capa de servicios de datos. El rendimiento general de un sistema con este tipo de modelo es el siguiente:

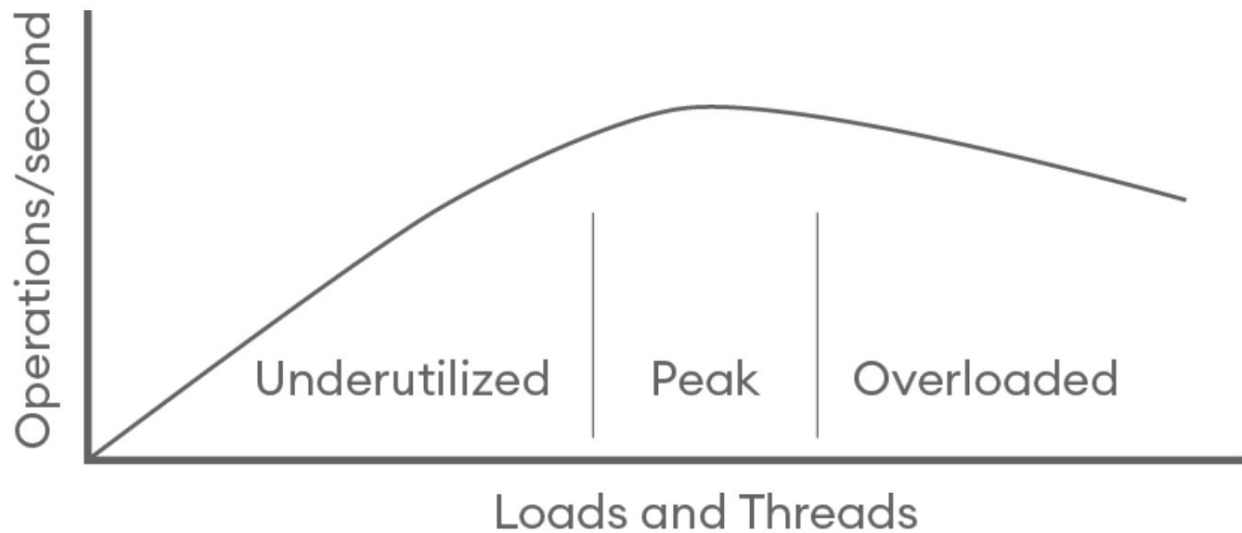


Figura 1.12: El rendimiento mejora a medida que la carga y los subprocesos crecen hasta que el sistema se sobrecarga

A medida que aumenta la carga y el número de subprocesos activos, el rendimiento en IOPS también aumenta hasta que el sistema llega a un punto en el que los subprocesos empiezan a competir por recursos compartidos. Más allá de este punto, un mayor aumento de la carga y el número de subprocesos puede provocar una disminución del rendimiento. La mayoría de los sistemas presentan esta característica. El rendimiento máximo suele depender de los siguientes factores:

- La cantidad de trabajo que se realiza para cada solicitud
- La relación entre subprocesos y núcleos de CPU
- La disputa por los recursos compartidos

La mejor manera de comprender este gráfico de carga/rendimiento para su aplicación o carga de trabajo es ejecutar experimentos de estrés/carga antes de poner su producto en producción.

Para resolver este problema, agregamos un segundo servidor de aplicaciones; el rendimiento general mejora, pero no alcanza nuestro objetivo de rendimiento.

Volvemos a analizar la carga de los componentes y ahora vemos que la CPU en el servidor que ejecuta el proceso mongod está completamente utilizada.

El rendimiento pasa de la parte subutilizada del gráfico a la parte sobrecargada .
Ahora vemos que el servidor MongoDB es el cuello de botella.

Ahora que hemos duplicado la carga de los servidores de aplicaciones, podemos reducirla de forma más precisa reduciendo el número de operaciones concurrentes o subprocesos que se ejecutan en el servidor. Puede gestionar esto desde sus servidores de aplicaciones, pero otra forma de hacerlo es reducir el límite de `maxPoolSize` en su controlador de base de datos o en su framework/ mapeador objeto-relacional (ORM). Esto reducirá el número máximo de conexiones a la base de datos, lo que a su vez reducirá el número máximo de subprocesos que pueden estar activos simultáneamente desde su servidor de aplicaciones. Esto devuelve el rendimiento de `mongod` al punto máximo del gráfico de la Figura 1.12 . Con estos dos cambios, hemos alcanzado nuestro objetivo de rendimiento.

Éste es otro ejemplo de lo que parece un cambio contra-intuitivo.
Reducir algo puede mejorar el rendimiento.

Un proceso incremental para la optimización

En general, puedes seguir un proceso de siete pasos con un bucle de cinco pasos. Cuando hemos visto a personas tener dificultades para ajustar y escalar el rendimiento, es porque comienzan en el medio, se saltan pasos o no saben dónde se encuentran en el proceso:

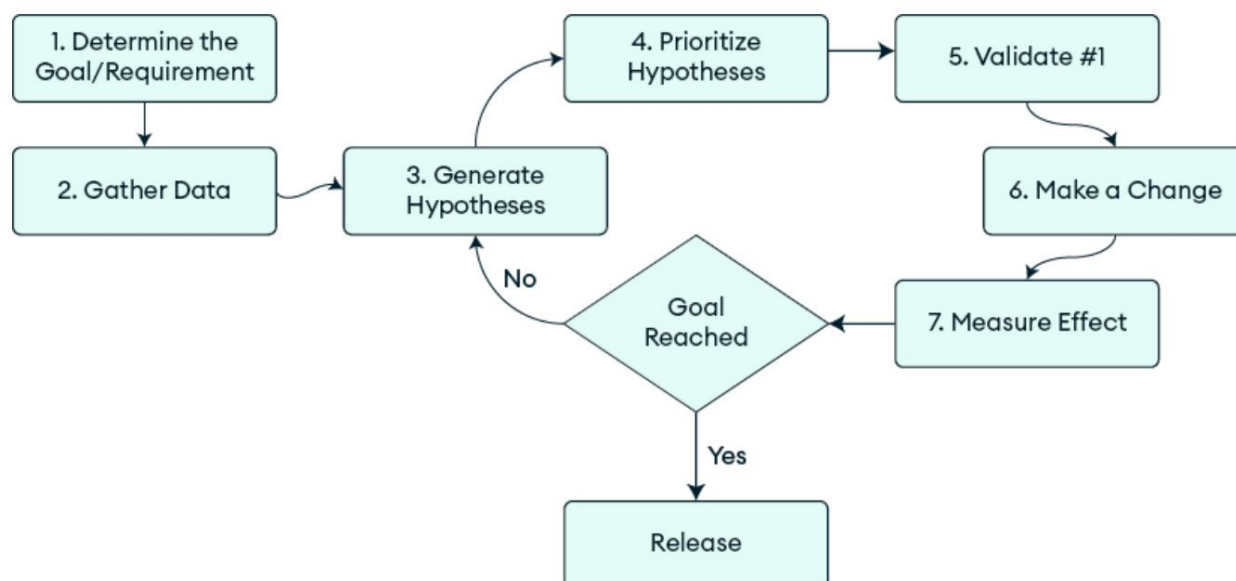


Figura 1.13: Un proceso incremental de optimización y ajuste

Repasemos estos pasos en detalle:

1. Determinar el objetivo/requisito : Siempre hay margen de mejora en un sistema complejo. Sin un objetivo acordado al inicio del proceso, optimizar el rendimiento puede convertirse en una tarea interminable. El objetivo puede convertirse en un objetivo variable a medida que cambian el mercado o los requisitos. Obtenga un acuerdo con los patrocinadores y las partes interesadas sobre los requisitos de rendimiento antes de iniciar el proyecto. Un requisito de rendimiento debe ser específico , que significa según el método SMART, medible, alcanzable, relevante y con plazos definidos . Lograr la mayor velocidad posible no es muy específico, alcanzable ni está limitado en el tiempo. 1 millón de consultas por segundo en una CPU de 4 núcleos a 1 GHz, con 3000 IOPS, no es alcanzable. El requisito también debe ser relevante para el usuario final.

Este también es un buen momento para considerar los límites de escalabilidad del sistema. Suponiendo que los datos no puedan crecer ilimitadamente indefinidamente,

¿Cómo expirarán o se archivarán los datos con el tiempo? ¿Cuáles son los objetivos de rendimiento al archivarlos?

2. Recopilar datos : Comience recopilando datos. El conjunto de trabajo es el tamaño de los datos e índices de uso frecuente. ¿Cabe el conjunto de trabajo en la memoria? Veremos ejemplos de este cálculo más adelante en el libro. En general, tenga en cuenta los costos de cada operación.

Por ejemplo, si su aplicación tiene una función de administrador que ejecuta una consulta ocasional sobre un conjunto de datos grande, es probable que los datos no estén en ninguna de las múltiples cachés. En el peor de los casos, la base de datos tendrá que recuperar cada documento de una fuente diferente.

Ubicación del disco, lo que genera operaciones de E/S. Su sistema tendrá un límite en el ancho de banda de E/S, que se comparte entre todas las consultas que se ejecutan simultáneamente.

3. Generar hipótesis : A continuación, genere hipótesis o posibles explicaciones para el cuello de botella actual. Normalmente, habrá más de una hipótesis posible. Es recomendable reunir a un grupo de personas para una lluvia de ideas. Cada persona tendrá diferentes niveles de experiencia en diferentes partes del sistema.

Sin embargo, cada uno tendrá sus propias teorías basadas en su experiencia. A veces, las discusiones pueden estancarse en un solo punto. Al principio, concéntrese en la cantidad; intente que la gente sugiera posibles hipótesis sin juicios ni análisis.

4. Priorizar hipótesis : Luego, analice cada hipótesis e intente priorizarlas. Anote cualquier evidencia o dato que pueda respaldar una hipótesis. Nuevamente, es importante limitar el tiempo de la discusión y no dedicar demasiado tiempo a una sola hipótesis.
5. Validar n.º 1 : ahora que tenemos una hipótesis n.º 1 o una hipótesis principal, Sospechoso, necesitamos identificar una prueba que pueda probar o refutar

La hipótesis. Por ejemplo, si creemos que una parte específica del sistema podría estar limitada por la CPU, intentemos aumentar el tamaño del servidor donde se ejecuta.

6. Realizar un cambio : A continuación, realice un cambio que mejore la situación y compruebe la hipótesis. Por ejemplo, podría aumentar el número de IOPS o reducir o aumentar el número de subprocesos en su aplicación. Idealmente, debería hacer un cambio a la vez.
7. Medir el efecto : Finalmente, mida el cambio en el rendimiento. Esperemos que sea una mejora. Si se cumple el requisito de rendimiento general, puedes lanzarlo y celebrarlo. Si no, tendrás que deshacer los cambios que no funcionaron, iterar y analizar la siguiente hipótesis y el siguiente cuello de botella.

La optimización no es una tarea única. Es un proceso estructurado e iterativo. Al seguir los pasos en orden y evitar la tentación de adelantarse, puede evitar errores comunes y lograr mejoras mensurables en el rendimiento del sistema.

Resumen

Este capítulo comenzó describiendo las características de los sistemas en general y cómo pueden comportarse de forma sorprendente al modificarlos. A continuación, analizamos los sistemas de software y los principios de rendimiento en general, antes de analizar los servicios de datos y MongoDB a gran escala. Enumeramos algunos de los posibles cuellos de botella o factores limitantes, analizamos dos ejemplos específicos y describimos un proceso que puede utilizarse para optimizar el rendimiento de un sistema.

También exploramos la arquitectura de MongoDB, diseñada pensando en el rendimiento. Su modelo de documentos conecta las bases de datos relacionales rígidas con los almacenes de clave-valor simples, ofreciendo flexibilidad sin sacrificar potencia. Analizamos componentes como el motor de almacenamiento, el optimizador de consultas y el mecanismo de replicación.

Al consolidar lo que tradicionalmente requeriría múltiples sistemas especializados, MongoDB ayuda a las organizaciones a reducir la complejidad operativa y, al mismo tiempo, satisfacer sus diversas necesidades de datos. El enfoque no consiste en sacrificar potencia por simplicidad, sino en hacer que las capacidades potentes sean más accesibles a través de un diseño inteligente.

En el próximo capítulo, profundizaremos en la arquitectura de MongoDB al explorar cómo aborda el diseño de esquemas y cómo puede diseñar para obtener un rendimiento óptimo.

Referencias

1. Meadows, Donella H. Pensamiento sistémico: una introducción . Chelsea Publicaciones verdes, 2008.

2

Diseño de esquemas para el rendimiento

MongoDB se creó para acortar la distancia entre los almacenes de clave-valor rápidos y escalables horizontalmente, y las bases de datos relacionales tradicionales. Para lograr el máximo rendimiento, es necesario comprender las fortalezas de MongoDB y diseñar su aplicación para aprovecharlas.

Ya sea que gestione grandes volúmenes de lectura, escritura o ambos, el objetivo es equilibrar la legibilidad, la facilidad de mantenimiento y la velocidad. El diseño flexible de esquemas de MongoDB ofrece oportunidades únicas, pero requiere una planificación minuciosa para evitar problemas comunes.

En este capítulo, abordaremos los principios fundamentales del diseño de esquemas de MongoDB para un alto rendimiento. Comenzaremos por abordar los principios clave del diseño de esquemas en MongoDB y aclarar algunos malentendidos comunes. A continuación, veremos cómo el modelo de documentos de MongoDB facilita la creación de esquemas flexibles y eficientes.

Esquemas. Después, revisaremos errores comunes y antipatronos que pueden afectar el rendimiento o dificultar la gestión de tus datos. Finalmente, compartiremos patrones prácticos de diseño de esquemas para ayudarte a optimizar las lecturas y escrituras. Para resumir, aplicaremos estas ideas a una aplicación de ejemplo para que puedas ver cómo funcionan en la práctica.

Este capítulo cubrirá los siguientes temas:

- Principios básicos del diseño de esquemas
- Puntos fuertes del modelo de esquema de MongoDB
- Uso de la validación de esquemas para la integridad de los datos
- Patrones de diseño de esquemas organizados por beneficios

Comprender los principios básicos del diseño de esquemas

El diseño de esquemas en MongoDB se basa en cómo la aplicación accede y modifica los datos; no existe una única estructura correcta que se aplique a todos los casos de uso. Esto supone un cambio con respecto al modelo relacional, donde la normalización es el punto de partida predeterminado y la desnormalización un paso para optimizar el rendimiento. En MongoDB, se empieza por comprender la carga de trabajo y luego se diseña el esquema para que se ajuste a ella.

En esta sección, cubriremos principios clave como la ubicación de datos, las compensaciones de lectura y escritura, el tamaño de los documentos y los errores comunes. Estas pautas le ayudarán a crear modelos de datos que sean eficientes y estén alineados con el comportamiento de su aplicación.

No existe una única forma correcta. Esto es lo más importante acerca del diseño de esquemas que quizás no sepas: no existe una única forma correcta de estructurar tus documentos .

¡Sí, has leído bien! Cada caso de uso exige un equilibrio único entre la eficiencia de lectura y escritura. La clave es pensar en tu...

Primero la carga de trabajo de la aplicación; los datos que se recuperan frecuentemente juntos deben almacenarse juntos para evitar búsquedas costosas .

Esto es fundamentalmente diferente de cómo podría haber aprendido a gestionar datos en bases de datos relacionales. En ese modelo, la normalización de datos es el enfoque esperado desde el principio. Solo cuando el rendimiento no pueda seguir el ritmo de un modelo de datos completamente normalizado, comenzará a desnormalizar selectivamente algunos datos según su uso. Con MongoDB, partimos de la pregunta "¿ Cómo se usarán estos datos?" y creamos un modelo de datos que se adapte mejor a nuestro acceso.

patrones.

Piense en la planificación urbana. Algunas ciudades aíslan zonas residenciales, comerciales e industriales en áreas diferenciadas. Pero en una ciudad bien diseñada, las áreas no se organizan estrictamente por tipo, sino como barrios donde las necesidades diarias están al alcance de sus habitantes. De igual forma, MongoDB le ofrece la flexibilidad de organizar sus datos según el uso real que haga su aplicación, no mediante una categorización arbitraria.

Colocación de datos

Una vez que haya identificado las entidades con las que trabajará su aplicación, la decisión principal es si integrar datos relacionados o almacenarlos.

Por separado y referenciarlo. Te preguntarás repetidamente si un objeto es una entidad independiente que debe almacenarse en su propia colección o si es un atributo de otro objeto y debe almacenarse como parte de él. De igual manera, para cada atributo de un objeto, deberás considerar si incrustarlo directamente o almacenarlo por separado.

Aquí es donde entra en juego el principio de que los datos que se utilizan conjuntamente deben almacenarse conjuntamente . Puede parecer simple a primera vista, pero resulta eficaz cuando se comprenden sus implicaciones. Así es como se aplica en la práctica:

- Incrustar datos que siempre se recuperan juntos : esto reduce la cantidad de viajes de ida y vuelta a la base de datos y mantiene las lecturas eficientes.

Por ejemplo, tiene sentido incorporar elementos de líneas de pedido dentro de un documento de pedido si normalmente se accede a ellos juntos.

- Separe los datos de acceso poco frecuente : Mantenga los campos grandes, pero poco utilizados, fuera del documento principal. Esto ayuda a evitar la sobrecarga si la mayoría de las consultas no los necesitan.

Almacenar atributos grandes que solo se necesitan ocasionalmente en un documento aumenta la E/S del disco y desperdicia RAM. Por ejemplo, almacenar una imagen completa de un usuario en el documento principal no es eficiente si solo se muestra una miniatura pequeña en la mayoría de las páginas de usuario.

El peor error probablemente sería tratar MongoDB como una base de datos relacional, normalizando los datos de múltiples colecciones y luego uniéndolos para cada consulta. Por otro lado, la sobreincrustación también puede ser subóptima si sobrecarga los documentos excesivamente. Terminará necesitando mucha más caché, operaciones de entrada/salida por segundo (IOPS) y ancho de banda de red que si estructurara los datos de forma más acorde con las necesidades de su aplicación.

Leer y escribir compensaciones

Al diseñar su esquema, puede encontrarse con requisitos contradictorios entre la optimización para lecturas y escrituras. Esto no es...

Exclusivo de MongoDB, pero el modelo de documento ofrece más opciones para resolver estos conflictos. A continuación, se presentan algunos factores comunes a considerar al equilibrar el rendimiento de lectura y escritura en el diseño del esquema:

- **Indexación** : Los índices aceleran las lecturas, pero añaden sobrecarga a las escrituras y consumen otros recursos. Añadir varios índices a una colección puede mejorar drásticamente el rendimiento de las consultas, pero encarece las operaciones de escritura. Es un equilibrio clásico, pero que vale la pena si los índices están correctamente diseñados para la aplicación.
- **Acuerdos de Nivel de Servicio (SLA) estrictos** : Priorice las operaciones con los requisitos de latencia más estrictos. A veces se optimiza para escrituras y otras para lecturas, pero rara vez ambas por igual. Por ejemplo, un sistema de procesamiento de pedidos puede priorizar las escrituras rápidas, mientras que un sistema de informes puede optimizar la eficiencia de lectura.
- **Reducción de E/S** : Minimizar la cantidad de lecturas y escrituras de datos. Cada bloque de datos al que se accede tiene un coste. Si se eliminan las lecturas o escrituras innecesarias, todas las operaciones serán más económicas y rápidas.

Estas consideraciones también influyen en si se deben modelar datos utilizando documentos grandes o pequeños, lo que afecta la eficiencia con la que su aplicación puede leer, escribir y actualizar datos.

Documentos pequeños versus grandes. Comprender cómo

MongoDB almacena los documentos en disco es crucial para optimizar el rendimiento. El motor de almacenamiento WiredTiger de MongoDB, que se describe con más detalle en Motores de almacenamiento, [Capítulo 7](#)

Utiliza una arquitectura orientada a páginas, donde los documentos se almacenan en páginas que se cargan en la memoria según sea necesario.

He aquí por qué esto es importante:

- **Tamaño mínimo de página :** WiredTiger tiene un tamaño mínimo de página sin comprimir de 32 KB. Los documentos menores a este límite (menos de 32 KB sin comprimir) comparten una página con otros documentos, lo que permite recuperar varios documentos pequeños en una sola lectura.
- **Fragmentación de documentos relacionados :** Si los datos relacionados se dividen en varios documentos, la aplicación tendrá que realizar varias consultas para recuperar el conjunto de datos completo. Además, estos documentos pueden residir en páginas diferentes, lo que requiere más operaciones de E/S para recuperar todos los datos relacionados.
- **Amplificación de escritura :** Al actualizar documentos grandes, incluso con cambios pequeños, es necesario escribir todo el documento en el disco durante los puntos de control. Cuanto más grande sea el documento, mayor será la probabilidad de que se produzcan escrituras más extensas de lo necesario.

Comprender esto nos lleva a nuestro siguiente principio importante: las cargas de trabajo de alta velocidad de lectura se benefician de los patrones de documentos incrustados . Esto se debe a que los datos relacionados se almacenan en un solo lugar, en una sola página. Por otro lado, para cargas de trabajo de alta velocidad de escritura, donde se actualizan frecuentemente pequeñas porciones de datos, usar múltiples patrones de documentos puede ser más eficiente.

Mitos comunes

Aclaremos algunos malentendidos comunes sobre el diseño de esquemas de MongoDB:

- **Desnormalización :** Desnormalizar datos no siempre significa más almacenamiento o duplicación de datos. En algunos casos, incrustar datos relacionados en un solo lugar reduce el uso general de espacio y la cantidad de índices necesarios. Por ejemplo, incrustar partidas individuales en un documento de pedido elimina la necesidad de...

Los índices de ID, tanto en los pedidos como en las colecciones de artículos, pueden ayudar a reducir el tamaño total de los datos. Integrar datos de dominio de acceso frecuente que rara vez o nunca cambian también reducirá las búsquedas adicionales y mejorará el rendimiento general.
- **Esquema dinámico :** MongoDB ofrece flexibilidad, pero no es obligatorio usarla. Puede aplicar un esquema estricto a través de su capa de aplicación, mediante las propias funciones de validación de esquemas de MongoDB o incluso ambas. Un esquema flexible es una opción, no un requisito.
- **Diseño intencional :** Si no diseñaste conscientemente tu estructura de datos, no es realmente un esquema. Alguien que te pasa JSON y lo almacenas directamente en la base de datos no es un diseño de esquema; simplemente almacena la carga útil. Un verdadero diseño de esquema refleja las necesidades y requisitos específicos de tu aplicación.
- **La indexación no es automática :** No es un mito común, pero existe. Cada colección crea automáticamente un índice único en el campo `_id` requerido , que es la clave principal de cada registro, y si no se proporciona con el documento al insertarlo, se generará automáticamente. Sin embargo, una parte importante del diseño del esquema es determinar qué atributos secundarios se utilizarán para las consultas y asegurarse de crear todos los índices que lo sean.

necesario para atender eficazmente esas consultas. Aprenderá más.
acerca de los índices en [Capítulo 3](#),

¿Recuerdas cuando los profesionales de bases de datos relacionales debatían?

¿Cada tabla necesitaba una clave sustituta o una clave natural? Ahora,
tener discusiones más interesantes sobre si incrustar o hacer referencia
datos relacionados , conversaciones que impactan directamente la aplicación
¡actuación!

Puntos fuertes de MongoDB diseño de esquema

En comparación con las bases de datos relacionales, el modelo de documento de MongoDB
permite patrones que simplifican el acceso a los datos, reducen la complejidad de las consultas,
mejorar el rendimiento para cargas de trabajo de lectura y escritura intensivas,
y adaptarse más fácilmente a los cambiantes requisitos de las aplicaciones.

En lugar de imponer una estructura rígida, MongoDB permite a los desarrolladores
Modelar datos de manera que reflejen casos de uso y acceso del mundo real.

patrones. Esta flexibilidad es la base del enfoque de MongoDB para

Diseño de esquemas que permite a los desarrolladores crear aplicaciones optimizadas
para velocidad y escalabilidad. En esta sección, abordaremos las fortalezas clave.

del diseño del esquema de MongoDB, incluido el manejo de uno a muchos
relaciones, incorporando entidades débiles, apoyando dinámicas

atributos, aprovechando el almacenamiento en caché y las instantáneas, optimizando para
casos de uso comunes y que permiten una evolución fluida del esquema.

Relaciones de uno a muchos

Las relaciones uno a muchos en MongoDB se consideran esenciales, con arrays, objetos y arrays de objetos que disfrutan de capacidades completas de consulta, actualización e indexación. Esta capacidad está integrada en la base del modelo de documentos de MongoDB, en lugar de añadirse posteriormente. Esto hace que MongoDB sea perfecto para almacenar datos estructurados y repetitivos, como comentarios en una entrada de blog, etiquetas de un producto o variaciones en la mercancía. Por ejemplo, en una plataforma de blogs, un documento de entrada de blog puede contener sus comentarios en un array, lo que permite a la aplicación recuperar la entrada y sus comentarios en una sola consulta. Al almacenar datos relacionados juntos, MongoDB elimina la necesidad de costosas uniones, reduce la complejidad de las consultas y minimiza las operaciones de E/S de disco, mejorando así el rendimiento general.

Integración de entidades débiles. Las entidades débiles (elementos que dependen de una entidad principal para su contexto) son ideales para integrarse en MongoDB. Por ejemplo, las líneas de pedido en una plataforma de comercio electrónico dependen intrínsecamente de su pedido principal. Integrar estas líneas de pedido en el documento del pedido garantiza que todos los datos relacionados se almacenen juntos, sin necesidad de búsquedas ni uniones independientes. Un pedido sin sus líneas de pedido tiene poco valor, y mantenerlas juntas permite a los desarrolladores consultar y actualizar los datos del pedido de forma eficiente. Este enfoque mejora el rendimiento al reducir las operaciones de E/S y la complejidad de las consultas, lo que agiliza la recuperación de datos y simplifica la lógica de la aplicación.

Atributos dinámicos

Las bases de datos relacionales a menudo tienen dificultades para modelar entidades donde los nombres de los campos o la cantidad de atributos no se conocen de antemano.

Estos desafíos suelen implicar costosas migraciones de esquemas para añadir nuevas columnas o requerir el almacenamiento de atributos en formatos complejos y propensos a la ineficiencia. Sin embargo, MongoDB adopta el modelado dinámico de datos de forma nativa. Por ejemplo, un catálogo de productos puede incluir atributos variables como "color" para prendas de vestir o "cilindrada" para vehículos, sin necesidad de definiciones rígidas ni cambios estructurales. Los documentos pueden almacenar fácilmente los nombres de los atributos como claves con sus valores correspondientes o utilizar matrices de pares clave-valor; ambos métodos permiten una indexación y consultas eficientes. Esto elimina la sobrecarga operativa y garantiza consultas más rápidas, incluso a medida que los datos evolucionan.

Cachés e instantáneas

En las bases de datos tradicionales, la normalización de datos evita la redundancia, pero MongoDB permite la duplicación selectiva para optimizar los patrones de acceso frecuente. Por ejemplo, almacenar la dirección de envío de un cliente directamente en cada documento de pedido garantiza una recuperación rápida de esta información sin tener que consultar una colección aparte. De igual forma, almacenar algunas de las acciones más recientes de un usuario, como publicaciones o reseñas, en su documento principal reduce las consultas repetidas y agiliza el acceso a los datos solicitados con frecuencia. Este uso de la duplicación de datos actúa como una caché eficaz o una instantánea puntual, reduciendo la sobrecarga de E/S y garantizando consultas más rápidas para aplicaciones con alto tráfico de usuarios.

Optimización para casos de uso comunes

Algunos de los ejemplos mencionados anteriormente permiten optimizar el sistema para operaciones que ocurren miles de veces por minuto, como que los usuarios consulten sus pedidos o las publicaciones de sus amigos, y permiten despreocuparse de operaciones que ocurren muy de vez en cuando (¿con qué frecuencia cambia un usuario su nombre? ¿Es realmente necesario que esa operación sea instantánea?). Esta optimización permite a los desarrolladores centrarse en flujos de trabajo clave, como recuperar publicaciones de usuarios o detalles de pedidos, y al mismo tiempo priorizar operaciones poco frecuentes, como que los usuarios cambien la información de su cuenta. La flexibilidad del esquema de MongoDB permite a los desarrolladores centrarse en optimizar estas operaciones frecuentes, reduciendo la sobrecarga de procesamiento innecesaria y garantizando un rendimiento fluido y rápido para las transacciones diarias.

Evolución del esquema

¿Alguna vez has intentado añadir una columna a una tabla grande en una base de datos relacional en producción? Con el esquema dinámico de MongoDB, puedes empezar a usar nuevos campos inmediatamente y, al mismo tiempo, actualizar documentos antiguos de forma diferida o no actualizarlos, según tus necesidades.

Las aplicaciones deben gestionar con fluidez los nuevos campos o formas en los documentos.

Esto garantiza la retrocompatibilidad y transiciones fluidas durante las actualizaciones de esquemas.

Las capas de acceso a datos eficaces en las aplicaciones MongoDB comprenden que los esquemas pueden evolucionar con el tiempo sin necesidad de una conversión completa de datos.

Las fortalezas del diseño de esquemas de MongoDB brindan flexibilidad y eficiencia para las aplicaciones modernas, lo que permite a los desarrolladores crear modelos de datos personalizados que maximizan el rendimiento en el mundo real.

Ya sea que se trate de manejar relaciones de uno a muchos o de incorporar vulnerabilidades débiles,

Al admitir entidades, admitir atributos dinámicos, aprovechar el almacenamiento en caché y las instantáneas, optimizar las operaciones de alta frecuencia o adaptarse a la evolución del esquema, MongoDB permite a los desarrolladores crear aplicaciones de alto rendimiento que se adaptan a las necesidades cambiantes. Al simplificar las consultas, reducir la latencia y optimizar los patrones de acceso a los datos, MongoDB ofrece la potencia y la versatilidad necesarias para cargas de trabajo escalables y modernas. Su modelo de documentos no solo es práctico, sino un pilar fundamental para la optimización del rendimiento de las aplicaciones del mundo real.

Validación de esquemas

Si bien gran parte del diseño de esquemas se centra en cómo se estructuran y se accede a los datos, un aspecto crucial, aunque a menudo se pasa por alto, es la validación de esquemas. Que MongoDB sea flexible no significa que se deba permitir el almacenamiento de cualquier estructura de datos. La validación de esquemas es una herramienta importante para mantener la integridad de los datos y su impacto en el rendimiento es prácticamente nulo. El coste de rendimiento de validar documentos a medida que se insertan o actualizan es insignificante en comparación con el coste total de la escritura, y ofrece varias ventajas prácticas:

- Detecte errores de forma temprana : la validación evita que datos incorrectos o malformados ingresen a su base de datos
- Aplicar reglas comerciales : puede definir requisitos como valores mínimos o patrones válidos directamente en su modelo de datos
- Documente su esquema : las reglas de validación sirven como documentación viva para su modelo de datos

- Evolución fluida : puede adaptar fácilmente las reglas de validación a medida que evoluciona su esquema

Si bien MongoDB ofrece gran flexibilidad, la validación de esquemas garantiza que esta flexibilidad no afecte la integridad de los datos. Al definir reglas claras para su modelo de datos, no solo previene errores y aplica la lógica de negocio, sino que también sienta las bases para esquemas sostenibles y en constante evolución. Analicemos ahora errores comunes en el diseño de esquemas, muchos de los cuales pueden evitarse con una validación minuciosa y una comprensión sólida de la estructura y los patrones de uso de sus datos.

Errores comunes en el diseño de esquemas

El modelo de documentos flexible de MongoDB ofrece importantes ventajas si se utiliza con cuidado. Las mismas características que permiten un alto rendimiento y la agilidad del desarrollador pueden generar ineficiencias o cuellos de botella si se aplican incorrectamente. Errores comunes en el diseño de esquemas pueden reducir el rendimiento, aumentar la complejidad o anular las ventajas del modelo de MongoDB. Comprender estos inconvenientes le ayudará a tomar decisiones de diseño más informadas y a evitar costosas reescrituras en el futuro.

Sobrenormalización Esto sucede a

menudo cuando los desarrolladores con experiencia en bases de datos relacionales aplican los mismos principios de normalización a MongoDB.

Aquí hay algunas señales de que podrías estar sobrenormalizando:

- Su código realiza frecuentes operaciones de búsqueda \$lookup (la versión de JOIN de MongoDB)
- Tiene varias colecciones que representan entidades a las que normalmente se accede juntas
- Las consultas de lectura requieren reunir datos de múltiples colecciones
- Necesita transacciones de múltiples documentos para actualizar campos relacionados almacenados en documentos separados que podrían almacenarse juntos y actualizarse de forma atómica sin transacciones.

Si tiene la misma cantidad de colecciones en MongoDB que tendría en una base de datos relacional, es muy probable que haya sobrenormalizado su esquema.



Costo de la sobrenormalización

Estaba revisando una aplicación donde los desarrolladores habían creado una colección separada para las preferencias del usuario, otra para los perfiles del usuario, otra para las configuraciones del usuario... Ya te haces una idea. ¡Cada carga de página de usuario requería siete consultas diferentes! Consolidamos los campos de acceso frecuente en un único documento de usuario y los tiempos de carga de la página se redujeron de 600 ms a menos de 100 ms.

Sobreincrustación. Este es el problema opuesto: cuando los desarrolladores se entusiasman con el modelo de documentos de MongoDB e intentan incluir demasiado en un solo documento. Aquí hay algunas señales de que podría estar sobreincrustando:

- El tamaño del documento sigue creciendo sin límites
- Tienes documentos muy grandes
- Estás almacenando datos a los que rara vez accedes dentro de documentos a los que accedes con frecuencia
- Tiene matrices que crecen indefinidamente (como comentarios en publicaciones populares)
- Escribir consultas para devolver solo partes específicas del documento se vuelve complicado
- Las actualizaciones de campos profundamente integrados requieren operadores de actualización complejos
- Crear índices efectivos para campos profundamente anidados se vuelve un desafío

Si sus documentos se acercan al límite de tamaño de 16 MB, eso es un fuerte indicio de que debe reconsiderar el diseño de su esquema.

Otros antipatrones comunes. Además de la sobrenormalización y la sobreincrustación, existen varios patrones de diseño de esquemas que suelen causar problemas, o lo que llamamos antipatrones. Reconocer y evitar estos obstáculos puede ayudarle a crear aplicaciones más eficientes, fáciles de mantener y escalables:

- Exigir transacciones para todo : Diseñar esquemas que siempre requieren transacciones multidocumentales anula las ventajas del modelo documental. Con una integración adecuada, la mayoría de las operaciones pueden ser atómicas sin transacciones.

- Mala estrategia de indexación : no saber cómo indexar adecuadamente sus patrones de consulta genera exploraciones de colecciones completas y un rendimiento deficiente.
- Actualizaciones complejas de datos incrustados : si no puede actualizar fácilmente campos profundamente anidados, su esquema puede ser demasiado complejo o estar sobreincrustado.
- Consultas ineficientes : si no puede consultar fácilmente solo las partes relevantes de sus documentos, reconsidere el diseño de su esquema.
- Tratar a MongoDB como una base de datos relacional : distribuir datos relacionados entre colecciones y volver a unirlos con \$lookup para cada consulta anula los beneficios del modelo de documento.
El uso excesivo de \$lookup ralentiza las consultas y se puede evitar con una incrustación estratégica o una desnormalización.
- Crecimiento ilimitado de matrices : la incorporación de matrices que podrían crecer indefinidamente (como comentarios en una publicación viral) puede generar problemas de tamaño del documento y un rendimiento deficiente.
- Ignorar patrones de escritura : optimizar solo para lecturas cuando su aplicación escribe mucho puede generar contención y cuellos de botella en el rendimiento.
- Sobreindexación : crear índices en cada campo por si acaso agrega sobrecarga a las escrituras y no necesariamente mejora el rendimiento de la consulta.
- Almacenamiento de datos binarios grandes en documentos : si bien MongoDB puede almacenar datos binarios, los archivos muy grandes (>16 MB) deben usar GridFS o almacenarse externamente, con referencias en sus documentos.

Ahora que está familiarizado con los errores comunes en el diseño de esquemas, centrémonos en los patrones que optimizan la eficiencia de las lecturas, escrituras, consultas, almacenamiento y más.

Patrones de diseño de esquemas según sus beneficios. Ahora que hemos cubierto los fundamentos, exploremos un conjunto de patrones de diseño de esquemas prácticos, cada uno organizado según sus beneficios específicos, como la mejora del rendimiento de lectura, la eficiencia de escritura o la flexibilidad de consultas. Estos patrones no son solo teóricos, sino que se basan en casos prácticos reales y desafíos comunes.

A lo largo de esta sección, utilizaremos ejemplos de una aplicación de redes sociales llamada Socialite para ilustrar patrones de diseño de esquemas. Socialite consta de tres almacenes de datos principales:

- Almacén de contenido : contiene publicaciones, multimedia y comentarios.
- Gráfico de usuarios : administra los usuarios y las relaciones entre ellos (seguidores, seguidos)
- Caché de la línea de tiempo : optimizado para la visualización rápida de contenido relevante cuando un usuario inicia sesión

Esta separación permite a Socialite optimizar cada tienda para sus patrones de acceso específicos. A medida que exploremos diferentes patrones de diseño de esquemas, volveremos a estos componentes para demostrar algunas aplicaciones prácticas. A continuación, analizaremos algunas ventajas de los esquemas específicos de MongoDB.

Patrones para optimizar el rendimiento de lectura. En esta sección, analizaremos los patrones de diseño de esquemas que resultan beneficiosos para optimizar el rendimiento de lectura. Estos incluyen el patrón de incrustación, que permite almacenar juntos datos relacionados de acceso frecuente; el patrón de referencia extendida, que equilibra la incrustación y la referencia para mayor flexibilidad y rendimiento; y el patrón de subconjunto, que ayuda a recuperar solo las partes más relevantes de un documento sin cargar datos innecesarios.

Patrón de incrustación. En

MongoDB, el patrón de incrustación implica incluir datos relacionados dentro del mismo documento en lugar de normalizarlos y referenciarlos desde otra colección. En el siguiente ejemplo, este patrón se utiliza en un documento de muestra de la colección de usuarios para almacenar información jerárquica y relacionada:

```
{
  _id: ObjectId("5f4e5b6d3c1e2a1b3c4d5e6f"),
  nombre de usuario:
    "jane_smith", nombre:
    "Jane Smith", correo electrónico:
    "jane.smith@example.com", perfil: { biografía:
      "Apasionada de los viajes y fotógrafa",
      ubicación: "San Francisco, CA", fecha de nacimiento:
      ISODate("1988-04-12T00:00:00Z"), avatar: "https://socialite.com/
      images/avatars/jane_sm
    }, preferencias: {
      tema: "oscuro",
```

```
    Notificaciones de correo electrónico: verdadero,  
    Configuración de privacidad: {  
      PerfilVisibilidad: "amigos",  
      Estado de la actividad: "todos"  
    }  
  }  
}
```

Repasemos este documento para entender cómo funciona la incrustación.

Aquí se ha aplicado el patrón:

- Datos de perfil como subdocumento : El campo de perfil está incrustado Como subdocumento, que contiene detalles como la ubicación , fecha de nacimiento , biografía y el avatar . En lugar de almacenar esta información en... una colección separada y haciendo referencia a ella, todos los perfiles de usuario relacionados Los datos residen juntos como una unidad cohesiva. Esto mantiene los datos Localizado para un rápido acceso y recuperación.
- Datos de preferencias como subdocumento : De manera similar, las preferencias El campo incluye las configuraciones específicas del usuario, como el tema y , Notificaciones de correo , las configuraciones de privacidad anidadas . electrónico Al incorporar estas preferencias directamente en la sección principal documento, evita la complejidad de las colecciones separadas y se une, lo que facilita la consulta y actualización de la configuración del usuario en un solo lugar operación.
- subconfiguración de privacidad anidada : `privacySettings` , que es una documento dentro de las preferencias , demuestra una visión más profunda incrustación, ya que almacena la visibilidad del perfil y el estado de la actividad Esto demuestra la flexibilidad de MongoDB para crear bases de datos multinivel. incrustaciones manteniendo una estructura lógica.

Al usar el patrón de incrustación para los subdocumentos de perfil y preferencias , el diseño logra la desnormalización de datos, reduce la necesidad de uniones costosas y garantiza la localización de los datos para optimizar el rendimiento en lecturas y escrituras. Este patrón funciona bien en escenarios donde se accede frecuentemente a datos relacionados, eliminando la necesidad de múltiples consultas.

Patrón de referencia extendido

El patrón de referencia extendida se utiliza en MongoDB cuando necesitamos hacer referencia a otro documento en una colección relacionada mientras que también incluyendo algunos de sus campos clave para mayor comodidad y rendimiento al realizar consultas. Este enfoque ayuda a evitar búsquedas innecesarias, especialmente cuando los campos de acceso frecuente están fácilmente disponibles.

El siguiente ejemplo es un documento de la colección de publicaciones que contiene una referencia al autor y datos relacionados con el autor:

```
{ _id: ObjectId("5f4e5b6d3c1e2a1b3c4d6f4h"), contenido: "¡Acabo  
de visitar el puente Golden Gate! #viajes postDate: ISODate("2023-04-07T09:15:30Z"),  
media: ["https://socialite.com/images/posts/golden_gate likes:  
42, comments: 7, author: { _id: ObjectId("5f4e5b6d3c1e2a1b3c4d5e6f"), username:  
"jane_smith",  
name: "Jane  
Smith",  
avatar: "https://socialite.com/images/avatars/jane_sm"
```

Árbitro // //
Dupli //
Dupli

```
}
```

En este ejemplo, el documento de publicaciones utiliza la referencia extendida Patrón mediante la inclusión de un subdocumento de autor :

- Referencia incrustada : El subdocumento del autor contiene una `Campo _id` , que es una referencia al Autor/Usuario completo documento almacenado en otra parte de la colección de un usuario . Esto permite vinculando la publicación a su autor.
- Campos clave duplicados para visualización : en lugar de simplemente almacenar los `_id` valor del usuario, detalles adicionales relacionados con el autor como `nombre de usuario` , `nombre` , y el avatar se incluyen directamente dentro del Documento de publicaciones . Estos campos se copian del documento referenciado. Documentar a los usuarios para mejorar el rendimiento y permitir la visualización detalles del autor sin necesidad de realizar consultas adicionales o una operación de búsqueda.
- Operaciones de lectura optimizadas : al incorporar el nombre de `usuario en` , `nombre` , y `avatar` , el documento de publicaciones , se garantiza que sea común consultas, como mostrar el contenido de las publicaciones junto con el autor. La información básica se puede manejar sin problemas y sin necesidad de ayuda adicional. llamadas a la base de datos. Esto reduce la latencia y mejora la eficiencia para Aplicaciones que muestran frecuentemente publicaciones y sus autores.

El uso del patrón de referencia extendido aquí logra un equilibrio entre Normalización y desnormalización. Si bien `_id` proporciona una definición Enlace al documento original del autor/usuario para actualizaciones o información más detallada recuperación de datos, los campos duplicados, como el `nombre de usuario` y el `avatar` , garantizan una representación rápida de las publicaciones con información esencial del autor. Este patrón es particularmente útil en aplicaciones de lectura intensiva como plataformas de redes sociales.

Patrón de subconjunto

El patrón de subconjunto se utiliza en MongoDB para almacenar datos a los que se accede con frecuencia.

datos por separado de los datos a los que se accede con poca frecuencia, optimizando las lecturas

Para consultas comunes, manteniendo la información más detallada o con menor frecuencia.

La información necesaria está disponible en un documento relacionado.

El siguiente es un ejemplo de datos de una sola publicación en los medios que es

almacenados en dos documentos separados:

```
// Documento 1: Datos multimedia de acceso frecuente
{
  _id: Id. del objeto("5f4e5b6d3c1e2a1b3c4d6f5i"),
  tipo de publicación: "video",
  Título: "Mi viaje a Yosemite",
  ID de autor: Id. de objeto("5f4e5b6d3c1e2a1b3c4d5e6f"),
  URL de la miniatura: "https://socialite.com/thumbnails/yosemit",
  duración: "3:42",
  recuento de vistas: 1287,
  Me gustaCount: 328,
  Número de comentarios: 57
}
// Documento 2: Datos de publicaciones de medios a los que se accede con poca frecuencia
{
  _id: Id. del objeto("5f4e5b6d3c1e2a1b3c4d6f6j"),
  ID de publicación: ID de objeto ("5f4e5b6d3c1e2a1b3c4d6f5i"),
  Vídeo de alta resolución: "https://socialite.com/videos/yosemite-tr",
  rawMetadata: {
    códec: "H.264",
    resolución: "1920x1080",
    fps: 60,
    tasa de bits: "12 Mbps"
  },
  Historial de procesamiento: [
    { marca de tiempo: ISODate("2023-04-05T14:20:00Z"), acción:
    { marca de tiempo: ISODate("2023-04-05T14:25:12Z"), acción:
    { i
    ISOD ("2023 04 05T14 30 45Z")
    i
```

```

    { marca de tiempo: ISODate("2023-04-05T14:30:45Z"), acción:
  ]
}

```

Repasemos este ejemplo para comprender cómo se ha aplicado el patrón de subconjunto. Este patrón está diseñado para dividir un conjunto de datos en dos documentos separados (uno con datos de acceso frecuente y otro con datos de acceso poco frecuente) para optimizar el rendimiento de las consultas comunes, manteniendo al mismo tiempo el acceso a la información detallada cuando sea necesario.

El primer documento (Documento 1) contiene los campos más consultados sobre la publicación en los medios, incluidos los siguientes:

- postType : el tipo de medio (por ejemplo, "video")
- title : el nombre de la publicación del
- medio Métricas de interacción, como viewCount comoContar , y recuento de comentarios
- authorId : Una referencia al perfil del autor
- Información multimedia ligera, como la URL de la miniatura y duración

Este documento está diseñado para casos de uso como la generación de un feed de publicaciones, la visualización de contenido en tendencia o la carga de resúmenes de publicaciones multimedia. Al ser un documento breve y centrarse en los campos de acceso frecuente, las consultas son rápidas y sencillas.

El segundo documento (Documento 2) almacena datos detallados que rara vez se necesitan en las consultas cotidianas, como los siguientes:

- La URL completa del video, almacenada en highResVideo para transmitir o descargar el archivo de video original.

- `metadatos sin procesar`, que contiene detalles técnicos sobre los medios, como su códec, resolución, FPS y tasa de bits, que son relevantes para el procesamiento de backend pero no para las consultas del usuario.
- Una `matriz processingHistory` que registra el flujo de trabajo del archivo multimedia, incluidas las marcas de tiempo de acciones como carga, procesamiento y publicación.
- Una referencia al primer documento con `postId`, que establece una relación entre ambos. Esto permite que las aplicaciones obtengan el documento detallado solo cuando sea necesario, como al preparar el vídeo para su transmisión o al acceder a datos históricos de procesamiento.

Este documento es más extenso y detallado. Se almacena por separado para evitar que las consultas comunes (por ejemplo, listar publicaciones en un feed) carguen información técnica o histórica innecesaria.

El uso del patrón de subconjunto permite dividir los datos en dos documentos separados: uno con campos de acceso frecuente y el otro, almacenado por separado, con información extensa y de acceso poco frecuente. Este enfoque equilibra la eficiencia para consultas frecuentes con la capacidad de recuperar datos detallados cuando se necesitan, lo que lo convierte en una solución ideal en escenarios donde diferentes partes del conjunto de datos tienen patrones de acceso significativamente diferentes.

Patrones para optimizar el rendimiento de escritura

En esta sección, analizaremos patrones de diseño de esquemas beneficiosos para optimizar el rendimiento de escritura. Estos incluyen el patrón de control de versiones de documentos, que permite gestionar eficientemente los cambios en documentos que se actualizan con frecuencia almacenando versiones anteriores en el mismo documento o en una colección independiente; el patrón de agrupamiento, que agrupa varios registros relacionados en un solo documento para reducir el número de escrituras y mejorar la eficiencia de las cargas de trabajo con alta carga de escritura; y el patrón de prefijo de claves, que configura el valor del campo `_id` para evitar la creación de índices secundarios.

Patrón de control de versiones de documentos

En lugar de actualizar documentos existentes y almacenar lo que cambió, cree nuevas versiones y mantenga un puntero a la versión actual.

Esto es particularmente útil cuando existe el requisito de mantener siempre el historial completo de cambios en una entidad.

El siguiente es un ejemplo de configuración de implementación de funciones en el Aplicación social que muestra dos documentos que contienen dos versiones diferentes:

```
    Versión actual (en la colección principal) //
  {
    _id: "feature_story_sharing", nombre:
    "Función para compartir historias", está
    habilitado: verdadero,
    porcentaje de usuario: 75,
    configuración:
      { maxDurationSeconds: 30, filtros
        permitidos: ["sepia", "noir", "vintage"], tamaño máximo de archivo:
          15728640
      }
  }
```

```

    },
    versión: 3,
    Última actualización: Fecha ISO("2023-04-12T16:30:00Z")
  }
  // Colección de historia (versiones anteriores)
  {
    _id: Id. del objeto("5f4e5b6d3c1e2a1b3c4d7f2o"),
    featureId: "compartir_historias_de_funciones",
    Nombre: "Función para compartir historias",
    isEnabled: verdadero,
    porcentaje de usuario: 50, // 50% era ahora 75%
    configuración: {
      Duración máxima en segundos: 30,
      Filtros permitidos: ["sepia", "noir"], tamaño máximo // filtro "vintage"
      de archivo: 15728640
    },
  },
  versión: 2,
  actualizado en: Fecha ISO("2023-04-10T09:15:00Z")
}

```

Repasemos este ejemplo para entender cómo funciona el documento.

Aquí se ha aplicado el patrón de control de versiones. El control de versiones del documento

El patrón se utiliza para rastrear los cambios en un documento a lo largo del tiempo almacenándolo.

la versión actual en una colección principal y versiones anteriores de la documento en una colección histórica separada :

- Documento de la versión actual (la colección **main**) La primera

El documento representa la configuración activa de la función, almacenando lo siguiente:

- El estado actual de la función (isEnabled: verdadero, Porcentaje de usuario: 75)
- La última configuración (permitidosFilters ahora incluye "vintage")

- El número de versión actual (versión: 3) y la marca de tiempo (lastUpdated), que indica cuándo se modificó por última vez

Este documento se almacena en la colección principal , que se centra en la versión actual y sirve a las aplicaciones que necesitan acceder o modificar la configuración de funciones más reciente. Esto optimiza la lectura y las actualizaciones rápidas, ya que representa los datos más recientes.

- Documento de versiones anteriores (la colección **history** el segundo documento proporciona una instantánea de una versión anterior de la configuración de funciones, que incluye lo siguiente:
 - Configuraciones históricas, como userPercentage: 50 y allowedFilters , Antes de que se añadiera el filtro "vintage"
 - El campo versión: 2 muestra claramente que se trata de una versión anterior de la función.
 - La marca de tiempo actualizada documenta cuándo esta versión fue reemplazada por una más nueva

Este documento se almacena en una colección de historial independiente , dedicada al mantenimiento de versiones de documentos. Almacenar versiones anteriores por separado garantiza que la colección principal mantenga su ligereza, a la vez que permite el acceso a datos históricos para casos de uso como auditorías, depuración o análisis comparativos.

El patrón de control de versiones de documentos evita la necesidad de una segunda escritura para registrar los cambios, separando la versión actual de una configuración de funciones de sus versiones anteriores. Este patrón es ideal para aplicaciones que requieren mantener un registro de auditoría, rastrear cambios a lo largo del tiempo o permitir reversiones, como

sistemas de gestión de configuración, plataformas de publicación de contenido o cualquier sistema que necesite datos históricos sin comprometer el rendimiento de los flujos de trabajo activos.

Patrón de agrupamiento Para

series de tiempo o datos de gran volumen, puede utilizar el patrón de agrupamiento para agrupar puntos de datos relacionados en "cubos" para reducir la cantidad de documentos, cuando por algún motivo no es posible utilizar una colección de series de tiempo especiales.

El siguiente es un ejemplo de un caché de línea de tiempo en la aplicación Socialite que rastrea la actividad diaria de un usuario:

```
// Con agrupamiento (optimizado para visualizaciones rápidas de líneas de tiempo)

{ _id: ObjectId("5f4e5b6d3c1e2a1b3c4d5f2f"), userId:
  ObjectId("5f4e5b6d3c1e2a1b3c4d5e6f"), date:
  ISODate("2023-04-10T00:00:00Z"), activities: [ { type:
    "post", postId:

      ObjectId("5f4e5b6d3c1e2a1b3c4d6f4h"), timestamp:
      ISODate("2023-04-10T10:00:00Z"), content: "¡Acabo de publicar
      una nueva foto!", mediaUrl: "https://socialite.com/
      images/photo123.jpg likes: 42 }, { type: "follow", timestamp: Fecha

      ISO("2023-04-10T10:15:12Z"),
      UsuarioObjetivo: { id: IdObjeto("5f4e5b6d3c1e2a1b3c4d6f7j"),
      nombredeusuario:
        "aventuras_de_viaje"

    }
  }
}
```

```

    },
    {
      Tipo: "me gusta",
      marca de tiempo: ISODate("2023-04-10T10:25:30Z"), ID
      de publicación: Id . de objeto("5f4e5b6d3c1e2a1b3c4d6f8k"),
      Autor de publicación: "mountain_explorer",
      Fragmento de publicación: "Día en la cima del Monte Rainier..."
    }
    ... más actividades dentro de este // día
  ],
  número de actividades: 27
}

```

Repasemos este ejemplo para entender cómo se ha aplicado aquí el patrón de agrupamiento:

- **Agrupación de actividades** : En lugar de almacenar cada actividad del usuario (como publicaciones, "Me gusta" y "Seguidos") como documentos individuales, todas las actividades del usuario en una fecha específica (fecha: ISODate("2023- 04-10T00:00:00Z")) se agrupan en un solo documento dentro de la matriz de actividades . Cada entrada de la matriz representa una actividad, lo que facilita el seguimiento de las acciones relacionadas dentro de ese periodo.
- **Metadatos para agregación** : el documento incluye un campo `activityCount` , que proporciona un resumen agregado de la cantidad de actividades almacenadas dentro del depósito para una referencia rápida.

En este ejemplo, el patrón de agrupación se aplica agrupando las actividades diarias de un usuario en un solo documento, en lugar de crear documentos individuales para cada actividad. Este patrón reduce drásticamente las operaciones de escritura y agiliza enormemente la visualización de la cronología de un usuario, ya que una sola consulta puede recuperar un día completo de...

actividad. El patrón de agrupamiento es adecuado para casos de uso que involucran datos basados en el tiempo o agrupados lógicamente, como registros de actividad, eventos seguimiento, lecturas de sensores IoT o datos de series temporales, donde lo natural Existe agrupación de registros y la consulta de registros agrupados es común.

Patrón de prefijo de clave

El patrón clave de prefijos implica agregar prefijos significativos o identificadores de un campo (normalmente el campo `_id`) para agrupar datos relacionados, Permiten realizar consultas eficientes y reducen la necesidad de consultas secundarias. índices. El patrón de prefijo de claves es mejor para cargas de trabajo con muchos Documentos pequeños y relacionados que necesitan recuperarse de manera eficiente juntos sin utilizar índices adicionales.

El siguiente ejemplo de la aplicación Socialite incluye documentos que contienen datos sobre las notificaciones de usuario:

```
{
  _id: "usuario_5f4e5b6d3c1e2a1b3c4d5e6f",           // Principal usuario doc
  nombre de usuario: "jane_smith",
  correo electrónico: "jane.smith@example.com"
  // ... otros campos de usuario
}
{
  _id: "user_5f4e5b6d3c1e2a1b3c4d5e6f_notif_001", tipo:      // No
  "me gusta",
  Mensaje: "A Alex le gustó tu publicación".
  marca de tiempo: ISODate("2023-04-10T10:30:00Z"),
  leer: falso
}
{
  _id: "usuario_5f4e5b6d3c1e2a1b3c4d5e6f_notif_002", tipo:    // Año
  "comentario",
  "Ella d"
```

```
Mensaje: "Sarah comentó tu publicación", marca de tiempo:
ISODate("2023-04-10T11:15:00Z"), lectura: falso
```

```
}
```

Repasemos este ejemplo para comprender cómo se ha aplicado el patrón de prefijo de claves. En este caso, el patrón se utiliza para estructurar el campo `_id` y asociar las notificaciones con un usuario específico, prefijando `_id` con el identificador del usuario:

- Prefijo del documento principal : El documento principal del usuario tiene un valor `_id` (`user_5f4e5b6d3c1e2a1b3c4d5e6f`), donde el prefijo ``user_`` lo identifica claramente como un registro de usuario. Este valor `_id` estructurado ayuda a organizar los documentos de forma lógica y simplifica las consultas dirigidas a tipos específicos de registros.
- Notificaciones relacionadas : Notificaciones relacionadas con el uso del usuario. El mismo prefijo (`user_5f4e5b6d3c1e2a1b3c4d5e6f`) seguido de `_notif_002` . Esto adicionales con sufijo (`_notif_001`) se garantiza que todas las notificaciones asocien directamente con el usuario y se puedan consultar eficientemente mediante `_id` .

Con el índice predeterminado del campo `_id` , puede recuperar eficientemente todas las notificaciones de un usuario mediante una consulta de rango . Además, si el prefijo después de ``user_`` es un ID de objeto, las notificaciones también se ordenan automáticamente por fecha y hora.

Patrones para la optimización de consultas y análisis. En esta sección, analizaremos los patrones de diseño de esquemas que son beneficiosos para la optimización de consultas y análisis. Estos incluyen:

patrón calculado, que precalcula y almacena los patrones derivados o datos agregados para reducir el costo de los cálculos sobre la marcha durante ejecución de consultas; el patrón de control de versiones del esquema, que realiza el seguimiento cambios en las estructuras de los documentos a lo largo del tiempo para garantizar la compatibilidad entre versiones y facilitar el análisis de conjuntos de datos en evolución; y Patrón polimórfico, que permite almacenar diferentes tipos de datos relacionados. datos en una colección unificada aprovechando esquemas flexibles, lo que permite Consultas y análisis eficientes sin necesidad de múltiples colecciones.

Patrón calculado

Con el patrón calculado, precalcula y almacena la agregación resultados para evitar cálculos costosos durante las operaciones de lectura. Esto incluye contadores incrementales cuando ocurren operaciones, de modo que Los totales siempre reflejan las métricas actuales.

El siguiente es un documento de muestra de la base de datos Socialite que Contiene análisis de publicaciones:

```
{
  _id: Id. del objeto("5f4e5b6d3c1e2a1b3c4d6f7k"),
  Contenido: "¡Mira mi nuevo portafolio de fotografía!",
  Fecha de publicación: Fecha ISO("2023-04-10T14:30:00Z"),

  // Métricas precalculadas
  Métricas de compromiso: {
    Vistas totales: 1432,
    Espectadores únicos: 1053,
    Me gustaCount: 287,
    Número de comentarios: 43
    compartirConteo: 18,
    Tiempo promedio de visualización: 12.5, en // segundos
  }
}
```

yo R 0 086 // % F i h k i

```

    tasa de conversión: 0.086 }, // % OMS los espectadores tomaron acción

// Demografía de los participantes (precalculada)
Desglose de la audiencia: {
  distribución de género: {
    masculino: 0,48,
    mujeres: 0,51,
    otros: 0.01
  },
  grupos de edad: {
    "18-24": 0,32,
    "25-34": 0,45,
    "35-44": 0,15,
    "45+": 0,08
  }
}
}
}

```

Repasemos este ejemplo para entender cómo funciona el patrón calculado.

Se ha aplicado. Aquí, el patrón se utiliza para almacenar datos precalculados.

Métricas de participación y desgloses demográficos de la audiencia

Junto con el contenido principal de la publicación:

- Métricas de participación : el documento incluye un cálculo precalculado

Campo engagementMetrics que contiene datos agregados como

total de vistas , uniqueViewers , likeCount , recuento de comentarios ,

shareCount , avgTimeViewed , y tasa de conversión . En lugar de

calculando estas métricas dinámicamente para cada consulta, son

almacenados directamente en el documento para su recuperación instantánea.

- Demografía de la audiencia : el campo " audienceBreakdown"

Proporciona datos precalculados sobre la distribución de género de los espectadores

y porcentajes por grupo de edad. Esto elimina la necesidad de realizar

realizar cálculos analíticos o ejecutar costosas canalizaciones de agregación en tiempo de consulta.

Al precalcular estos datos e incorporarlos al documento,

Consultas relacionadas con estadísticas de participación y datos demográficos de la audiencia.

Se puede ejecutar rápidamente, sin necesidad de acceder a registros de eventos sin procesar.

u otros conjuntos de datos relacionados.

Este patrón elimina la necesidad de consultas de agregación complejas.

durante las operaciones de lectura. Tenga en cuenta que esto aumenta el costo de lo que

Normalmente sería solo una lectura o una escritura agregando un adicional

escribir; para evitar este costo, puede poner en cola la escritura adicional para que se

se realizó como una escritura asincrónica en lugar de hacerla obligatoria durante

la operación original.

Patrón de control de versiones del esquema

Al aplicar el patrón de control de versiones del esquema, se incluye un

Campo de versión para rastrear cambios de esquema a lo largo del tiempo, lo que permite transiciones suaves durante la evolución del esquema.

El siguiente ejemplo incluye dos documentos de usuario de muestra de

La base de datos de Socialite:

```
Nuevo documento 3 con versión de esquema //  
{  
  _id: Id. del objeto("5f4e5b6d3c1e2a1b3c4d5e6f"),  
  versión del esquema: 3,  
  nombre de usuario: "jane_smith",  
  nombre: // En esquema v3, nosotros dividir nombre en primero/último  
    { nombre: "Jane",  
      último: "Smith"  
    },  
}
```

el "j

yo@

yo

"

```

    correo electrónico: "jane.smith@example.com",
    // ... otros campos
  }
  // Documento antiguo con la versión 2 del esquema
  {
    _id: Id. del objeto("5f4e5b6d3c1e2a1b3c4d5e7a"),
    versión del esquema: 2,
    nombre de usuario: "john_doe",
    Nombre completo: "John Doe", // En esquema v2, nosotros usado a soltero
    correo electrónico: "john.doe@example.com",
    // ... otros campos
  }

```

Repasemos este ejemplo para entender cómo funciona el control de versiones del esquema.

Aquí se ha aplicado el patrón:

- Identificación de la versión : El campo `schemaVersion` explícitamente indica a qué versión se ajusta cada documento, lo que permite lógica a nivel de aplicación para manejar variaciones en el esquema durante consultas, actualizaciones o lecturas.
- Evolución del esquema : En el ejemplo, los dos documentos son asociados con diferentes versiones del esquema. Para la versión 3, el nombre se almacena en dos campos (`nombre` y `apellido`), mientras que en Versión 2, el nombre se almacena como un único campo `fullName` .

Este patrón permite que su aplicación maneje documentos con

diferentes versiones del esquema correctamente. Puede que no sea necesario usar un campo `schemaVersion` a menos que sea necesario para la validación del esquema o

Algunas partes de la aplicación no pueden funcionar simplemente con el uso de la ausencia de un campo como indicación de la versión del esquema.

Patrón polimórfico

El patrón polimórfico almacena entidades similares pero no idénticas en la misma colección para realizar consultas flexibles.

El siguiente ejemplo incluye documentos de la base de datos Socialite que contienen datos del feed de actividad, incluidas las publicaciones, los seguidores y las uniones a grupos:

```

actividad {
  // Publicar
  {
    _id: ObjectId("5f4e5b6d3c1e2a1b3c4d5e6f"),
    ActivityType: "publicación",
    UserId: ObjectId("5f4e5b6d3c1e2a1b3c4d5e7a"),
    Timestamp: ISODate("2023-04-10T15:30:00Z"),
    Content: "¡Acabo de publicar una nueva foto!",
    MediaUrl: "https://socialite.com/images/photo123.jpg "
  }

  // Seguir la actividad
  { _id: ObjectId("5f4e5b6d3c1e2a1b3c4d5e7b"), tipo de
    actividad: "seguir", ID de
    usuario: ObjectId("5f4e5b6d3c1e2a1b3c4d5e7a"), marca
    de tiempo: ISODate("2023-04-10T16:45:00Z"), ID de
    usuario objetivo: ObjectId("5f4e5b6d3c1e2a1b3c4d5e8c"),
    nombre de usuario objetivo: "alex_wong"
  }
  // Actividad de unirse al grupo
  { _id: ObjectId("5f4e5b6d3c1e2a1b3c4d5e8d"), tipo de
    actividad: "group_join", ID de
    usuario: ObjectId("5f4e5b6d3c1e2a1b3c4d5e7a"), marca
    de tiempo: ISODate("2023-04-10T17:15:00Z"), ID de
    grupo : ObjectId("5f4e5b6d3c1e2a1b3c4d5e9e"), nombre
    del grupo: "Aficionados a la fotografía" }

```

Repasemos este ejemplo para entender cómo funciona el polimorfismo.

Aquí se ha aplicado el patrón:

- Colección compartida : todas las actividades se almacenan en la misma Colección. Cada documento representa un tipo de actividad distinto, pero comparte campos comunes como `_id` , `activityType` , `userId` y campos adicionales , `marca de` específicos de la actividad.
- tiempo , campos específicos de tipo : el campo `activityType` especifica el tipo de actividad (`publicación` , `valor de` , o `group_join`). Basado en esto seguimiento , campos específicos del tipo se incluyen en cada documento. Para Por ejemplo, una actividad de publicación incluye `contenido` y `mediaUrl` .

Ahora, en lugar de mantener colecciones separadas para cada tipo de Actividad, las consultas pueden apuntar a la colección única usando `activityType` como filtro.

Este patrón permite realizar consultas eficientes en diferentes tipos de actividades manteniendo campos específicos del tipo.

Patrón de archivo para almacenamiento mejoramiento

En esta sección, veremos los patrones de archivo que son beneficiosos

Para optimizar el almacenamiento. Este patrón ayuda a reducir el almacenamiento activo.

costos al mover datos a los que se accede con poca frecuencia o históricos a una carpeta separada Colección o base de datos optimizada para el almacenamiento a largo plazo.

Para aplicar el patrón de archivo, mueva los archivos antiguos o a los que se accede con poca frecuencia. datos para separar colecciones para mejorar el rendimiento de la principal recopilación.

El siguiente ejemplo incluye documentos de mensajes de dos colecciones: una para mensajes activos que son recientes y a los que se accede con frecuencia, y otra para el almacenamiento a largo plazo de documentos de mensajes antiguos:

```
// Recopilación de mensajes activos

{ _id: ObjectId("5f4e5b6d3c1e2a1b3c4d7f5q"), conversationId:
  ObjectId("5f4e5b6d3c1e2a1b3c4d7f6r"), sender:
  ObjectId("5f4e5b6d3c1e2a1b3c4d5e7a"), recipient:
  ObjectId("5f4e5b6d3c1e2a1b3c4d5e6f"), content: "¿Seguimos reunidos
  mañana?", timestamp: ISODate("2023-04-10T14:30:00Z"), read:
  true

}

// Colección de mensajes archivados (con más de 90 días de antigüedad)
{
  _id: ObjectId("5f4e5b6d3c1e2a1b3c4d7f7s"), conversationId:
  ObjectId("5f4e5b6d3c1e2a1b3c4d7f6r"), sender:
  ObjectId("5f4e5b6d3c1e2a1b3c4d5e7a"), recipient:
  ObjectId("5f4e5b6d3c1e2a1b3c4d5e6f"), content: "Hola, ¿cómo estás?",
  timestamp: ISODate("2023-01-05T10:15:00Z"),
  read: true, archivedAt: ISODate("2023-04-05T00:00:00Z")

}
```

Repasemos este ejemplo para entender cómo se ha aplicado aquí el patrón de archivo:

- Colección de mensajes activos : El primer documento forma parte de la colección activa, que almacena datos recientes a los que se accede o actualiza con frecuencia, como los mensajes intercambiados entre usuarios. Este documento representa un mensaje enviado el 10/04/2023.

y permanece en la colección activa porque se considera parte de interacciones en curso.

- Colección de mensajes archivados : El segundo documento forma parte de una colección de mensajes archivados, que almacena mensajes antiguos que ya no forman parte de interacciones activas. Este documento representa un mensaje enviado el 05/01/2023 el [REDACTED], que fue archivado 05/04/2023 tras superar el límite de 90 días. El campo "archivadoAt" [REDACTED] captura la fecha y hora en que este mensaje se trasladó al archivo.

Al separar los datos en colecciones activas y archivadas, este patrón mantiene sus colecciones activas optimizadas y, al mismo tiempo, preserva los datos históricos. Puede archivar datos en otra colección del mismo clúster, en otro clúster o en el archivo en línea de Atlas.

Aplicación en el mundo real: La

Aplicación para socialités

En aplicaciones del mundo real, lograr un rendimiento óptimo a menudo requiere más que simplemente adoptar un único patrón de diseño de esquema; implica combinar múltiples patrones. ¡La flexibilidad de MongoDB permite a los desarrolladores hacer precisamente eso! Esta sección explora cómo la combinación de patrones de diseño de esquema puede mejorar el rendimiento, ilustrado con ejemplos prácticos de escenarios de Socialite.

aplicación.

Escenario 1: Perfil y actividad del usuario

alimentar

En el siguiente escenario, la aplicación Socialite debe funcionar de manera eficiente mostrando el perfil de un usuario junto con su actividad reciente. El

El enfoque tradicional sería almacenar datos de usuario, publicaciones, comentarios, Me gusta y sigue en colecciones separadas, y luego realiza tareas complejas.

se une a estas colecciones cada vez que se ve una página de perfil.

Este método puede ser lento y consumir muchos recursos, especialmente porque

El volumen de datos crece. Sin embargo, con MongoDB, podemos optimizar

Este proceso se realiza mediante una combinación de patrones de diseño de esquemas: patrón calculado y el patrón del subconjunto.

El siguiente ejemplo demuestra este enfoque. Muestra una documento único de la colección de usuarios :

```
// Documento de usuario
{
  _id: Id. del objeto("5f3d7eb8c242d20df83d3a8c"),
  nombre de usuario: "jane_smith",
  nombre: "Jane Smith",
  biografía: "Fotógrafo y entusiasta de los viajes",
  Imagen de perfil: "https://socialite.com/images/profiles/ja
seguidorCount: 1245,      // Patrón calculado
seguidorCount: 362,      // Patrón calculado

// Patrón de subconjunto: actividades más recientes
Actividades recientes: [
  {
    tipo: "publicación",
    marca de tiempo: ISODate("2023-04-07T10:22:15Z"),
    Contenido: "¡Acabo de publicar fotos de mi viaje a Japón!"
    URL de la imagen: "https://socialite.com/images/posts/japan
```

```

    likeCount: 48 },

    { type: "like",
      timestamp: ISODate("2023-04-07T09:15:30Z"), targetPost: { id:

        ObjectId("5f3d7eb8c242d20df83d3a94"), author:
          "travel_adventures", snippet: "Mi día en el
            Monte Fuji..."
        }
      }
    }
  // 3-5 actividades más recientes
]
}

```

Repasemos este ejemplo para entender cómo se han aplicado aquí los patrones calculados y de subconjunto:

- Patrón calculado :
 - Los campos como `followerCount` y `followingCount` se calculan con antelación y se almacenan directamente en el documento del usuario.
 - Reduce la necesidad de agregaciones o consultas sobre la marcha para contar seguidores/seguídos durante las vistas de perfil
- Patrón de subconjunto :
 - Incorpora una matriz de `actividades recientes` directamente en el documento del usuario
 - Incluye solo las publicaciones, los me gusta o las acciones más recientes, optimizadas para un acceso rápido durante la carga de la página de perfil.
 - El historial de actividad más completo se difiere a una colección separada para consultas menos frecuentes

Este diseño optimiza la carga de la página de perfil al incluir las actividades recientes directamente en el documento del usuario. El historial completo de actividades se almacena en una colección independiente que solo se consulta cuando un usuario visualiza más actividades.

Escenario 2: Sistema de chat. En el siguiente

escenario, la aplicación Socialite necesita implementar un sistema de mensajería que gestione eficientemente tanto el chat en tiempo real como el acceso al historial de mensajes del usuario. Podríamos vernos tentados a almacenar todos los mensajes en una sola colección y consultarla repetidamente para obtener actualizaciones en tiempo real y datos históricos, pero esto puede generar cuellos de botella en el rendimiento a medida que aumenta el volumen de mensajes. Sin embargo, con MongoDB, podemos optimizar esto combinando los patrones de subconjunto, agrupamiento y archivo.

El siguiente ejemplo demuestra este enfoque e incluye dos documentos, el primero de una colección de mensajes activos y el segundo de una colección de mensajes archivados:

```
// Recopilación de mensajes activos: estadísticas de conversaciones actuales
{
  _id: ObjectId("5f4e5b6d3c1e2a1b3c4d7f6r"), participantes:

  [ ObjectId("5f3d7eb8c242d20df83d3a8c"), // jane_smith
    ObjectId("5f4e5b6d3c1e2a1b3c4d5e7a") ], lastMessage: john_doe //

  { sender:
    ObjectId("5f4e5b6d3c1e2a1b3c4d5e7a"), content: "¡Nos vemos
    mañana!", timestamp:
    ISODate("2023-04-12T18:45:22Z") },
}
```

```

unreadCount: {
  "5f3d7eb8c242d20df83d3a8c": 0,
  "5f4e5b6d3c1e2a1b3c4d5e7a": 1
},

// Mensajes recientes (patrón de subconjunto)
Mensajes recientes: [
  {
    remitente: ObjectId("5f4e5b6d3c1e2a1b3c4d5e7a"),
    Contenido: "¡Nos vemos mañana!"
    marca de tiempo: ISODate("2023-04-12T18:45:22Z"),
    leer: falso
  },
  {
    remitente: ObjectId("5f3d7eb8c242d20df83d3a8c"),
    Contenido: "Sí, las 3 p. m. me vienen bien."
    marca de tiempo: ISODate("2023-04-12T18:44:15Z"),
    leer: verdadero
  }
  // 10-15 mensajes más recientes
]
}
// Colección de mensajes archivados: para mensajes más antiguos,      USO
{
  _id: Id. del objeto("5f4e5b6d3c1e2a1b3c4d8f1t"),
  ID de conversación: Id. de objeto("5f4e5b6d3c1e2a1b3c4d7f6r"),
  fecha: ISODate("2023-04-11T00:00:00Z"),      // Un cubo p
  mensajes: [
    {
      remitente: ObjectId("5f3d7eb8c242d20df83d3a8c"),
      Contenido: "¿Qué tal si nos reunimos mañana?"
      marca de tiempo: ISODate("2023-04-11T15:30:12Z"),
      leer: verdadero
    }
    Más mensajes del de esto //
  ]
}

```


Repasemos este ejemplo para comprender cómo se han aplicado aquí los patrones de subconjunto, archivo y agrupación:

- Patrón de subconjunto :
 - El campo `recentMessages` del primer documento de la colección de mensajes activos almacena los 10 a 15 mensajes más recientes directamente dentro del documento.
 - Permite un acceso rápido a los mensajes más recientes sin necesidad de realizar una consulta independiente a un almacén de datos más grande.
- Patrón de archivo :
 - El segundo documento utiliza el patrón de archivo para almacenar mensajes antiguos. El campo `"conversationId"` vincula el contenedor a una conversación específica, lo que garantiza una agrupación lógica por `"conversationId"`. El campo `"date"` representa el día que abarca el contenedor, lo que permite la organización cronológica de los mensajes.
 - Este patrón garantiza el almacenamiento y la recuperación eficientes de mensajes más antiguos a los que se accede con menos frecuencia y reduce la sobrecarga al consultar o analizar conversaciones históricas, ya que las consultas pueden apuntar a segmentos específicos en función de la fecha y el `conversationId`.
- Patrón de agrupamiento :
 - los mensajes más antiguos se agrupan en grupos diarios en la matriz de `mensajes` en una colección separada con un documento por conversación por día. Organiza los mensajes
 - históricos de manera eficiente para reducir la sobrecarga de consultar conjuntos de datos masivos y mejora la escalabilidad para conversaciones de larga duración.

Este enfoque permite un acceso rápido a mensajes recientes y al mismo tiempo mantiene la escalabilidad organizando y consultando archivos de mensajes históricos de manera eficiente.

Resumen

Al alinear su esquema con los patrones reales de uso de datos (cómo lee, escribe o actualiza documentos), puede minimizar la E/S y la contención de recursos. La estructura flexible de MongoDB puede ser útil, pero solo si se usa con cuidado, con la combinación adecuada de incrustación, referencia e indexación.

Aquí hay muchas conclusiones importantes. Es fundamental comprender la carga de trabajo y considerar los patrones de lectura y escritura antes de diseñar el esquema. Asegúrese de pensar en documentos, no en tablas. Aproveche la flexibilidad del modelo de documento en lugar de imponer patrones relacionales. Recuerde diseñar para operaciones comunes y optimizar las operaciones con los requisitos de rendimiento más estrictos.

Prueba con volúmenes de datos realistas: qué funciona con 100 documentos

Podría fallar con 10 millones. Por último, sea siempre intencional: un diseño de esquema deliberado siempre superará el almacenamiento arbitrario de datos.

El diseño de esquemas en MongoDB es tanto un arte como una ciencia. Al comprender la carga de trabajo de su aplicación y equilibrar la eficiencia de lectura y escritura, puede crear esquemas eficientes y fáciles de mantener.

MongoDB cierra la brecha entre los almacenes de clave-valor rápidos y escalables horizontalmente y las bases de datos relacionales con numerosas funcionalidades.

Para aprovechar al máximo su potencia, diseñe su esquema para aprovechar el documento.

Puntos fuertes del modelo: integrar datos relacionados, evitar búsquedas innecesarias y optimizar para sus patrones de acceso específicos. En el siguiente capítulo, nos basaremos en estos principios de diseño de esquemas para explorar estrategias de indexación que mejoren aún más el rendimiento de su aplicación.

3

Índices

¿Cuál es la operación de base de datos más común? Es una consulta o la lectura de uno o varios documentos. Insertamos documentos una vez, los actualizamos a veces y los leemos con frecuencia. Por lo tanto, es importante garantizar que esas lecturas sean lo más rápidas posible.

En este capítulo, aprenderá a usar índices para mejorar el rendimiento de sus consultas. Comenzará por comprender qué son los índices y cómo los usa MongoDB. Después, explorará los planes de consulta y cómo interpretarlos para identificar operaciones lentas o ineficientes. También practicará la creación y el uso de índices para acelerar las consultas y verá cómo MongoDB elige qué índice usar. Al finalizar este capítulo, podrá tomar decisiones informadas sobre cuándo y cómo usar índices para optimizar el rendimiento.

Este capítulo cubrirá los siguientes temas:

- Índices y tipos de índices que admite MongoDB
- Cardinalidad, selectividad y cómo afectan a los índices
- Mejores prácticas para optimizar índices compuestos
- Por qué los índices parciales y las consultas de índices cubiertos son importantes para el rendimiento

Introducción a los índices

Comencemos con un ejemplo básico. Supongamos que tiene una colección de pedidos y quiere encontrar el pedido con el valor de ID 295 :

```
db.orders.find({ orderId: 295 })
```

Sin un índice, MongoDB debe examinar cada documento de la colección para comprobar si contiene un campo `orderId` y, a continuación, si su valor es 295. Para una colección con millones de documentos, esto puede ser extremadamente lento. Esto genera una complejidad temporal de $O(n)$, lo que significa que el tiempo necesario aumenta linealmente con el número de documentos.

Esto es similar a empaquetar todo en cajas para una mudanza. Si no te tomas el tiempo de organizar, agrupar y etiquetar las cajas, encontrar un artículo específico más tarde significa tener que revisarlo todo. Esto no es muy eficiente, pero te ahorra tiempo al empacar. Los índices de bases de datos funcionan de la misma manera. Debes decidir si es más importante dedicar tiempo a organizar y etiquetar artículos al empacar (escribir) o si puedes permitirte dedicar tiempo extra a buscar un artículo (leer). En la mayoría de los casos, lo más inteligente es dedicar algo de tiempo al principio para que sea más fácil encontrarlos después.

¿Qué es un índice?

En el mundo de las bases de datos, un índice es una estructura de datos que permite encontrar registros rápidamente sin tener que explorar toda la colección. MongoDB utiliza un tipo de estructura de datos llamada árbol B para la mayoría de sus índices.

Un árbol B es una estructura de datos de árbol autoequilibrado que mantiene los datos ordenados y permite diversas operaciones en tiempo logarítmico, que es $O(\log n)$. Esto significa que, a medida que los datos crecen, el tiempo para encontrar...

El documento aumenta mucho más lentamente en comparación con un escaneo de colección, que se escala linealmente con el tamaño de sus datos o tiempo $O(n)$.

Echemos un vistazo a los detalles. Si creamos un índice en nuestro campo `orderId`, el árbol B simplificado podría verse similar a la Figura 3.1 :

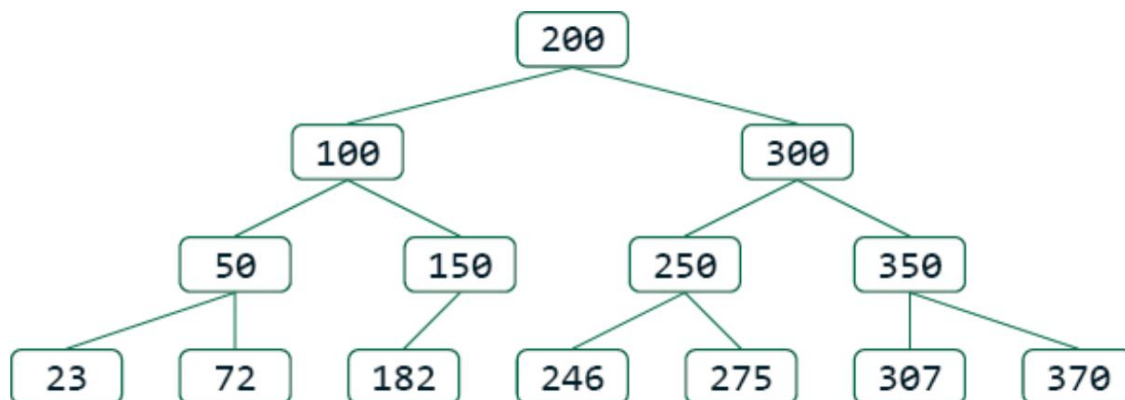


Figura 3.1: Vista simplificada del árbol B en el campo `orderId`

Al buscar el ID de pedido 295 MongoDB seguiría estos pasos:

1. Comience en el nodo raíz, [200] .
2. Compara 295 con 200. Como $295 > 200$, ve a la derecha.
3. Llegar al nodo [300] .
4. Compara 295 con 300. Como $295 < 300$, ve a la izquierda.
5. Llegada al nodo [250] .
6. Compara 295 con 250. Como $295 > 250$, ve a la derecha.
7. Continúe hacia la derecha hasta encontrar 295 o determinar que no existe.

Este proceso toma un tiempo de $O(\log n)$, lo cual es considerablemente más rápido que el tiempo $O(n)$ requerido para un escaneo de colección.

Los nodos hoja almacenan los valores indexados, así como los punteros (ID de registro) a los documentos que contienen dichos valores. Por lo tanto, cuando llegamos al nodo hoja que contiene el valor 295, encontramos el recordID asociado ,

Este es un puntero o referencia a la ubicación real del documento. Si varios documentos comparten el mismo valor, el nodo hoja contiene varios ID de registro.

Un árbol B suele tener más de dos ramas desde cada nodo y, al igual que un árbol B+ típico, los árboles B de MongoDB solo contienen datos en los nodos hoja; los nodos internos son solo para navegación.

Curiosamente, la estructura de datos específica utilizada no es tan importante como el hecho de que los valores indexados se almacenan en una estructura de datos ordenada que puede ser buscada y recorrida eficientemente en orden. Si se considera un índice como una lista ordenada o un árbol de búsqueda binario, no está muy lejos de la realidad. Lo importante es que puede acceder fácilmente a los datos y encontrarlos ordenados, y al estar ordenados, es relativamente rápido (tiempo $O(\log n)$) acceder a un valor o rango de valores en particular.

Eficiencia de los recursos y compensaciones

Los índices no solo aceleran las consultas, sino que también son clave para usar los recursos del sistema eficientemente. Al realizar un análisis completo de una colección, se obliga a MongoDB a cargar toda la colección en memoria (en concreto, en la caché de WiredTiger). Esto no solo tarda más, sino que también consume un recurso limitado (memoria caché) y puede causar mucha E/S de disco debido a la rotación constante.

Como alternativa, al usar un índice, MongoDB solo necesita cargar las partes relevantes del índice (normalmente más pequeñas que una colección) y luego recuperar los documentos específicos que coinciden con los criterios de consulta. Esto ahorra memoria valiosa e IOPS que pueden utilizarse para otras operaciones, lo que aumenta la eficiencia de toda la base de datos.

Además de búsquedas más rápidas, los índices también cumplen con los requisitos para devolver los resultados en un orden específico. Si los documentos se leen en orden desde el índice, se pueden evitar costosas operaciones de ordenación en memoria. En un

En pocas palabras, los índices aceleran las consultas, minimizan la E/S del disco, reducen los requisitos de RAM y reducen la carga general de la CPU.

Pero los índices no son gratuitos; hay una contrapartida. Cada índice que se añade requiere una actualización automática cada vez que se modifica un valor indexado, se crea un nuevo documento o se elimina uno existente. Esto ralentiza las operaciones de escritura. Mantener los índices en memoria consume más RAM, y escribir tanto en índices como en documentos aumenta el uso del disco.

El impacto en el rendimiento de la latencia de escritura de cada índice puede oscilar entre el 5 % y el 70 %, dependiendo del número de campos, el tamaño y el tipo de índice, y de la carga de trabajo, según las pruebas realizadas por el equipo de rendimiento de MongoDB. No existe una fórmula exacta, y la única forma fiable de saber cuánto afecta la latencia de escritura es comparar los datos y la carga de trabajo reales. Por eso, es importante ser estratégico al crear índices, evitar mantener índices redundantes y no indexar todo.

Uso de recursos

Mencionamos que los índices requieren espacio en disco y RAM, pero ¿cuánta RAM utilizan? Puedes ver el tamaño de tu índice usando `$collStats`, pero el tamaño del índice no significa necesariamente que deba caber todo.

En RAM. No todos los índices son iguales en cuanto al uso de recursos. Existe el mito de que todos los índices deben caber en RAM para lograr el mejor rendimiento, pero esto solo sería cierto si siempre se accediera al índice completo. Esto podría no ser necesario en una aplicación bien diseñada.

Imagine que su aplicación procesa pedidos con IDs que aumentan secuencialmente. Los nuevos pedidos siempre tienen IDs más altos que los existentes. En su B-

índice del árbol, todas las nuevas escrituras ocurren en el lado derecho del árbol, como se muestra en la Figura 3.2 .

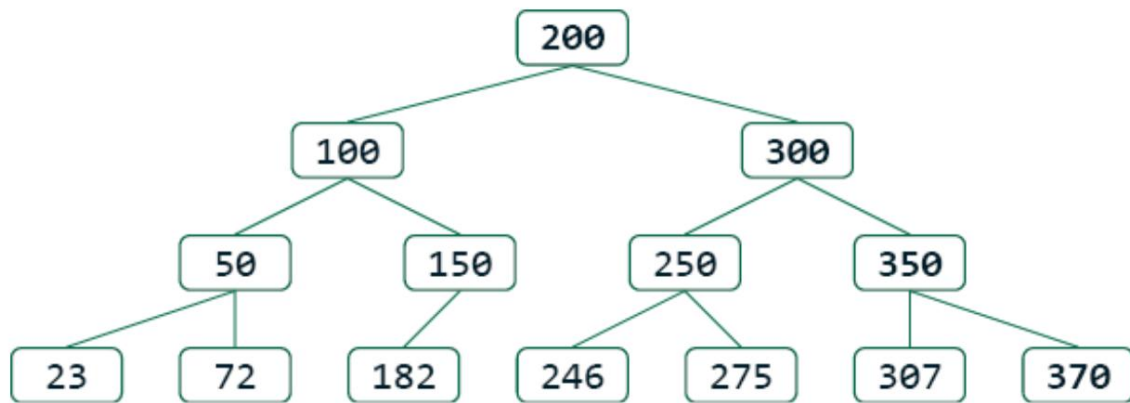


Figura 3.2: Árbol B con toda la actividad reciente en el lado derecho

Los nuevos pedidos siempre se insertan a la derecha del índice (valores más altos), y la mayoría de las actualizaciones se realizan en pedidos recientes, sin mencionar que los pedidos antiguos rara vez se leen. Esto crea

Lo que llamamos un índice equilibrado . La parte del índice con órdenes más antiguas se accede con poca frecuencia y, por lo tanto, no se cargará en la RAM y puede permanecer inactiva en el disco.

Solo la parte del índice a la que se accede (la parte activa que contiene los datos recientes) debe estar en memoria.

Los campos que pueden crear un índice equilibrado incluyen cualquier valor que aumente naturalmente, por ejemplo:

- Identificadores que se incrementan automáticamente, como los identificadores de pedidos
- Marcas de tiempo y/o fechas
- ID de objetos generados automáticamente por MongoDB (porque incluyen un componente de marca de tiempo como los primeros cuatro bytes)

Por el contrario, al indexar un valor aleatorio, los patrones de acceso abarcarán todo el árbol B, lo que eventualmente requerirá que todo el índice se cargue en memoria, lo que lo hace mucho menos eficiente. Los tipos de valores aleatorios que pueden obligar a que todo el índice permanezca en RAM incluyen los siguientes:

- UUID
- La mayoría de los valores de cadena (nombres, nombres de usuario, etc.)
- Cualquier valor hash

Por lo tanto, si bien el tamaño del índice es importante, la forma de acceder a él lo es aún más. Diseñar cuidadosamente los patrones de acceso a los datos puede reducir significativamente el uso de memoria y mejorar el rendimiento.

Conceptos erróneos comunes sobre los índices en MongoDB

Existen algunos mitos comunes sobre MongoDB (y NoSQL en general). Uno de ellos es que MongoDB es tan rápido que no necesita índices.

Esto podría ser cierto solo si su conjunto de datos es tan pequeño que cabe en la RAM y cada consulta es rápida, incluso si se realiza un análisis completo de la colección. Esto nunca ocurriría en un sistema de producción real. Otro mito es que todos los campos se indexan automáticamente. En MongoDB, solo la clave principal (el campo `_id`) se indexa automáticamente. Para lograr un rendimiento aceptable en entornos de producción, debe analizar sus patrones de consulta y crear los índices secundarios adecuados.

Tipos de índices en MongoDB

MongoDB ofrece múltiples tipos de índices para optimizar diferentes patrones de consulta. La mayoría utiliza árboles B, pero no todos. Este capítulo se centrará en aquellos que sí utilizan árboles B como base de estructura.



El tipo de índice puede referirse a los campos que está utilizando indexación o qué atributos le asigna al índice. El primero especifica los campos que se almacenarán en el árbol B; Este último se centra más en los atributos del índice en su conjunto.

Echemos un vistazo a algunos de los tipos de índices más comunes.

Índices de campo único

Un índice de campo único en MongoDB es un índice creado en un solo campo de una colección. Así se crea un índice básico en un solo campo:

```
db.collection.createIndex({ nombreDeCampo: 1 })
```

En la sección ¿Qué es un índice? ya vimos un ejemplo usando

`{orderId:1}`, que acelera las consultas de ID de pedidos específicos, así como Ordena o clasifica consultas por ID de pedido.

Un índice de campo único que existe en cada colección regular es el `_id` índice, que refuerza la unicidad, aunque no la muestra explícitamente

El atributo único cuando se ejecuta un comando para enumerar los índices:

```
// Comando mostrar todos los índices a en recopilación
db.collection.getIndexes()
// salida que muestra solo el índice _id
[ { "v" : 2, "clave" : { "_id" : 1 }, "nombre" : "_id_" } ]
```

Así es como puedes ver los índices que existen para una colección en particular.

La mayoría de los índices que cree en la vida real no serán índices de un solo campo;

Sin embargo, cada colección regular tendrá al menos un índice por defecto:

un índice único en el campo `_id`, que es la clave principal.

Índices compuestos

Los índices compuestos se crean en varios campos. A continuación, se muestra un ejemplo que crea un índice en tres campos:

```
db.collection.createIndex({ estado: 1, país: 1, zipString
```

Los índices compuestos almacenan múltiples valores de campo de cada documento, por lo que cada entrada tiene múltiples valores concatenados. El árbol B almacena los datos ordenados por los valores combinados. Veamos un ejemplo de indexación por estado, país y código postal:

Valores de estado, país y código postal en su colección	El orden en que se muestran estos valores se almacenan en el índice
"Procesando","EE. UU.,""10007" "Listo","EE. UU.,""95401" "Listo","FR","" "Listo","EE. UU.,""90210" "Listo","EE. UU.,""14850"	"listo","FR","" "listo","US","14850" "listo","US","90210" "listo","US","95401" "procesando","US","10007"

Tabla 3.1: Cómo afecta el orden del índice a la recuperación de documentos

Primero, los registros se ordenan por estado, luego por país y, finalmente, por código postal . Probablemente pueda observar que el orden de los campos al crear un índice compuesto es muy importante. Permite usar el índice compuesto para consultar solo el primer campo o cualquier campo de prefijo. Más información al respecto en breve.

Índices multiclave

Un índice multiclave no es técnicamente un tipo de índice separado que... crear explícitamente; aún así se crea un índice en uno o más campos. Pero si cualquiera de esos campos contiene una matriz, MongoDB crea un índice entrada para cada elemento de la matriz y etiqueta el índice como si fuera una clave múltiple Índice. He aquí un ejemplo sencillo:

```
// Si los documentos tienen: { etiquetas: ["electrónica", "venta", "nuevo"]
db.collection.createIndex({ etiquetas: 1 })
// Crea entradas de índice para "electrónica", "venta", // // todas las cuales y "nuevo
// apuntan al mismo ID de registro a
```

Los índices de campo único, compuestos y otros se pueden etiquetar como multiclave

Si algún campo indexado contiene una matriz, revisaremos las implicaciones de

Esto se tratará a lo largo del resto de este capítulo y nuevamente [Capítulo 12](#), en Conceptos avanzados de consulta e indexación .

Índices dispersos

Si especifica que el índice debe ser disperso , entonces solo se mostrarán los documentos que tienen los campos que se están indexando se insertarán en el árbol B. Esto es a diferencia de los índices regulares (no dispersos), que indexarán cada documento de la colección, incluso aquellos que no tienen los campos siendo indexado.

A continuación se explica cómo crear un índice disperso en MongoDB:

```
es // escaso un atributo del índice siendo creado
db.collection.createIndex({a:1}, {sparse:true})
```

Lo anterior crea un índice en el campo `a`, que sólo contiene documentos donde el campo `a` está presente.

Debido a la semántica nula del lenguaje de consulta MongoDB (`MQL`), los documentos indexados no dispersos en los que falta un campo se indexan de la misma manera.

Lo mismo que los documentos en los que el campo existe con un valor de `null`. En un índice disperso, una entrada nula significa que el campo está presente y configurado como nulo.

Exploraremos esto más a fondo en [Capítulo 12](#) Consulta avanzada e indexación.

Conceptos.

Índices comodín

Los índices comodín le permiten especificar un índice sin conocer el nombres de los campos por adelantado. Esto no es raro con un dinámico esquema:

```
// Campos de índice dentro del subdocumento attr
db.collection.createIndex({ "attr.$**": 1 })
// Documento de muestra
//
//    ...,
//    atributo: {
//        género: "Misterio",
//        autor: "John Smith",
//        páginas: NumberInt(298),
//        ...
//    }
// }

usando // la consulta índice
db.collection.find({"attr.pages":{$gt:500}})
```

Esto le permite realizar consultas utilizando un índice en cualquiera de los atributos almacenados.

bajo `attr` sin tener que crear un índice separado para cada uno de ellos

Los índices comodín son dispersos de forma predeterminada.

Índices parciales

En ocasiones, puede que desee indexar y consultar solo un subconjunto de sus documentos. Los índices parciales solo indexarán los documentos que coincidan con un filtro específico. Este comando crea un índice en dos campos, pero solo indexa los documentos cuyo estado sea "en proceso" :

```
db.orders.createIndex( { país:  
  1, fecha de pedido: 1 },  
  { partialFilterExpression: { estado: "procesando" } }  
)
```

Una consulta con {status:"processing"} entre otros predicados de consulta podrá utilizar este índice parcial.

El uso de índices parciales puede reducir significativamente el tamaño del índice y ahorrar recursos. Esto es especialmente útil cuando las consultas se dirigen solo a una pequeña parte de la colección. Por ejemplo, si consulta con frecuencia solo documentos activos , que constituyen un pequeño subconjunto de sus datos, un índice con un filtro parcial de {active:true} será compacto y eficiente. Cuando un documento se actualiza a {active:false} , se elimina automáticamente del índice.

Esta no es una lista exhaustiva de todos los tipos de índices y atributos; puede leer más sobre ellos en la documentación de MongoDB Server en <https://www.mongodb.com/docs/manual/indexes/> .

Diseño de índices eficientes

Los índices son una piedra angular de la estrategia de optimización del rendimiento de MongoDB, ya que permiten realizar consultas y ordenar de forma rápida y eficiente grandes conjuntos de datos. Sin embargo, diseñar índices efectivos requiere más que comprender

Su función básica exige una cuidadosa consideración de los patrones de consulta, la cardinalidad de los campos y las necesidades operativas. Los conceptos erróneos comunes sobre los campos de baja cardinalidad y las consultas de desigualdad suelen confundir a los desarrolladores, pero las estrategias de indexación adecuadas pueden mejorar drásticamente el rendimiento. Los índices compuestos, la compresión de índices, las consultas cubiertas y los índices parciales proporcionan herramientas poderosas para optimizar las operaciones de bases de datos, y la directriz Igualdad, Ordenamiento, Rango (ESR) simplifica las decisiones complejas en torno al ordenamiento de campos.

En esta sección, exploraremos los principios fundamentales del diseño de índices y cubriremos temas críticos como cardinalidad versus selectividad, diseño de índices compuestos, compresión de prefijos, índices parciales y cómo los índices admiten canales de agregación, brindándole las herramientas para diseñar índices que maximicen el rendimiento de su aplicación.

Cardinalidad y selectividad

Existe la idea errónea de que los campos de baja cardinalidad (campos con pocos valores únicos) no deben incluirse en los índices. Esto simplemente no es cierto.

La cardinalidad en bases de datos se refiere a la cantidad de valores distintos que puede tener un campo. Un campo booleano tiene dos cardinalidades: verdadero o falso. Un campo de ID único tiene una cardinalidad igual al número de documentos de la colección. El apellido tiene una cardinalidad mayor que el nombre.

La cardinalidad no es tan importante para la eficacia de la consulta como la selectividad, el número de documentos en la colección que coinciden con un valor específico en su consulta.

Por ejemplo, imagine una colección de pedidos con un campo de estado , que puede ser "en proceso" o "terminado" . En un momento dado, menos del 0,01 % de los pedidos tienen el estado "en proceso", ya que la mayoría...

están "hechos" . Un índice de estado sería excelente para realizar consultas.
 procesar pedidos porque es muy selectivo, aunque el campo tiene
 Baja cardinalidad. Por otro lado, usar ese índice para encontrar todos los órdenes
 que se han llevado a cabo es de ayuda limitada porque coincide al 99,99%
 de todos los documentos de la colección.

Esto nos lleva a un malentendido relacionado: las consultas de desigualdad
 (como `$ne`) no pueden usar índices eficazmente. ¡Eso es falso! Si consulta
 para `{status:{ne:"done"}}` contra la misma colección de pedidos , será
 ser tan selectivo como consultar `{estado:"procesando"}` . El
 El motor de consultas de MongoDB lo maneja de forma ligeramente diferente (se procesa como un
 consulta de rango), pero al ser altamente selectiva, sigue siendo muy eficiente, incluso
 aunque utilizará un plan de consulta diferente:

```

Para {estado:{ne:"hecho"}}, // el Usos del escaneo de índice:
"límites de índice": {
  "estado": [
    "[MinKey, \"hecho\"]",
    "[\"hecho\", MaxKey]"
  ]
}
// Para {estado: "procesando"}, El escaneo de índice utiliza:
"límites de índice": {
  "estado": [
    "[\"procesando\", \"procesando\"]"
  ]
}

```

Estos son ejemplos de resultados del método de explicación , que explicaremos a continuación.

Hablaremos de más detalles más adelante en [Capítulo 12](#) Consultas avanzadas e indexación.

Conceptos : donde empezamos a intentar depurar nuestras consultas y validar cómo
 nuestros índices contribuyen al rendimiento. Saber si una operación...

Una consulta de igualdad o rango es muy importante para nuestro próximo tema de ordenar campos en índices compuestos.

Construcción de índices compuestos

Para optimizar los índices compuestos, es necesario comprender cómo los utilizan las diferentes consultas. Un índice compuesto indexa múltiples campos en un orden definido, y ese orden afecta significativamente la forma en que MongoDB utiliza el índice.

Un índice compuesto en MongoDB incluye varios campos y ordena los documentos primero por el valor del primer campo indexado, luego por el segundo, y así sucesivamente. Internamente, está organizado como una estructura de árbol ordenada, lo que permite un recorrido eficiente cuando la consulta coincide con el inicio del patrón de clave del índice. Por ejemplo, un índice compuesto en `{a:1,b:1,c:1}` permite a MongoDB localizar eficientemente documentos que coinciden con las consultas que filtran por lo siguiente:

```
{ a: <valor> } { a:
<valor>, b: <valor> } { a: <valor>,
b: <valor>, c: <valor> }
```

Esto se conoce como coincidencia de prefijo. La consulta debe coincidir con el prefijo de los campos de índice para poder usar el índice. Una consulta que omite el primer campo, como `{b:<valor>}` o `{c:<valor>}`, usa este índice. En esos casos, no puedo MongoDB realiza un análisis completo de la colección.

Veamos un ejemplo más práctico. Supongamos que crea un índice compuesto como este: `{categoría:1,precio:1,stock:1}`.

Este índice funciona bien para consultas como `{category:"books"}` o `{category:"books",price:{<lt:20}}`. Usar una lista ordenada puede ayudarnos a...

Visualice cómo se puede utilizar el índice:

```
"libros", 10.399
"libros", 13.205
"libros", 15.11
"libros", 22.0
"bolígrafos",
2.175
"bolígrafos", 5.42
"bolígrafos", 12.109 "bolígrafos", 15.98
```

Cuando la lista está ordenada por categoría y luego por precio, es fácil encontrar rápidamente todos los libros o todos los bolígrafos, y una vez que encontramos una sola categoría, es fácil encontrar el rango de "precio menor a \$20" y devolver esos registros en orden por precio.

Sin embargo, puede ver que si simplemente solicitara todos los artículos menores a \$20 sin especificar una categoría, tendríamos que revisar cada valor de categoría, lo que haría que el índice fuera mucho menos útil. Esto se debe a que la consulta omite el campo de categoría. Dado que el índice se ordena primero por categoría, MongoDB no tiene una forma eficiente de aislar el rango relevante de documentos solo por precio. Además, el índice es completamente inutilizable para responder a una consulta como "¿qué artículos tienen stock 0?". Podríamos realizar un escaneo de colección para esa consulta.

Comprender cómo funciona la coincidencia de prefijos es importante para diseñar índices compuestos que se alineen con los patrones de consulta más comunes de su aplicación.

Consultas de igualdad

Un patrón de consulta común implica el uso de la igualdad en varios campos. Por ejemplo, supongamos que almacena direcciones, principalmente en Estados Unidos.

donde se aplique lo siguiente:

- El país suele ser "EE. UU." (cardinalidad baja), el
- estado tiene alrededor de 50 posibilidades (cardinalidad media) y
- zipString tiene más de 42 000 valores únicos (cardinalidad alta).

Digamos que siempre consultamos con igualdad en los tres campos:

```
db.collection.find({país: "EE. UU.", estado: "NY", código postal: " "})
```

¿Qué índice compuesto tendría un mejor desempeño?

- Opción 1: { país: 1, estado: 1, zipString: 1 }
- Opción 2: { zipString: 1, estado: 1, país: 1 }

La sabiduría convencional, o tu instinto, podría decir la Opción 2 porque el campo más selectivo se ordena primero. Pero resulta que no tendría éxito.

Las consultas se ejecutan más rápido. Para consultas con condiciones de igualdad en todos los campos, ambos índices tendrán el mismo rendimiento. Es importante comprender este concepto: al seleccionar en todos los campos, el árbol B para la Opción 1 y la Opción 2 tendrá el mismo tamaño porque deben indexar el mismo número de documentos; por lo tanto, tendrán la misma profundidad, y la búsqueda de un valor específico tendrá exactamente el mismo valor de $O(\log n)$ para ambas.

Dicho esto, debería preferir la Opción 1 si todo lo demás es igual. Cuando los campos de baja cardinalidad (como country , que podría ser solo "US" o "CA") aparecen primero, MongoDB puede comprimir largas series de valores repetidos con mayor eficacia. Debido a la compresión de prefijos , colocar primero los campos de baja cardinalidad reducirá significativamente el tamaño del índice tanto en el disco como en... RAM:

```
"país_1_estado_1_cadena_zip_1": 620 KB  
"cadena_zip_1_estado_1_país_1": 804 KB
```

¡Eso representa una diferencia de tamaño de casi el 30%! Índices más pequeños significan menos espacio en disco.

uso, cargas más rápidas y un uso de memoria más eficiente.

Todavía puede haber razones válidas para ordenar los campos de una determinada manera. Para

Por ejemplo, el índice de la Opción 2 puede atender consultas solo en `zipString` o en

`zipString` y `state`. Por lo tanto, podría haber buenos casos de uso para

Elegir la Opción 2 en lugar de la Opción 1, pero esperando "consultas más rápidas" con 1: la igualdad en los tres campos no es una de ellas.

Las cosas se vuelven más interesantes (y más complejas) cuando empezamos a sumar consultas de rango y operaciones de clasificación en la mezcla.

Ordenaciones y consultas de rango

Los índices nos permiten responder de manera eficiente a las consultas de rango porque podemos

Encuentre rápidamente el valor en un extremo del rango y luego proceda a través de

el índice, ordenando los documentos hasta llegar al primer valor anterior

nuestro límite superior, como sigue:

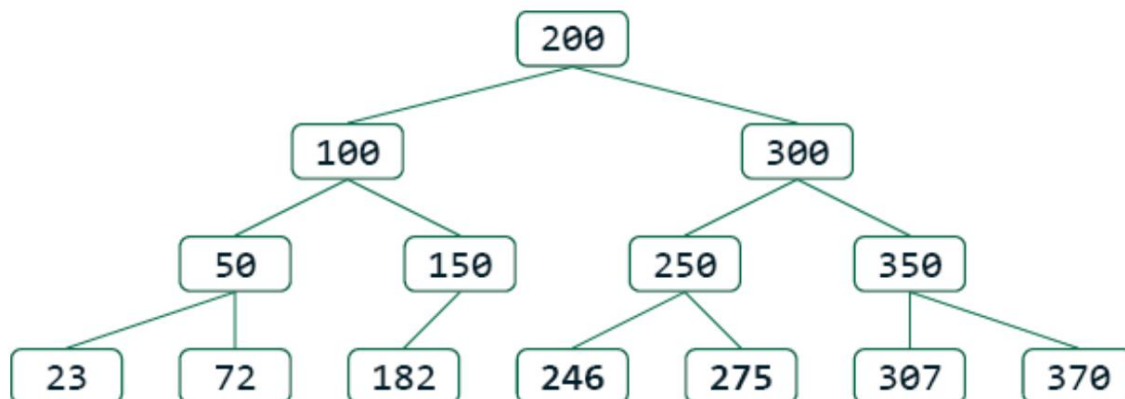


Figura 3.3: Uso de índices para consultas de rango y ordenamiento eficientes

Para una consulta sobre `{field:{$gt: 205, $lt:290}}`, encontramos nuestro valor inferior

rango, 205, y luego recorremos los nodos de las hojas hasta que encontramos un

valor mayor que nuestro rango superior, 290. Tenga en cuenta que estamos obteniendo estos

Los valores se ordenan según el índice. Por lo tanto, si la consulta tuviera sort: nosotros `{field:1}` , se obtendrían los valores ordenados gratuitamente. Si la expresión de buscaríamos el valor del superior estuviera en `{field:-1}` , primero , ordenación rango 290 y luego recorreríamos el índice de derecha a izquierda (es decir, hacia atrás en lugar de hacia adelante) hasta superar el , la parte inferior límite de 205 , lo que nos daría los valores en orden descendente.

¿Qué ocurre si tenemos una consulta de igualdad en un campo y una consulta de rango en otro? Analicemos el índice de `{category:1,price:1}` y la consulta `{category:"books",price:{<:20}}` . Ya vimos que podemos encontrar fácilmente documentos con igualdad en la categoría y rango (u ordenamiento) en el precio :

```
"libros", 10
"libros", 13
"libros", 15
"libros", 22
"bolígrafos",
2 "bolígrafos",
5 "bolígrafos",
12 "bolígrafos", 15
```

Dado que el campo de igualdad va primero, podemos navegar directamente a la parte del árbol ordenado que coincide con ese valor, y el segundo campo permanece ordenado, por lo que aplicar un filtro de rango es igual de sencillo. Pero ¿qué ocurre si el índice está en `{price:1,category:1}` , lo que significa que el filtro de rango está en el primer campo y el de igualdad en el segundo?

El índice ahora se vería así:

```
2,"bolígrafos"
5,"bolígrafos"
10,"libros"
12,"bolígrafos"
```

```
13,"libros"
15,"libros"

15,"bolígrafos" 22,"libros"
```

En este índice, cuando navegamos hacia el rango del índice donde el precio es menor a 20 , Descubriremos que la categoría está completamente mezclada. Esto significa que tendremos que...

Escanear más entradas de índice con un precio inferior a 20, descartando las que no coinciden con el predicado de igualdad, de lo que haríamos con una estructura de índice diferente. ¿Y si necesitáramos ordenar el segundo campo después?

¿Aplicar rango en el primer campo? Nuevamente, como los valores no están en orden, necesitaríamos trabajar mucho más en memoria.

El mismo concepto se aplica si tenemos igualdad y ordenación. Si la igualdad se da en el primer campo del índice compuesto, los valores del segundo campo siguen "ordenados" en la parte del índice a la que hemos accedido. Pero si la ordenación se da en el primer campo del índice, aunque podemos ordenar esos valores al escanear el índice completo, debemos revisar y descartar constantemente las entradas donde el segundo campo no es igual a nuestro predicado.

¿Qué sucede si tenemos los tres: igualdad, predicado de rango y una ordenación en otro campo? Volvamos a nuestro ejemplo con tres campos en el índice {category:1,price:1,stock:1} y estos valores:

```
"libros", 10.399
"libros", 13.205
"libros", 15.11
"libros", 22.0
"bolígrafos",
2.175
"bolígrafos", 5.42
"bolígrafos", 12.109 "bolígrafos", 15.98
```

Podemos ver que si consultamos la categoría "libros", los precios están en orden, pero las acciones no. Si el índice cambiara el orden de precios, obtendríamos lo siguiente: las acciones se convierten en {categoría:1,acciones:1,precio:1}.

```
"libros",0,22
"libros",11,15
"libros",205,13
"libros",399,10
"bolígrafos",42,5
"bolígrafos",98,15
"bolígrafos",109,12
"bolígrafos",175,2
```

Ahora podemos encontrar rápidamente artículos de categorías específicas sin stock, u ordenar categorías específicas por stock, pero ya no tenemos los valores de precio ordenados. Por lo tanto, si tuviéramos consultas de rango y ordenación, o dos, podríamos determinar si el documento coincide.

A partir del índice, no podríamos limitar el número de entradas de índice que examinamos solo a las que coinciden.

Todo esto nos lleva a algunos principios generales que podemos aplicar.

La guía ESR

Es posible que haya oído hablar de las directrices ESR para índices compuestos:

- E : Campos de igualdad primero
- S : Ordenar campos a continuación
- R : Campos de rango últimos

Si bien esta no es una regla estricta, sirve como guía útil, y una parte de ella es absoluta: colocar los campos de igualdad primero. Aplicar la igualdad a los campos principales del índice compuesto nos permite seguir aprovechando

El resto de los valores de campo están ordenados. Sin embargo, si los campos de ordenación o de rango deben ir a continuación depende de sus necesidades específicas:

- Si es fundamental evitar ordenaciones en memoria, coloque los campos de ordenación antes de los campos de rango (ESR)
- Si su predicado de rango en la consulta es muy selectivo, colóquelo antes de los campos de ordenamiento (ERS) para reducir la cantidad de documentos a ordenar y permitir un ordenamiento eficiente en memoria.

Por lo tanto, la igualdad debe venir primero porque la estructura de un índice ordenado con múltiples campos nos permite aplicar un filtro de igualdad mientras preservamos el orden de los campos restantes.

Maximizar recursos con índices parciales

A veces, no necesitamos indexar todos los documentos, especialmente si una consulta solo apunta a un subconjunto pequeño y bien definido de su colección. En algunos casos, su consulta de igualdad múltiple podría especificar siempre un campo específico con un solo valor. Por ejemplo, en una colección de pedidos en proceso, probablemente habrá muchos menos pedidos con el estado "en proceso" que aquellos marcados como "terminados". Por otro lado, estos son los pedidos que probablemente consultará con más frecuencia. En tal caso, crear un índice que admita consultas sobre pedidos "en proceso" (e ignore los pedidos "terminados") puede ser mucho más eficiente que un índice estándar para cada documento.

Para respaldar las consultas de pedidos que se están procesando donde obtenemos estado, siguiente índice: y país, código postal, podemos crear el

```
{estado:1, código postal:1, estado:1, país:1}
```

Este índice ya se beneficiará de la compresión de prefijos, pero sería...

No nos beneficiamos de estar en equilibrio correcto, aunque sólo estemos consultando documentos con procesamiento de estado, porque cuando el estado es Actualizado al índice, Ahora tenemos que actualizar páginas en la parte de la "Listo", que no se lee habitualmente. Ahorraríamos más espacio y recursos. indexando únicamente los documentos con estado "en proceso", como esto:

```
db.coll.createIndex(
  {zipString:1,estado:1,país:1},
  {partialFilterExpression:{estado:"procesando"}})
```

Ahora, solo un pequeño subconjunto de toda su colección se almacena en el índice, y cuando un pedido se actualiza para tener el estado establecido en, va a "listo", se desindexa o se elimina automáticamente del índice, manteniendo su tamaño pequeño.

Consultas cubiertas: el rendimiento sagrado

Una consulta cubierta es aquella en la que se especifican todos los campos necesarios para satisfacer la consulta. están en el índice, por lo que MongoDB no necesita mirar el valor real. documentos en absoluto. Estas consultas suelen requerir menos recursos. Aquí es un ejemplo de lo que se necesitaría para tener una consulta cubierta:

```
// Nosotras creaste índice:
db.orders.createIndex({ orderId: 1, estado: 1 })
// Esto está cubierto, consulta El índice ya contiene el estado
db.orders.find({ orderId: 295 }, { _id: 0, estado: 1 })
// Esto está cubierto, no consulta. nosotros Necesito obtener documentos para
db.orders.find({ orderId: 295 }, { _id:0, total: 1 })
```

Sin embargo, existen algunas limitaciones. Las consultas cubiertas generalmente no funcionan con índices multiclave cuando el campo que se necesita devolver podría ser un array. Esto se debe a que MongoDB necesita revisar el documento para determinar si el valor a devolver es un escalar o un array. Algunas consultas nulas no se pueden cubrir porque el índice no distingue entre campos que contienen valores nulos y campos que no existen.

Ninguna forma de la cláusula \$exists se puede determinar a partir de un índice regular debido a la misma semántica nula. Por lo tanto, si desea obtener una consulta indexada eficiente, evite las cláusulas \$exists por regla general, a menos que utilice un índice disperso. Analizaremos este tema con más detalle en Conceptos avanzados de consultas e indexación [Capítulo 12](#).

Orden de índice ascendente versus descendente

Si te fijaste, todos los ejemplos usaron {field:1} en lugar de {field:-1}. Esto se debe a que normalmente no necesitamos almacenar el índice en orden descendente. El motor de consultas puede escanear el árbol B de izquierda a derecha o de derecha a izquierda, según necesite los resultados en orden ascendente o descendente. Solo hay una excepción: ordenar varios campos en direcciones opuestas.

Digamos que tenemos la siguiente consulta:

```
db.coll.find({categoría:"libros"}).sort({género:1, precio:-1})
```

Para crear un índice que admita esto completamente, tendríamos que hacer que el índice sea uno de los siguientes:

```
{categoría:1, género:1, precio:-1} O {categoría:1, género:-1, p
```

Esto se debe a que el orden de clasificación especificado en la consulta utiliza dos campos de clasificación en direcciones opuestas (una ascendente y otra descendente); por lo tanto, El índice de esos dos campos también debe estar en orden opuesto para respaldar Ordenar . Dado que el índice se puede escanear en orden ascendente o descendente, Podemos invertir el orden de cualquiera de los dos campos de clasificación .

Canalizaciones de indexación y agregación

Los índices son elegibles para ser utilizados por todos los comandos MongoDB que necesitan encontrar Un subconjunto de documentos en una colección: buscar , actualizar , eliminar e , y incluso agregar . Pero esto último suele causar cierta confusión: cuando ¿Puede utilizar índices para filtrar datos y cuándo no?

Piense en lo que hacen los índices: ayudan a localizar documentos por su ID de registros. Una vez que una canalización de agregación transforma los documentos en nuevas formas, esos índices ya no se aplican porque estamos tratando con Documentos nuevos en memoria que no corresponden a nuestro indexado estructura de la colección. Pero cuando la agregación necesita obtener documentos de una colección que realmente existe, entonces los índices son tan críticos como para cualquier otra operación de búsqueda .

Los índices se utilizan en las etapas \$match y \$sort de agregación cuando Se aplican a los documentos de la colección que se agregan. Los índices son También lo utiliza cualquier etapa que pase a otra colección, como por ejemplo \$unionWith (que ejecuta una nueva canalización) o \$lookup (que ejecuta una consulta en otra colección basándose en un valor en la colección actual canalización). Analizaremos más de cerca cómo determinar qué índices son siendo utilizado en [Capítulo 4](#) Agregaciones y Avanzado [Capítulo 12](#) , Conceptos de consulta e indexación .

Resumen

Los índices son herramientas potentes que, si se usan correctamente, pueden acelerar significativamente las consultas de MongoDB. Si bien mejoran el rendimiento de lectura, tienen una pequeña desventaja: escrituras más lentas. En la mayoría de los casos, esta desventaja compensa con creces las mejoras de rendimiento. Es importante comprender el comportamiento de los diferentes tipos de índices. Por ejemplo, los índices con balance correcto, como los basados en ObjectID u otros valores secuenciales, son más eficientes en el uso de memoria.

Al elegir los campos a indexar, concéntrese en la selectividad (la capacidad de un campo para restringir los resultados) en lugar de solo en la cardinalidad. En índices compuestos, priorizar los campos de baja cardinalidad puede mejorar la compresión de prefijos, pero también es fundamental considerar la coincidencia de prefijos para minimizar el número de índices creados. Los índices parciales pueden ser revolucionarios al reducir drásticamente el tamaño del índice cuando solo se necesita indexar un subconjunto de documentos. Además, las consultas cubiertas (donde todos los campos obligatorios están en el índice) suelen ser las más eficientes, así que téngalas en cuenta al diseñar sus índices.

Al aplicar estas prácticas recomendadas, estará bien preparado para crear una base de datos MongoDB de alto rendimiento que satisfaga las necesidades de su aplicación. En el siguiente capítulo, exploraremos el potente marco de agregación de MongoDB y profundizaremos en temas más avanzados, como la forma en que el optimizador de consultas selecciona índices.

4

Agregaciones

Si alguna vez ha intentado procesar datos para obtener información significativa, sabe que los documentos sin procesar por sí solos no son suficientes. Necesita segmentar, fragmentar, transformar y resumir. Aquí es donde destaca el marco de agregación de MongoDB.

Imagínelo como una cadena de montaje de datos: los documentos entran por un extremo, experimentan una serie de transformaciones a medida que pasan por varias etapas y emergen transformados por el otro.

Es potente y flexible, pero si no tienes cuidado, puede convertirse en un cuello de botella en el rendimiento.

En este capítulo, exploraremos cómo crear pipelines de agregación potentes y de alto rendimiento. Analizaremos estrategias de optimización de pipelines, técnicas de gestión de memoria, consideraciones específicas de cada etapa y enfoques para gestionar grandes conjuntos de datos, todo ello con el rendimiento como guía.

Este capítulo cubrirá los siguientes temas:

- Los fundamentos del marco de agregación de MongoDB
- Técnicas de optimización para el rendimiento de la canalización de agregación
- Mejores prácticas para trabajar con grandes conjuntos de datos
- Agregación en entornos distribuidos y colecciones fragmentadas

- Monitoreo, elaboración de perfiles y resolución de problemas del rendimiento de la agregación

Marco de agregación de MongoDB

El framework de agregación de MongoDB es la navaja suiza de la transformación y el análisis de datos en el mundo de las bases de datos. Pero antes de profundizar en las técnicas de optimización, entendamos qué hace que esta herramienta sea tan especial.

En sus inicios, las opciones de agregación de MongoDB eran, digamos, minimalistas . Los desarrolladores dependían del comando `mapReduce` o de la gestión de agregaciones en el código del cliente para realizar el resumen de datos.

Si bien eran funcionales, estos enfoques solían ser torpes, lentos y no especialmente fáciles de usar.

Presentamos la canalización de agregación, una innovación introducida en MongoDB

2.2. De repente, los desarrolladores podían encadenar una serie de etapas, cada una transformando los datos de alguna manera, como una línea de montaje en una fábrica o una tubería de línea de comandos de Unix. En versiones posteriores, el marco de agregación se ha vuelto más potente, añadiendo nuevas etapas, operadores, expresiones y optimizaciones de rendimiento para satisfacer las necesidades de procesamiento de datos cada vez más sofisticadas.

Si tiene experiencia en SQL, quizás esté acostumbrado a consultas con múltiples `JOIN` . La canalización de agregación operaciones de escritura `SELECT`, `GROUP BY` y ofrece capacidades similares , pero con una particularidad: en lugar de una consulta monolítica, se crea una canalización de etapas, cada una de las cuales realiza una tarea o transformación específica. Los documentos fluyen desde una colección a la primera etapa.

Luego de allí pasan a etapas posteriores y, finalmente, se devuelven al cliente como resultado.

Este enfoque modular facilita la lectura y el mantenimiento de tareas complejas de procesamiento de datos, especialmente al trabajar con estructuras documentales complejas, comunes en MongoDB. Además, en las agregaciones de MongoDB se pueden realizar tareas que, como trabajar con matrices complejas de documentos, harían que una consulta SQL tradicional resultara demasiado compleja.

Conceptos básicos de la canalización de agregación. Imagine sus

datos como un

río, y cada etapa de la canalización como una presa, un filtro o una turbina que modifica el flujo de alguna manera. Los documentos entran en la canalización, se transforman en cada etapa y emergen finalmente como el conjunto de resultados deseado. Esto es muy similar a usar la canalización de línea de comandos de Unix para realizar múltiples transformaciones en un archivo de texto grande:

```
grep xxx archivo.log | cortar .. | sed | ordenar | encabezado -20 cat archivo.log |  
awk '{imprimir $2, $NF}' | ... >> nuevoarchivo.log
```

El orden de las etapas es muy importante. Filtrar con antelación reducirá drásticamente la cantidad de trabajo posterior. Cada etapa trabaja exclusivamente con los documentos que recibe de la etapa anterior, y su resultado se convierte en la entrada para la siguiente etapa de la secuencia.

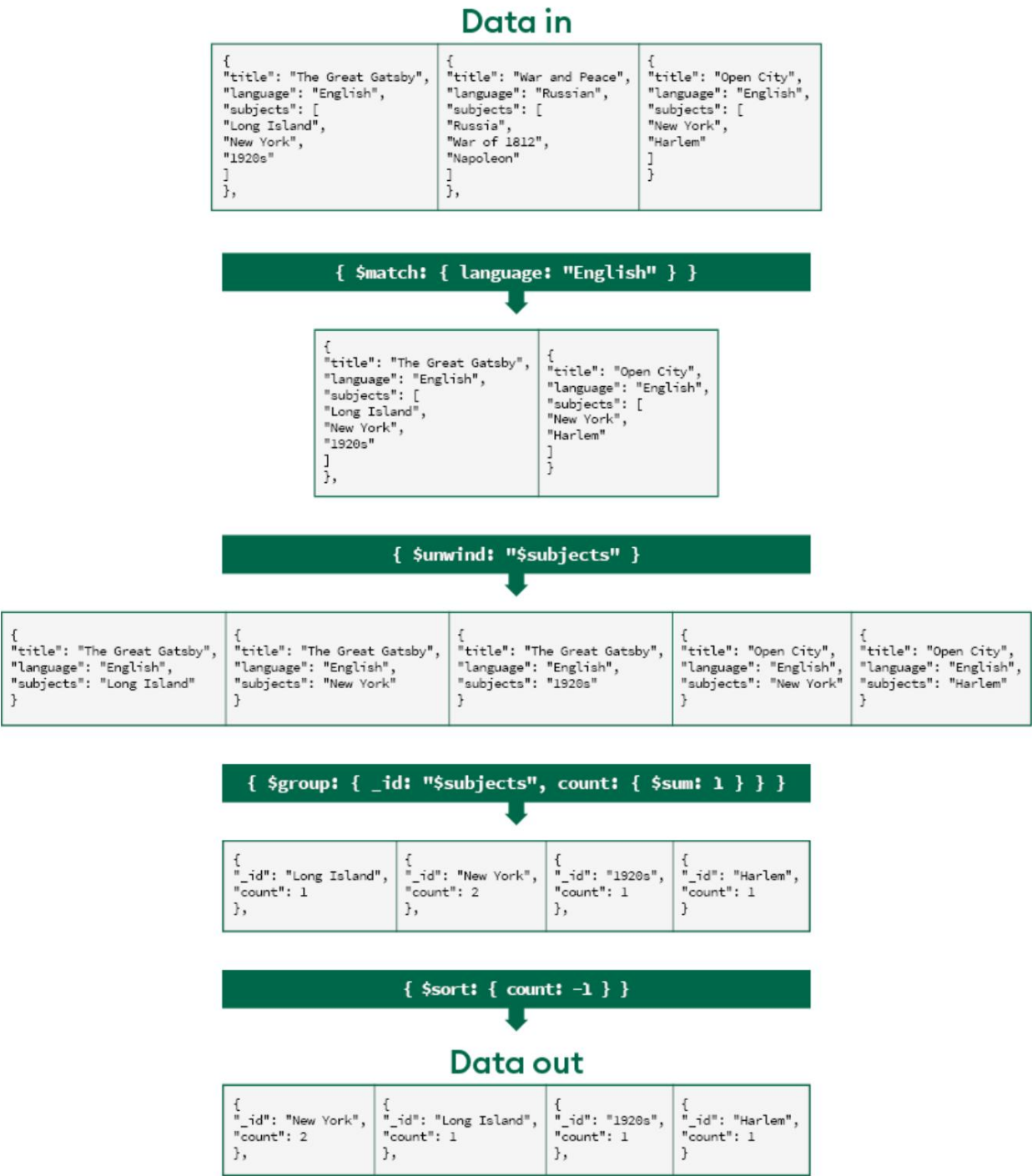


Figura 4.1: Los documentos se transforman a medida que fluyen a través de las etapas del proceso de procesamiento.

Todo en la canalización utiliza una sintaxis específica para realizar su magia:

- Las etapas definen qué tipo de transformación ocurre, como por ejemplo: `$match` stage para filtrar documentos, `$group` para agrupar o agregándolos, `$set` para agregar nuevos campos o cambiar valores de campo existentes y `$sort` para cambiar el orden de documentos en el flujo. MongoDB ofrece muchas de estas etapas, cada una Cumplen un propósito específico. Puedes explorar la lista completa en <https://www.mongodb.com/docs/manual/reference/operator/aggregation/>.
- Las expresiones de agregación como `$eq`, `$gt`, `$in`, `$convert` se utilizan para evaluar, calcular y asignar nuevos valores.
- Las expresiones de ruta le brindan una forma de usar valores existentes en campos actuales.
- Las expresiones de acumulador como `$sum`, `$avg`, `$push`, `$min` y `$max` se utilizan dentro de las etapas de agrupación para realizar agregaciones sobre datos agrupados.

Las expresiones en el marco de agregación utilizan una sintaxis similar a JSON. Verá muchos signos de dólar (`$`); indican operadores, expresiones o rutas de campo. Las expresiones se pueden anidar, lo que permite cálculos potentes y flexibles. Por ejemplo, podrías usar `$add` para sumar dos campos, `$cond` o `$switch` para implementar condicional lógica, o una amplia gama de operadores integrados para aritmética, cadenas manipulación, manejo de fechas y operaciones lógicas.

Puede hacer referencia a campos dentro de sus documentos utilizando la ruta de campo notación (campos simples con `"fieldName"` campos anidados con `"$object.nestedField"`). El framework también admite varios tipos de variables, incluidas variables del sistema como `$$ROOT` a hacer referencia a todo el documento o `$$NOW` para obtener la hora actual, y le permite configurar sus propias variables para facilitar la lectura.

Consideraciones de rendimiento

Si bien el marco de agregación es potente, no es mágico. Gran parte del rendimiento depende de cómo se construyen las canalizaciones. Una canalización bien diseñada puede ser increíblemente rápida, mientras que una mal diseñada puede colapsar la base de datos. La clave está en pensar en cómo fluyen los datos a través de la canalización y cómo cada etapa afectará el rendimiento.

Por ejemplo, filtrar documentos de forma temprana con `$match` puede reducir drásticamente la cantidad de datos que las etapas posteriores necesitan procesar. El uso eficaz de índices también es crucial. Al igual que con las consultas regulares, una canalización que pueda usar un índice para encontrar rápidamente los documentos que necesita tendrá un rendimiento mucho mayor que una que tenga que analizar toda la colección.

Como veremos en la siguiente sección, el optimizador de consultas de MongoDB hace mucho para mejorar el rendimiento de manera automática, pero comprender los principios detrás de estas optimizaciones lo ayudará a escribir mejores canalizaciones.

Flujo de la tubería de agregación

Tras analizar la canalización, el optimizador de consultas de MongoDB la analiza e intenta optimizarla al máximo. Puede reordenar las etapas para mejorar el rendimiento (asegurando que no se alteren los resultados), combinar operaciones compatibles, aplicar filtros más cercanos a los datos para usar los índices disponibles eficazmente y crear un plan de ejecución óptimo basado en la estructura de la canalización. También analiza qué campos de los documentos originales utiliza la canalización para limitar la cantidad de datos que se incorporan desde la colección.

Internamente, MongoDB envía lotes de documentos a través de la canalización en streaming. Cada etapa recibe un lote de documentos de la etapa anterior, aplica su transformación y pasa los resultados a la siguiente. Este diseño minimiza el uso de memoria y permite un procesamiento eficiente de grandes conjuntos de datos.

Sin embargo, no todas las etapas pueden pasar cada lote a la siguiente, ya que algunas necesitan ver toda la información que les llega antes de poder pasarla. Piense en `$sort`, que no sabe cuál es el valor más alto o más bajo hasta que ve el último documento, o en `$group`, que no sabe si cada agrupación está completa hasta que ha visto todos los documentos.

Documento. Escuchará que estas etapas se denominan etapas de bloqueo, a diferencia de las etapas de transmisión. Las etapas de transmisión permiten que cada lote de documentos se transmita una vez finalizado su trabajo, pero las etapas de bloqueo deben recibir todos los documentos antes de que puedan pasar a la siguiente etapa.

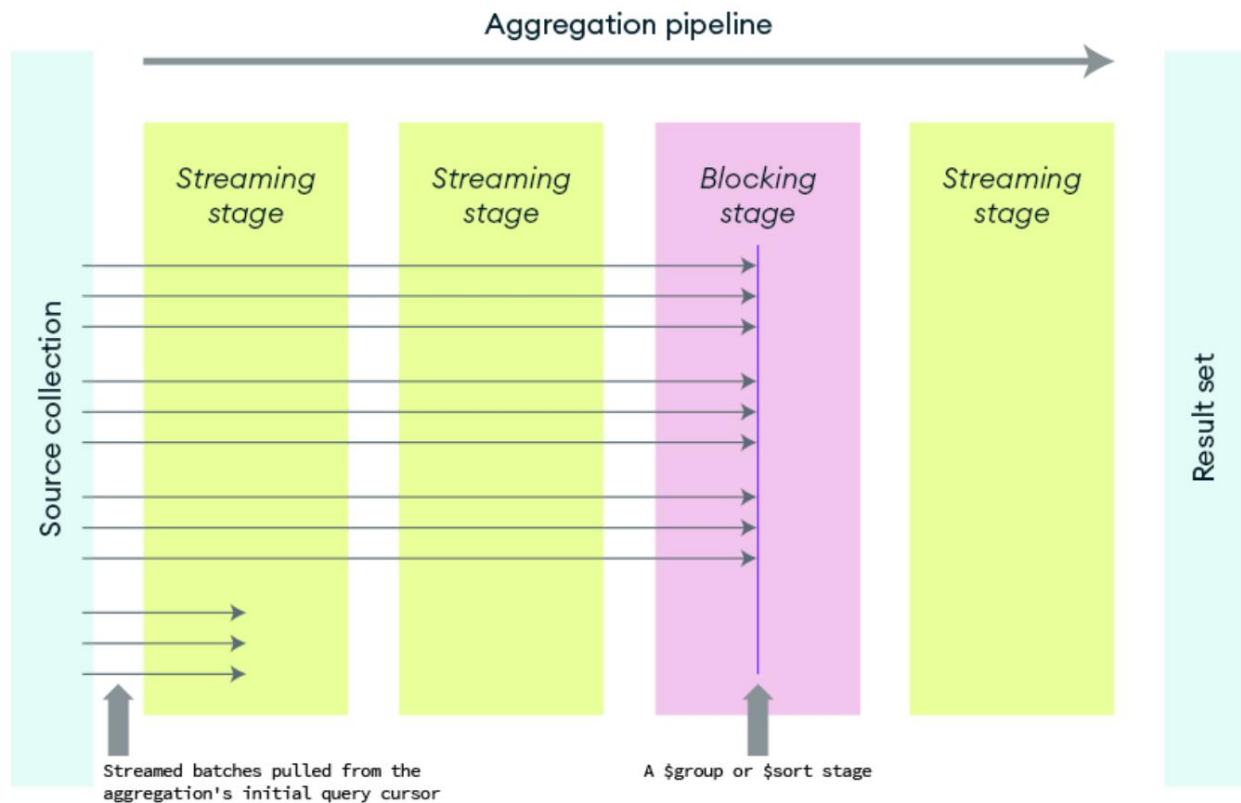


Figura 4.2: Canal de agregación con etapas de transmisión y bloqueo

En clústeres fragmentados, cuando la canalización incluye documentos de más de un fragmento, MongoDB ejecuta la mayor parte posible de la canalización en cada fragmento para distribuir la carga de trabajo antes de fusionar los resultados. Por ejemplo, las etapas de filtrado y transformación se realizan en los fragmentos, lo que reduce la cantidad de datos que se envían a través de la red. Mientras que operaciones como la agrupación o la ordenación solo realizan el trabajo inicial en los fragmentos, pero posteriormente pueden requerir la fusión de resultados intermedios y la finalización del trabajo en el enrutador MongoDB .

MongoDB está trabajando arduamente para acelerar su pipeline. ¿Qué optimizaciones le ofrece esto?

Optimización de las canalizaciones de agregación

El orden de las etapas en su pipeline de agregación puede afectar significativamente el rendimiento. Si bien el optimizador de consultas de MongoDB es bastante inteligente y puede reordenar las etapas automáticamente, comprender estas optimizaciones puede ayudarle a diseñar mejores pipelines.

El motor de consultas de MongoDB ejecuta una canalización ligeramente diferente a la que se pasa al optimizador de consultas. Puede comparar la canalización real que se ejecutará con la que escribió utilizando el método `explain()`, que revela cómo MongoDB ha transformado su canalización para mejorar la eficiencia. Ejecute la agregación con `explain("executionStats")` o `explain("queryPlanner")` para examinar la canalización transformada que MongoDB ejecuta. Utilizar "executionStats" también le mostrará el número de documentos que fluyen de una etapa a otra, así como el tiempo total empleado hasta ese momento. Esto le permite ver qué etapas consumen mucho tiempo y deben examinarse para una posible revisión, y cuáles contribuyen tan poco a la latencia general que intentar ajustarlas no será perceptible al ejecutar la canalización completa.

```
db.collection.explain("Estadísticas de ejecución").agregado([ {$match {
  etapas:
    [ { '$cursor': { queryPlanner:
      { parsedQuery: {'$and':[ {a:{$eq:1}}, {b:{$gte winningPlan:
        { etapa:
          'PROJECTION_SIMPLE',
          transformBy: { b: 1, d: 1, e: 1, y: 1, _id: 0 inputStage:
            { etapa: 'IXSCAN',
              keyPattern: { a: 1, b:
                1 },
              // ...
            },
          estadísticas de ejecución: {
```

```

    ejecuciónÉxito: verdadero,
    nDevuelto: 45,
    tiempoMillisDeEjecución: 2,
    totalKeysExamined: 45,
    totalDocsExamined: 45,
    etapasDeEjecución:
      { etapa: 'PROJECTION_SIMPLE',
        nDevuelto: 45,
        tiempoMillisDeEjecuciónEstimado: 0,
        transformBy: { b: 1, d: 1, e: 1, y: 1, _id: 0
          // ...
        } }, nDevuelto: Long('45'),
    EstimaciónMillisDeTiempoDeEjecución: Long('2') },

    { '$establecer': { x: '$y' },
      nDevuelto: Long('45'),
      EstimaciónMillisDeTiempoDeEjecución: Long('2') },

    { '$establecer': { a: '$b' },
      nDevuelto: Long('45'),
      EstimaciónMillisDeTiempoDeEjecución: Long('2') },

    { '$proyecto': { a: verdadero, c: { '$añadir': [ '$d', '$e' ] } nDevuelto:
      Long('45'),
      EstimaciónMillisDeTiempoDeEjecución: Long('2') } } }
  /* ... */

```

Aquí puedes ver las etapas ejecutadas. La primera etapa se llama , aunque campos parece que está ejecutando la etapa \$match . \$cursor muestra los que devuelve en el campo transformBy , y puedes ver que las etapas \$set y \$project prácticamente no tardaron nada, ya que cada etapa muestra el tiempo acumulado invertido.

lo que significa que cada una de las etapas que no son \$cursor tardó 0 milisegundos en ejecutar.

Técnicas de optimización

Veamos varias técnicas de optimización y cómo pueden ayudar a que su canalización funcione más rápido.

Filtrar datos con antelación. Aunque

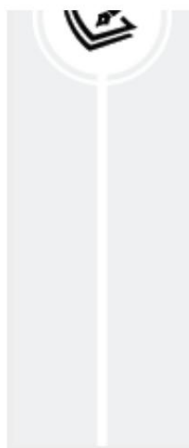
MongoDB reordena automáticamente las etapas, comprender los principios que subyacen a estas reordenaciones puede ayudarle a crear pipelines más eficientes. La idea general es reducir el volumen de datos lo antes posible:

- Coloque \$match lo antes posible y asegúrese de que pueda ser compatible con un índice, lo que garantizará que solo los documentos necesarios se incorporarán al flujo de trabajo desde el principio, lo que reducirá la cantidad de documentos que deben leerse desde el caché (o peor aún, desde el disco) y procesarse.
- Coloque \$sort lo más temprano posible en la canalización y asegúrese de que pueda ser compatible con un índice

A diagram showing a vertical pipeline with a stage highlighted. The stage is represented by a vertical bar with a circular arrow at the bottom, indicating a loop or iteration.

Evite la etapa inicial del proyecto para preservar la optimización del pipeline

No es necesario eliminar manualmente los campos que no se utilizarán. Esto es algo que la etapa de optimización del pipeline ya hace automáticamente al examinar qué campos de su pipeline están...



utilizando y evitando recuperar campos no utilizados. Esta optimización automática de la canalización se puede derrotar de manera involuntaria si se agrega una etapa `$project` de inclusión explícita temprana innecesaria, por lo que no use `$project` hasta el final de la canalización para cambiar el nombre o reformular de otro modo los resultados en la forma que el cliente espera.

MongoDB puede combinar ciertas etapas en una única etapa más eficiente. Esta fusión reduce la sobrecarga y acelera la ejecución. Como práctica recomendada, puede agrupar operaciones similares usted mismo y evitar etapas innecesarias para facilitar el trabajo del optimizador. Sin embargo, no dedique demasiado tiempo, ya que algunas de estas etapas pueden ser muy rápidas. Por otro lado, algunas etapas pueden acelerarse significativamente si se hace lo correcto :

- Cuando la etapa `$sort` va seguida inmediatamente de una etapa `$limit`, MongoDB optimiza esta etapa combinándolas y rastreando únicamente los N primeros documentos ordenados, lo que reduce significativamente el uso de memoria para la ordenación. Esto se conoce como ordenación *Y* top-N y es mucho más eficiente que ordenar todo el conjunto de datos y luego conservar solo los N primeros valores.
- Por lo general, varias etapas `$match` adyacentes se fusionan en una sola etapa `$match`, combinando sus condiciones de filtro con un AND implícito, mismo. Pero, por lo que no debería ser necesario combinarlas usted asegúrese de tener un índice compuesto para admitir las condiciones en el filtro combinado.

En general, cuando las etapas que el sistema de consultas MongoDB puede optimizar se ubican al principio del proceso, se postergan hasta el final.

El subsistema de consultas, y solo los documentos que la consulta devuelve se procesan en la canalización. Al observar la salida de Explain , esto generalmente se manifiesta por la primera etapa, que es \$cursor , y absorbe todas las etapas iniciales, como los casos \$match y \$limit \$sort , donde no hay más etapas de agregación y la , en ello. En canalización completa de agregación podría haberse expresado como un comando find . En realidad, se ejecutará igual que el comando find equivalente, y Explain no mostrará ninguna etapa de agregación adicional.

En esta sección, viste cómo ciertas etapas pueden transferirse al subsistema de consultas cuando se colocan al principio del pipeline. Deben optimizarse de la misma manera que las consultas equivalentes. Veamos ahora algunas de las otras etapas potencialmente desafiantes y escenarios.

Evite los \$unwind y \$group innecesarios

La etapa \$unwind se utiliza para aplanar matrices y, a menudo, se malinterpreta como un factor negativo en el rendimiento. Sin embargo, en realidad, no es inherentemente lenta ni problemática. El problema radica en el tratamiento de sus resultados. Cuando se desenrolla o aplanar una matriz para aplicar ciertas etapas a los documentos resultantes, pero luego estos se agrupan de nuevo (usando \$group) con "\$_id" como clave de grupo (reconstruyéndolos en los documentos originales), siempre se da el caso de que fue un error usar \$unwind + \$group . En su lugar, se deberían haber usado una o más expresiones de procesamiento de matrices en los documentos sin deconstruirlos.

La expresión `$map` le permite manipular cada elemento en un matriz y devuelve una matriz de los resultados, lo que permite transformar la matriz Elementos en su lugar. Otras expresiones de matriz incluyen `$filter`, que , cual devuelve solo los elementos de la matriz que cumplen una condición dada, `$sortBy` para ordenar elementos de la matriz y `$reduce` para agregar la matriz elementos dentro de un documento sin desenrollarlos.

A veces, por supuesto, desenrollar antes de `$group` es la única forma correcta. Opción, especialmente cuando necesita agrupar en una matriz individual elementos en varios documentos. En tales casos, si no Si desea conservar todos los elementos, puede usar `$match` con `$elemMatch` en el elemento de la matriz antes de `$unwind` . Esto le permite usar un índice en el campo de matriz para mejorar el rendimiento.

Diseñar operaciones grupales eficientes

La etapa `$group` puede consumir mucha memoria, especialmente con claves de agrupación de alta cardinalidad o cuando cada documento del grupo va a ser grande. `$group` debe mantener el estado actual para cada grupo único grupo que encuentra. Si el campo por el que está agrupando (el campo `_id`) dentro de `$group`) tiene muchos valores únicos, es posible que sea necesario realizar esta etapa almacenar más datos en la memoria de lo permitido (por defecto, 100 MB es El límite en memoria para cada etapa individual, que analizamos dentro de poco).

Para controlar el tamaño del grupo, mantenga únicamente los campos y valores que sean necesarios en la agrupación y evitar el uso de `$push` de todos los documentos entrantes a menos que sea absolutamente necesario.

Diferentes acumuladores también pueden tener diferente rendimiento.

Características. Acumuladores como `$suma` , `$promedio` , `$mín` , `$máx` , , ,

`$primero` , y `$last` suelen ser eficientes, ya que solo necesitan mantener un único valor actual por grupo. Por el contrario, `$push` y `$addToSet` consumen más memoria al acumular muchos elementos en matrices, especialmente si son grandes.

Si la entrada a `$group` ya está ordenada por la clave de grupo y otro valor, el motor puede optimizar algunos grupos sin almacenar simultáneamente todas las claves de grupo distintas en memoria. Por ejemplo, usar un orden descendente `$sort` seguido de `$group` con `$first` para encontrar el valor más alto por grupo es significativamente más eficiente que usar una etapa `$group` que use `$max` para calcular el mismo valor más alto. Por supuesto, esta optimización depende de tener el índice en los valores `$group + $first` para usar:

```
db.coll.explain("Estadísticas de ejecución").agregado([ {$sort:{estado:1, ID
de grupo:1}}, {$grupo:{_id:"$estado", primerGrupo:
{$primero:"$ID de grupo"
{
  "etapas" : [ {
    "$cursor" :
      { "queryPlanner" :
        { "namespace" : "test.coll",
          "optimizationTimeMillis" : 0, "winningPlan" :
            { "isCached" : falso, "stage" :
              "PROYECCIÓN_CUBIERTA",
              "transformBy" : { "groupId" : 1, "state" : 1, "_id" : 0
            },
            "inputStage" : { "stage" :
              "DISTINCT_SCAN",
```

```

"keyPattern" :
  { "estado" : 1,
    "groupId" : 1
  },
"nombreíndice" : "estado_1_groupId

// ...

```

La salida explicada muestra el plan `DISTICT_SCAN` que se utiliza para encontrar el primer valor de cada grupo.

Evite problemas comunes de rendimiento de \$lookup

La etapa `$lookup` permite realizar uniones entre colecciones, pero puede suponer un cuello de botella en el rendimiento si no se usa con cuidado. Es fundamental comprender dos tipos de `$lookup` :

- Las búsquedas de campos locales/externos (la forma estándar que usa `localField` y `foreignField`) comparan campos entre colecciones usando la semántica de búsqueda habitual . MongoDB las ejecuta de la misma manera que si se tuviera una etapa `$match` como primera etapa de la canalización interna , que usa el valor de `localField` en la búsqueda con el valor de `find` .

La condición `{foreignField:{$eq:<localField-value>}}` es válida si `localField` es un escalar, o `{foreignField:{$in:<localField-array>}}` es válida si `localField` es un array. Esto permite que todos los índices de la colección " from " o " foreign " se utilicen de la misma forma que en los comandos de búsqueda normales con los operadores `$eq` o `$in` .

Uso de la opción de canalización para uniones mejoradas



La opción de canalización se puede utilizar con sintaxis local / externa para agregar más etapas para implementar condiciones de unión adicionales, reducir el tamaño de los documentos devueltos (que van en una matriz as) y realizar cualquier otro procesamiento que requiera la unión.

- Las búsquedas expresivas (que no utilizan `localField` ni `foreignField` y solo utilizan la opción `pipeline`) generalmente especifican las condiciones de unión en una etapa `$match` explícita que debería ser la primera en el pipeline y pueden hacer referencia a campos de los documentos de entrada (asignados a variables en la opción `let`) en el sub-pipeline.

Cuando no existe ninguna condición que dependa de los campos de los documentos de entrada, se genera una subconsulta no correlacionada , que solo se ejecuta una vez y sus resultados se almacenan en caché, en lugar de ejecutarse una vez por cada documento entrante en la canalización. Esta sintaxis solo debe usarse cuando no existe una condición de unión de igualdad.



estándar versus expresiva `$lookup`

La única diferencia entre las búsquedas estándar y expresivas es que una incluye `localField` y `foreignField` , mientras que la otra no. Ambas pueden especificar una canalización para agregar etapas a la ejecución de `$lookup` .

Para los formatos `localField` y `foreignField` , la optimización más crítica es tener un índice en el `foreignField` en la colección externa (`from`). Esto permite a MongoDB encontrar eficientemente los documentos coincidentes. Si los documentos externos son grandes, pero solo necesitará...

Para algunos campos, use ``$project`` como última etapa dentro de la agregación definida en el campo ``canalización`` para conservar solo los campos necesarios. Si espera que solo se devuelva un valor en el campo ``as`` y no desea conservarlo como un array de uno, es totalmente gratuito, desde el punto de vista del rendimiento, ejecutar ``$ unwind`` inmediatamente después de ``$ lookup`` en el campo ``as``, ya que el optimizador absorberá esta etapa en la ejecución de ``$lookup``. Existen optimizaciones adicionales disponibles si planea usar ``$unwind`` en el array ``as``; por ejemplo, una etapa ``$match`` en ``"as.field"`` posterior a la combinación ``$lookup`` y ``$unwind`` también se incluirá en ``$lookup`` como filtro adicional.

```
db.a.explain().aggregate([ { $lookup:{ de:
  "b", campo local:
    "a._id", campo
    extranjero: "_id", como:
    "detalles" }}, {$unwind:"$detalles"},
  {$match:{"detalles.date":
    {$gt:ISODate("2025-05-20T00:00

])
{  "etapas" :
    [ {"$cursor" :
      { "queryPlanner" : {
        // ...
      },
      // ...
    }},
    {"$lookup" :
      { "from" : "b",
        "as" : "detalles",
        "localField" : "a._id",
        "foreignField" : "_id", "let" : { },
```

```

        "tubería" :
        [ {"$coincidencia" :
          { "fecha" : {"$gt":ISODate("2025-05-20T00:00 }}
        ],
        "desenrollando" :
        { "preserveNullAndEmptyArrays" : falso
        }
      }
    }
  }
}

```

La salida de explicación muestra que las etapas `$unwind` y `$match` se absorben en la etapa `$lookup` .

Utilice la opción de canalización expresiva solo con `$lookup` cuando no exista ninguna condición de igualdad en la unión. En este caso, asegúrese de que la primera etapa de la subcanalización sea una etapa `$match` que utilice campos indexados en la colección externa. Es probable que la etapa `$match` utilice la sintaxis `$expr` para incorporar variables de cada documento de entrada, por lo que debe asegurarse externa que `$expr`, de que el campo o los campos referenciados en la colección configuró mediante ``let`` estén en un índice.

Uso eficiente de `$project` y `$addFields`

`$project` y `$addFields` tienen diferentes propósitos y tienen distintas implicaciones de rendimiento. Use `$project` para la inclusión cuando necesite remodelar documentos controlando explícitamente qué campos incluir y cómo nombrarlos, generalmente solo en la última etapa del proceso. Use `$project` para la exclusión cuando desee anular la configuración de varios campos. En este caso, puede usar su alias `$unset` para mayor claridad.

Utilice `$addField` (o su alias `$set`) cuando desee agregar nuevos campos o actualizar/calcular campos existentes mientras mantiene todos los demás campos existentes sin cambios. `$addField` también se puede utilizar para eliminar campos al mismo tiempo, asignando al campo innecesario el valor `$$REMOVE`.

Para cálculos muy complejos, puede resultar más claro dividir el trabajo en varias etapas, cada una realizando una parte del cálculo. Esto también puede mejorar la legibilidad y el mantenimiento. No es estrictamente necesario, ya que usar la expresión `$let` y crear variables temporales en la etapa `$addField` puede facilitar la escritura de expresiones muy complejas. Sin embargo, varias etapas `$set` secuenciales no aumentarán significativamente el tiempo de ejecución de la canalización, y probablemente no lo aumentarán de forma medible.

Tener todas estas técnicas en mente será especialmente útil cuando el tamaño de su conjunto de datos crezca significativamente.

Trabajar con grandes conjuntos de datos

El marco de agregación de MongoDB está diseñado para manejar todo, desde colecciones pequeñas hasta conjuntos de datos que formarían una base de datos.

El sudor de los científicos. Pero a medida que tus datos crecen, también lo hacen los desafíos. Exploremos estrategias para trabajar con grandes conjuntos de datos y mantener tus procesos funcionando sin problemas.

Límites de la canalización de agregación

MongoDB impone varios límites para evitar que sus agregaciones se descontrolen:

- Restricciones de tamaño de los resultados : cada documento en el resultado debe respetar el límite de 16 MB. El límite de tamaño máximo del documento También se aplica a documentos que fluyen entre etapas. La etapa `$facet` solo puede devolver un único documento, que no puede superar los 16 MB.
- Límite de etapas : Los pipelines pueden tener hasta 1000 etapas. Si alcanzas este límite, probablemente sea momento de replantear tu enfoque.
- Límite de memoria : Las etapas de agregación individuales están limitadas por defecto a 100 MB de RAM. Si una etapa supera este límite, la operación de agregación utilizará automáticamente los archivos temporales del disco, ya que `allowDiskUse` está habilitado por defecto desde la versión 6.0 de MongoDB. Esta opción puede desactivarse si es necesario por motivos de rendimiento; sin embargo, las canalizaciones con etapas que superen este umbral fallarán y devolverán un error.

Estos límites están diseñados para proteger su sistema de cargas de trabajo descontroladas, no para restringir un desarrollo minucioso. Al comprenderlos y trabajar dentro de ellos, podrá diseñar canales de agregación eficientes y confiables, incluso a gran escala.

Administrar restricciones de memoria con `allowDiskUse`

Las etapas de bloqueo como `$group` o `$sort` sin indexar pueden consumir mucha memoria, por lo que tienen un límite de memoria de 100 MB por defecto. Al alcanzar este límite, los datos almacenados en memoria se transfieren automáticamente al disco mediante archivos temporales. Este comportamiento se controla mediante la opción `allowDiskUse`, que por defecto tiene el valor `true`. Tiene dos opciones para evitar usar el disco para las ordenaciones:

- Optimice su pipeline para reducir el uso de memoria (el mejor enfoque para el rendimiento)
- Deshabilite explícitamente el uso del disco con `allowDiskUse:false` si prefiere que las operaciones fallen con un error en lugar de ralentizarse al tener que leer y escribir en el disco.

Si bien tener "`allowDiskUse`" habilitado por defecto proporciona una red de seguridad para operaciones con uso intensivo de memoria, es importante tener en cuenta que la E/S de disco es significativamente más lenta que las operaciones en memoria. Se recomienda deshabilitarlo explícitamente configurando "`allowDiskUse:false`" en escenarios donde la baja latencia es crítica y es preferible que las operaciones fallen rápidamente en lugar de ralentizarse debido al uso del disco.

Sólo ciertas etapas de agregación están sujetas a este límite; todas son etapas de bloqueo:

- Operaciones de `$sort` sin indexar en grandes conjuntos de datos : Ordenar una gran cantidad de documentos sin el beneficio de un índice y sin un `$limit` inmediato posterior requiere que todos los documentos que se ordenarán se mantengan en la memoria
 - `$group` con claves de alta cardinalidad y/o documentos de resultado grandes para cada clave : Si los campos utilizados en el `_id` de una etapa `$group` tienen muchos valores únicos y/o si cada agrupación tiene que mantener una gran cantidad de datos, la etapa debe mantener una gran cantidad de datos para cada grupo distinto en memoria
 - `$bucket`, `$bucketAuto` y `$setWindowFields` : estas etapas son variantes de `$group` y pueden consumir una cantidad significativa de memoria en las mismas circunstancias que `$group`
-



Consideraciones sobre el bloqueo de etapas y el uso de la memoria

No todas las etapas de bloqueo pueden o probablemente superarán este límite. Por ejemplo, `$count` es una etapa de bloqueo que necesita contar todos los documentos entrantes, pero como solo genera un documento como resultado, su uso de memoria es insignificante. La etapa `$sortByCount` es una etapa de bloqueo que agrupa toda su entrada en pequeños documentos que contienen solo dos campos, pero luego debe ordenarlos en memoria. Si no se especifica una etapa `$limit` después de `$sortByCount` y se tienen más de un millón de valores de clave de grupo distintos, el resultado podría superar los 100 MB, pero ese escenario es poco probable. Probablemente se usaría `$limit` en lugar de devolver millones de resultados al cliente.

Agregación en entornos distribuidos

La fragmentación, que abordamos en [Capítulo 6](#) [Información sobre fragmentación], es la clave del escalamiento horizontal de MongoDB, pero añade complejidad a las canalizaciones de agregación. Cuando los datos se distribuyen en múltiples fragmentos, comprender cómo funciona la agregación en este entorno distribuido es clave para mantener el rendimiento y evitar cuellos de botella.

Optimización de la agregación para colecciones fragmentadas

En un clúster fragmentado, las canalizaciones de agregación pueden ejecutarse en paralelo entre los fragmentos. MongoDB intenta transferir la mayor cantidad de trabajo posible a cada fragmento que contiene los datos necesarios para la agregación, minimizando así la cantidad de datos que deben transferirse a través de la red.

Si una consulta de agregación puede ser satisfecha con datos que residen en un solo fragmento (o un subconjunto de ellos), MongoDB dirigirá la consulta únicamente al fragmento o fragmentos relevantes. Este es el escenario más eficiente, ya que minimiza la comunicación entre fragmentos y la sobrecarga de coordinación.

Esto suele ocurrir cuando la consulta incluye un filtro en la clave del fragmento o la colección que se agrega solo existe en uno o en un pequeño subconjunto de fragmentos.

Si no se puede lograr la agregación objetivo (por ejemplo, si no hay filtro en la clave del fragmento y los datos residen en varios fragmentos), MongoDB realiza una operación de dispersión-recopilación. Envía la primera parte de la canalización a todos los fragmentos que contienen datos de la colección objetivo. A continuación, MongoDB recopila los resultados y los fusiona ejecutando el resto de la canalización. También existen escenarios en los que la etapa de fusión debe realizarse en un fragmento seleccionado en lugar de en MongoDB.

La elección de la clave de fragmento puede determinar el rendimiento de la agregación. Una clave de fragmento bien elegida permite dirigir muchas consultas de agregación a un solo fragmento, lo que mejora drásticamente el rendimiento. Por el contrario, una clave de fragmento mal elegida puede provocar que la mayoría de las agregaciones se conviertan en operaciones de dispersión-recopilación, lo que aumenta la latencia y la carga del clúster.

Comprensión de las operaciones locales de fragmentos frente a las fusionadas

No todas las etapas de la canalización son iguales. Algunas pueden ejecutarse en fragmentos individuales, mientras que otras deben fusionarse en mongos . Use el comando `explain()` para determinar dónde se ejecuta cada etapa. El resultado mostrará cómo se divide la canalización y qué partes se ejecutan en los fragmentos y cuáles en mongos .

Todas las etapas de streaming pueden ejecutarse en fragmentos, así como las primeras partes de las etapas de bloqueo. En algunos casos especiales, incluso `$group` puede ejecutarse completamente en un fragmento si se agrupa por la clave del fragmento. La parte final de la primera etapa de bloqueo, que no puede ejecutarse completamente en el fragmento, y todas las etapas posteriores, deberán ejecutarse en la ubicación de fusión, que suele ser mongos .

Mantenga el máximo procesamiento posible en los fragmentos para reducir el tráfico de red y acelerar su flujo de trabajo. Si una agregación puede ejecutarse completamente en un solo fragmento, se evita la latencia de red y la sobrecarga de fusión en mongos .

Si tiene una etapa `$group` y la clave de grupo es (o incluye) la clave del fragmento, cada fragmento puede realizar la operación de grupo completa localmente, con la canalización continuando localmente, y Mongos solo necesita devolver los resultados al cliente sin procesamiento adicional. Si la clave de grupo es diferente, o si hay otra etapa de bloqueo, los fragmentos realizan la parte inicial de esa etapa y Mongos realiza una fusión final y todas las etapas posteriores, lo que supone más trabajo para Mongos .

Las fusiones suelen ser inevitables, pero se puede minimizar su impacto. Las primeras etapas de la canalización (`$match`) deberían

Filtrar documentos de forma agresiva. Cuantos menos datos envíe cada fragmento a Mongos para su fusión, más rápida será la operación.

Monitoreo y perfilado del rendimiento de agregación. Incluso la canalización de agregación más

minuciosamente diseñada puede presentar problemas de rendimiento. Por eso, el monitoreo y el perfilado son habilidades esenciales para cualquier profesional de MongoDB.

Vio brevemente cómo el método `explain()` le permite acceder al funcionamiento interno de una canalización de agregación. Revela el plan de ejecución, mostrando cómo se procesa cada etapa, qué índices se utilizan y dónde se realiza el trabajo pesado. Mostraremos mucho más sobre los conceptos

[Capítulo 12](#), avanzados de consultas e indexación .

Para obtener la salida de explicación de una agregación, añada `.explain()` al nombre de la colección antes del comando `added()` , use la opción `{explain:true}` dentro de la función `added` o, en Mongosh , añada `.explain()` al final del método `added`. Los modos de verbosidad comunes incluyen los siguientes:

- **`explain("queryPlanner")`** muestra el plan seleccionado por el optimizador de consultas, incluidas las etapas como se ejecutarán y cómo se utilizan los índices
- **`explain("executionStats")`** ejecuta la consulta y devuelve estadísticas detalladas sobre la ejecución, como la cantidad de documentos devueltos (`nReturned`), el total de documentos examinados (`totalDocsExamined`), el tiempo total de ejecución después de cada etapa (`execTimeMillis`) y el uso del índice

- `explain("allPlansExecution")` que `executionStats` pero incluye información adicional y estadísticas sobre los planes de consulta rechazados en la etapa `$cursor`

El objetivo principal de usar `explain()` es identificar oportunidades de mejora. Busque `COLLSCAN` (análisis de colección) como `inputStage` para operaciones `$match` o `$sort` donde esperaba un `IXSCAN` (análisis de índice). `COLLSCAN` implica que MongoDB tuvo que leer todos los documentos de la colección para encontrar los relevantes, lo cual suele ser ineficiente.

Etapas como `$group` o `$sort` que muestran un aumento considerable en `execTimeMillis` con respecto a la etapa anterior y procesan muchos documentos podrían consumir mucha memoria. Valores altos de `totalDocsExamined` en comparación con `nReturned` para una etapa `$match` sugieren que el filtro no es muy selectivo o no utiliza un índice eficazmente.

Puede ver si la etapa `$lookup` usó índices y, de ser así, su eficiencia. También puede asegurarse de que los índices utilizados sean los correctos.

Esperaba ser seleccionado, y si no lo es, puede ver si tiene índices redundantes que puedan eliminarse. También puede ver si una etapa de bloqueo se desbordó en disco y cuánto espacio de disco utilizó. Si la operación se realizó completamente en memoria, puede ver cuánta memoria se utilizó y si se está acercando al límite de 100 MB, momento en el que la velocidad se ralentizará considerablemente.

La experiencia y la elaboración de perfiles a menudo revelan patrones comunes que conducen a problemas de rendimiento.

Utilizando vistas materializadas

A veces, una agregación compleja será más lenta que el SLA requerido para devolver sus resultados, en cuyo caso puede ejecutar la agregación periódicamente para almacenar en caché sus resultados en otra colección, que los clientes pueden consultar directamente.

Utilice `$out` o `$merge` para almacenar resultados intermedios mientras estas etapas escriben los resultados del proceso de agregación en una colección específica, que luego puede consultarse, indexarse o usarse como entrada para agregaciones posteriores.

Resumen

Resumamos lo aprendido en este capítulo en principios clave que te ayudarán a crear pipelines de agregación potentes y de alto rendimiento. Primero, filtra con anticipación. La optimización más efectiva suele ser la más sencilla. Cada documento que puedas excluir con anticipación implica menos trabajo en cada etapa posterior. No se trata solo de añadir una etapa `$match` al principio, aunque esa es la optimización más efectiva. También se trata de añadir `$limit` cuando sabes que no usarás todos los documentos para minimizar el flujo de datos, no crear campos innecesarios y revisar tu pipeline con `explain` para comprobar que solo se transfieren los datos necesarios de una etapa a otra.

A continuación, aproveche los índices estratégicamente. Los índices no solo se utilizan para las operaciones `find()`; son vitales para el rendimiento de la agregación.

Por ejemplo, las etapas iniciales de `$match` pueden usar índices al igual que las consultas regulares; las primeras operaciones de `$sort` usarán índices, cuando sea posible; `$lookup` usará índices en las colecciones `from`; y algunas operaciones

de `$group` pueden aprovechar los índices. La clave es asegurar que la estructura de su canalización lo permita.

MongoDB usará esos índices. Por ejemplo, si necesita filtrar y ordenar, asegúrese de tener un índice compuesto que admita ambas operaciones.

Además, diseñe teniendo en cuenta la fragmentación. En un entorno fragmentado, comprender cómo se distribuyen las etapas de agregación entre los fragmentos puede determinar el rendimiento. Para obtener mejores resultados, filtre por la clave del fragmento al principio para identificar solo fragmentos específicos, minimice el número de fragmentos involucrados y tenga en cuenta qué etapas fuerzan una fusión (es decir, detienen la ejecución paralela). Por último, monitoree y mida el rendimiento. Continúe experimentando, midiendo y perfeccionando su enfoque. El pipeline más elegante no es necesariamente el que tiene menos etapas, sino el que equilibra la legibilidad, la facilidad de mantenimiento y el rendimiento para brindar información de manera precisa y eficiente.

Una vez optimizado el procesamiento de sus datos, es hora de centrar nuestra atención en las vías que transportan sus datos.

El siguiente capítulo explora la replicación, la preferencia de lectura y la preocupación por la escritura, que son esenciales para mantener la disponibilidad, la consistencia y el rendimiento de los datos en sistemas distribuidos.

5

Replicación

En este capítulo, aprenderá cómo MongoDB utiliza la replicación para el funcionamiento continuo y la conmutación por error automática. Los conjuntos de réplicas organizan la sincronización de datos entre múltiples nodos, lo que garantiza que su aplicación permanezca en línea cuando surjan problemas de hardware o red.

Verá por qué la combinación correcta de nodos es crucial para la confiabilidad, y cómo características como la preferencia de lectura y la preocupación por la escritura le permiten ajustar su sistema para lograr rendimiento, consistencia y durabilidad.

Aprenderá sobre procesos como elecciones y conmutación por error, y se adentrará en configuraciones avanzadas de conjuntos de réplicas que aíslan la carga de trabajo, reducen la sobrecarga de la red y evitan que los análisis interfieran con las consultas operativas. Al finalizar este capítulo, contará con las herramientas para diseñar, configurar y supervisar conjuntos de réplicas para implementaciones robustas de MongoDB.

Este capítulo cubrirá los siguientes temas:

- Cómo los conjuntos de réplicas garantizan alta disponibilidad y conmutación por error automática
- Los roles de los nodos primarios, secundarios y árbitros
- Estrategias para equilibrar el rendimiento, la consistencia y la durabilidad con configuraciones de lectura/escritura
- Supervisión del estado de la replicación, diagnóstico de retrasos y gestión de la conmutación por error

- Funciones avanzadas como replicación encadenada, nodos de análisis y dimensionamiento óptimo del registro de operaciones

Comprensión de los conjuntos de réplicas de MongoDB

La replicación de MongoDB garantiza la disponibilidad de los datos y la resiliencia del sistema al mantener copias de los datos en múltiples servidores, organizadas en un conjunto de réplicas.

Dentro de este conjunto, un único servidor principal gestiona las operaciones de escritura, mientras que los servidores secundarios mantienen copias de los datos, lo que permite solicitudes de lectura y proporciona capacidades de conmutación por error. La preferencia de lectura controla qué servidores se utilizan para las operaciones de lectura, lo que proporciona un mayor control sobre los nodos a los que se envían las operaciones de lectura.

La preocupación por la escritura determina el nivel de confirmación de la base de datos.

proporciona después de una operación de escritura, lo que le permite ajustar el equilibrio entre la velocidad de escritura y la durabilidad de los datos.

La combinación de estos elementos crea un sistema capaz de tolerar fallos del servidor y gestionar grandes cargas de lectura. En resumen, la replicación realiza copias de datos, la preferencia de lectura determina desde dónde leer y la preocupación por la escritura determina el nivel de confirmación necesario tras escribir los datos, lo que resulta en un sistema de base de datos robusto.

Componentes de un conjunto de réplicas

Un conjunto de réplicas típico de MongoDB incluye una cantidad impar de nodos para proporcionar tolerancia a fallas y alta disponibilidad, y consta de un nodo principal y uno o más nodos secundarios:

- **Primario** : El nodo principal se encarga de gestionar todas las operaciones de escritura. Mantiene el estado más reciente de los datos y propaga los cambios a los nodos secundarios.
- **Secundarios** : Los nodos secundarios replican continuamente los datos del principal para mantener copias actualizadas. También pueden realizar operaciones de lectura si la aplicación está configurada para permitir lecturas secundarias.
- **Árbitros (opcional)** : Un nodo árbitro participa en las elecciones, pero no almacena datos. No se recomienda su uso en configuraciones de producción; sin embargo, si las limitaciones del sistema impiden almacenar datos, se pueden usar para romper empates en escenarios de votación cuando se tiene un número par de nodos que contienen datos.

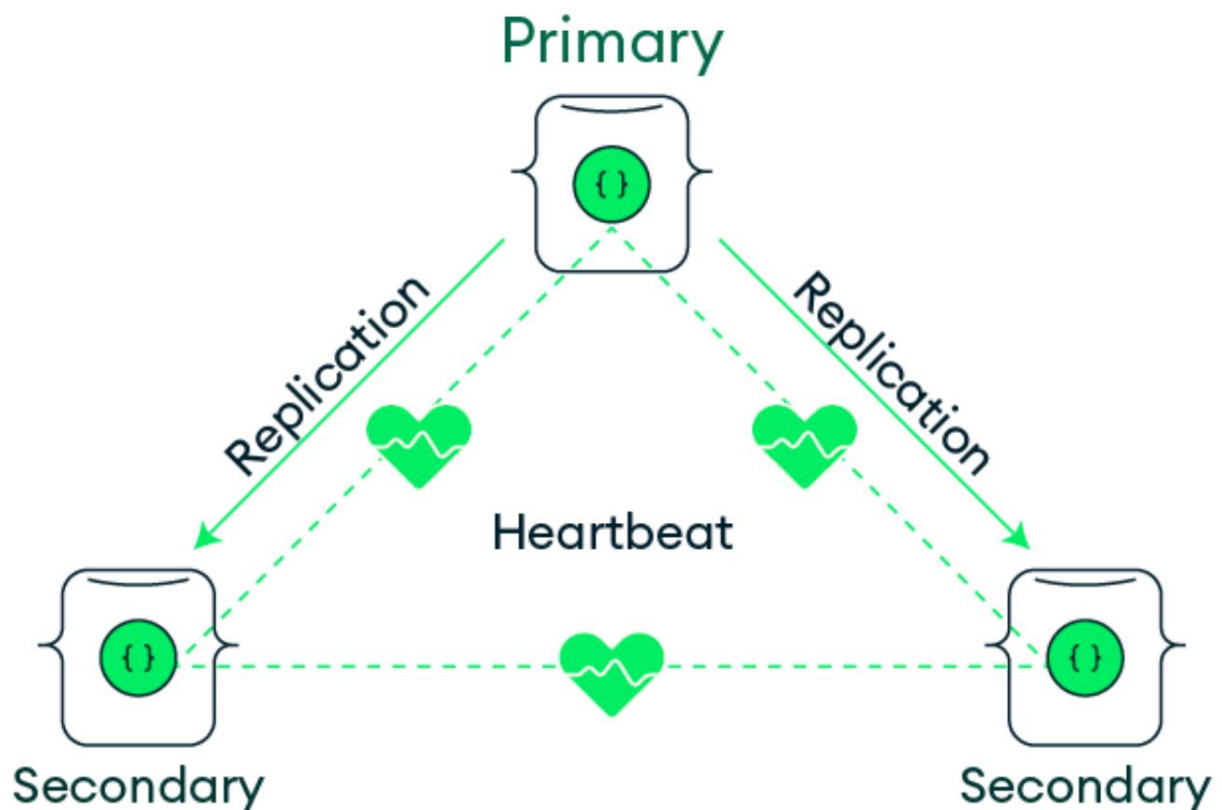


Figura 5.1: Arquitectura del conjunto de réplicas de MongoDB

Los conjuntos de réplicas utilizan un proceso de elección para garantizar una alta disponibilidad.

Si el nodo principal deja de estar disponible, se activa una elección para seleccionar un nuevo nodo principal. El proceso de elección sigue estos pasos:

1. Un nodo convoca una elección si cree que puede convertirse en el primario (por ejemplo, si un secundario no puede ver a un primario).
2. Los nodos emiten votos en función de factores como la configuración de prioridades, estado de replicación y disponibilidad.
3. El nodo que recibe la mayoría de votos es elegido como el nuevo primario.

Al diseñar un conjunto de réplicas de MongoDB para un rendimiento óptimo, debe considerar cuidadosamente la cantidad de nodos, su ubicación y cómo configurar las preferencias de elección y lectura. Una configuración de conjunto de réplicas bien diseñada puede ayudarle a equilibrar los requisitos de rendimiento, tolerancia a fallos y latencia, a la vez que garantiza una alta disponibilidad para su aplicación.

Tener un número impar de miembros con derecho a voto en un conjunto de réplicas de MongoDB es esencial para garantizar que el proceso electoral se complete correctamente. Cuando solo hay un número par de miembros con derecho a voto, es más probable que no se alcance el quórum y que la elección de una nueva primaria se retrase.

En algunas implementaciones, se pueden agregar nodos secundarios adicionales para distribuir las cargas de trabajo de lectura o facilitar el análisis sin afectar el rendimiento del nodo principal. Esta configuración garantiza la alta disponibilidad de los datos y, al mismo tiempo, permite el aislamiento de las cargas de trabajo mediante la segmentación de lectura.



Los secundarios son para redundancia, no para escalabilidad



Si bien la preferencia de lectura le permite seleccionar recursos secundarios para las lecturas, esto no está pensado como un mecanismo para escalar lecturas en todo el clúster. Los secundarios proporcionan al clúster alta disponibilidad y resiliencia.

Replicación y alta disponibilidad

Siempre que un clúster tenga un mínimo de tres nodos (el valor predeterminado en MongoDB Atlas), será tolerante a fallas y resiliente desde el primer momento.

En el caso de una interrupción en el funcionamiento de un miembro principal, un proceso de elección de conjunto de réplicas detecta automáticamente la falla, elige un miembro secundario como reemplazo y promueve a ese miembro secundario para que se convierta en el nuevo miembro principal.

Todo el proceso de elección y conmutación por error ocurre en segundos, sin intervención manual.

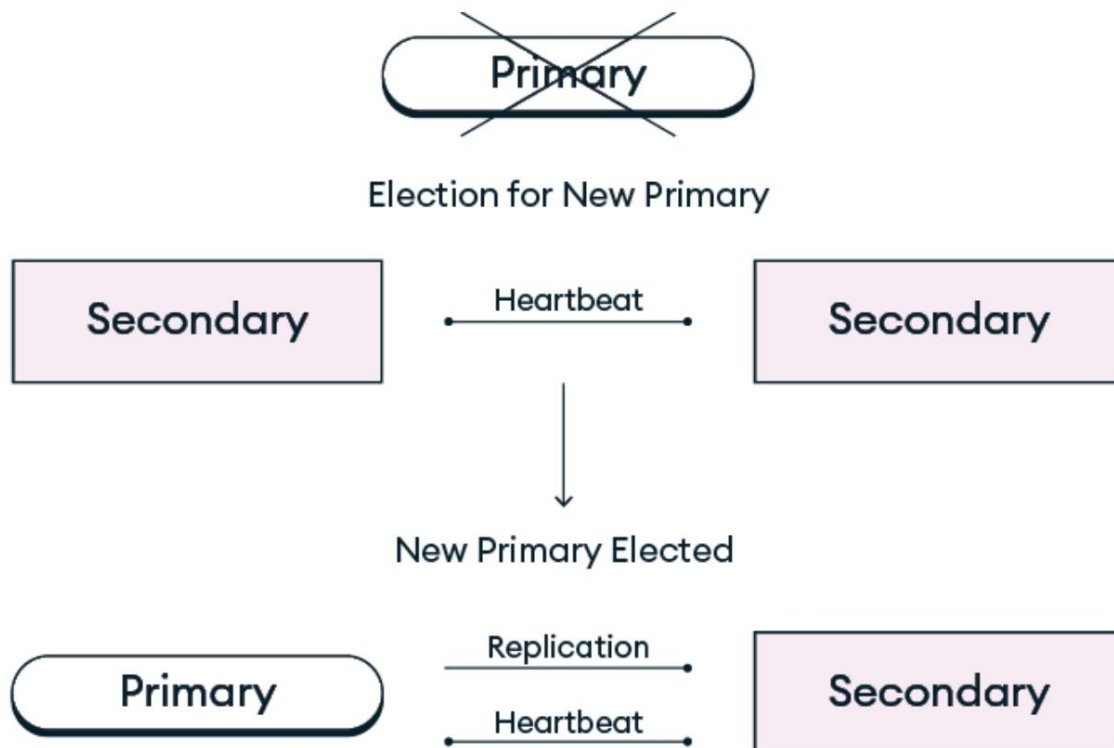


Figura 5.2: Proceso de conmutación por error del conjunto de réplicas de MongoDB

El tiempo de conmutación por error depende de varios factores, incluida la latencia de la red. El proceso de elección de MongoDB está diseñado para evitar conmutaciones por error innecesarias y ofrecer flexibilidad para diferentes necesidades de implementación.

Para minimizar el tiempo de inactividad durante las conmutaciones por error, la configuración adecuada de las elecciones, incluidas las prioridades y los tiempos de espera apropiados, es crucial para garantizar una transición rápida y sin problemas del liderazgo entre los miembros del conjunto de réplicas:

- **Prioridad de nodo :** asigne valores de prioridad más altos a los nodos para influir en los resultados de las elecciones.
- **Tiempos de espera de elecciones :** configure `electionTimeoutMillis` para lograr un equilibrio entre una conmutación por error rápida y evitar tiempos de espera innecesarios.

Elecciones debido a problemas temporales de red. El valor predeterminado de 10 000 (10 segundos) debería ser adecuado para la mayoría de las configuraciones.

Los miembros del conjunto de réplicas intercambian mensajes de latido cada 2 segundos para supervisar el estado de cada uno. Si la conexión falla debido a un problema de red...

Error: según el miembro monitoreado, un miembro principal que no reciba un latido de la mayoría de los miembros con derecho a voto se retirará, mientras que un miembro secundario que no reciba un latido del miembro principal puede convocar elecciones. Si no hay un fallo de red y no se recibe una respuesta dentro del umbral de `electionTimeoutMillis`, se tomará la misma medida.

Comprensión del proceso de elección de MongoDB. Como se mencionó anteriormente, un conjunto de réplicas de MongoDB utiliza un proceso de elección totalmente automatizado para determinar qué miembro del conjunto se convertirá en el nuevo miembro principal. Los conjuntos de réplicas pueden activar una elección en respuesta a diversos eventos, como los siguientes:

- Pérdida de conectividad con un miembro principal durante más del tiempo de espera configurado (para MongoDB Atlas, el valor predeterminado es 5 segundos, o 10 segundos en caso contrario)
- Agregar un nuevo nodo al conjunto de réplicas
- Realizar mantenimiento del conjunto de réplicas

Durante cualquiera de estos eventos, algoritmos sofisticados controlan el proceso de elección de miembros. El algoritmo de elección procesa diversos parámetros, incluyendo un análisis histórico para identificar a los miembros del conjunto de réplicas que han aplicado las actualizaciones más recientes.

El estado del servidor principal, el estado de latido (ping o conectividad) y las prioridades definidas por el usuario asignadas a los miembros del conjunto de réplicas. Este proceso garantiza que el miembro secundario más adecuado se convierta en el principal. Una vez elegido el nuevo miembro principal del conjunto de réplicas, los miembros secundarios restantes comienzan a replicar automáticamente desde la fuente de sincronización configurada, como se explica en la próxima sección "Replicación encadenada". Si el servidor principal original vuelve a estar en línea, reconocerá que ya no es el principal y se reconfigurará para convertirse en un miembro secundario del conjunto de réplicas y comenzar a recuperarse.

Configuración del conjunto de réplicas

En esta sección, profundizamos en configuraciones especializadas de conjuntos de réplicas que van más allá de las configuraciones básicas de primario-secundario. Estas técnicas avanzadas pueden ayudarle a optimizar el rendimiento, reducir la sobrecarga operativa o satisfacer requisitos específicos de carga de trabajo. Si bien cada función introduce complejidades adicionales, pueden resultar invaluable para los equipos que buscan optimizar sus implementaciones de MongoDB.



Nota

La reconfiguración del conjunto de réplicas no está disponible en MongoDB Atlas.

Replicación encadenada . De forma predeterminada, los servidores secundarios pueden extraer cambios de datos (entradas del registro de operaciones) del servidor principal o de otro servidor secundario. La replicación encadenada permite que un servidor secundario obtenga datos del registro de operaciones de otro servidor secundario. Este enfoque...

Puede ayudar a reducir la carga de red en la red principal en implementaciones grandes o distribuidas. La replicación encadenada funciona de la siguiente manera:

1. MongoDB determina el secundario más cercano o mejor para servir como la fuente de sincronización.
2. El secundario de sincronización recupera continuamente las entradas del registro de operaciones de esta fuente secundaria en lugar de la primaria.
3. Los cambios se propagan a través del conjunto de réplicas con reducción
Uso del ancho de banda en el nodo principal.

Si bien la replicación encadenada tiene muchas ventajas, también puede tener algunas desventajas:

Beneficios	Desventajas
Facilita implementaciones distribuidas globalmente : los nodos secundarios en regiones remotas pueden extraer de una fuente geográficamente más cercana, lo que reduce la latencia.	Mayor retraso en las cadenas : si un nodo secundario aguas abajo se queda atrás, los nodos que dependen de él en la cadena también se quedarán atrás.
Optimiza el tráfico de la red : Particularmente beneficioso en implementaciones multirregionales donde el ancho de banda de la red entre regiones puede ser limitado o costoso.	Complejidad de la resolución de problemas : las rutas de replicación más largas pueden complicar el diagnóstico de problemas de replicación
	Retrasos en cascada : Los problemas de red que afectan a los nodos intermedios pueden

	impacto múltiple secundarias aguas abajo
--	--

Tabla 5.1: Ventajas y desventajas de la replicación encadenada en conjuntos de réplicas de MongoDB

Puede permitir (o no) la replicación encadenada ajustando la Configuración del conjunto de réplicas desde una conexión MongoDB Shell a su grupo:

```
cfg = rs.conf();
cfg.configuraciones = cfg.configuraciones || {};
// true permite que los secundarios se repliquen desde otros secundarios
// falso los restringe a replicar solo desde el primario
cfg.settings.chainingAllowed = verdadero;
rs.reconfig(cfg);
```



Nota

La replicación encadenada está habilitada de forma predeterminada; sin embargo, es deshabilitado para conjuntos de réplicas de una sola región en MongoDB Atlas.

Al utilizar la replicación encadenada, la monitorización se vuelve aún más Crítico. El siguiente script, que también se puede ejecutar desde MongoDB, La conexión de Shell a su clúster imprimirá la fuente y el estado de sincronización Información sobre cada miembro del conjunto de réplicas:

```
// Comprueba el sincronización fuente para cada miembro
rs.status().members.forEach(función(miembro) {
  imprimir("Miembro: " + nombre.de.miembro +
    ", Fuente de sincronización: " + (member.syncSourceHost || "non
```

```

    }, Estado: " + miembro.stateStr);
});

```

Asegúrese de monitorear el retraso de replicación y la latencia de escritura mayoritaria más de cerca si el encadenamiento está habilitado, especialmente en entornos de alta latencia.

Etiquetas de conjunto de réplicas y nodos de análisis. MongoDB Atlas ofrece nodos de análisis, que son nodos especializados de solo lectura que se utilizan para aislar consultas que no se desea que afecten a la carga de trabajo operativa. Son útiles para gestionar datos analíticos, como consultas de informes ejecutadas por herramientas de inteligencia empresarial (BI). En el contexto de un conjunto de réplicas, los nodos de análisis son similares a otros miembros secundarios, pero tienen una prioridad de 0 y no pueden elegirse como principales. Sin embargo, mediante la configuración de etiquetas de conjunto de réplicas, pueden excluirse de las cargas de trabajo operativas habituales y asignarse específicamente a otras mediante una preferencia de lectura adecuada.

Por ejemplo, para configurar un miembro para análisis, se podría aplicar la siguiente reconfiguración del conjunto de réplicas:

```

conf = rs.conf();
conf.members[0].tags = { "uso": "producción" };
conf.members[1].tags = { "uso": "informes" };
conf.members[2].tags = { "uso": "producción" }; rs.reconfig(conf);

```

El nodo de análisis, que hemos identificado como que tiene una etiqueta de uso: se uso de informe puede orientar a cargas de trabajo específicas mediante el

configuración de preferencia de lectura adecuada, como en el siguiente ejemplo:

```
db.collection.find({}).readPref("secundario", [{ "uso": "
```

Las etiquetas de conjunto de réplicas no se pueden configurar directamente en MongoDB Atlas; sin embargo, Atlas ofrece un conjunto de etiquetas de conjunto de réplicas predefinidas que se pueden usar para el aislamiento de la carga de trabajo, algunas de las cuales son las siguientes:

Predefinido	Descripción	Ejemplo
nombre de la etiqueta		
nodeType	Tipo de nodo. Valores posibles son los siguientes: - ELEGIBLE - SOLO LECTURA - ANALÍTICA	<pre>{"nodeType": "ANALYTICS"}</pre>
proveedor	Proveedor de la nube en el que se encuentra el nodo es provisionado. Valores posibles son los siguientes: - AWS - GCP - AZUR	<pre>{"proveedor": "AWS"}</pre>

region	Región nubosa en cual el nodo reside.	<code>{"región": "EE. UU._ESTE_2"}</code>
---------------	---------------------------------------	---

Tabla 5.2: Nombres de etiquetas predefinidos comunes para los miembros del conjunto de réplicas de MongoDB

Para obtener una lista completa de las etiquetas de conjunto de réplicas predefinidas y las posibles regiones valores disponibles en MongoDB Atlas, consulte

<https://www.mongodb.com/docs/atlas/reference/replica-un-conjunto-de-etiquetas>.

Si una aplicación realiza operaciones complejas o de larga duración, como como extracto , transformar , y carga (ETL) o informes, es posible que desee

Para aislar las consultas de la aplicación del resto de su funcionamiento.

carga de trabajo al dirigir exclusivamente esas consultas a los nodos de análisis.

Esto se puede configurar en función de las etiquetas predefinidas anteriores para un aplicación que se conecta a un clúster MongoDB Atlas, de la siguiente manera:

```
mongodb+srv://<nombre_de_usuario_de_la_base_de_datos>:<contraseña_de_la_base_de_datos>@abc123.mongodb.n
```

Configurar su aplicación con la cadena de conexión anterior

garantizará que todas las lecturas se dirijan únicamente a miembros secundarios que tengan la

Etiqueta nodeType:ANALYTICS aplicada.



Nota

Los nodos de análisis en MongoDB Atlas están configurados con una prioridad , lo que les impide ser de 0 primaria elegida.

Internos y rendimiento de la replicación

La replicación en MongoDB implica mecanismos para mantener los datos sincronizados entre los nodos mientras se gestionan los retrasos y el uso de recursos. Factores como el registro de operaciones, el control de las tasas de escritura y los retrasos de direccionamiento entre nodos influyen en el rendimiento general. Equilibrar estos elementos es fundamental para mantener la consistencia y la disponibilidad de los datos. En esta sección, analizaremos los aspectos clave de la replicación interna y las consideraciones de rendimiento en MongoDB.

Control de flujo

El control de flujo es una característica de MongoDB que limita la velocidad a la que el servidor principal aplica nuevas operaciones cuando los servidores secundarios se quedan demasiado atrás. Esto ayuda a evitar un retraso excesivo en la replicación, a costa de una latencia ligeramente mayor en lo que se refiere al reconocimiento de escrituras.

MongoDB proporciona parámetros internos para ajustar el comportamiento del control de flujo. Por ejemplo, el parámetro `flowControlTargetLagSeconds` define el umbral de retardo aceptable antes de que se active el control de flujo, como se muestra aquí:

```
// Verifique la configuración de control de flujo actual
db.adminCommand({ obtenerParámetro: 1, flowControlTargetLagSec

Configurar los ajustes de control de flujo //

db.adminCommand({ setParameter: 1,
flowControlTargetLagSeconds: 10 });
```


La configuración predeterminada de 10 segundos debería ser óptima para la mayoría de los casos de uso y no se recomienda ajustarla.



Nota

El comando administrativo `setParameter` es no disponible en MongoDB Atl como.

Replicación y el registro de operaciones. El registro de operaciones (oplog) es un componente central de la replicación de MongoDB. Se trata de una colección especial con límite donde los nodos registran todas las operaciones de escritura. Los nodos secundarios utilizan el registro de operaciones para replicar los cambios y mantenerse sincronizados con el principal.

A continuación se muestran algunas características clave del oplog:

- Recopilación limitada : El registro de operaciones tiene un tamaño fijo por defecto, que funciona según el principio de "primero en entrar, primero en salir" (FIFO). Las entradas antiguas se eliminan automáticamente cuando el registro de operaciones alcanza su límite de tamaño.
- Proceso de replicación : los nodos secundarios aplican continuamente entradas de registro de operaciones del nodo principal para replicar los cambios.
- Consideraciones de rendimiento : El tamaño adecuado del registro de operaciones es crucial. Si el registro de operaciones es demasiado pequeño, los nodos secundarios podrían retrasarse y requerir una resincronización completa. Por el contrario, un registro de operaciones demasiado grande puede consumir almacenamiento innecesario.

Supongamos que su carga de trabajo genera 2 GB de actualizaciones por hora. Para garantizar una replicación fluida, el registro de operaciones debe tener un tamaño que permita almacenar varias horas de operaciones, normalmente entre 24 y 48 horas. Esto

Reduce el riesgo de resincronizaciones prolongadas si un nodo secundario se desconecta temporalmente.

Por ejemplo, si configura un tamaño de registro de operaciones de 10 GB, esto proporcionará Aproximadamente 5 horas de historial de funcionamiento. Este búfer permite nodos secundarios para ponerse al día después de breves interrupciones de la red o períodos de mantenimiento sin necesidad de una resincronización completa de datos, pero puede ser demasiado corto si el nodo falla de una manera que toma más de 5 Horas para volver a ponerlo en línea.

Un registro de operaciones más grande proporciona una ventana más larga para que los secundarios se pongan al día Si se quedan atrás debido a problemas de red, limitaciones de recursos o Mantenimiento. Esto es particularmente importante en aplicaciones de alto rendimiento. entornos o durante el tiempo de inactividad planificado para los nodos secundarios.

Para determinar el tamaño de registro de operaciones adecuado para su implementación, Considere lo siguiente:

- El volumen de escritura promedio y máximo de su aplicación
- La duración máxima esperada de posibles interrupciones
- Confiabilidad de la red entre centros de datos (para ubicaciones geográficas implementaciones distribuidas)
- El número de nodos secundarios (más secundarios pueden requerir un ventana más grande)

MongoDB proporciona herramientas para monitorear el uso del registro de operaciones y estimar su ventana actual:

```
// Comprobar el tamaño del registro de operaciones y la ventana de replicación
db.printReplicationInfo();
// Ejemplo de salida:
// tamaño del registro de operaciones configurado: 5120 MB
// a Longitud del registro inicio/fin: 86400 segundos (24 horas)
```

```
// Hora del primer evento del oplog: Mié 02 2025 10:03:25 GMT+00
// último registro de operaciones del evento de abril: // Jue 03 de abril de 2025 10:03:25 GMT+000
// ahora: Jue 03 de abril de 2025 10:05:01 GMT+0000
```

Los límites del registro de operaciones se pueden establecer de dos maneras: tamaño y tiempo, como Sigue:

- **tamaño** : el tamaño máximo del oplog en megabytes, que puede se puede configurar con cualquier valor entre 990 MB y 1 petabyte
- **minRetentionHours** : El número mínimo de horas a conservar una entrada de oplog

Como el registro de operaciones es una colección limitada, cuando se configuran

Una vez cumplidos los límites, se eliminarán las entradas más antiguas del registro de operaciones.

Cambiar el tamaño configurado del oplog de su valor actual a 16

GB podría hacerse de la siguiente manera:

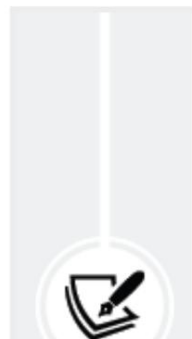
```
db.adminCommand({ "replSetResizeOplog": 1, tamaño: Doble(16
```

De manera similar, si en lugar de un tamaño fijo quisieras asegurar exactamente 48

Siempre había horas de contenido de oplog disponibles, el comando sería

la siguiente:

```
db.adminCommand({ "replSetResizeOplog": 1, minRetentionHou
```



Riesgo de agotamiento del espacio en disco con

minRetentionHours

Cuando se configura **minRetentionHours** , el registro de operaciones puede crecer sin restricciones para conservar las entradas del registro de operaciones



Durante el número de horas configurado. Esto puede resultar en una reducción o agotamiento del espacio en disco del sistema debido a la combinación de un alto volumen de escritura y un largo período de retención.

Cambiar el tamaño del registro de operaciones o el periodo mínimo de retención de un miembro del conjunto de réplicas con `replSetResizeOplog` no modifica el tamaño del registro de operaciones de ningún otro miembro del conjunto de réplicas. Debe ejecutar `replSetResizeOplog` en cada miembro del conjunto de réplicas del clúster para cambiar el tamaño del registro de operaciones o el periodo mínimo de retención de todos los miembros.

Gestión del retardo de replicación. El retardo de replicación se produce cuando los nodos secundarios no pueden mantener el ritmo de las operaciones que procesa el nodo principal. Esto puede provocar lecturas obsoletas y tiempos de conmutación por error más largos si el nodo con retraso necesita convertirse en el principal. Para minimizar el retardo de replicación, considere implementar las siguientes estrategias:

Estrategia	Acción	Consideraciones
Optimizar la red	Coloque los secundarios más cerca a los enlaces de red principal o de actualización.	Considere dedicado enlaces de red entre miembros del conjunto de réplicas. Implementar la priorización del ancho de banda para el tráfico de replicación.

Escala recursos	Aumente la capacidad de E/S de CPU, memoria o disco.	Actualizar el hardware en los nodos existentes.
Mejorar la lectura carga de trabajo segmentación	Descarga intensiva consultas a secundarias dedicadas y adaptadas para análisis, manteniendo el resto del conjunto de réplicas equilibrado.	Configure las preferencias de lectura para dirigir cargas de trabajo específicas a los nodos apropiados.

Tabla 5.3: Estrategias para reducir el retraso de replicación en conjuntos de réplicas de MongoDB

Al abordar de forma proactiva el retraso de replicación a través de estas estrategias, puede garantizar conmutaciones por error más rápidas, un rendimiento de lectura más consistente y una mayor estabilidad general dentro de su conjunto de réplicas de MongoDB.

El monitoreo y ajuste periódicos basados en patrones de carga de trabajo ayudarán a mantener una salud de replicación óptima a medida que su implementación escala.

Estrategias de lectura y escritura

MongoDB ofrece estrategias flexibles para optimizar las lecturas y escrituras en conjuntos de réplicas. La configuración de preferencias de lectura determina qué nodos gestionan las consultas de lectura, equilibrando la consistencia, la disponibilidad y la latencia.

La preocupación por la escritura define cuántos nodos deben reconocer una escritura, lo que afecta la durabilidad de los datos frente a la latencia de escritura.

Ajustar cuidadosamente estas estrategias ayuda a gestionar la latencia y mantener la resiliencia. En esta sección, analizaremos los enfoques y consideraciones clave para las operaciones de lectura y escritura en MongoDB.

Preferencia de lectura

Un conjunto de réplicas contendrá más de un servidor que puede atender una operación, y las bibliotecas de cliente de MongoDB pueden realizar la selección del servidor según una preferencia de lectura configurada para garantizar que las operaciones se enruten al servidor deseado.

De forma predeterminada, una aplicación dirige sus operaciones de lectura al miembro principal de un conjunto de réplicas (es decir, el modo de preferencia de lectura es "principal"); sin embargo, los clientes pueden especificar una preferencia de lectura para enviar operaciones de lectura a los secundarios.

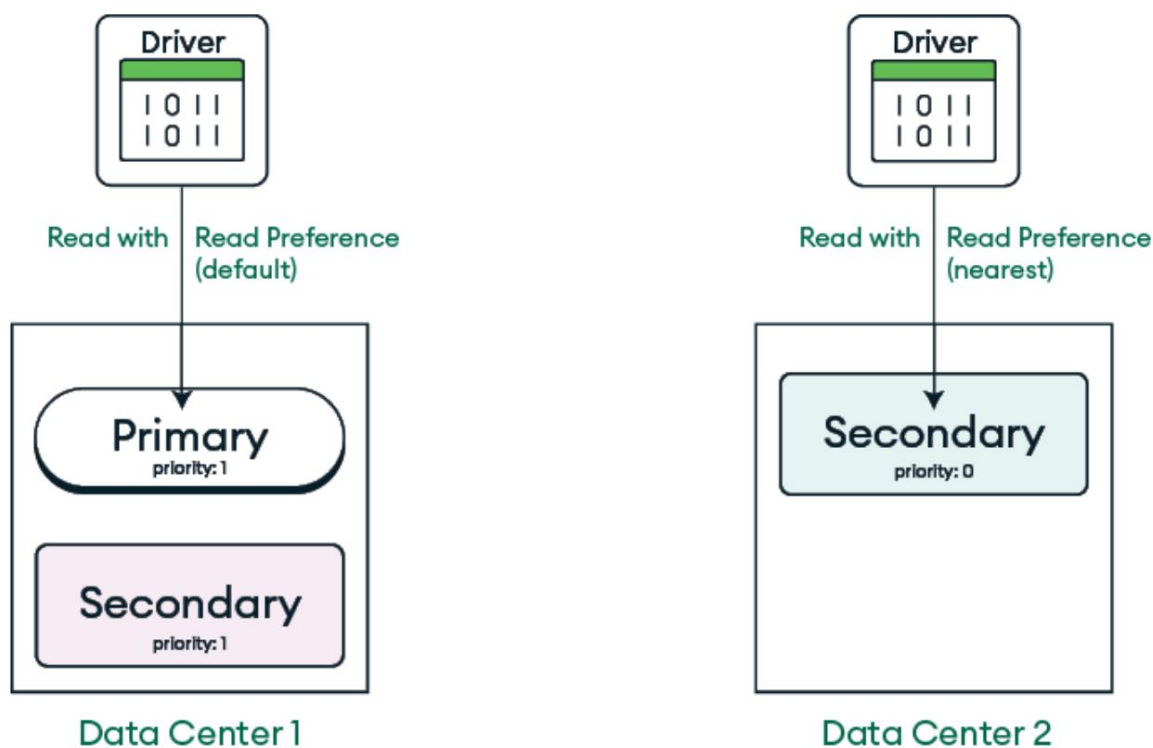


Figura 5.3: Comportamiento de preferencia de lectura de MongoDB en los centros de datos

La preferencia de lectura consta del modo de preferencia de lectura y, opcionalmente, una lista de conjuntos de etiquetas y la opción `maxStalenessSeconds`. Dado que los miembros del conjunto de réplicas pueden retrasarse con respecto al principal (debido a factores como la red)

congestión, bajo rendimiento del disco, operaciones de larga duración, etc.), la opción de preferencia de lectura `maxStalenessSeconds` le permite especificar un retraso de replicación máximo o un tiempo de inactividad para lecturas desde secundarios.

Cuando el tiempo de inactividad estimado de un secundario operaciones de lectura. , excede el tiempo límite, el cliente deja de usarlo para `maxStalenessSeconds` De manera predeterminada, no hay un tiempo de inactividad máximo y los clientes no considerarán el retraso de un secundario al elegir dónde dirigir una operación de lectura; sin embargo, al configurar esta opción, debe especificar un valor de 90 segundos o más.

Hay muchos modos de preferencia de lectura disponibles, que proporcionan distintos grados de flexibilidad en lo que respecta a maximizar la disponibilidad y la consistencia, o minimizar la latencia.

Leer modo de preferencia	Descripción
primario	La preferencia de lectura predeterminada , donde todas las operaciones leen desde el conjunto de réplicas principal actual. Para maximizar la consistencia y evitar lecturas obsoletas, se debe utilizar este modo de preferencia de lectura.
primariaPr referido	En la mayoría de las situaciones, las operaciones leen desde el miembro principal, pero si no está disponible, las operaciones leen desde los miembros secundarios. Este modo de preferencia de lectura permite la máxima disponibilidad, ya que las operaciones de lectura serán consistentes.

	Se entrega siempre que haya un primario disponible. Si el primario no está disponible, aún puede consultar los secundarios.
secundario	Todas las operaciones se leen desde los miembros secundarios del conjunto de réplicas.
secundario Privilegiado	Las operaciones suelen leer datos de los miembros secundarios del conjunto de réplicas. Si el conjunto de réplicas solo tiene un miembro principal y ningún otro, las operaciones leen datos de este.
más cercano	Para minimizar la latencia, las operaciones leen de un miembro aleatorio elegible del conjunto de réplicas, independientemente de si es principal o secundario, según un umbral de latencia especificado. La operación considera lo siguiente al calcular la latencia: - La opción de cadena de conexión localThresholdMS - La opción de preferencia de lectura maxStalenessSeconds - Cualquier lista de conjuntos de etiquetas especificados

Tabla 5.4: Modos de preferencia de lectura de MongoDB

Para las aplicaciones que están distribuidas geográficamente, una preferencia de lectura más cercana puede mejorar aún más el rendimiento de lectura, ya que las operaciones se enrutarán a cualquier miembro con la ventana de latencia más pequeña, según lo define localThresholdMS (valor predeterminado de 15 milisegundos).

Si uno o más miembros del conjunto de réplicas están asociados con etiquetas, puede especificar una lista de conjuntos de etiquetas (matriz de conjuntos de etiquetas) en la preferencia de lectura para orientar esos miembros.

Por ejemplo, considere la siguiente lista de conjuntos de etiquetas con tres conjuntos de etiquetas:

```
[ { "región": "Sur", "centro de datos": "A" }, { "rack": "ra
```

Primero, MongoDB intenta encontrar miembros etiquetados con ambas "región": "Sur" y "centro de datos": "A". Si se encuentra un miembro, el

Los conjuntos de etiquetas restantes no se consideran. En su lugar, MongoDB utiliza este conjunto de etiquetas para encontrar a todos los miembros elegibles.

Si no se encuentra un miembro, se prueba { "rack": "rack-1" }, seguido de la etiqueta final { }, que coincidirá con cualquier miembro elegible.



Nota

Las etiquetas y `maxStalenessSeconds` no son compatibles con un modo de preferencia de lectura principal.

Las configuraciones de preferencias de lectura permiten que las aplicaciones dirijan a los miembros del conjunto de réplicas principal o secundario según necesidades específicas. Esto puede mejorar el rendimiento y la disponibilidad al distribuir las cargas de lectura entre múltiples nodos, reduciendo la latencia y garantizando el acceso continuo a los datos incluso si el nodo principal no está disponible.

Escribir preocupación y durabilidad

En los conjuntos de réplicas de MongoDB, la preocupación por la escritura es una opción de configuración importante que determina el nivel de confirmación que un cliente requiere de la base de datos al realizar una operación de escritura. Este mecanismo afecta directamente el delicado equilibrio entre la durabilidad de los datos y el rendimiento del sistema.

A partir de MongoDB 5.0, la solicitud de escritura predeterminada implícita es `w:"majority"`. Con esta solicitud, se solicita confirmación de que la mayoría calculada de los miembros con derecho a voto que contienen datos ha escrito el cambio de forma duradera en su registro de operaciones local. La mayoría calculada se determina tomando el menor de los siguientes valores:

- La mayoría de todos los miembros con derecho a voto (incluidos los árbitros)
- El número de todos los miembros votantes portadores de datos.

La preocupación por la escritura en conjuntos de réplicas describe la cantidad de miembros que contienen datos (es decir, los primarios y secundarios, pero no los árbitros) que deben reconocer una operación de escritura antes de que la operación regrese como exitosa.

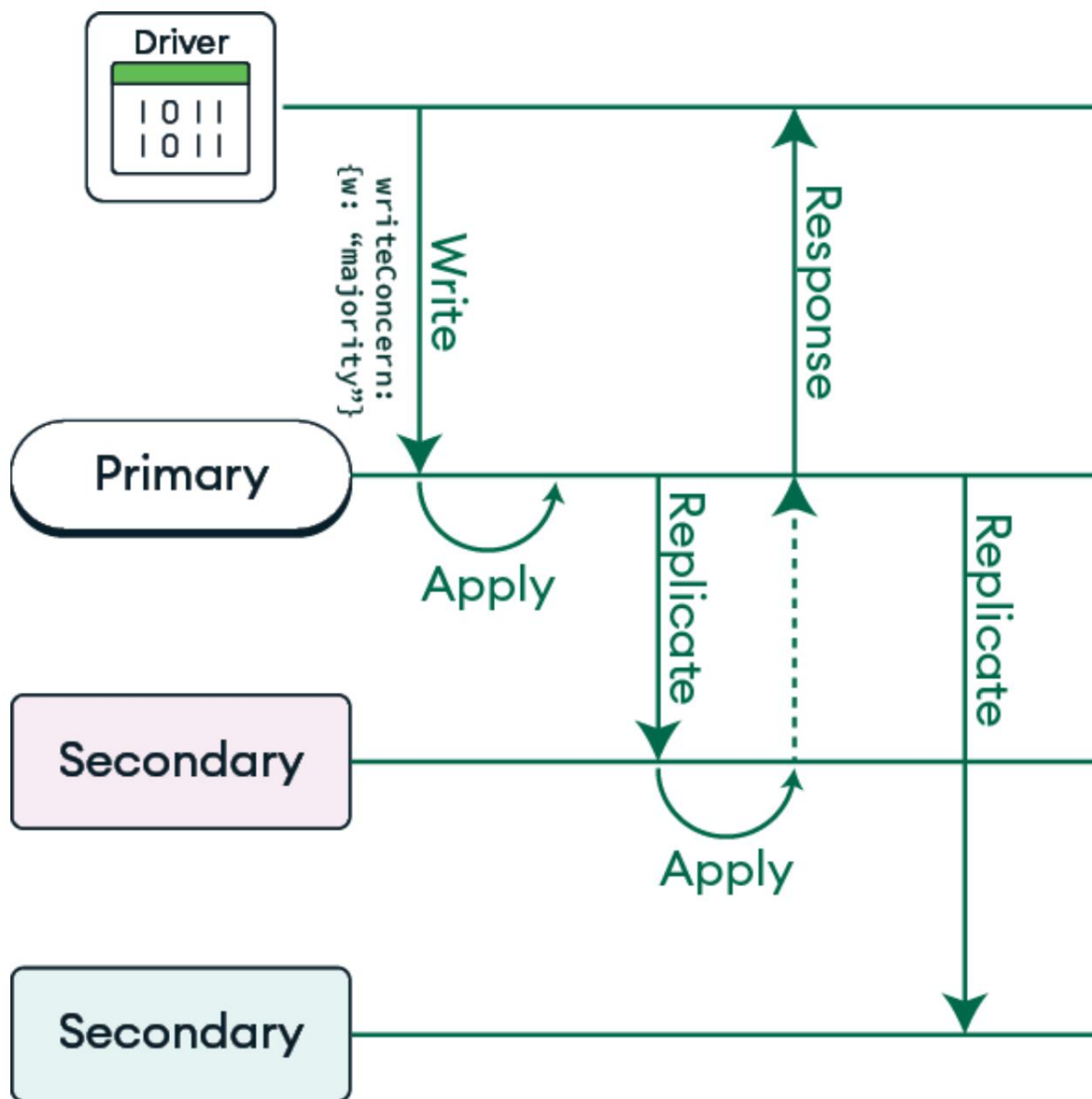


Figura 5.4: Escritura de inquietudes con reconocimiento mayoritario en MongoDB

Una aplicación que emite una operación de escritura espera hasta que la primaria recibe reconocimiento del número requerido de miembros para

La preocupación de escritura especificada. Para una preocupación de escritura de w mayor que 1

o w : "mayoría", los El primario espera hasta que se alcance el número requerido de

secundarios reconocen la escritura antes de devolver la preocupación de escritura

Reconocimiento. Para una preocupación escrita de $w:1$, La primaria puede volver

Escribe el acuse de recibo de la preocupación tan pronto como aplica localmente la escritura, ya que es elegible para contribuir a la preocupación de escritura solicitada.

Cuanto más miembros confirmen una escritura, menor será la probabilidad de que los datos escritos se reviertan si falla el primario. Sin embargo, especificar un nivel alto de preocupación por la escritura puede aumentar la latencia, ya que el cliente debe esperar hasta recibir el nivel de confirmación de preocupación por la escritura solicitado.

La preocupación por la escritura se especifica mediante dos parámetros principales: `w` y `wtimeout`. El primer parámetro (`w`) determina el número de nodos que deben confirmar la escritura y pueden aceptar configuraciones de valores:

- Valores numéricos (`1`, `2`, `3`): Número específico del conjunto de réplicas miembros)
- "mayoría": (predeterminado) La mayoría de los miembros votantes deben reconocer
- Etiquetas de conjunto de réplicas: subconjunto personalizado de miembros del conjunto de réplicas

El segundo parámetro (`wtimeout`) especifica un límite de tiempo, en milisegundos, para que una operación de escritura se propague a suficientes miembros como para alcanzar el objetivo de escritura tras su éxito en el clúster principal. Si se supera el valor de `wtimeout` y se devuelve un error de escritura al cliente, es probable que la operación de escritura se realice correctamente; sin embargo, el cliente no esperará más tiempo para la confirmación. Por defecto, `wtimeout` es "infinito", por lo que si desea que su aplicación devuelva un error en lugar de esperar hasta que la replicación a la mayoría calculada del clúster se realice correctamente, debe proporcionar su propio valor de `wtimeout`.

El siguiente ejemplo demuestra cómo configurar una preocupación de escritura personalizada en una operación de inserción que garantiza que la escritura se haya realizado.

reconocido por la mayoría calculada de nodos y devolverá un error de escritura si la escritura no se ha reconocido dentro de los 5 segundos:

```
db.collection.insertOne( { elemento:  
  "ejemplo" }, { writeConcern:  
  
    { w: "mayoría", wtimeout:  
      5000  
    }  
  
  } );
```

Todos los controladores de MongoDB brindan la capacidad de especificar inquietudes de escritura tanto a nivel de conexión como por operación, lo que permite un control detallado.

Puede modificar la preocupación de escritura predeterminada global ejecutando el comando `setDefaultWriteConcern`. Desde MongoDB Shell, suponiendo que tenga los privilegios necesarios, puede modificar este valor de la siguiente manera:

```
db.adminCommand( { "setDefaultWriteConcern" :  
  1, "defaultWriteConcern" : { "w" : 2  
  
  } })
```

El ejemplo anterior establece la preocupación de escritura global en w:2 . Si emite una operación de escritura con una preocupación de escritura específica, la operación de escritura utiliza su propia preocupación de escritura en lugar de la predeterminada.

Recomendamos dejar "w" en su valor predeterminado de "mayoría", aunque parezca que la latencia sería mejor sin esperar la replicación. Tenga en cuenta que si las copias secundarias empiezan a retrasarse, el control de flujo se activará y comenzará a limitar las escrituras.

Resumen

El sistema de replicación de MongoDB proporciona una base sólida para crear implementaciones de bases de datos altamente disponibles y tolerantes a fallas.

La arquitectura del conjunto de réplicas, combinada con la conmutación por error automática mediante elecciones, garantiza que su base de datos pueda soportar fallos de nodos, manteniendo al mismo tiempo la integridad de los datos. El registro de operaciones (oplog) actúa como la columna vertebral del proceso de replicación, permitiendo que los nodos secundarios se mantengan sincronizados con el principal.

Al diseñar su conjunto de réplicas, considere factores como la distribución geográfica de los datos y el tamaño adecuado del registro de operaciones para las características de su carga de trabajo. Estas decisiones afectarán significativamente la resiliencia, el rendimiento y la capacidad de su base de datos para satisfacer las necesidades específicas de su aplicación.

Recuerde que una configuración adecuada no es una tarea única, sino un proceso continuo que debe evolucionar con el crecimiento y los requisitos cambiantes de su aplicación. Mediante la observación continua y las mejoras iterativas, puede mantener un comportamiento de replicación óptimo incluso a medida que su implementación de MongoDB escala.

En el próximo capítulo, exploraremos cómo funciona la fragmentación de MongoDB con la replicación para proporcionar escalabilidad horizontal para implementaciones con grandes conjuntos de datos y requisitos de alto rendimiento.

6

Fragmentación

A medida que las aplicaciones modernas generan y almacenan cantidades cada vez mayores de datos, garantizar un alto rendimiento y escalabilidad se convierte en una prioridad para cualquier sistema de bases de datos. La arquitectura de fragmentación de MongoDB es una solución diseñada para proporcionar escalabilidad horizontal a aplicaciones con conjuntos de datos masivos y requisitos de rendimiento exigentes. A diferencia del escalamiento vertical tradicional, que suele implicar actualizaciones de hardware costosas y limitadas por el tamaño, la fragmentación ofrece una forma más eficiente y rentable de escalar al distribuir datos entre múltiples servidores. Comprender la arquitectura de fragmentación le ayudará a gestionar picos de actividad de los usuarios, la expansión global y la creciente complejidad de los datos.

Este capítulo profundiza en los principios fundamentales de la arquitectura de fragmentación de MongoDB, lo que le permitirá comprender cómo interactúa con la replicación para lograr una alta escalabilidad y confiabilidad. Comenzaremos examinando los componentes principales de un clúster fragmentado y sus funciones en el enrutamiento y la distribución de datos. A partir de ahí, exploraremos aspectos críticos como la selección de una clave de fragmentación óptima. Finalmente, abordaremos consideraciones prácticas como el refinamiento de la clave de fragmentación, la ubicación estratégica de datos y las técnicas para gestionar importaciones a gran escala y operaciones de refragmentación. Al finalizar este capítulo, tendrá un conocimiento profundo del ecosistema de fragmentación de MongoDB, lo que le

Diseñar arquitecturas escalables que soporten las cambiantes demandas de datos de su aplicación.

Este capítulo cubrirá los siguientes temas:

- Cómo la fragmentación permite la escalabilidad horizontal al distribuir datos entre conjuntos de réplicas
- Los roles y funcionalidades de los fragmentos, los procesos de Mongos y los servidores de configuración en un clúster fragmentado
- Estrategias para seleccionar una clave de fragmento eficiente para optimizar la distribución de datos y la focalización de consultas
- Comparación de métodos de fragmentación basados en rangos, hash y zonas para distintas cargas de trabajo
- Uso de re-sharding para redistribuir datos y abordar cuellos de botella de rendimiento sin tiempo de inactividad
- Técnicas para coubicar datos relacionados para mejorar el rendimiento de las consultas de transacciones y agregaciones
- Configuración y supervisión del balanceador para gestionar las migraciones de fragmentos de manera eficaz
- Enfoques para refinar las claves de fragmentos para resolver problemas como fragmentos gigantes o granularidad deficiente

Comprensión de la arquitectura de fragmentación central

La fragmentación de MongoDB funciona con la replicación para proporcionar escalabilidad horizontal en implementaciones con grandes conjuntos de datos y requisitos de alto rendimiento. Tradicionalmente, cuando una carga de trabajo superaba las capacidades del servidor de base de datos, este debía actualizarse a uno más grande.

Servidor. Esto se conoce como escalamiento vertical y está limitado tanto por el tamaño máximo de cada servidor como por la relación precio-rendimiento; ¡las supercomputadoras suelen ser muy caras!

En cambio, si puede distribuir la carga de trabajo entre varios servidores más pequeños, teóricamente puede escalarla horizontalmente casi infinitamente, con un aumento ideal de los costos lineal con el número de servidores. Esto es lo que permite la fragmentación de MongoDB cuando su carga de trabajo supera las capacidades de un único conjunto de réplicas.

Si su carga de trabajo tiene varias bases de datos distintas relativamente aisladas entre sí, no hay problema en implementar un conjunto de réplicas independiente para cada una. Por ejemplo, podría tener un conjunto de réplicas independiente para usuarios, otro para cuentas y un tercero para la actividad de los usuarios. Sin embargo, con el tiempo, es posible que un solo clúster ya no pueda gestionar un subconjunto específico de la carga de trabajo general de su aplicación. En ese momento, es necesario fragmentar o particionar la carga de trabajo en varios fragmentos, cada uno de los cuales constituye un conjunto de réplicas.

Así, cuando sus datos superan la capacidad de un único conjunto de réplicas, la fragmentación los distribuye entre varios conjuntos de réplicas, cada uno con un subconjunto de sus datos. Este escalado horizontal permite a MongoDB gestionar conjuntos de datos masivos y requisitos de alto rendimiento. Esta sección analiza la arquitectura de un clúster fragmentado y sus elementos de rendimiento importantes.

Componentes arquitectónicos de un clúster fragmentado

Un clúster fragmentado consta de varios componentes: fragmentos (cada uno un conjunto de réplicas responsable de un subconjunto de datos en el clúster), procesos mongos (que se encargan principalmente de enrutar las operaciones a diferentes fragmentos y devolver los resultados al cliente) y servidores de configuración (un conjunto de réplicas especial que mantiene metadatos sobre la configuración de la fragmentación y la ubicación de los datos en el clúster). A partir de la versión 8.0 de MongoDB, los servidores de configuración no necesitan ser servidores independientes y pueden integrarse con uno de los fragmentos.

Mongos es el componente más interesante aquí porque, desde la perspectiva del controlador o la aplicación cliente, actúa como si fuera la base de datos, aceptando solicitudes de operaciones CRUD y devolviendo resultados al controlador. Por otro lado, actúa como proxy del controlador cuando se comunica con los fragmentos, enviándoles operaciones CRUD y luego reenviándoles el resultado. Normalmente, hay varios procesos Mongos en un clúster fragmentado de producción. La Figura 6.1 muestra todos los componentes de un clúster fragmentado y cómo la aplicación... se conecta a él.

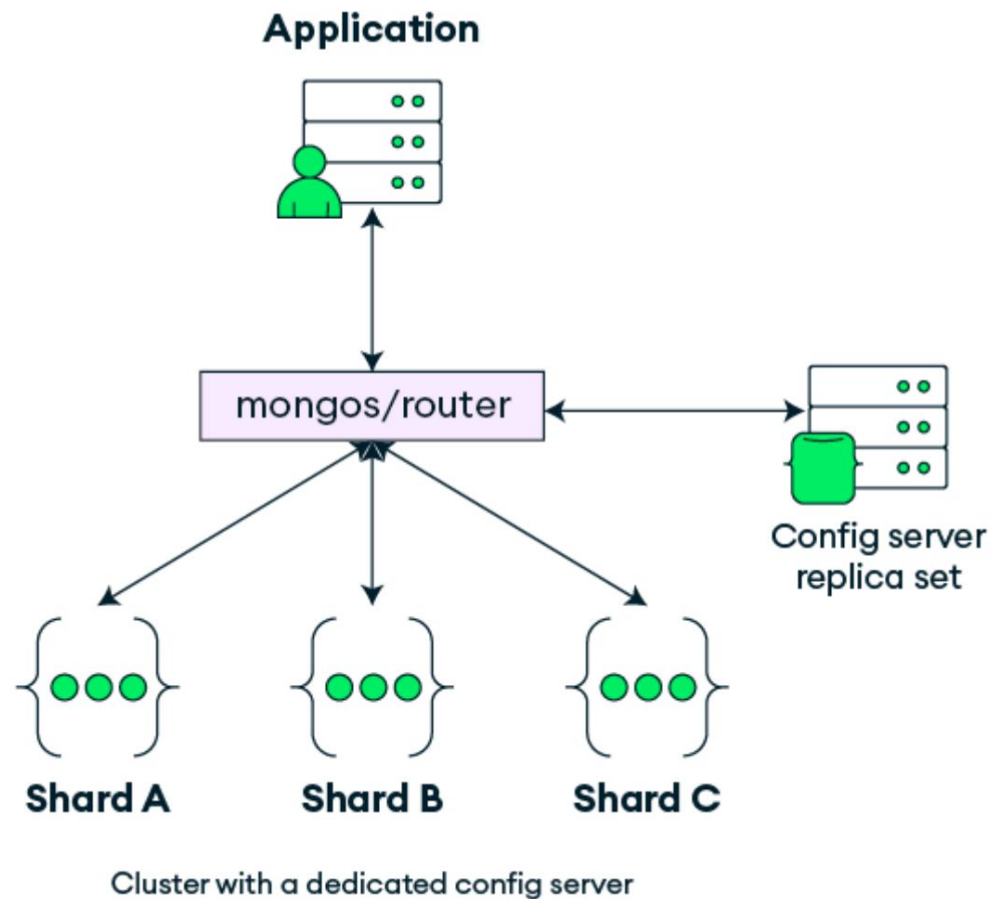


Figura 6.1: Arquitectura de clúster fragmentado

Pero ¿cómo sabe **Mongos** a qué fragmentos reenviar las operaciones? Basándose en la información almacenada en los servidores de configuración, **Mongos** mantiene una tabla de enrutamiento actualizada. Esto significa que, para cada colección, sabe dónde se encuentran sus datos. Si la colección está fragmentada, rastrea la ubicación de cada parte. Luego, reenvía las operaciones que recibe a uno o más fragmentos, según sea necesario. Si se trata de más de un fragmento, también debe fusionar o combinar los resultados de cada uno para devolver un único resultado al cliente.

Fragmentar una colección y seleccionar una

clave de fragmento

Cuando implementa por primera vez un clúster fragmentado, debe decidir cómo
Quieren distribuir los datos entre los fragmentos. Esto es fundamental para

Lograr escalabilidad horizontal gracias a un clúster fragmentado escalable

Distribuye los datos de manera que las operaciones se dividan equitativamente entre
todos los fragmentos. Esto significa que desea que todos sus datos de lectura y escritura...

Las operaciones se distribuirán de forma aproximadamente igualitaria entre todos los fragmentos.

Si bien puede optar por aislar diferentes colecciones en diferentes

Fragmentos: lo más común es que se emplee la fragmentación cuando se tiene una gran cantidad de datos.

suficiente colección que no puede ser manejada por un solo fragmento. En este momento

En ese punto, debes fragmentar o particionar la colección y obtienes

Decide cómo se particionará seleccionando una clave de fragmento.

El valor de la clave de fragmento se utiliza para dividir la colección lógicamente en

rangos, llamados fragmentos. Estos fragmentos se distribuyen a través de

fragmentos de una manera que apunta a equilibrar los datos de manera uniforme en todos los

fragmentos. En versiones anteriores de MongoDB, esto generalmente significaba que cada

El fragmento tenía aproximadamente la misma cantidad de fragmentos, pero en versiones más nuevas

versiones, los fragmentos solo se dividen cuando es necesario. Como resultado, mientras

Aunque los datos todavía se distribuyen de manera uniforme, esto no se traduce necesariamente en
el mismo número de fragmentos en cada fragmento.

Considere un ejemplo simple: si su clave de fragmento es nombre de usuario y el

Los valores del nombre de usuario van desde aaaa , entonces tus trozos

hasta zzzz y pueden dividirse en rangos como abc,etc, ef ,

balanceador, un proceso en segundo plano que se ejecuta en el clúster fragmentado,

Distribuir automáticamente los datos correspondientes a estos rangos de fragmentos a todos los fragmentos, lo que garantiza una distribución aproximadamente igual de los datos.



Limitaciones del balanceador al evaluar la actividad de la clave de fragmento

El equilibrador actúa únicamente en función del tamaño de los datos y no puede determinar qué rangos de valores de clave de fragmento pueden ser más activos que otros.

Ahora, MongoDB enrutará las operaciones que incluyan nombres de usuario que comiencen con una letra específica al fragmento que posee el rango de fragmentos que incluye este nombre de usuario.

Seguramente te estarás preguntando qué sucede con una solicitud que no incluye un nombre de usuario. En estos casos, MongoDB debe enviar la solicitud a todos los fragmentos. Este tipo de solicitud se denomina operación de difusión o de dispersión-recopilación, y no escala horizontalmente con la misma eficacia que las operaciones dirigidas a un solo fragmento.

¿Por qué es malo el método scatter-gather? Es un error común

creer que las consultas scatter-gather son eficientes simplemente porque cada fragmento solo procesa un pequeño subconjunto de los datos. Si bien esto puede parecer intuitivamente cierto y, de hecho, refleja el funcionamiento del proceso original de map-reduce (al dividir grandes cantidades de trabajo entre varios servidores), este enfoque no escala bien horizontalmente en la mayoría de los casos. Analicemos por qué.

Cuando una solicitud de consulta se dirige a un único fragmento, ese fragmento ejecuta la operación y envía los resultados a MongoDB, que los reenvía al cliente. Pero cuando una solicitud de consulta se dirige a todos los fragmentos (un

operación de dispersión-reunión), cada fragmento tiene que ejecutar la operación en su propia parte de los datos y enviar el resultado de vuelta a mongos .
Fusionar y reenviar al cliente. Observe algo sobre estos dos
¿Escenarios? En consultas dirigidas, solo un fragmento debe ejecutar la operación. En las consultas de dispersión y recopilación, cada fragmento debe ejecutar la operación.

Por lo tanto, en un clúster con 20 fragmentos, una única operación de dispersión-reunión es traducido en 20 operaciones. Aunque parezca más económico
operación, procesando solo 1/20 del total de datos, la operación
La sobrecarga de ejecución sigue siendo la misma para cada fragmento y Mongos tiene
Para realizar el trabajo extra de fusionar los resultados. En algunos casos, Mongos...
No se puede comenzar a devolver datos al cliente hasta que reciba respuesta.
cada fragmento que recibió la consulta.

Supongamos que su clúster está procesando decenas o cientos de miles de
Consultas. Así es como una sola solicitud dirigida a un fragmento en comparación con un...
La consulta de dispersión y recopilación que se escala en un clúster de 20 fragmentos se verá así:

Cliente consultas	Consultas dirigidas solo	Consultas de dispersión y recopilación solo
100	100	2.000
1.000	1.000	20.000
10.000	10.000	200.000
100.000	100.000	2.000.000

Tabla 6.1: Tabla que muestra el total de consultas que se ejecutan en los 20 fragmentos

Ahora, imagine duplicar el número de fragmentos en la Tabla 6.1 . La columna de consultas de destino se mantendrá, pero la de consultas de dispersión-recolección se duplicará. Por lo tanto, no se obtiene el beneficio del escalado lineal, ya que aumentar el número de fragmentos en el clúster también aumentará el número total de consultas. Como desventaja adicional, las consultas de dispersión-recolección que necesitan recibir información de todos los fragmentos se vuelven cada vez más lentas a medida que aumenta el número de fragmentos. Esto se debe a que también aumenta la probabilidad de que un fragmento experimente una interrupción de latencia (como una configuración incorrecta de la red, una elección o un fallo). Esto contribuye a una latencia adicional para cada consulta de transmisión, a diferencia de las consultas dirigidas, que solo afectan a un subconjunto de consultas dirigidas al fragmento afectado.

Selección estratégica de claves de fragmentos

La selección de la clave de fragmento es una decisión crucial que afecta directamente el rendimiento y la escalabilidad de un clúster fragmentado. Una buena clave de fragmento puede dirigir las operaciones a fragmentos específicos, proporcionar la granularidad adecuada para distribuir los datos de forma uniforme y minimizar el riesgo de crear fragmentos activos. Analicemos algunos elementos importantes de una buena clave de fragmento.

Clave de fragmento para operaciones de orientación Para comprender cómo la partición de datos influye en la distribución de operaciones, veamos otro ejemplo de cómo las colecciones se pueden particionar en fragmentos en función de la clave de fragmento definida.

Supongamos que tiene una colección de pedidos . Podría dividirla por ID del pedido , Esto es único y permitirá la creación de múltiples rangos de fragmentos, lo que permite una distribución precisa de los datos entre los fragmentos. Sin embargo, si order_id aumenta monótonamente, el último fragmento (el que va desde un rango alto de order_id hasta `maxInt`) siempre recibirá todas las nuevas inserciones.

Esto puede ser un problema o no. Si la creación de nuevos pedidos es el cuello de botella de su sistema, esto podría ser problemático, pero si se crean con poca frecuencia y la mayor parte de la carga proviene de la actualización, lectura o agregación de pedidos, puede que no sea un problema grave.

Ahora, consideremos una operación que lee todos los pedidos recientes de un cliente en particular. Como no se conoce el rango de pedidos de los clientes, se deberá difundir la consulta a todos los fragmentos que contienen fragmentos para la colección de pedidos, como se muestra en la Figura 6.2y eso es problemático para la escalabilidad horizontal.

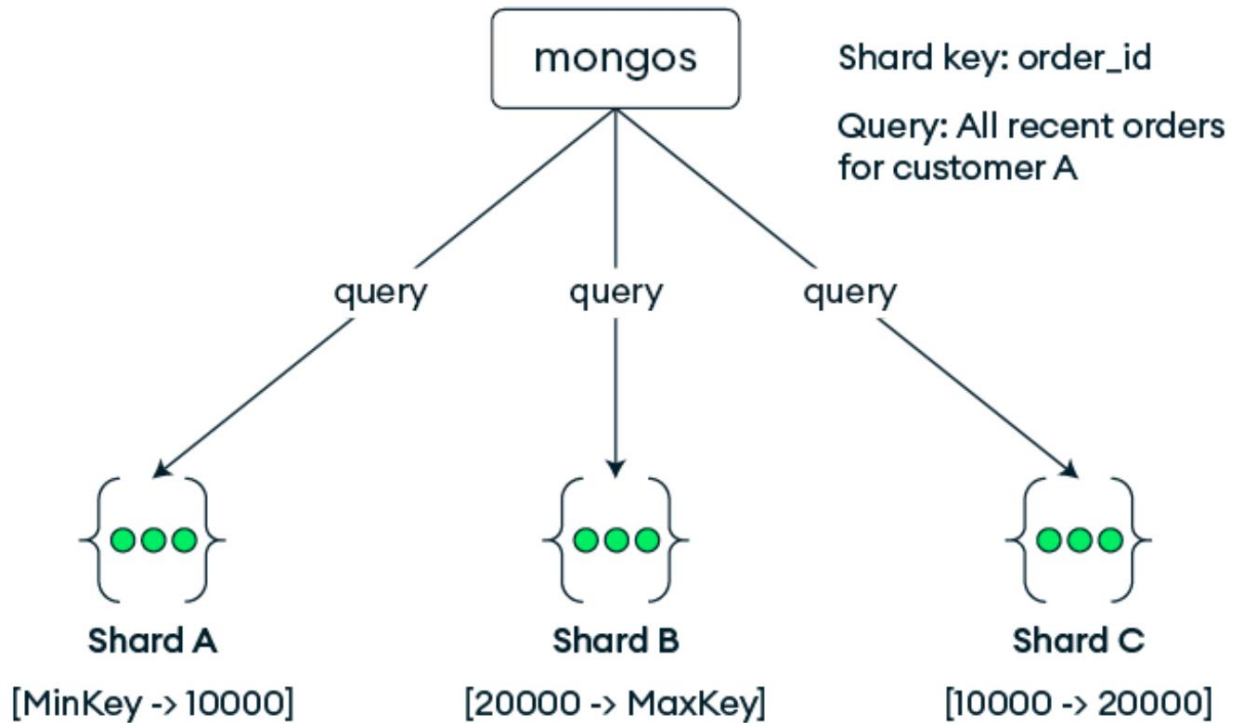


Figura 6.2: Consulta de difusión

Si sabe que su aplicación siempre tendrá el valor `customer_id` al procesar un pedido (una suposición razonable), sería mejor usar una clave de fragmento compuesta como `{customer_id:1,order_id:1}` .

Esto mantendrá los pedidos de cada cliente en un único fragmento (ya que tienden a estar dentro de un único fragmento), y las consultas sobre los pedidos de un cliente en particular se dirigirán a un único fragmento en lugar de dispersarse entre todos.

En la Figura 6.3, Todos los pedidos del cliente A están dentro del fragmento 6.3 se cubre desde el `customer_id` más bajo (inclusivo) hasta el cliente I (exclusivo), que está en el fragmento A. Como tal, la consulta de todos los pedidos recientes del cliente A solo se enviará al fragmento A. Como beneficio adicional, ya no se crearán nuevos pedidos en el mismo fragmento. Sin embargo, en este caso, todas sus operaciones en pedidos deben incluir

tanto los valores `customer_id` como `order_id` en la consulta para garantizar que estén dirigidos a un solo fragmento.

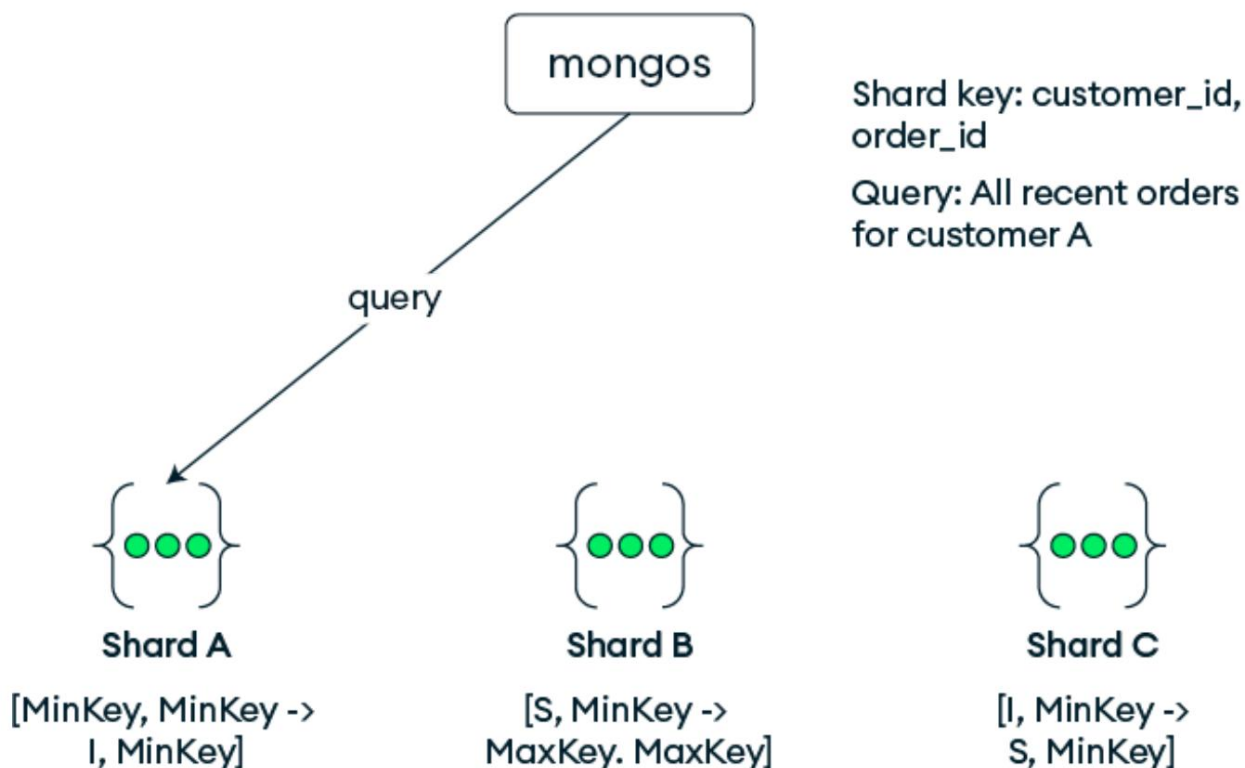


Figura 6.3: Consulta de segmentación

Escalar clústeres fragmentados consiste en concentrar las operaciones en la menor cantidad posible de fragmentos. Una clave de fragmento eficaz permitirá que la mayoría de las operaciones se dirijan a un solo fragmento.

Clave de fragmento con buena granularidad. Probablemente hayas escuchado que es importante elegir una clave de fragmento que ofrezca buena granularidad, principalmente porque permite dividir los fragmentos eficazmente. El ejemplo que vimos en la sección anterior lo ilustró bien. Dado que `order_id` es único, cualquier clave de fragmento que lo incluya como componente se puede dividir fácilmente en fragmentos más pequeños. Pero

¿Por qué es tan importante la granularidad y qué sucederá si su clave de fragmento no es lo suficientemente granular?

Imagine un sistema multiinquilino donde la clave del fragmento es "tenant_id" . A primera vista, parece una buena idea, ya que cada operación del inquilino siempre incluye el valor "tenant_id" , lo cual es ideal para dirigir las operaciones de un inquilino específico a un fragmento, y los nuevos inquilinos no se crean con la suficiente rapidez como para preocuparse por todas las inserciones que se realizan en un único rango de fragmentos. Sin embargo, ¿qué sucedería si un solo inquilino se convirtiera en un usuario avanzado, generando tantos datos en la colección que simplemente no caben en un fragmento?

La forma de localizar datos en dos fragmentos es decidiendo qué valores de la clave del fragmento se ubicarán en cada uno. Sin embargo, en nuestro caso, todos los datos del inquilino tienen el mismo valor de clave de fragmento, lo que hace que los datos no se puedan dividir , y el fragmento que los contiene ahora se denomina fragmento jumbo . En la Figura 6.4 , el inquilino_I contiene una gran cantidad de datos, y el fragmento desde el inquilino_I (incluido) hasta el inquilino_J (excluido) se denomina jumbo. El fragmento B contiene más datos que el fragmento A; sin embargo, dado que el fragmento no es divisible, el balanceador no puede transferir datos del fragmento B al fragmento A para equilibrarlos. En este caso, refinar o ampliar la clave del fragmento puede ser útil añadiendo otro campo como componente de la clave después de "tenant_id" .

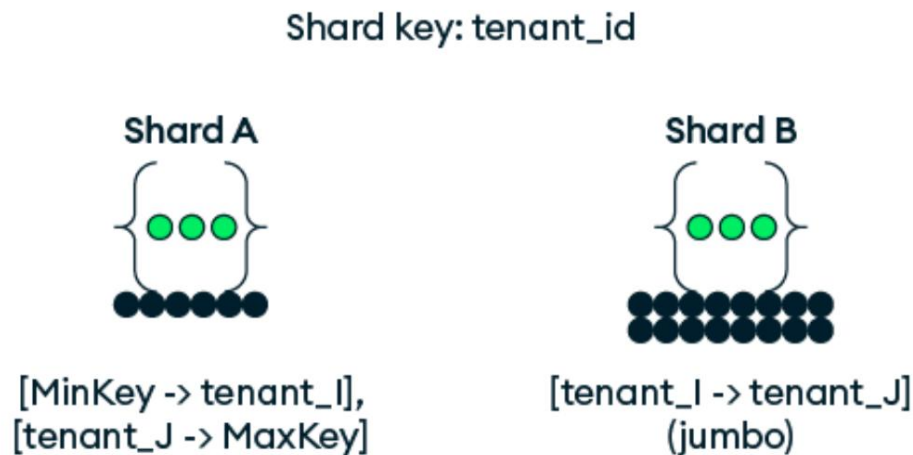


Figura 6.4: Clave de fragmento en el id del inquilino

Afortunadamente, existe un comando para refinar la clave de fragmento sin tiempo de inactividad, lo que permite ampliarla cuando no es lo suficientemente granular. Esta operación es económica, ya que permite añadir uno o varios campos sin tener que mover datos. Una vez que la clave de fragmento se amplía con otro campo (por ejemplo, `_id` u otro campo que tenga sentido en el modelo de datos), MongoDB puede dividir el fragmento para el inquilino grande según el valor `_id`, y algunos de sus datos se pueden equilibrar o mover a otro fragmento. , En la Figura 6.5, tras refinar la clave de fragmento, el fragmento jumbo anterior se puede dividir y algunos de los datos se pueden mover al otro fragmento.

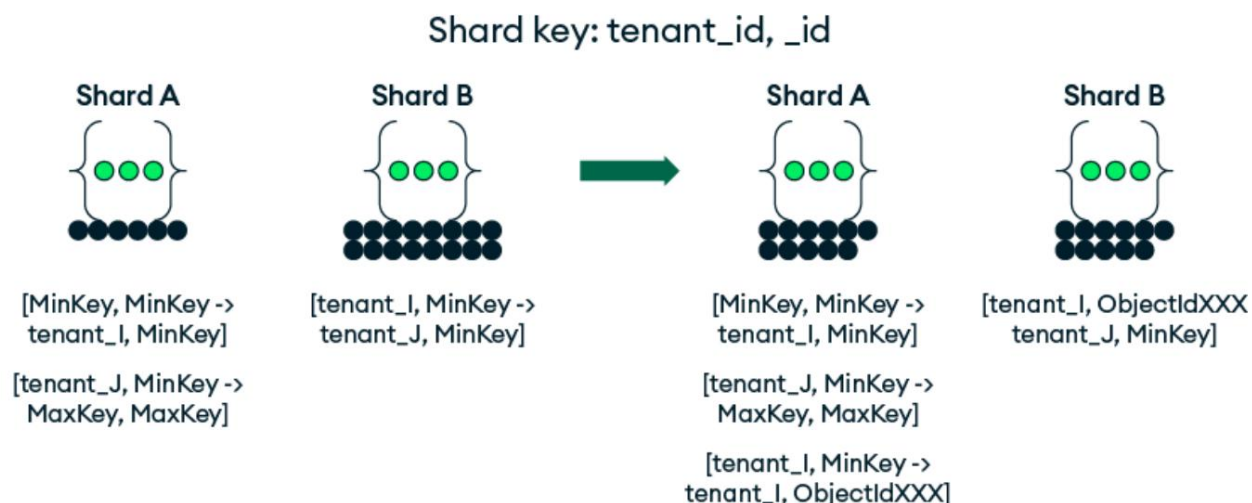


Figura 6.5: Refinar la clave del fragmento y esperar a que se equilibren los datos

La mejor clave de fragmento será lo suficientemente granular para permitir la división de rangos de datos para una mejor distribución entre los fragmentos.

Evite aumentar o disminuir los valores de clave de fragmento

Quizás haya leído que una clave de fragmento que aumenta monótonamente es una mala idea. De hecho, como se destacó en el ejemplo anterior, es algo que debería preocuparle si es probable que se produzca un cuello de botella en la inserción de nuevos documentos. Veamos un ejemplo donde esto probablemente ocurriría y por qué sería un problema.

Piense en el ejemplo de la aplicación Socialite de [Capítulo 2](#), Esquema Design for Performance . Considere la colección de contenido que recibe las publicaciones de los usuarios. Esta colección experimenta un alto número de escrituras, principalmente de usuarios que insertan nuevas publicaciones. Ahora imagina fragmentar la colección de contenido según el valor `_id` de cada publicación, usando el tipo ObjectId predeterminado . Dado que los primeros cuatro bytes

de `ObjectId` representa la marca de tiempo actual, podría pensar que es útil tenerlo como proxy para el tiempo de creación y que no es una mala idea tener todas las publicaciones recientes en el mismo fragmento para fines de segmentación. Después de todo, los usuarios normalmente sólo quieren leer las publicaciones más recientes, ¿verdad? Sin embargo, el resultado de esto es que cada inserción va al rango de fragmentos más alto .

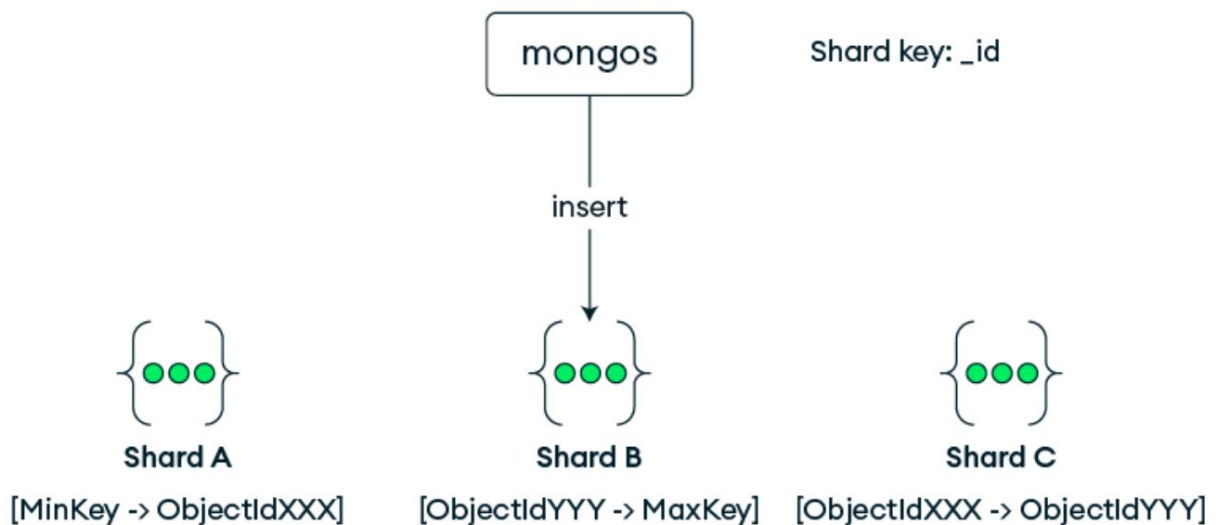


Figura 6.6: Fragmento activo causado por una clave de fragmento que aumenta monótonamente

Como se puede ver en la Figura 6.6, Los nuevos insertos van a girar lo que sea el fragmento superior en el fragmento activo, mientras que los demás fragmentos no reciben ninguna escritura (ya que las publicaciones no suelen actualizarse ni eliminarse). Incluso si el diseño del esquema incluye la actualización de las publicaciones con información como el número de "me gusta", la actividad se concentrará solo en las publicaciones recientes (¿con qué frecuencia los usuarios retroceden meses para dar "me gusta" a una publicación de alguien de entonces?). Por lo tanto, las únicas escrituras que no se dirigen al fragmento superior serían las eliminaciones (cuando los usuarios deciden limpiar o archivar contenido antiguo), ¿verdad? Pero incluso estas crearían un fragmento activo al acceder a los valores del rango inferior.

Tampoco obtendrías ningún beneficio al enfocarte en las lecturas, porque

Cuando buscas publicaciones para mostrarle a alguien, generalmente son:

Las publicaciones de tus amigos o las tuyas propias. La consulta se vería

algo como esto:

```
// Consulta las últimas 20 para obtener publicaciones // a subconjunto de usuarios
Esto da como resultado una operación de dispersión-reunión.
db.posts.find({userId:{$in:["..."]}}).sort({_id:-1}).lim
```

Este tipo de consulta, para obtener las últimas 20 publicaciones para mostrar en un subconjunto de usuarios, probablemente resulte en una consulta de dispersión-recolección. No solo eso, sino que

Esta consulta requiere esperar a que todos los fragmentos ordenen sus resultados.

fusionarlos en mongos y comenzar a devolver resultados al cliente.

Dado que lo más importante en una aplicación de redes sociales con un feed es

empezar a mostrar resultados lo más rápido posible, esto no es aceptable.

La solución probablemente sería usar una clave de fragmento mejor. La mayoría

Una opción obvia sería fragmentar por {userId:1, _id:1} , lo que

Probablemente coloque el contenido de cada usuario en el mismo fragmento, lo que significa que

A medida que cientos o miles de usuarios crean nuevas publicaciones,

Probablemente se distribuirá a todos los fragmentos que poseen los fragmentos para aquellos

usuarios. Si su ID de usuario es aleatorio (como lo sería cualquier nombre seleccionado por el usuario)

sea), su distribución también será aleatoria. Pero incluso si usa

Algún tipo de sistema generado, único y de aumento monótono.

valor, e incluso si sus nuevos usuarios tienden a ser más activos al principio, esto

Se compensaría con la creación de todos los usuarios ya existentes.

contenido y distribuir los escritos de manera más o menos uniforme en todo el

fragmentos.

Este es un enfoque similar al que vimos en el ejemplo anterior de `órdenes`, donde anteponer `customer_id` a `order_id` hace una mejor Clave de fragmento que solo `order_id`. En sistemas multiusuario o multiinquilino, considere siempre que el campo principal de la clave de fragmento sea su ID de usuario o inquilino para maximizar la segmentación de consultas para cada usuario o arrendatario.

Tenga en cuenta que no es necesariamente recomendable aleatorizar un valor de clave de fragmento que aumenta monótonamente mediante un hash. En la siguiente sección, analizaremos algunas ventajas y desventajas que debe considerar antes de decidir aplicar un hash a su clave de fragmento.

Si todavía no está seguro de su elección de clave de fragmento antes de fragmentar o volver a fragmentar una colección, siempre puede usar el comando `configureQueryAnalyzer` para recopilar una muestra de consultas en su colección y luego ejecutar el comando `analyzeShardKey` para evaluar una clave de fragmento para esta colección.

Tipos de fragmentación

Fundamentalmente, MongoDB admite un mecanismo fundamental para distribuir datos entre fragmentos: la fragmentación basada en rangos. Sin embargo, también ofrece dos variantes especiales que se adaptan mejor a diferentes casos de uso: la fragmentación hash y la fragmentación basada en zonas. Exploreemos cómo funcionan, cómo se relacionan entre sí y cuándo cada una sería una opción adecuada.

Fragmentación basada en rangos

La fragmentación de rangos es el enfoque predeterminado y más intuitivo. Esto es lo que hemos descrito en este capítulo hasta ahora. La fragmentación de rangos divide los datos según el orden natural de los valores de la clave de fragmento y luego distribuye los rangos de datos a diferentes fragmentos. Este es el método adecuado para la mayoría de los casos de uso; solo asegúrese de elegir una clave de fragmento adecuada para sus patrones de lectura y escritura. Todos los ejemplos que hemos analizado hasta ahora utilizan la fragmentación de rangos estándar. Todos los demás métodos de fragmentación utilizan la fragmentación de rangos como base, pero ofrecen funciones adicionales para modificar los valores utilizados para los rangos o para influir en su equilibrio en el clúster.

Fragmentación hash: La

fragmentación hash toma la clave de la partición, la ejecuta mediante una función hash y luego aplica rangos a esos valores hash. El resultado es similar a reorganizar todas las páginas antes de colocarlas en volúmenes, lo cual es ideal para distribuir los datos aleatoriamente. Todas las operaciones realizan este hash automáticamente, por lo que la aplicación no necesita cambiar nada en la forma en que consulta los datos.

Imagine almacenar registros o eventos en una colección enorme donde el valor `_id` de cada evento aumenta constantemente. Con la fragmentación por rango, todos esos registros deben insertarse en el mismo fragmento. Pero si usa `{`, el sistema evitando así los puntos calientes los distribuirá aleatoriamente, `_id: "hashed"`, calientes de inserción.

Existen múltiples inconvenientes y costos adicionales al aplicar hashes al valor de la clave de fragmento. Las consultas de rango en la clave de fragmento con hashes son ineficientes, y hay un índice de acceso aleatorio adicional que mantener en cada inserción. Y lo más importante, debe recordar que no se gana...

Cualquier granularidad adicional mediante el hash del valor de la clave de fragmento. Si su distribución es deficiente debido a la baja granularidad de la clave de fragmento (insuficientes valores distintos), el hash no cambiará esto en absoluto.

Aún tendrá una distribución deficiente con la sobrecarga añadida de una clave hash.

En los casos en que una clave de fragmento con hash sigue siendo la mejor opción, la versión 7.0 de MongoDB introdujo la posibilidad de eliminar el índice con hash, siempre que se disponga de un índice diferente que admita la clave de fragmento y la colección ya esté equilibrada. Una vez eliminado el índice de la clave de fragmento con hash, no se realiza ningún otro equilibrado, pero se evita tener que mantener el índice con hash en cada inserción.

Fragmentación por zonas. La fragmentación

por zonas permite controlar qué rangos de datos residen en qué conjuntos de fragmentos, según una política creada por el usuario. Permite incorporar la geografía o simplemente limitar una colección fragmentada a un subconjunto de fragmentos. Para usar la fragmentación por zonas, se deben seguir dos pasos: asignar etiquetas de zona a un fragmento o conjunto de fragmentos y, a continuación, especificar los rangos de claves de fragmento que se asociarán a estas zonas específicas, lo que permite controlar dónde se almacenan físicamente los datos.

Considere una empresa global de SaaS que desea almacenar datos de usuarios europeos en la UE para garantizar el cumplimiento normativo. Al zonificar los rangos de fragmentos de la región "UE" con los fragmentos ubicados en Fráncfort y de la región "NA" con los fragmentos ubicados en Norteamérica, mantiene los datos locales, de fácil acceso y en cumplimiento con las políticas.

Así es como se ven esos pasos en los comandos de base de datos que fragmentan la colección, asignar zonas apropiadas a los fragmentos y agregar la clave del fragmento rangos a zonas:

```
un Fragmento // Recopilación y asignación de zonas para datos geográficos
sh.shardCollection("db.coll", {país:1, id de usuario:1})
sh.addShardToZone("shard0", "NA")
sh.addShardToZone("fragmento1", "UE")
sh.addTagRange("db.coll", {país:"DE",id de usuario:MinKey},{co
sh.addTagRange("db.coll", {país:"EE. UU.", id de usuario: MinKey},{co
```

Puede encontrar más ejemplos en la documentación de MongoDB:

- Segmentación de datos por ubicación :

<https://www.mongodb.com/docs/manual/tutorial/sha>

[segmentación de datos por ubicación](#)

- Segmentación de datos por aplicación o cliente :

<https://www.mongodb.com/docs/manual/tutorial/sha>

[fragmentos de segmentación de ruta/](#)

- Escrituras locales distribuidas para cargas de trabajo de solo inserción :

<https://www.mongodb.com/docs/manual/tutorial/sha>

[escrituras de alta disponibilidad rding/](#)

- Distribuir colecciones mediante zonas :

<https://www.mongodb.com/docs/manual/tutorial/sha>

[rding-distribuir-colecciones-con-zonas/](#)

Una vez que la cantidad de fragmentos en su clúster crezca, querrá...

Ejercer más control sobre dónde se encuentra cada fragmento (y no fragmento)

Se encuentra la colección. Por ejemplo, puede tener dos archivos fragmentados enormes.

colecciones en 10 fragmentos, pero en lugar de tener cada colección en

cada uno de los 10 fragmentos, es posible que prefieras tener cada colección solo en 5 fragmentos propios. Esto puede ser útil de varias maneras:

Si la carga de trabajo generada por cada colección es diferente, podría asignar diferentes recursos a través de escalamiento asimétrico de fragmentos y cualquier Operaciones de dispersión y recopilación que se envían a cada fragmento de la colección Las vidas solo se enviarían a la mitad de los fragmentos. Dado que la disminución El número total de operaciones en el clúster ayuda a la escalabilidad, esto es un Una forma sencilla de lograrlo. Además, si tienes un imprevisto Si tienes problemas con uno de tus fragmentos, solo una de las dos colecciones funcionará. verse afectados en lugar de ambos.

Para distribuir dos colecciones a un subconjunto de fragmentos, la base de datos Los comandos podrían verse así:

```
// Ejemplo de distribución de colecciones a subconjuntos de fragmentos
sh.addShardToZone("fragmento0", "C1")
sh.addShardToZone("fragmento1", "C1")
sh.addShardToZone("fragmento2", "C2")
sh.addShardToZone("fragmento3", "C2")
sh.addTagRange("db.coll1",{<shKey>:MinKey},{<shKey>:Ma
sh.addTagRange("db.coll2",{<shKey>:MinKey},{<shKey>:Ma
```

Estos comandos primero etiquetan cada fragmento con una etiqueta C1 o C2 ,
y luego asignar la colección coll1 a los fragmentos etiquetados como C1 y
Colección coll2 a fragmentos etiquetados como C2 . Usarías tu...

Nombres de campos de clave de fragmento para <shKey> . Este control añade complejidad.
Debe administrar zonas y límites de rango y cómo se mapean
a zonas, monitorear el equilibrio entre fragmentos y planificar un crecimiento desigual.
Si bien existen ventajas en aislar cada colección fragmentada a su...
propio subconjunto de fragmentos, hay dos casos en los que debería hacerse

Se deben evitar estas restricciones y, en su lugar, las colecciones deben ubicarse en los mismos fragmentos con rangos de claves que mantengan unidos los datos relacionados. Una de ellas es admitir transacciones de múltiples documentos en dos colecciones que permanecen en un solo fragmento; la otra es evitar que las operaciones de búsqueda entre dos colecciones tengan que funcionar a través de la red.

Analizaremos esto con más detalle en la sección Colocación de fragmentos de recopilación fragmentados juntos , más adelante en este capítulo.

Administración avanzada de fragmentación

Al trabajar con clústeres fragmentados, existen varias consideraciones operativas que, si bien no forman parte de la mecánica básica de fragmentación, son importantes para mantener la eficiencia y la capacidad de respuesta de su implementación a lo largo del tiempo. Estas incluyen la refragmentación cuando la situación se descontrola, la predivisión en nuevas colecciones para grandes cargas de datos, la gestión del balanceador, el traslado de colecciones no fragmentadas y la comprensión de la mejor manera de distribuir los datos para obtener el mejor rendimiento.

Refragmentación: si, cuándo y cómo

Elegir la clave de fragmento correcta la primera vez es ideal, pero seamos sinceros, a veces nos equivocamos la primera vez y, en otras ocasiones, los patrones de uso reales evolucionan. Lo que empezó como una buena clave puede convertirse en un cuello de botella. Quizás elegiste una clave de fragmento con hash en un campo que ya estaba aleatorizado sin darte cuenta de la sobrecarga adicional que esto causa, y ahora quieres cambiarla a una clave de fragmento con rango normal. O tal vez usaste `order_id` para tu colección de pedidos y ahora te das cuenta de que la mayoría de tus consultas para un

Los pedidos de clientes particulares se agrupan de forma dispersa y no están dirigidos, y usted desea cambiar la clave de fragmento a `{customer_id, order_id}` porque permite una mejor orientación de las consultas.

La función de refragmentación de MongoDB te da una segunda oportunidad. Sin interrupciones, puedes cambiar la clave de fragmento a una más adecuada que permita reescribir tus datos en diferentes fragmentos sin interrupciones. La refragmentación es una operación en segundo plano; lee y reescribe todos tus datos en segundo plano, pero al completarse, obtienes una colección mejor distribuida y un sistema que puede volver a funcionar.

La redistribución no solo sirve para cambiar la clave de fragmento; puede usarse con una clave de fragmento existente para redistribuir o reescribir la colección en fragmentos existentes con mayor rapidez que la que tardaría el balanceador en mover fragmentos y eliminar huérfanos. A partir de la versión 8.0 de MongoDB, puede ejecutar el comando `reshardCollection` en su colección con la misma clave de fragmento estableciendo la opción `forceRedistribution` en `true`. Considere usarlo si agrega o elimina varios fragmentos a la vez, o después de fragmentar una colección existente sin fragmentar. En la Figura 6.7, hay tres fragmentos en este clúster y hay una colección sin fragmentar en el fragmento A. Ahora, los datos de esta colección han crecido y desea fragmentarla. Como un fragmento solo puede participar en una migración de fragmentos a la vez, el balanceador solo puede mover los datos del fragmento A al fragmento B o al fragmento C a la vez. La velocidad de balanceo podría ser lenta. Además, después de migrar un fragmento, el fragmento A necesitaría eliminar su copia original de los datos en este fragmento, y el espacio en disco no se recupera automáticamente. Si vuelve a fragmentar esta colección inmediatamente después de fragmentarla, el mecanismo de refragmentación crea una colección temporal y los tres fragmentos pueden leer los datos del fragmento A y

Reescribirlo en la colección temporal en paralelo. Al finalizar la repartición, se renombrará la colección temporal y se eliminará la colección original. El espacio de disco se recupera automáticamente en el fragmento A.

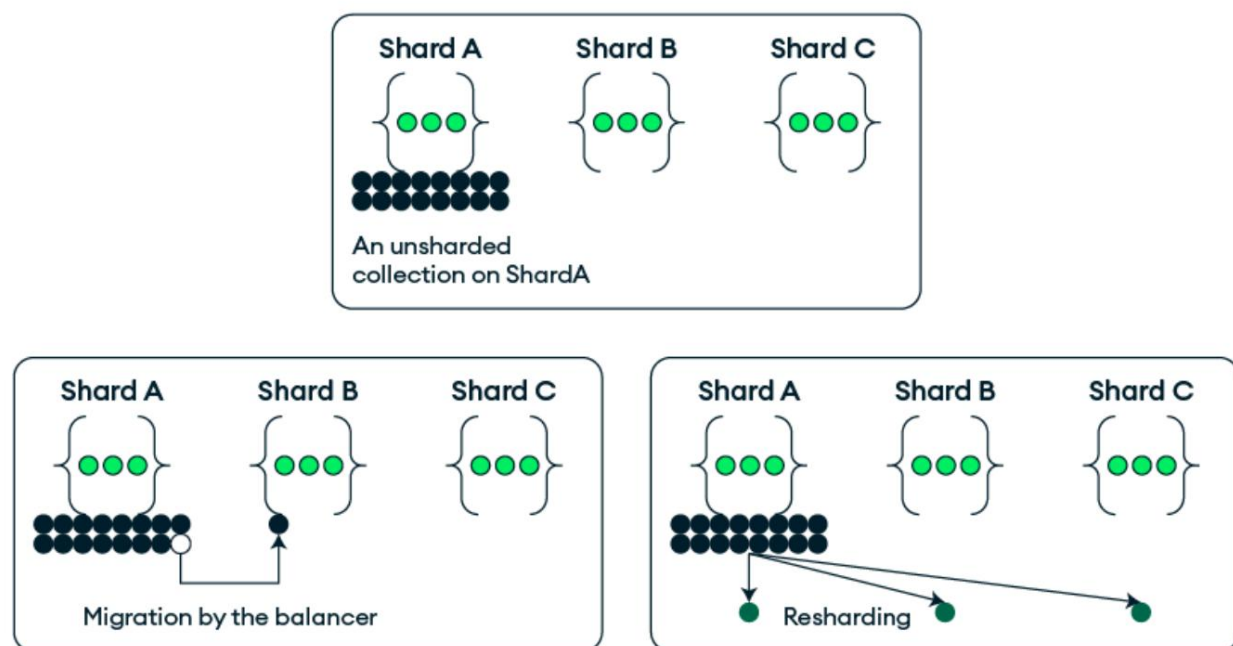


Figura 6.7: Migración de fragmentos versus refragmentación

Consulte la documentación para obtener más detalles, restricciones y ejemplos:

<https://www.mongodb.com/docs/manual/core/sharding-reshard-a-collection/>.

El comando para refinar (o ampliar) su clave de fragmento que mencionamos en la sección Clave de fragmento con buena granularidad es diferente del comando de resharding, ya que ampliar la clave de fragmento con campos adicionales no requiere mover ningún dato de sus fragmentos actuales.

Lea más sobre cómo refinar la clave de fragmento en la documentación:

<https://www.mongodb.com/docs/manual/core/sharding-refine-a-shard-key/#std-label-shard-key-refine>.

Consideraciones sobre el balanceador

En segundo plano de un clúster fragmentado de MongoDB, el balanceador garantiza que los datos se distribuyan uniformemente entre los fragmentos. Su función es supervisar la distribución de datos y mover fragmentos cuando la situación se vuelve irregular, manteniendo el sistema equilibrado. Sin embargo, aunque está activado por defecto y es fácil olvidarlo, el balanceador no es gratuito, y comprender su comportamiento puede ser clave para evitar efectos secundarios no deseados.

En primer lugar, existe una sobrecarga. Cada migración de fragmentos que activa el balanceador implica copiar datos de un fragmento a otro, actualizar los metadatos en los servidores de configuración y eliminar la copia original. Si bien estas operaciones son eficientes y seguras, consumen E/S, CPU y ancho de banda de red. En un clúster con poca carga, es posible que ni siquiera lo note. En un sistema de producción con mucha actividad, especialmente durante las horas de máxima demanda, estas migraciones en segundo plano pueden competir con las operaciones de la aplicación por recursos y ralentizar el proceso.

Por eso, MongoDB permite establecer ventanas de balanceo, es decir, horarios específicos durante los cuales se permiten las migraciones de fragmentos. Por ejemplo, se puede restringir el balanceo a las horas nocturnas o en periodos de baja demanda, cuando el tráfico es más bajo. Esto permite mantener un rendimiento óptimo durante el día y, al mismo tiempo, que el sistema redistribuya los datos cuando sea más seguro hacerlo sin un impacto apreciable.

Pero incluso con las ventanas de balanceo configuradas, conviene estar atento. El balanceador puede actuar en función del tamaño de los datos, no de la frecuencia de acceso. Esto significa que podría no detectar fragmentos activos que generan más tráfico, incluso si contienen menos documentos. Revisar periódicamente la distribución, no solo de los datos, sino también de la carga, es esencial para evitar interrupciones silenciosas.

degradación del rendimiento. El `balancerStatus` y Los comandos ,
`currentOp` y `sh.status()` son tus amigos aquí, así como el monitoreo

Gráficos que muestran el número de operaciones en cada fragmento. El último

Lo que hay que recordar es que el balanceador es excelente para mover objetos pequeños.
fragmentos de datos para restablecer el equilibrio en un estado algo desequilibrado

Colección fragmentada. Cuando se necesita que se llenen fragmentos completos.

Con los datos, la fragmentación puede ser más rápida.

Pre-división: si, cuándo y cómo

La mayoría de las veces, el balanceador de MongoDB hace un gran trabajo al reaccionar a
aumentando los datos y manteniéndolos distribuidos. Pero en algunos escenarios, como
Como se trata de grandes importaciones iniciales, no desea insertar todo en una sola
Fragmentar y esperar a que el balanceador se ponga al día. Aquí es donde entra en
juego la predivisión.

La división previa depende del rango de valores de la clave de su fragmento. Necesitará

Cree un script que divida el rango completo de valores de clave de fragmento en

varios trozos y los distribuye uniformemente entre los fragmentos que

recibirá nuevos datos mediante el comando `moveRange` . Si su fragmento...

El valor de la clave es una cadena; utilice rangos como `MinKey-k` o `ks` si , y `s-`

`MaxKey` , desea comenzar con fragmentos en tres fragmentos. Si su

los valores son de otros tipos, calcule el rango aproximado de valores

en su conjunto de datos y luego divídalos según corresponda. Recuerde que

No es necesario ser muy preciso, ya que el equilibrador lo manejará fácilmente.

Desequilibrios más pequeños una vez que se hayan cargado suficientes datos. Lo que

Lo que queremos evitar es que todos los datos vayan a un único fragmento al mismo tiempo.

comienzo.

Generalmente, conviene usar la predivisión antes de una importación masiva a una nueva colección vacía, o durante la fase inicial de configuración de un nuevo clúster. La predivisión es más sencilla cuando se comprende de antemano cómo se distribuyen los datos y se quiere empezar con buen pie.

La predivisión es proactiva. Cambia tu mentalidad de la gestión reactiva de la carga a la planificación deliberada de datos y, en las manos adecuadas, es una de las maneras más eficaces de garantizar que tu clúster arranque y se mantenga en buen estado.

Mover colecciones no fragmentadas No todas las colecciones

de su base de datos estarán fragmentadas. De hecho, todas las colecciones más pequeñas estarán fragmentadas, y solo las colecciones demasiado grandes para caber en un fragmento deberían estarlo. Al igual que no todas las colecciones fragmentadas necesitan vivir en todos los fragmentos, las colecciones no fragmentadas no necesitan vivir juntas en un fragmento principal. De hecho, normalmente sería mejor distribuir las equitativamente entre sus fragmentos, porque incluso los datos no fragmentados pueden causar problemas si hacen que la distribución de su clúster sea desigual. Imagine pequeñas colecciones diarias para eventos en los que cada día vuelca los datos en una colección no fragmentada separada (y eventualmente descarta las antiguas). Si todos los días están en el mismo fragmento, ese fragmento eventualmente se verá abrumado, no por la fragmentación, sino por un desequilibrio de las colecciones no fragmentadas.

En estas situaciones, el comando `moveCollection` (compatible a partir de la versión 8.0 de MongoDB) debería ser su herramienta preferida. Permite mover manualmente una colección completa sin particionar de un fragmento a otro. No es automático y bloquea brevemente las escrituras durante el...

mover si lo estás ejecutando en una colección activa (generalmente solo por un tiempo)

unos pocos segundos), pero puede ser un salvavidas a la hora de reequilibrar las cargas de trabajo

Sin pasar por la complejidad de fragmentar una colección que

No es necesario fragmentarlo. Como beneficio adicional, si anticipas esto y

Ejecute `moveCollection` justo al principio cuando lo cree por primera vez

La colección, mover una colección vacía será rápido y de bajo costo.

costo.

En la Figura 6.8, , originalmente, las colecciones AH no fragmentadas en la base de datos de pruebas

todas las bases de datos están en el fragmento 0. Podemos ejecutar los siguientes comandos para...

Mueva algunos de ellos al fragmento 1 para distribuir los datos en este clúster:

```
// Mover fragmentos específicos de colecciones no fragmentadas
// Útil para equilibrar datos no fragmentados en el clúster al otro lado de
db.adminCommand({moverCollection:"testdb.E", aShard:"shard
db.adminCommand({moverColección:"testdb.F", aFragmento:"fragmento
db.adminCommand({moverColección:"testdb.G", aFragmento:"fragmento
db.adminCommand({moverColección:"testdb.H", aFragmento:"fragmento
```

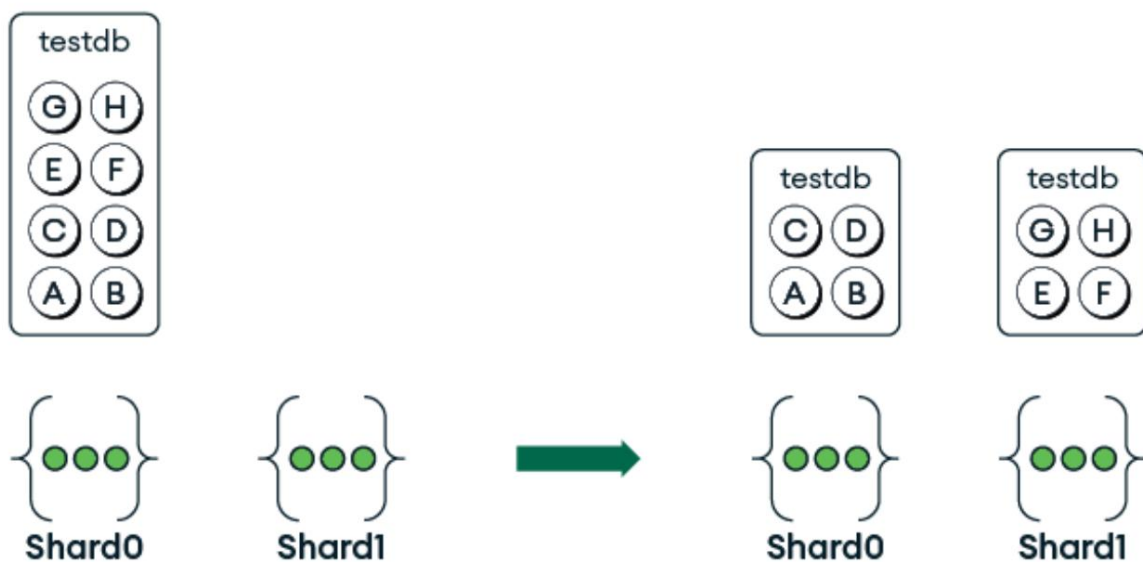


Figura 6.8: Mover colecciones no fragmentadas

Colocación de fragmentos de colecciones fragmentadas juntos

Como se mencionó en la sección Fragmentación basada en zonas , hay dos casos en los que puede ser beneficioso para el rendimiento desactivar la balanceador y colocar manualmente (mediante scripting) dos o más colecciones en los mismos fragmentos con rangos de claves de fragmento que mantienen datos relacionados juntos. El primer caso en el que puede ser de gran ayuda es transacciones de múltiples documentos, donde las transacciones de un solo fragmento son significativamente más eficientes que los de múltiples fragmentos, y el otro es agregaciones con operaciones \$lookup entre dos colecciones, donde mantener rangos relacionados en los mismos fragmentos permite la canalización ejecutar localmente en lugar de tener que enviar datos a través de la red a otro fragmento.

Un ejemplo de ello es el caso del comercio de referencia TPC-C. datos, que tienen colecciones para almacenes, distritos, clientes y pedidos. Podemos usar la fragmentación de rango en cada una de estas colecciones, lo que compartir prefijos para localizar los mismos valores en los mismos fragmentos:

Clave de fragmento de colección	
ALMACÉN w_id	
DISTRITO w_id	, hizo
CLIENTE w_id	, hizo , c_id
PEDIDOS	w_id , hizo , c_id , o_id

Tabla 6.2: Colecciones y sus claves de fragmento para el punto de referencia TPC-C

Para lograr la colocación, deshabilitaríamos el balanceador para estas colecciones, luego divídalas en los mismos valores de prefijo (`w_id`) y mover los rangos de fragmentos con los mismos valores a los mismos fragmentos. Alternativamente, podemos asociar los rangos de zona en el mismo prefijo (`w_id`) valores con la misma zona usando los siguientes comandos, y deje que el balanceador divida y mueva los rangos de fragmentos en consecuencia:

```
// Colocación conjunta de fragmentos de recopilación fragmentados para TPC-C
sh.shardCollection("db.ALMÉN", {w_id:1})
sh.shardCollection("db.DISTRITO", {w_id:1, d_id:1})
sh.shardCollection("db.CLIENTE", {w_id:1, d_id:1, c_id:1})
sh.shardCollection("db.ÓRDENES", {w_id:1, d_id:1, c_id:1, o_id:1})
// Asociar cada fragmento // a a zona única
sh.addShardToZone("fragmentoA", "ZonaA")
sh.addShardToZone("fragmentoB", "ZonaB")
sh.addShardToZone("fragmentoC", "ZonaC")
// Para cada colección, asociarla con la misma
sh.addTagRange("db.ALMÉN", {w_id:MinKey}, {w_id:33}, "Zona A")
sh.addTagRange("db.DISTRITO", {w_id:MinKey, d_id:MinKey}, {w_id:33, d_id:MinKey}, "Zona A")
sh.addTagRange("db.CLIENTE", {w_id:MinKey, d_id:MinKey, c_id:MinKey}, {w_id:33, d_id:MinKey, c_id:MinKey}, "Zona A")
sh.addTagRange("db.ÓRDENES", {w_id:MinKey, d_id:MinKey, c_id:MinKey, o_id:MinKey}, {w_id:33, d_id:MinKey, c_id:MinKey, o_id:MinKey}, "Zona A")
sh.addTagRange("db.ALMÉN", {w_id:33}, {w_id:66}, "Zona B")
sh.addTagRange("db.DISTRITO", {w_id:33, d_id:MinKey}, {w_id:66, d_id:MinKey}, "Zona B")
sh.addTagRange("db.CLIENTE", {w_id:33, d_id:MinKey, c_id:MinKey}, {w_id:66, d_id:MinKey, c_id:MinKey}, "Zona B")
sh.addTagRange("db.ÓRDENES", {w_id:33, d_id:MinKey, c_id:MinKey, o_id:MinKey}, {w_id:66, d_id:MinKey, c_id:MinKey, o_id:MinKey}, "Zona B")
sh.addTagRange("db.ALMÉN", {w_id:66}, {w_id:MaxKey}, "Zona C")
sh.addTagRange("db.DISTRITO", {w_id:66, d_id:MinKey}, {w_id:MaxKey, d_id:MinKey}, "Zona C")
sh.addTagRange("db.CLIENTE", {w_id:66, d_id:MinKey, c_id:MinKey}, {w_id:MaxKey, d_id:MinKey, c_id:MinKey}, "Zona C")
sh.addTagRange("db.ÓRDENES", {w_id:66, d_id:MinKey, c_id:MinKey, o_id:MinKey}, {w_id:MaxKey, d_id:MinKey, c_id:MinKey, o_id:MinKey}, "Zona C")
```

La figura 6.9 muestra que después de definir los rangos de zona, el balanceador divide y mueve los fragmentos al fragmento correspondiente asociado

con la zona.



Figura 6.9: Los mismos rangos para diferentes colecciones en el mismo fragmento

Si bien esto puede parecer complejo a nivel operativo, en realidad sería un script y, después de probarlo en un entorno de prueba, el proceso en producción sería simplemente ejecutar el script para inicializar la distribución de datos automáticamente.

Al particionar los mismos valores de ID de almacén en los mismos fragmentos, maximizamos la probabilidad de que las transacciones que involucren el mismo

Los clientes, pedidos, distritos y almacenes se gestionarán en un único fragmento.

De la misma forma, minimizaríamos la búsqueda entre fragmentos para las consultas de agregación que necesitan buscar los detalles de entidades relacionadas.

Al igual que cada decisión sobre su esquema e índices de MongoDB depende de sus requisitos y de las compensaciones que pueda hacer, también depende de la decisión de fragmentar para mejorar el rendimiento y la escalabilidad de su clúster. Seleccionar la clave de fragmentación teniendo en cuenta sus SLA es fundamental para garantizar que esté optimizando lo más importante. Normalmente no hay una solución perfecta, solo compensaciones. La clave

Es comprender los patrones de acceso dominantes de su aplicación. ¿Lee los datos por entidad o los escribe en bloque? ¿Prevé que algunas entidades dominen la carga de trabajo? El equilibrio es posible, pero comienza con el modelado intencional y continúa con la monitorización, la iteración y, a veces... sí, la redistribución.

Resumen

La arquitectura de fragmentación de MongoDB es una herramienta potente y flexible para lograr escalabilidad horizontal en implementaciones con conjuntos de datos masivos y requisitos de rendimiento exigentes. Al comprender los componentes principales de un clúster fragmentado, la importancia de la selección de claves de fragmentación y la variedad de métodos de fragmentación disponibles, puede diseñar un sistema que distribuya y dirija las operaciones de forma eficiente.

Además, las técnicas avanzadas analizadas, como el refinamiento de claves de fragmentos, el uso de zonificación para la ubicación estratégica de datos, la predivisión durante las importaciones masivas y el equilibrio de cargas de trabajo entre fragmentos, contribuyen a mantener un clúster eficiente y con capacidad de respuesta a medida que sus datos y cargas de trabajo escalan. Una fragmentación eficaz no se limita a distribuir datos, sino también a anticipar y gestionar los matices de los patrones de consulta, la sobrecarga operativa y las necesidades cambiantes.

Con este conocimiento, estará bien posicionado para crear arquitecturas escalables y robustas que puedan soportar las crecientes demandas de su aplicación.

En el próximo capítulo sobre motores de almacenamiento, cambiaremos nuestro enfoque de la distribución de datos al almacenamiento de datos, profundizando en el motor de almacenamiento WiredTiger, el motor de almacenamiento predeterminado y más comúnmente utilizado para MongoDB.

7

Motores de almacenamiento

Como vimos en [Capítulo 1](#) [Sistemas y arquitectura MongoDB](#), WiredTiger es un componente del servidor MongoDB y el motor de almacenamiento más común para MongoDB. Los motores de almacenamiento actúan como interfaz entre la estructura lógica de los datos (tal como los ven los usuarios, las aplicaciones y las capas superiores de la base de datos) y el almacenamiento físico de dichos datos. Por lo tanto, son un factor crucial para diagnosticar y resolver problemas de rendimiento. Este capítulo tiene como objetivo brindarle una comprensión profunda del motor de almacenamiento WiredTiger, permitiéndole tomar decisiones informadas sobre su configuración. Al finalizar este capítulo, comprenderá a fondo el funcionamiento interno de WiredTiger en cuanto al rendimiento y los conocimientos necesarios para optimizarlo para lograr una mayor eficiencia.

Los temas tratados en este capítulo incluyen los siguientes:

- Una descripción general de los motores de almacenamiento y su función en MongoDB
- La arquitectura de WiredTiger, incluido su formato de almacenamiento, control de concurrencia y estrategias de almacenamiento en caché
- Cómo WiredTiger administra su caché, garantiza la durabilidad de los datos y gestiona la recuperación ante fallos inesperados
- Los algoritmos de compresión compatibles con WiredTiger y las compensaciones entre la eficiencia de la compresión y el uso de recursos

- Opciones de configuración clave para WiredTiger y cómo ajustarlas para su carga de trabajo específica y configuración de hardware

Explorando los motores de almacenamiento

Diferentes motores de almacenamiento emplean diversas estructuras de datos, algoritmos y técnicas para optimizar el rendimiento, la escalabilidad, la confiabilidad y otras características del sistema de base de datos.

Algunos tipos comunes de motores de almacenamiento incluyen los siguientes:

- Motor de almacenamiento orientado a filas : estos motores almacenan datos en filas, donde cada fila representa un registro y contiene múltiples columnas o campos.
- Motor de almacenamiento orientado a columnas : estos motores almacenan datos en columnas, donde cada columna representa un campo y contiene valores de varias filas.
- Motor de almacenamiento clave-valor : Estos motores almacenan datos como conjuntos de pares clave-valor, donde cada clave única está asociada a un valor correspondiente. La aplicación puede acceder a un valor mediante su clave. Por ejemplo, una clave podría ser una cadena o un ID de usuario, y el valor podría ser un conjunto complejo de datos anidados.

WiredTiger es un motor de almacenamiento de clave-valor que se puede configurar para almacenar datos en filas o columnas. Otros ejemplos populares de motores de almacenamiento de bases de datos son InnoDB, MyISAM y RocksDB. La elección del motor de almacenamiento puede afectar significativamente el rendimiento, la escalabilidad y la funcionalidad de un sistema de bases de datos. Los distintos motores de almacenamiento están optimizados para distintas cargas de trabajo y casos de uso, y pueden ofrecer ventajas y desventajas.

diferencias entre diversos factores, como el rendimiento de lectura/escritura, la compresión de datos, el soporte de transacciones y la consistencia de los datos.

Por ejemplo, InnoDB es actualmente el motor de almacenamiento predeterminado para MySQL. Ofrece funciones como transacciones, bloqueo a nivel de fila y compatibilidad con claves foráneas. MyISAM, un motor de almacenamiento más antiguo, es conocido por su simplicidad y velocidad para operaciones de lectura intensiva, pero no admite transacciones ni bloqueo a nivel de fila.

En teoría, WiredTiger podría usarse como motor de almacenamiento para otras bases de datos, y MongoDB podría usar otros motores de almacenamiento. MongoDB ha definido una API de motor de almacenamiento que permite a los desarrolladores implementar adaptadores para otros motores de almacenamiento. Por ejemplo, MongoRocks es un módulo de motor de almacenamiento de RocksDB para MongoDB.

Una base de datos específica podría usar solo un subconjunto de las funciones de un motor de almacenamiento y permitir a los usuarios configurar un subconjunto de sus parámetros de configuración. Por ejemplo, al momento de escribir este artículo, MongoDB no utiliza funciones como el almacenamiento en columnas de WiredTiger ni estructuras de datos como árboles de fusión con estructura logarítmica. Esto se debe a que MongoDB no implementa actualmente funciones que utilicen estas capacidades, pero podría hacerlo en el futuro. Las opciones de configuración del servidor MongoDB admiten un subconjunto de las opciones de configuración de WiredTiger.

Descripción general de WiredTiger. WiredTiger es el motor de almacenamiento predeterminado y más utilizado para MongoDB. Está disponible desde la versión 3.0 de MongoDB.

Antes de eso, el motor de almacenamiento predeterminado para MongoDB era MMAPv1, que utilizaba archivos mapeados en memoria para gestionar los datos. Si bien MMAPv1 era funcional, presentaba algunas limitaciones.

La estructura de datos fundamental de WiredTiger es el árbol B, que mantiene los datos ordenados y permite la búsqueda, el acceso secuencial, las inserciones y las eliminaciones en tiempo logarítmico. Cada nodo de un árbol B puede contener múltiples claves (entradas de datos), y la cantidad de claves que un nodo puede contener está determinada por el grado u orden del árbol. La búsqueda de una clave en un árbol B implica navegar desde la raíz hasta una hoja, siguiendo los punteros que representan rangos de claves. Esto es eficiente porque la altura del árbol es logarítmica en relación con el número de claves. En comparación con los árboles de búsqueda binarios, los árboles B son menos profundos porque permiten más de dos hijos por nodo. Esto reduce el número de accesos a disco necesarios al recorrer el árbol, lo cual es crucial para bases de datos donde las operaciones de E/S son costosas.

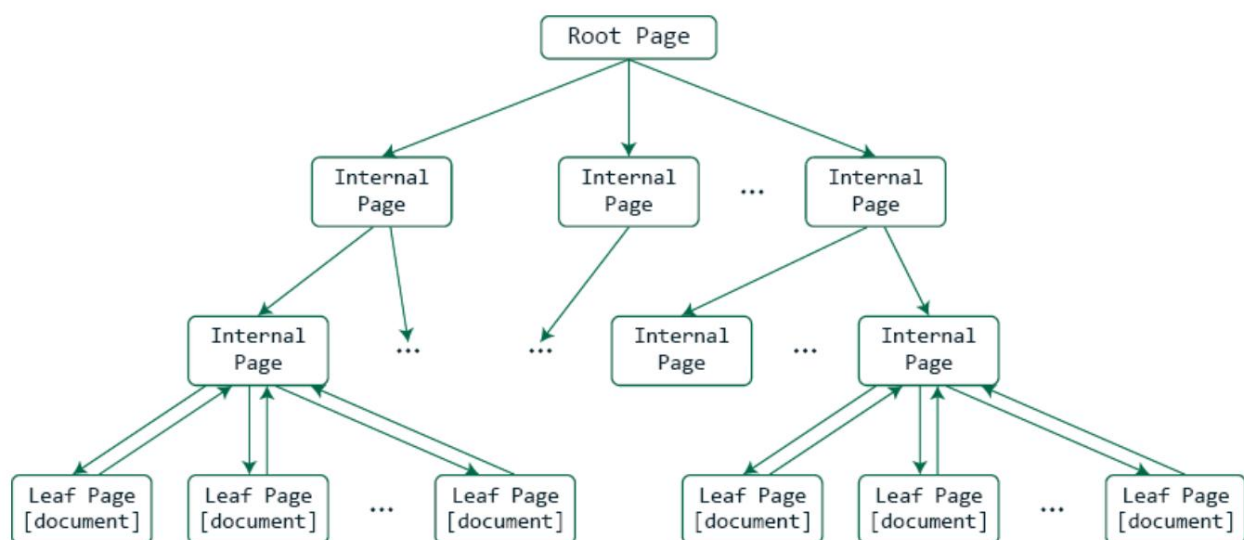


Figura 7.1: El árbol B de WiredTiger

Cada nodo del árbol B contiene claves y valores. Los nodos hoja almacenan datos reales, mientras que los nodos internos guían el proceso de búsqueda a través del árbol. El componente fundamental de WiredTiger es el par clave-valor. Múltiples pares clave-valor se pueden combinar en páginas.

Cada dato está asociado a una clave única, lo que permite una recuperación y actualización eficientes. Las claves internas se denominan ID de registro.



Páginas en el kernel de Linux versus el motor de almacenamiento WiredTiger

El concepto de páginas existe tanto en el kernel de Linux como en el motor de almacenamiento WiredTiger. Si bien el término página se utiliza en ambos, representa conceptos distintos en cada uno.

Una página en el kernel de Linux es la unidad fundamental de gestión de memoria.

Por ejemplo, al intentar recuperar un documento con un campo indexado específico, como `_id`, primero se busca en un árbol B de índices utilizando la clave `_id` para obtener el valor del ID del registro. A continuación, este ID del registro se utiliza como clave para buscar en la colección y devolver el valor correspondiente a un documento de MongoDB.

WiredTiger emplea el control de concurrencia multiversión (MVCC) para gestionar el acceso concurrente a los datos en árboles B y páginas. Esto permite que múltiples lectores y escritores operen simultáneamente, ofreciendo mayor concurrencia que los esquemas más simples.

Los datos se almacenan de forma persistente en archivos que contienen las estructuras de árbol B junto con los metadatos. Estos archivos pueden crecer dinámicamente según sea necesario. En el disco, encontrará un archivo separado para cada índice y colección de MongoDB.

Otro conjunto de archivos WiredTiger se utiliza para almacenar el diario , que a menudo se denomina registro de escritura anticipada (WAL) en otros sistemas de bases de datos. Cada operación que modifica la base de datos se escribe primero en la

diario para garantizar la durabilidad de las escrituras, en caso de que el servidor falle o pierda energía.

Al acceder por primera vez a una colección o índice, la raíz y la primera capa de nodos internos se cargan en una caché. A medida que se realizan operaciones de lectura y escritura, la caché en memoria se llena con páginas según demanda. De forma predeterminada, la caché de WiredTiger utiliza el 50 % de la memoria disponible menos 1 GB. Por ejemplo, en un sistema con un total de 8 GB de RAM, la caché de WiredTiger utilizaría 3,5 GB de RAM ($0,5 * (8 \text{ GB} - 1 \text{ GB}) = 3,5 \text{ GB}$).

La implementación actual no permite la fijación explícita de documentos o colecciones en la caché. La caché se comparte entre todos.

Bases de datos y colecciones en un servidor MongoDB. WiredTiger crea periódicamente puntos de control consistentes que capturan una instantánea persistente de la base de datos en un momento específico.

WiredTiger es totalmente compatible con transacciones ACID, lo que garantiza la consistencia e integridad de los datos incluso ante fallos. Las siguientes secciones integrarán todos estos conceptos describiendo las operaciones de búsqueda, actualización e inserción.

Una operación de búsqueda

Cuando una aplicación ejecuta una consulta en la base de datos, esta se traduce en una o más operaciones de búsqueda del motor de almacenamiento, cada una de las cuales hace lo siguiente:

1. Suponiendo que estamos leyendo un documento identificado por un campo indexado, WiredTiger recorre el árbol B del índice desde la raíz hasta la hoja que contiene la clave.

2. Si alguna página no está en la caché, se carga desde el almacenamiento tanto en la caché del sistema de archivos del sistema operativo como en la caché de WiredTiger. Finalmente, WiredTiger encuentra el ID del registro correspondiente al campo indexado.
3. El ID de registro del paso anterior se utiliza para recorrer el Árbol B de la colección.
4. Si alguna página no está en la memoria caché, se carga desde el almacenamiento tanto en la memoria caché del sistema de archivos del sistema operativo como en la memoria caché de WiredTiger, donde se descifran y descomprimen.
5. Cuando WiredTiger encuentra el ID del registro, devuelve el documento de la página.

Esta operación garantiza que las páginas se utilicen desde la memoria si están en el caché y evita E/S de almacenamiento innecesarias.

Una operación de actualización. Cuando la

aplicación actualiza un documento, la base de datos lo traduce en una operación de actualización en el motor de almacenamiento. Esto es lo que suele ocurrir:

1. El documento se lee en caché como se describe en el punto anterior. sección.
2. La página está marcada como sucia y los nuevos valores del documento se almacenan en una lista de actualización en la memoria.
3. Si la escritura es parte de una transacción, espera hasta que se complete la transacción. comprometido.
4. Se escribe una entrada en el registro de WiredTiger con suficiente información para rehacer la operación en caso de fallo. Esta escritura del registro se vacía en el dispositivo de almacenamiento.

5. Más tarde, cuando se ejecuta un punto de control o se necesita volver a cargar la página,

Una vez expulsada de la caché, WiredTiger revisa la lista de actualizaciones de la página y la guarda en memoria. Las versiones anteriores del documento se guardan en un archivo de historial aparte.

Este mecanismo proporciona durabilidad, recuperación ante fallos y seguimiento de versiones, todos ellos esenciales para operaciones de base de datos confiables.

Una operación de inserción

Para las operaciones de inserción, WiredTiger realiza lo siguiente:

1. Recorre el árbol B para encontrar el nodo hoja apropiado para insertar una clave para el registro/documento.
2. Al escribir el nodo en el almacenamiento, si está lleno, se divide. La operación de división puede revertirse en cascada hacia arriba en el árbol si el nodo principal está lleno.

Al igual que las lecturas y actualizaciones, las inserciones se basan en un recorrido eficiente del árbol B y en la gestión dinámica de nodos para mantener el rendimiento y la integridad estructural de los datos. Todas las operaciones con los datos implican el recorrido de las estructuras de datos del árbol B de los índices y los datos de la colección.

Expulsión, puntos de control y recuperación. La caché tiene un tamaño limitado, por lo que WiredTiger necesita expulsar páginas de la caché cuando necesita espacio para datos más recientes. WiredTiger utiliza subprocesos en segundo plano para gestionar el proceso de expulsión. Estos subprocesos evalúan continuamente el estado de la caché y realizan tareas de expulsión, ejecutándose junto con otras operaciones de la base de datos.

Las páginas se seleccionan para su eliminación según una variación del algoritmo de uso menos reciente , donde las páginas con accesos más recientes son candidatas para la eliminación. Esto significa que las páginas con acceso frecuente deben permanecer en memoria/caché durante más tiempo.

WiredTiger rastrea tres tipos de datos de caché: datos totales, datos sucios y actualizaciones. Para cada uno, cuenta con dos umbrales para gestionar la expulsión de la caché:

- Objetivo : cuando el caché alcanza este nivel, WiredTiger comienza actividades de desalojo para reducir gradualmente el uso del caché.
- Disparador : Si se supera este nivel, WiredTiger activa los subprocesos de operaciones de la base de datos para liberar espacio rápidamente y evitar que la caché se llene. Esto reduce la cantidad de subprocesos que pueden ejecutar operaciones de base de datos simultáneamente.

Estos umbrales se pueden configurar al ejecutar MongoDB autogestionado. Veremos ejemplos más adelante en la sección "Configuración para mejorar el rendimiento" de este capítulo. Al usar Atlas, MongoDB selecciona estos umbrales según el tamaño de la instancia que se esté ejecutando.

Un punto de control toma todas las estructuras de datos en memoria y escribe cualquier modificación en el almacenamiento. Se puede configurar la frecuencia con la que se crean los puntos de control. Veremos un ejemplo más adelante en este capítulo. El resto del servidor de bases de datos continúa leyendo y escribiendo páginas durante el proceso del punto de control.

El proceso del punto de control implica los siguientes pasos:

1. WiredTiger garantiza que todas las transacciones que se han confirmado hasta ese momento se almacenen de forma duradera en la imagen de disco de la base de datos escribiendo todas las páginas sucias desde el caché.

2. Se eliminan todas las escrituras pendientes en los archivos.
3. Se rotan los archivos de diario que contienen los registros de transacciones.
4. WiredTiger marca el final del punto de control en sus registros y metadatos.

Esto permite una recuperación segura a partir de este punto en adelante.

Desde una perspectiva de rendimiento, si se escribe un gran volumen de datos entre dos puntos de control, muchas operaciones de almacenamiento y archivos se realizarán durante el segundo. En sistemas con limitaciones de IOPS o ancho de banda, esto puede provocar caídas periódicas del rendimiento.

Se requiere una recuperación si el equipo que ejecuta el servidor MongoDB falla o se queda sin energía sin apagarse correctamente. Al reiniciarse el servidor, comienza el proceso de recuperación cargando primero el estado de la base de datos desde el último punto de control completado. Si el apagado anterior no fue correcto, los cambios adicionales almacenados en el registro de WiredTiger (journal) se reproducen sobre el último punto de control. Por lo tanto, cuanto mayor sea el intervalo entre puntos de control, mayor será el tiempo de recuperación.

Compresión y cifrado. Si bien la compresión ayuda a ahorrar espacio de almacenamiento, implica un mayor uso de la CPU. Si su carga de trabajo ya está limitada por el uso de la CPU en un servidor con MongoDB, puede desactivar la compresión en cualquiera de los siguientes casos:

- No le preocupa el tamaño de los archivos en sus dispositivos de almacenamiento
- Los datos que estás almacenando no son muy comprimibles

Como alternativa, puede cambiar a un algoritmo de compresión diferente para reducir el uso de la CPU, aunque esto suele resultar en archivos de mayor tamaño debido a una compresión menos efectiva. MongoDB actualmente admite tres opciones de compresión:

Algoritmo de compresión	UPC uso	Eficiencia de compresión	Notas
Rápido	Más bajo	Más bajo	Opción más rápida con un costo mínimo de CPU; compresión menos efectiva
zstd	Medio	Mejor que Rápido	Una elección equilibrada entre la velocidad y la relación de compresión
zlib	Lo mejor de lo mejor		Más lento, pero ofrece la mayor compresión.

Tabla 7.1: Algoritmos de compresión y sus características de rendimiento

La opción de compresión principal se utiliza para comprimir tanto los datos del diario como los de la colección. Los índices utilizan una forma de compresión de prefijo. Esto es eficaz para datos con campos indexados que tienen una inicial común. secuencias.

Como se mencionó anteriormente, cada carga de trabajo y conjunto de datos es único, por lo que es importante realizar sus propios experimentos para determinar la mejor configuración. Por ejemplo, si usa zlib con un...

Conjunto de datos grande y comprimible. Cada lectura de almacenamiento puede expandirse a más páginas dentro de la caché de WiredTiger. Esto puede provocar un aumento de expulsiones y, potencialmente, más errores de caché al releer o actualizar documentos.

MongoDB Enterprise utiliza algoritmos estándar de la industria, como el Estándar de Cifrado Avanzado (AES), para el cifrado de datos en reposo, específicamente AES-256 en modo de Encadenamiento de Bloques de Cifrado (CBC) configurado con relleno PKCS#7. MongoDB Atlas utiliza el cifrado en reposo de forma predeterminada para todos los datos almacenados en sus clústeres. Esto significa que cualquier persona con acceso físico a los dispositivos de almacenamiento no puede leer los datos. Los procesadores modernos cuentan con aceleración de hardware para estos algoritmos, lo que minimiza la sobrecarga de seguridad.

Si usted mismo administra su implementación de MongoDB y sospecha que no se está utilizando la aceleración de hardware AES, puede confirmarlo creando un perfil de MongoDB y analizando gráficos de llama.

Configuración para mejorar el rendimiento

Esta sección describe varias opciones de configuración del motor de almacenamiento WiredTiger que pueden ayudar a mejorar el rendimiento. En general, Atlas no permite la manipulación directa de muchas configuraciones de WiredTiger. Las administra y optimiza automáticamente según los niveles de la instancia. Gracias a la simplicidad y eficiencia de Atlas, puede confiar en su automatización avanzada para optimizar el rendimiento según la demanda. Si no se satisfacen sus necesidades específicas, puede consultar con el soporte de MongoDB para obtener orientación adaptada a su aplicación.

Sin embargo, si ejecuta un clúster MongoDB autogestionado, puede configurar y experimentar con la configuración de WiredTiger. Al iniciar el servidor MongoDB, se seleccionarán los valores predeterminados. Como puede imaginar, es imposible para MongoDB seleccionar valores predeterminados óptimos para todos los conjuntos de datos y cargas de trabajo. Si sabe que WiredTiger es el factor limitante actual de su carga de trabajo, aquí tiene algunos experimentos que puede realizar para diagnosticar y mejorar el rendimiento.

Cambiar el tamaño de la caché de WiredTiger

Si sabe que su carga de trabajo está limitada por la E/S, puede aumentar el tamaño de la caché de WiredTiger para reducir la cantidad de E/S necesaria. Por ejemplo, si ejecuta MongoDB en una instancia en la nube, puede usar los paneles de métricas del proveedor de la nube para ver cuántas IOPS utiliza su carga de trabajo. Con 4200 IOPS aprovisionadas, un conjunto de datos mayor que la caché y una carga de trabajo compuesta por un 95 % de lecturas y un 5 % de actualizaciones, podría observar métricas de estado estable como las siguientes:

- 3500 solicitudes de lectura de almacenamiento por
- segundo 700 escrituras de almacenamiento por segundo (incluidos archivos de diario/registro y de
- datos) 15 000 consultas de lectura de base de datos por
- segundo 800 operaciones de actualización de base de datos por segundo

Aumentar el tamaño de la memoria caché de WiredTiger puede reducir la cantidad de operaciones de E/S necesarias por operación de base de datos, por lo que, si bien la carga de trabajo todavía está limitada a 4200 operaciones de almacenamiento por segundo, se pueden ejecutar más operaciones de base de datos porque acceden a los datos en la memoria caché en lugar de recargarlos desde el almacenamiento.

Para ajustar el tamaño de la caché a 20 GB, puede utilizar el siguiente comando:

```
db.adminCommand({establecer parámetro: 1, wiredTigerEngineRuntimeC
```

Este comando se puede utilizar en una instancia de MongoDB en ejecución desde el shell.

Sin embargo, también hay situaciones en las que reducir el tamaño de la caché puede mejorar el rendimiento. Por ejemplo, si tiene una carga de trabajo con un alto consumo de lectura que accede a documentos aleatorios y comprimibles en un conjunto de datos mayor que la caché, reducir el tamaño de la caché de WiredTiger permite obtener más espacio para la caché del sistema de archivos, donde los datos permanecen comprimidos. Es más probable que cada lectura aleatoria no alcance la caché de WiredTiger, pero sí la del sistema de archivos, lo que reduce el número de operaciones de E/S y aumenta el número de operaciones de base de datos por segundo.

Por ejemplo, en algunos casos, reducir el tamaño de la caché de WiredTiger tres veces puede mejorar el rendimiento de la base de datos, en operaciones por segundo, hasta en un 60 %. Sin embargo, este cambio también tiene sus desventajas, como la reducción del rendimiento de una carga de trabajo de inserción en casi un 20 %.

Cambiar el retardo de sincronización:

Reducir el retardo de sincronización aumenta la frecuencia de los puntos de control y resulta beneficioso cuando se necesitan operaciones de escritura en disco consistentes, más pequeñas y frecuentes para evitar picos de rendimiento. También reduce el tiempo de recuperación si un servidor falla y se reinicia.

Si tiene una carga de trabajo de escritura intensiva y nota caídas periódicas del rendimiento que coinciden con picos en las operaciones de almacenamiento, esto podría indicar que reducir el retraso de sincronización podría ayudar.

En la Figura 7.2, Vemos una ilustración de esta situación. Periódicamente, durante los puntos de control (marcados con C en el diagrama), las operaciones de E/S de almacenamiento aumentan y el número de operaciones de base de datos disminuye.

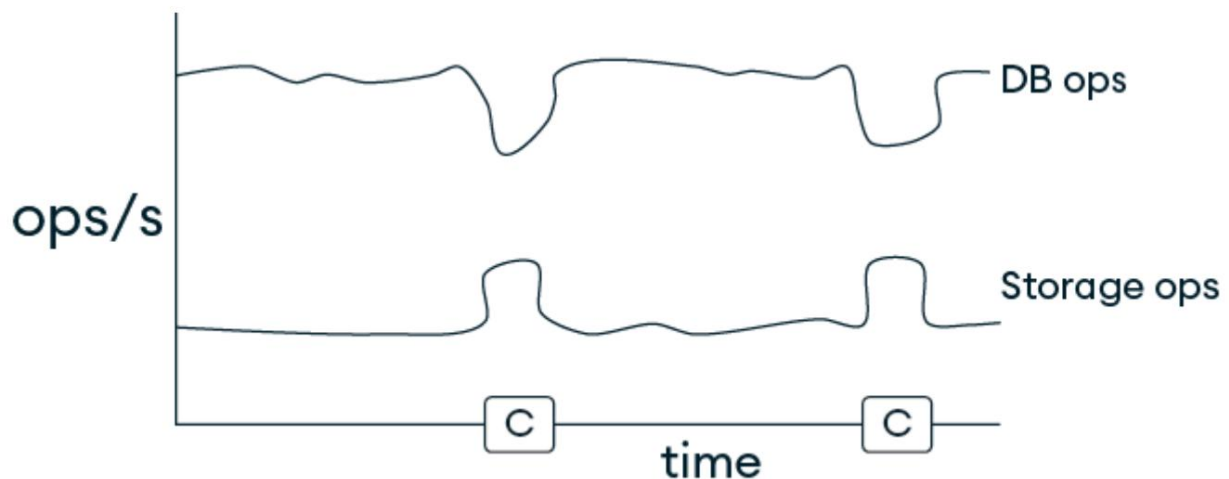


Figura 7.2: Caídas periódicas del rendimiento durante los puntos de control

Puede reducir el retardo de sincronización (en segundos) modificando el valor en el archivo de configuración. El valor predeterminado es 60 segundos. Si ya está en ejecución, deberá reiniciar el servidor MongoDB después de modificar el archivo de configuración:

```
almacenamiento:  
  retardo de sincronización: 30
```

Alternativamente, puede configurar syncdelay con una opción de línea de comando:

```
mongod --syncdelay 30
```

Esto debería cambiar los gráficos de rendimiento a algo similar a la Figura 7.3 . Las caídas de rendimiento y los picos de E/S no son tan pronunciados durante los puntos de control.

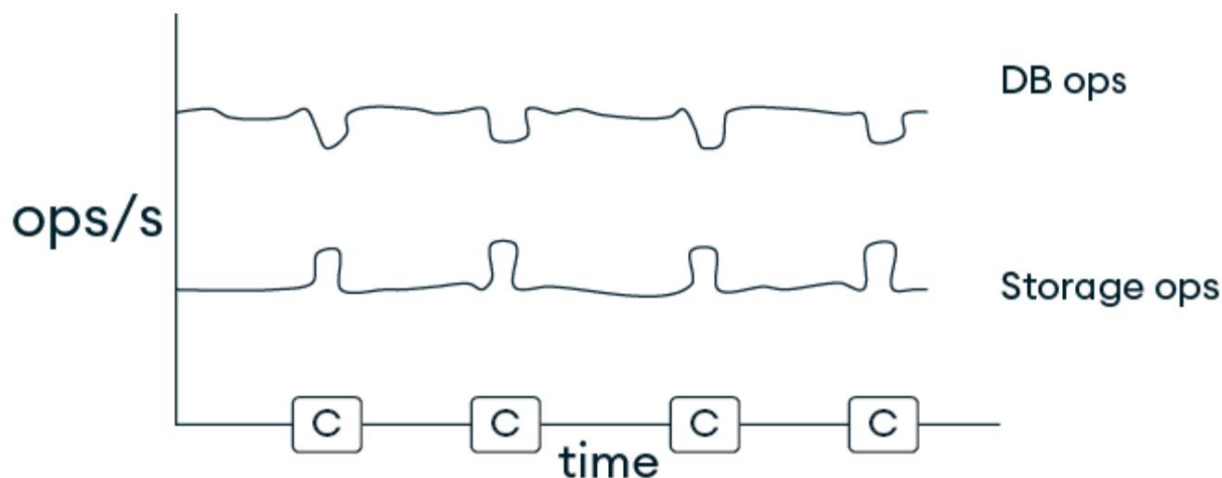


Figura 7.3: Caídas de rendimiento más pequeñas con puntos de control más frecuentes

Por otro lado, aumentar el retardo de sincronización puede ser beneficioso para cargas de trabajo con uso intensivo de escritura cuando se desea reducir el número total de operaciones de E/S de disco, especialmente durante el procesamiento masivo de datos. Por ejemplo, en cargas de trabajo por lotes nocturnas que actualizan muchos documentos varias veces, podrían requerirse menos operaciones de E/S si un documento se actualiza más de una vez entre puntos de control. Esto reduce el impacto de los puntos de control en la carga de trabajo principal y también puede reducir la sobrecarga derivada de operaciones menos frecuentes de gestión de transacciones y vaciado de almacenamiento.

Modificar minSnapshotHistoryWindowInSeconds

El parámetro `minSnapshotHistoryWindowInSeconds` en MongoDB especifica el intervalo de tiempo mínimo (en segundos) durante el cual el motor de almacenamiento conserva el historial de instantáneas. Un intervalo más amplio permite consultar el valor de un documento en un momento específico utilizando la información de lectura de instantáneas durante un período más largo.

Sin embargo, mantener un historial de instantáneas más largo, especialmente para documentos actualizados con frecuencia, requiere más almacenamiento y puede afectar el rendimiento de escritura, ya que el sistema debe rastrear más versiones históricas de los datos.

Este parámetro se puede configurar tanto en tiempo de ejecución con el comando `setParameter` como al inicio con la configuración `setParameter`. Por ejemplo, para configurar el parámetro a 5 segundos en tiempo de ejecución, puede usar lo siguiente:

```
db.adminCommand({establecerParámetro: 1, minSnapshotHistoryWindow
```



Evite modificar la ventana del historial de instantáneas en los nodos del servidor de configuración

En clústeres fragmentados, evite cambiar el valor predeterminado en los nodos del servidor de configuración, ya que esto puede provocar que las operaciones internas fallen.

Cambiando el funcionamiento del desalojo

Si bien Atlas no permite la manipulación directa de datos internos,

Las configuraciones de desalojo de WiredTiger, como `eviction_dirty_target` o `eviction_trigger`,

controlan cómo comienza el desalojo.

Gradualmente (objetivo) y luego utiliza subprocesos de operación de la base de datos (disparador) para reducir el uso de caché. Atlas administra y optimiza automáticamente estas configuraciones según el nivel de la instancia.

Al autogestionar MongoDB, es posible que observe caídas periódicas en el rendimiento, como se ilustra en la Figura 7.4 . Durante un punto de control, puede producirse un gran número de IOPS de escritura en el dispositivo de almacenamiento, que a veces superan la capacidad asignada, lo que resulta en una disminución en las operaciones de la base de datos por segundo.

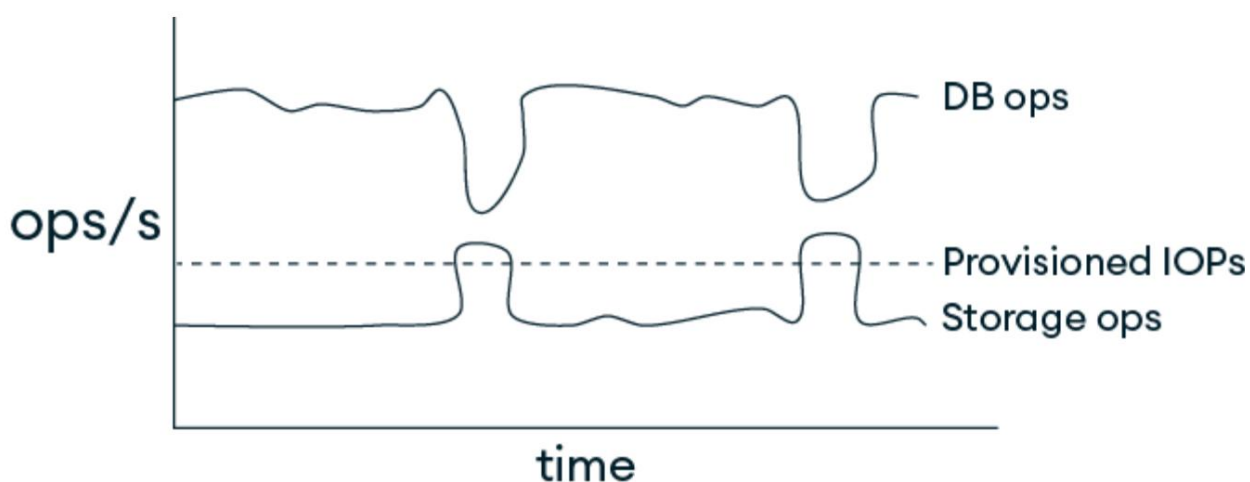


Figura 7.4: Caídas periódicas del rendimiento durante los puntos de control

Reducir `syncdelay` , como se mencionó anteriormente, puede ayudar a acortar el intervalo entre puntos de control, pero podría no ser suficiente para reducir los picos de almacenamiento. En su lugar, un enfoque más eficaz consiste en ajustar dos configuraciones de desalojo para limpiar y desalojar datos de la caché más rápidamente. Esto implica reducir `eviction_dirty_target` de su valor predeterminado del 5 % de la caché al 2 % y aumentar el número de subprocesos de trabajo de desalojo:

```
"almacenamiento":
{ "wiredTiger":
  { "engineConfig": {
```

```
"configString": "desalojo_sucio_objetivo=2, desalojo=( {} }
```

```
}
```

Ahora, cuando supervisa el rendimiento, verá una mayor cantidad de operaciones de almacenamiento por segundo, pero picos más pequeños durante los puntos de control, y el impacto en las operaciones de base de datos por segundo es menos pronunciado. Esto se debe a que hay más subprocesos de trabajo en ejecución y expulsando páginas sucias en un umbral más bajo antes del punto de control, por lo que es necesario escribir menos datos durante el punto de control.

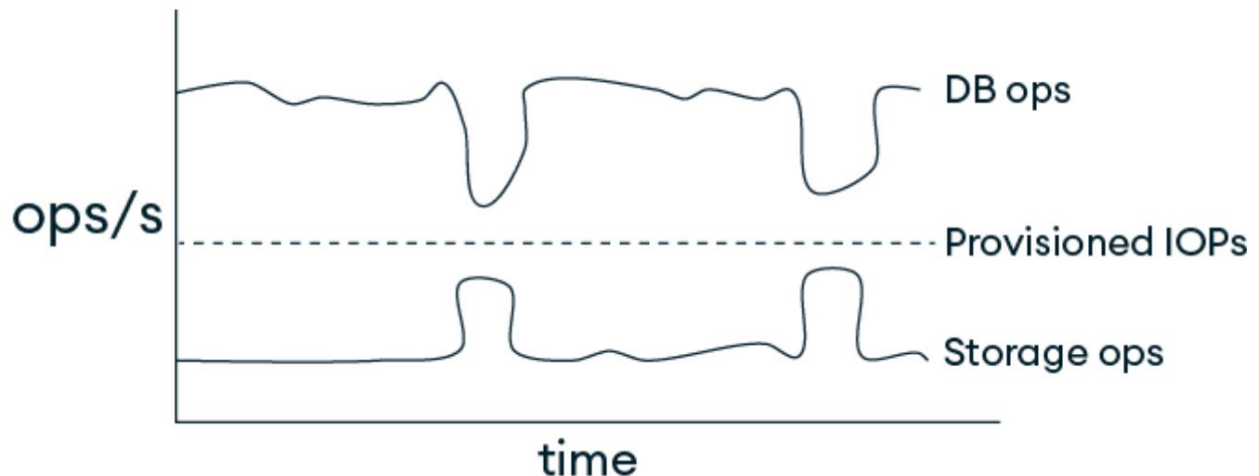


Figura 7.5: Caídas y picos menos pronunciados con cambios en las configuraciones de desalojo

Al igual que con muchas recomendaciones para optimizar el rendimiento, es importante experimentar con la carga de trabajo y el conjunto de datos. Por ejemplo, hemos visto cargas de trabajo con un gran número de operaciones de inserción donde aumentar `eviction_dirty_target` y reducir el número de subprocesos de trabajo de desalojo (hasta uno) puede mejorar significativamente el rendimiento.

Cambiar al motor de almacenamiento en memoria

El motor de almacenamiento en memoria permite evitar la E/S de disco, lo que proporciona un mayor rendimiento y una latencia más baja y predecible para las operaciones de la base de datos. Salvo algunos metadatos e información de diagnóstico, el motor de almacenamiento en memoria no escribe datos en el disco. De hecho, puede usarlo como caché, cargando y almacenando datos de otra base de datos.



Nota

El motor de almacenamiento en memoria solo está disponible en implementaciones autoadministradas y no es compatible con MongoDB Atlas.

De forma predeterminada, el motor de almacenamiento en memoria utiliza el 50 % de la RAM física menos 1 GB. Sin embargo, dado que los datos no se escriben en el disco (y no se necesita tanta caché del sistema de archivos), se puede aumentar el tamaño asignado para usar más memoria. Esto se puede configurar en el archivo de configuración (véase a continuación) o mediante la línea de comandos:

```
almacenamiento: motor:
  inMemory dbPath:
    <ruta>
    inMemory:
      engineConfig: inMemorySizeGB: <nuevoTamaño>
```

Las operaciones de escritura que hagan que los datos superen el tamaño de memoria especificado generarán un error. Si ejecuta una implementación autogestionada, puede configurar fragmentos o conjuntos de réplicas para que incluyan una combinación de motores de almacenamiento en memoria y WiredTiger.

Una configuración de motor de almacenamiento mixto dentro de un conjunto de réplicas debe diseñarse con cuidado, ya que introduce matices en la persistencia de datos, el comportamiento de conmutación por error y las características de rendimiento. Esta configuración suele ser adecuada para escenarios donde las ventajas del acceso rápido desde los miembros en memoria superan con creces la complejidad adicional de un modelo de persistencia híbrido.

Cambiar el tamaño máximo de la página de la hoja

Como vimos en [Capítulo 1](#) Los dispositivos de almacenamiento , de sistemas y arquitectura MongoDB tienen limitaciones de rendimiento y ancho de banda. Si tiene limitaciones de ancho de banda en lugar de rendimiento, puede mejorar el rendimiento reduciendo el tamaño máximo de página de hoja para WiredTiger.

El tamaño de página predeterminado actual es de 32 KB para colecciones y de 16 KB para índices. Esto significa que, si ejecuta una aplicación que realiza lecturas aleatorias en un conjunto de datos mucho mayor que la caché, cada operación que no accede a la caché solo lee un documento. Si sus documentos tienen un tamaño de solo 1 KB y solo se utiliza uno por página, está desperdiciando 31 KB de ancho de banda de disco (y espacio de caché) por cada operación de base de datos que accede al almacenamiento.

Por ejemplo, en una implementación de nube autogestionada, puede usar herramientas de monitoreo de rendimiento o `iostat` para verificar si su ancho de banda de almacenamiento frecuentemente está en el límite.

Puede cambiar la configuración `leaf_page_max` (para nuevas colecciones) a través del archivo de configuración o la línea de comando:

```
almacenamiento: motor:
  wiredTiger
    wiredTiger:
      collectionConfig: configString: "leaf_page_max=8192" # 8kB
```

También puedes especificarlo al crear una colección:

```
db.createCollection("miColección", { motor de
  almacenamiento:
    { wiredTiger:
      { cadena de configuración: "leaf_page_max=8192" # 8kB
    }
  }
})
```

Los beneficios de este cambio pueden variar según factores como el tamaño de la instancia, el tamaño del conjunto de datos, el tamaño promedio de los documentos y la compresibilidad de los datos. Por ejemplo, en experimentos con instancias grandes, hemos observado una mejora de hasta el doble en cargas de trabajo de lectura intensiva con documentos relativamente pequeños (1 KB) y un conjunto de datos con el doble del tamaño de la memoria disponible. Sin embargo, cuando el conjunto de datos crece hasta 10 veces el tamaño de la memoria, la mejora se reduce a aproximadamente 1,4 veces. En instancias más pequeñas, la mejora podría ser de tan solo 1,1 veces.

Como con cualquier cambio, existen posibles desventajas. Por ejemplo, si el rendimiento de inserción es importante para su aplicación, unas páginas hoja más pequeñas pueden resultar en más operaciones de E/S, ya que las páginas se llenan más rápido al insertar documentos. Además, unas páginas más pequeñas implican lo siguiente:

- Menos posibilidades de compresión, por lo que el conjunto de datos en el disco podría ser más grande
- Para cargas de trabajo que escanean una gran cantidad de datos, es necesario realizar más operaciones de E/S para escanear la misma cantidad de documentos.

En muchos dispositivos de almacenamiento en la nube, el tamaño de bloque óptimo es de 256 KB, por lo que las páginas más pequeñas podrían ser menos eficientes desde la perspectiva de todo el sistema.



Cambiar `leaf_page_max` requiere reescritura de datos

No puedes cambiar el valor `leaf_page_max` para datos existentes, por lo que para cambiar `leaf_page_max` para una colección, necesitarás leer los datos y volver a escribirlos.

Resumen

En resumen, WiredTiger es un motor de almacenamiento sofisticado que combina estructuras de datos eficientes, control de concurrencia avanzado y gestión de datos sólida para ofrecer una solución de almacenamiento de alto rendimiento, escalable y flexible para aplicaciones modernas.

MongoDB eligió los valores predeterminados actuales de WiredTiger para lograr un buen equilibrio entre una amplia gama de cargas de trabajo y conjuntos de datos. Sin embargo, su carga de trabajo y conjunto de datos son únicos, y realizar experimentos antes de migrar su aplicación a producción puede ayudarle a identificar configuraciones más eficientes que permitan un mejor rendimiento.

En el siguiente capítulo, profundizaremos en las capacidades reactivas de MongoDB explorando patrones avanzados, mejores prácticas y consideraciones prácticas para el uso de flujos de cambio en entornos de producción. Obtendrá información sobre el escalado basado en eventos.

arquitecturas, gestión de escenarios de alto rendimiento e integración de flujos de cambio con sistemas externos para crear aplicaciones robustas que respondan dinámicamente a los cambios en sus datos.

8

Flujos de cambio

Los flujos de cambios representan una de las capacidades más potentes de MongoDB para crear aplicaciones reactivas basadas en eventos. Considérelos como un flujo de datos en vivo que envía notificaciones constantes de actualizaciones a medida que se producen en sus colecciones y bases de datos, así como en toda su implementación de MongoDB. Este capítulo explora la arquitectura, las estrategias de implementación y las consideraciones de rendimiento de los flujos de cambios, ofreciendo una guía práctica para ayudarle a aprovechar esta función eficazmente en implementaciones de MongoDB de alto rendimiento.

Comenzaremos examinando la arquitectura de los flujos de cambio y su evolución desde el registro manual de operaciones (`oplog`). Aprenderá cómo MongoDB transforma las operaciones de escritura en eventos de cambio, cómo se ven estos eventos y cómo evolucionan a lo largo de su ciclo de vida. A continuación, veremos cómo implementar flujos de cambio eficazmente seleccionando el alcance de observación adecuado, aplicando filtros del lado del servidor con canales de agregación y utilizando estrategias de búsqueda de documentos que equilibren el rendimiento y la integridad. Finalmente, un ejemplo práctico demostrará cómo crear un servicio de monitorización de precios en tiempo real mediante flujos de cambio.

La segunda mitad del capítulo se centra en el rendimiento y la durabilidad: estrategias para la optimización de recursos, diseño de sistemas resilientes,

consumidores y la gestión de flujos de alto volumen. También abordamos consideraciones especiales para clústeres fragmentados, limitaciones de tamaño de documentos y ciclos de vida de colecciones. El capítulo concluye con una guía sobre la monitorización y los matices de los conjuntos de réplicas, así como un resumen de las técnicas de ajuste para garantizar que la implementación de su flujo de cambios esté lista para producción.

Este capítulo cubre los siguientes temas:

- Comprender la arquitectura de los flujos de cambio y cómo funcionan
- Utilizar los flujos de cambio de manera eficiente
- Gestión del rendimiento y la durabilidad
- Patrones avanzados y preparación para la producción

Comprender la arquitectura de los flujos de cambio

En esencia, un flujo de cambios es un cursor que permanece abierto y envía eventos de cambio de la base de datos a las aplicaciones conforme ocurren. Para aprovechar al máximo los flujos de cambios, es fundamental comprender su funcionamiento subyacente. Antes de la introducción de los flujos de cambios en MongoDB 3.6, los desarrolladores que necesitaban eventos de cambio recurrían al sondeo, creaban sistemas de notificación del lado de la aplicación o rastreaban manualmente la recopilación de registros de operaciones de MongoDB.

Estos enfoques presentaban varias desventajas. El sondeo desde la aplicación podía sobrecargar innecesariamente el servidor de base de datos si el intervalo de sondeo era más frecuente que los cambios. Por otro lado, si el intervalo de sondeo era menos frecuente, la aplicación corría el riesgo de...

Omitir algunos cambios o introducir retrasos, lo que requería mayor complejidad para gestionar estos escenarios. Analizar las entradas del registro de operaciones sin procesar también exigía conocimiento de los aspectos internos de MongoDB, lo que añadía mayor complejidad. Además, la implementación era frágil, ya que el formato del registro de operaciones es un detalle interno que podía cambiar entre versiones de MongoDB. Finalmente, el seguimiento manual del registro de operaciones ofrecía garantías limitadas: no existía la posibilidad de reanudación integrada en caso de pérdida de conectividad de red, ni garantías de ordenación consistente. Estas limitaciones convertían el seguimiento del registro de operaciones en una solución frágil que requería una gran experiencia para su correcta implementación y dejaba las aplicaciones vulnerables a cambios internos disruptivos entre versiones de MongoDB.

Los flujos de cambio resuelven estos problemas al proporcionar una interfaz oficial y compatible que abstraer las complejidades subyacentes.

Con él podrás disfrutar de características como las siguientes:

- Confiabilidad : No hay notificaciones de eventos duplicados o perdidos
- Documentos de eventos estructurados : un formato bien definido con información detallada sobre cada cambio
- Reanudabilidad : los tokens de reanudación integrados permiten una recuperación confiable después de desconexiones
- Garantías de ordenamiento : garantice un ordenamiento global coherente de los eventos, incluso en clústeres fragmentados
- Capacidades de filtrado : filtrado del lado del servidor mediante canales de agregación
- Más seguro : mientras que el registro de operaciones expone todos los cambios en el servidor, los flujos de cambios solo entregan actualizaciones a las que el usuario o el código tienen permiso de acceso.

- Más simple : los eventos se envían solo para escrituras con confirmación mayoritaria, por lo que su aplicación no necesita manejar reversiones
- Imágenes opcionales : Puede obtener una copia del documento modificado antes y/o después del cambio.

En conjunto, estas características ofrecen una base sólida y confiable para crear aplicaciones en tiempo real que respondan a los cambios de datos, al tiempo que mantienen la consistencia y la resiliencia, incluso ante desconexiones o fallas del servidor.

Cómo funcionan los flujos de cambio: desde operaciones de escritura hasta eventos Los flujos de cambio se basan en la infraestructura de replicación de MongoDB.

Comprender esta conexión ayuda a explicar tanto sus capacidades como sus limitaciones. A grandes rasgos, una aplicación cliente recibe una notificación de un cambio mediante los siguientes pasos:

1. Una parte de la aplicación cliente realiza una operación de escritura en un Colección MongoDB.
2. La operación se registra en el registro de operaciones del primario.
3. La función de flujo de cambios monitorea el registro de operaciones en busca de entradas con mayor número de confirmaciones.
4. Las entradas del oplog sin procesar se transforman en un formato más amigable para los desarrolladores.
5. Los eventos se envían a las aplicaciones cliente a través de un servidor abierto.cursor.

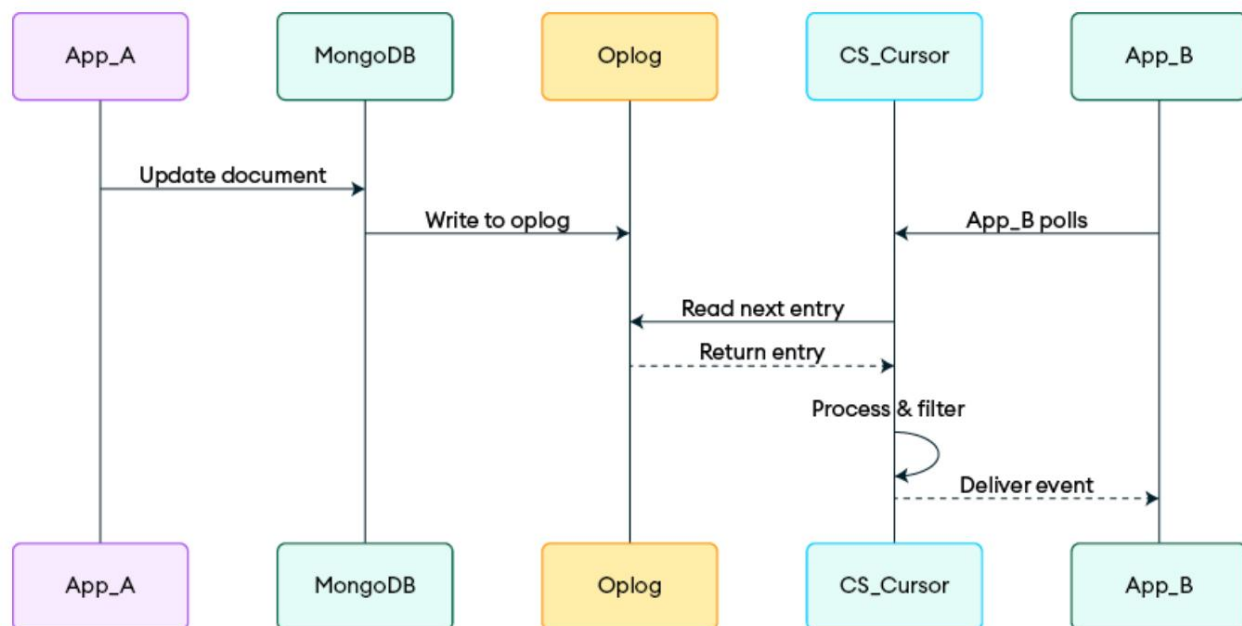


Figura 8.1: Secuencia de cómo se notifican los cambios de documentos a través de flujos de cambio

La Figura 8.1 muestra una vista general de los pasos necesarios para enviar notificaciones de cambio desde MongoDB a las aplicaciones cliente mediante flujos de cambios. App_A y App_B podrían representar la misma aplicación. El registro de operaciones forma parte de MongoDB, mientras que el cursor del flujo de cambios (CS_Cursor) existe tanto en el servidor MongoDB como en el cliente, en App_B .

Este proceso garantiza que todos los cambios en la base de datos se capturen de forma fiable y se entreguen en un orden coherente a las aplicaciones que necesitan responder en tiempo real. En clústeres fragmentados, el proceso se vuelve más complejo, ya que el enrutador debe coordinarse entre múltiples fragmentos, preservando al mismo tiempo el orden global de los eventos.

Estructura y ciclo de vida del evento

Cada evento del flujo de cambios contiene información sobre los cambios y proporciona herramientas para mantener la continuidad. Por ejemplo, aquí hay un

evento de flujo de cambio para un cambio en el campo de precio en un documento en una colección de productos :

```
{
  "_identificación": {
    "_data": "825F65A78B000000012B022C0100296E5A1004..."
  },
  "operationType": "actualizar",
  "ns": {
    "db": "inventario",
    "coll": "productos"
  },
  "clave del documento": {
    "_id": Id. del objeto("5f65a78b00000000000000000001")
  },
  "updateDescription": {
    "CamposActualizados": {
      "precio": 29,99,
      "Última modificación": ISODate("2023-09-15T14:22:01.123Z")
    },
    "removedFields": ["promoción"]
  },
  "documento completo": {      Solo presente opt   con documento completo //
    //   Estado actual completo del documento
  }
}
```

Como puede ver, el evento de flujo de cambio puede incluir el flujo completo documento, si se solicita durante la configuración del flujo de cambios. Sin embargo, Esto puede afectar significativamente el rendimiento, por lo que debe utilizarse juiciosamente. Un cursor de flujo de cambios avanza a través de lo siguiente Estados durante su ciclo de vida:

1. Apertura : Establecimiento de su posición en el oplog
2. Esperando : Permanecer inactivo hasta que se produzcan cambios.

- 3. Entrega : Envío de eventos al cliente
- 4. Reanudación : Restablecimiento de la transmisión después de una desconexión
- 5. Invalidación : Finalización después de ciertas operaciones

Comprender este ciclo de vida es clave para crear aplicaciones resilientes.

Implementar flujos de cambio de manera efectiva

Ahora que está familiarizado con la arquitectura de los flujos de cambio, veamos cómo implementar una aplicación que los utilice de manera efectiva para lograr un rendimiento óptimo en diferentes escenarios.

Elegir el alcance y la estrategia de filtrado adecuados: MongoDB ofrece flexibilidad tanto en el alcance de observación como en las capacidades de filtrado. La combinación de estos dos factores tiene un impacto significativo en el rendimiento. El alcance de observación puede limitarse a una colección, abarcar una base de datos o extenderse a toda la implementación. En general, cuanto más amplio sea el alcance, mayor será el impacto en el rendimiento, ya que se consumen más recursos y se debe procesar un mayor volumen de eventos de cambio.

En la Tabla 8.1 se describen cada ámbito, su uso de recursos, el volumen de eventos esperado y un caso de uso típico para cuando podría elegirlo en su aplicación:

Alcance	Evento de recursos	Casos de uso
---------	--------------------	--------------

	uso	volumen	
Colección Inferior		Eventos empresariales específicos y enfocados,	sincronización dirigida
Base de datos	Flujos de trabajo entre recopilaciones de nivel medio y moderado		auditoría de toda la base de datos
Despliegue superior		Registro de auditoría global de máxima calidad y	transmisión de eventos multiinquilino

Tabla 8.1: Alcances y características del flujo de cambios

Como se muestra en la Tabla 8.1, la elección del alcance de observación adecuado es

Un acto de equilibrio entre visibilidad y rendimiento. A nivel de colección

El alcance ofrece el uso más eficiente de los recursos y es ideal para objetivos específicos.

Casos de uso. Expansión a una base de datos o a un alcance de implementación.

aumenta la observabilidad pero también exige más de la base de datos y

su aplicación.

Mejores prácticas

Elija el alcance más estrecho que se ajuste a sus necesidades.

requisitos. Esto maximizará el rendimiento al

reduciendo el consumo de recursos en el servidor de base de datos

y minimizar el número de eventos de su aplicación

Necesita procesarse.

Sin embargo, es necesario equilibrar esto con el número de

cambiar los flujos porque cada uno, en efecto, está analizando y



Filtrar el registro de operaciones. Por ejemplo, si una base de datos tiene 20 colecciones y desea procesar cambios de 16 de ellas, es mejor tener un flujo de cambio en la base de datos con más de 16 flujos de cambio en cada colección.

Filtrado del lado del servidor con agregación tuberías

Una de las técnicas de rendimiento más efectivas para los flujos de cambio es un filtrado del lado del servidor mediante canales de agregación. Este enfoque reduce drásticamente el consumo de ancho de banda de la red y sobrecarga de procesamiento del lado del cliente de la aplicación.

Por ejemplo, si solo le interesan los cambios de precios de los productos documentos, puede configurar un flujo de cambios con una agregación canalización que coincide únicamente con los cambios en el campo de precio :

```
// Fragmento de código del controlador MongoDB Node.js
// Abre un flujo con una tubería de filtro
const pipeline = [
  // Solo capturar operaciones de actualización en Los productos recopilados
  { $match: { 'ns.coll': 'productos', tipo de operación: 'actualizar' } },

  // Los filtros adicionales solo incluyen actualizaciones de precios
  { $match: { 'updateDescription.updatedFields.price': { $exists: true } } },
];
const filteredStream = colección.watch(pipeline);
```

Este flujo de cambio filtrado emite solo eventos para actualizaciones del precio campo en la colección de productos , reduciendo significativamente la cantidad de datos enviados a través de la red y procesados por la aplicación cliente.

Estrategias y rendimiento de la búsqueda de documentos

Los flujos de cambio ofrecen varias opciones para acceder a los datos del documento, cada una con diferentes implicaciones de rendimiento:

- Solo delta : comportamiento predeterminado; incluye solo información de cambios.
- Búsqueda de estado actual : `fullDocument: 'updateLookup'` obtiene el documento actual.
- Imágenes de antes y después (MongoDB 6.0+) : Muestra el estado del documento antes y después de los cambios. Esta función opcional puede afectar el rendimiento, ya que la base de datos debe capturar y almacenar el documento antes o después del cambio.

El mejor enfoque depende de su caso de uso específico. Hágase las siguientes preguntas para guiar su decisión:

- ¿El valor delta en `updateDescription` es suficiente para el caso de uso de su aplicación?
- ¿Cuánto ancho de banda de red adicional puede permitirse?
- ¿Es aceptable la latencia de una búsqueda de documento adicional?

Elegir la opción correcta depende de las necesidades de precisión, rendimiento y eficiencia de recursos de su aplicación. Por ejemplo, la opción "updateLookup" obtendrá el documento completo un tiempo después de la actualización. Esto puede tener un impacto claro en el rendimiento si el documento es grande. Sin embargo, también puede requerir un manejo cuidadoso en su aplicación. Por ejemplo, si el documento cambia con frecuencia y hay una cola de cambios por procesar, podría terminar leyendo una versión con tres cambios posteriores. Esto también podría ocurrir si reproduce el flujo de cambios; el documento completo ...

La aplicación que recibe (cuando se usa 'updateLookup') es la actual versión del documento, no la del momento del cambio.

Creación de un servicio de seguimiento de precios

Para ilustrar estos conceptos en acción, implementemos un servicio que Monitorea los cambios de precios de los productos y notifica a las partes interesadas. El siguiente código configura una canalización de agregación para vigilar las actualizaciones al campo de precio y notifica a otras partes de la aplicación cuando el cambio excede un umbral definido:

```
// Fragmentos de código para el controlador MongoDB Node.js
// Configurar la colección usando preImages y antes de la
      changeStreamPreAndPostImages: { habilitado: true
    });
// Configurar el flujo de cambios con búsquedas y filtrado
const priceMonitor = async (minimumPriceChange) => {
  intentar {
    constante pipeline = [
      { $match: { operationType: 'actualizar' } },
      { $match: { 'updateDescription.updatedFields.price'
      { $project: { documentKey: 1, 'updateDescription.upd
    ];
    opciones = { fullDocumentBeforeChange: 'obligatorio' };
    const stream = colección.watch(pipeline, opciones);

    // Procesar cada cambio como él ocurre
    para esperar ( cambio constante de flujo) {
      const nuevoPrecio = cambio.actualizarDescripción.actualizadoFie
      const oldPrice = cambio.documentoCompletoAntesDelCambio.pri
      const priceDiff = Math.abs(precioNuevo - precioAnterior);

      // Procesar únicamente cambios de precios significativos
      si (diferenciaPrecio >= cambioPrecioMínimo) {
        esperar notificarPrecioSubscriptores(cambio.documentKey._i
```

```

    }
  }
} catch (error)
{ console.error(" Error de cambio de flujo:", error);
  // Implementar la lógica de reconexión aquí
}
};

```

Este ejemplo demuestra varias prácticas clave de rendimiento. Al implementar el filtrado del lado del servidor a través de la canalización de agregación, la aplicación reduce el tráfico de red entre MongoDB y el cliente. Además, la aplicación aplica un filtro específico para la empresa (el umbral `minimumPriceChange`) en el lado del cliente. Sin embargo, tenga en cuenta que capturar el estado del documento (`preImage`) antes de la actualización genera cierta sobrecarga.

Esto puede tener un impacto sutil en el rendimiento. De hecho, habilitar `changeStreamPreAndPostImages` provoca una amplificación de escritura, ya que las copias deben almacenarse. Sin embargo, es probable que estén en la caché cuando la aplicación acceda a ellas. Si usa `'updateLookup'` , evita que estas requieran `changeStreamPreAndPostImages` . Cuando escrituras adicionales la lectura se realiza posteriormente, es relativamente menos probable que se almacene en la caché. Desde el punto de vista del rendimiento, lo ideal sería usar `$match` en los campos de `updatedFields` . Si eso no es posible, experimente con `changeStreamPreAndPostImages` y `'updateLookup'` .

Gestión del rendimiento y la durabilidad

Desarrollar aplicaciones de alto rendimiento con flujos de cambios requiere una cuidadosa consideración tanto del uso de recursos como de la durabilidad de los datos. exploremos estrategias para optimizar ambos.

Estrategias de optimización de recursos: Los flujos de cambio

consumen recursos de toda la pila. En algunos casos, realizar procesamiento adicional en el servidor puede reducir la carga de trabajo de la aplicación. En otras situaciones, el procesamiento adicional en el cliente puede mejorar el rendimiento general del sistema. La Tabla 8.2 resume estas ventajas y desventajas:

Recurso área	Factores de impacto	Enfoques de optimización
Del lado del servidor	• Uso de CPU para el procesamiento de registros de operaciones • Uso de memoria para estado del cursor • Red ancho de banda consumo	• Filtrar agresivamente. • Utilice el más pequeño alcance apropiado. • Multiplexar varios flujos de cambio en uno solo y dejar que Filtro de aplicación para reducir el número de cursores consumiendo el oplog. • Dimensiona el oplog para evitar que los flujos queden demasiado atrás de las actualizaciones. En última instancia, el oplog es el buffer si la aplicación

		El procesamiento del flujo de cambios se queda atrás.
Lado de la aplicación	• Conexión uso de la piscina • Evento capacidad de procesamiento • Memoria utilizada para almacenamiento en búfer • Red ancho de banda uso	• Conexiones dedicadas • Utilice el procesamiento por lotes

Tabla 8.2: Equilibrio entre el procesamiento del servidor y de la aplicación

Como ocurre con la mayoría de las decisiones de rendimiento, no existe una solución universal. Optimizar el procesamiento del flujo de cambios requiere considerar cuidadosamente dónde es más crítico el uso de recursos en su arquitectura. Al comprender las ventajas y desventajas entre el procesamiento del lado del servidor y el del lado de la aplicación, puede adaptar su enfoque para equilibrar la eficiencia, la escalabilidad y la capacidad de respuesta.



Mejores prácticas

En implementaciones de gran volumen, considere configurar un servicio dedicado para procesar los flujos de cambio. Este enfoque consolida a varios consumidores en un solo...

Conexión de corriente, mejorando la eficiencia y reduciendo
Uso de recursos.

Manejo de flujos de eventos de gran volumen. En aplicaciones con tasas de escritura extremadamente altas, los flujos de cambios generarán un gran volumen de eventos. Para facilitar el escalado eficaz de estas aplicaciones, considere las siguientes tácticas:

- Implementar un filtrado agresivo del lado del servidor
- Implementar una infraestructura de aplicaciones dedicada al procesamiento del flujo de cambios
- Utilice estrategias de procesamiento por lotes que logren el equilibrio adecuado entre rendimiento y latencia.

Si su registro de operaciones es demasiado pequeño, este se reinicia y sobrescribe las entradas antiguas antes de que la aplicación pueda procesar sus eventos de flujo de cambios. Esto generará un error de flujo de cambios. Este riesgo es especialmente alto en aplicaciones donde el procesamiento del flujo de cambios puede desconectarse durante períodos prolongados.

Para diagnosticar esto, monitoree el tamaño de la colección `oplog.rs` y la ventana de registro de operaciones actual. Puede comprobarlo con el comando `db.getReplicationInfo()` en MongoDB Shell. Si la ventana de registro de operaciones es corta (por ejemplo, solo unas horas), existe una mayor probabilidad de que se produzca un error. Para solucionarlo, aumente el tamaño del registro de operaciones. En los conjuntos de réplicas, esto suele implicar ajustar el parámetro `oplogSizeMB` en la configuración o reinicializar el conjunto de réplicas con un registro de operaciones mayor. Como regla general,

Intente crear un registro de operaciones que pueda retener varios días, o incluso una semana, de operaciones, dependiendo de su objetivo de punto de recuperación (RPO).

Si sospecha que su aplicación de flujo de cambios se está quedando atrás de los cambios reales en la base de datos, esto puede provocar un procesamiento retrasado o datos obsoletos en los sistemas posteriores. Para evitarlo, considere los siguientes métodos:

- Monitoree la E/S de red tanto en el nodo principal de MongoDB como en su servicio de flujo de cambios. Busque picos en el uso de la red que coincidan con periodos de alta actividad de la base de datos.
- Considere la proximidad física de su aplicación a la base de datos. Asegúrese de que la conectividad de red entre su clúster de MongoDB y su aplicación de flujo de cambios sea óptima. Esto podría implicar ubicarlos en el mismo centro de datos o región de la nube.
- Si es posible, proporcione más ancho de banda de red tanto para el servidor de base de datos como para la aplicación del consumidor.
- Considere procesar eventos en lotes en lugar de hacerlo individualmente.

Su aplicación también necesita suficientes recursos computacionales para procesar eventos eficazmente. Si tiene recursos limitados y no puede seguir el ritmo del volumen de cambios, se retrasará cada vez más antes de finalmente salir del registro de operaciones y recibir un error.

De igual forma, el servidor MongoDB debe contar con recursos suficientes para generar y enviar eventos de cambio. Supervise la CPU, la memoria y la E/S de disco en el sistema que ejecuta su aplicación. Busque un uso elevado de la CPU, presión sobre la memoria o cuellos de botella de E/S. Intente minimizar las operaciones costosas dentro del bucle de procesamiento del flujo de cambios. En la base de datos, un uso elevado de la CPU o de la E/S puede indicar escritura.

Cuellos de botella. Por ejemplo, una indexación excesiva puede ralentizar las escrituras, lo que a su vez afecta la velocidad de escritura del registro de operaciones y, en consecuencia, altera el rendimiento del flujo.

Si su aplicación de flujo de cambios recibe con frecuencia eventos de invalidación , lo que la obliga a cerrarse y reabrirse, esto puede ser disruptivo y provocar una mayor latencia o la pérdida de eventos si no se gestiona correctamente. Un evento de invalidación significa que la colección o base de datos supervisada por el flujo de cambios ya no existe o ha sufrido un cambio estructural (como la eliminación o el cambio de nombre) que impide que el flujo continúe.

Para solucionar problemas, cuente los tipos de eventos que recibe su flujo de cambios. Si detecta eventos de invalidación , identifique la operación específica que los activó. Si es posible, evite eliminar o renombrar colecciones o bases de datos que estén siendo monitoreadas activamente por flujos de cambios, especialmente en entornos de producción, donde el procesamiento continuo es fundamental. Si estas operaciones son necesarias, asegúrese de que su aplicación de flujo de cambios esté diseñada para gestionar correctamente los eventos de invalidación restableciendo el flujo.

Consideraciones especiales para implementaciones fragmentadas. En

clústeres fragmentados, los

flujos de cambios requieren coordinación entre múltiples fragmentos, manteniendo al mismo tiempo un orden global coherente de eventos. Esto implica varias consideraciones de rendimiento únicas:

- Sobrecarga de ordenamiento global : Mantener un ordenamiento consistente en todos los fragmentos requiere coordinación. Esto se gestiona mediante

mongos y agrega carga a esa infraestructura.

- Impacto del fragmento frío : los fragmentos sin actividad de escritura aún participan en la coordinación del flujo de cambio.
- Latencia geográfica : los fragmentos distribuidos introducen la red Latencia en la entrega de eventos.
- Manejo de migración de fragmentos : cambios durante las migraciones de fragmentos requieren un manejo especial.
- Rendimiento a nivel de sistema : si una colección fragmentada tiene un alto niveles de actividad de escritura, luego mongos o el servidor de aplicaciones Es posible que el procesamiento de la transmisión no pueda seguir el ritmo de los cambios en todos los fragmentos.

Para optimizar el rendimiento en entornos fragmentados, el siguiente código configura dos de estos parámetros, maxAwaitTimeMS y batchSize :

```
// Fragmento de código del controlador MongoDB Node.js
// Configuración de flujos de cambio para un rendimiento óptimo en
constante shardedStreamOptions = {
  documento completo: 'updateLookup',
  maxAwaitTimeMS: 1000, tamaño // // Ajuste basado en requisitos de latencia
  del lote: 100 };                               Equilibrio entre rendimiento y

// Utilice secuencias a nivel de colección siempre que sea posible reducir a
const productStream = db.collection('productos').watch([], do
```

El parámetro batchSize establece el número máximo de documentos

que se pueden devolver en cada lote de un flujo de cambios (hasta un límite de 16 MB). Es similar al parámetro batchSize de los cursores. Este

A menudo requiere experimentación porque si el tamaño del lote es demasiado grande, El cursor puede asignar más recursos de los necesarios; si es demasiado pequeño, puede conducir a un aumento de viajes de ida y vuelta en la red.

El parámetro `maxAwaitTimeMS` establece el tiempo máximo en milisegundos que el servidor espera nuevos cambios de datos antes de devolver un lote vacío al cursor del flujo de cambios. Es similar al parámetro `maxAwaitTimeMS` de los cursores.

A continuación se presentan algunas técnicas recomendadas de optimización de clústeres fragmentados:

- Configure `periodicNoopIntervalSecs` (predeterminado: 10 segundos) para reducir la latencia de los fragmentos de escritura fría o baja.
Estas operaciones sin operaciones garantizan que las entradas del registro de operaciones se creen regularmente, incluso cuando no se producen escrituras reales en el fragmento principal, lo que permite a los miembros del fragmento actualizar sus relojes lógicos y posiciones de replicación, reduciendo así el tráfico de coordinación a fragmentos más fríos.
- Dirija los flujos de cambio a colecciones específicas siempre que sea posible.
Esto reduce significativamente la sobrecarga de comunicación entre fragmentos, especialmente para colecciones con alto nivel de escritura.
- Aplique filtrado del lado del servidor utilizando canalizaciones `$match` para recopilaciones de alta actividad.
- Coloque a los consumidores del flujo de cambios geográficamente cerca de sus fragmentos principales para minimizar la latencia de la red.
- Supervise la ventana de registro de operaciones de cada fragmento de forma independiente para detectar posibles problemas que puedan afectar los servicios basados en flujos de cambios.
- Limite el número de flujos de cambios específicos de cada documento . Por ejemplo, en lugar de crear cientos de flujos de cambios para supervisar documentos individuales, cree un único flujo de cambios y filtre los eventos relevantes en su aplicación.

En resumen, ejecutar flujos de cambios eficientemente en un clúster fragmentado requiere una atención cuidadosa tanto a la configuración como a la estrategia de implementación.

Al ajustar parámetros como `batchSize` y `maxAwaitTimeMS`, dirigir los flujos a colecciones específicas y reducir la coordinación innecesaria entre fragmentos, se puede mejorar significativamente el rendimiento y la fiabilidad.

Como en la mayoría de los sistemas distribuidos, la monitorización continua y el ajuste iterativo son clave para mantener un comportamiento óptimo a escala.

Patrones avanzados y preparación para la producción

Implementar flujos de cambio en producción requiere comprender las opciones avanzadas y las necesidades de monitorización. En esta sección, exploraremos consideraciones clave para las implementaciones en producción, basándonos en la documentación oficial de MongoDB.

Visibilidad de transacciones y agrupación de eventos. Al trabajar con transacciones en

MongoDB, los flujos de cambios siguen reglas de visibilidad específicas que afectan la entrega de eventos. Todos los cambios realizados dentro de una transacción solo son visibles para los flujos de cambios después de que esta se confirme. En ese momento, todos los cambios de esa transacción aparecen a la vez como eventos separados en el flujo, manteniendo su orden lógico dentro de la transacción.

Este comportamiento garantiza la consistencia de los datos, pero requiere que sus aplicaciones de consumo gestionen estas ráfagas de eventos de manera eficiente cuando son grandes.

Las transacciones se confirman. Considere implementar mecanismos de procesamiento por lotes y de limitación en su lógica de procesamiento para gestionar estos aumentos de forma eficiente.

Limitaciones de tamaño de documento y ciclo de vida de la colección.

Los documentos de respuesta del flujo de

cambios deben cumplir con el límite de 16 MB de MongoDB para documentos BSON. Si utiliza opciones como "fullDocument: 'updateLookup'" (que realiza una búsqueda para obtener el estado actual completo de un documento después de una operación de actualización) o trabaja con documentos grandes, podría alcanzar este límite, lo que provocaría un fallo en los eventos. Para documentos más grandes, en MongoDB 6.0.9 y posteriores, considere usar la etapa de agregación

`$changeStreamSplitLargeEvent` para dividir eventos de cambio grandes en varios fragmentos más pequeños que se puedan procesar individualmente sin exceder las restricciones de tamaño.

Cuando se eliminan o renombran colecciones o bases de datos mientras los flujos de cambios las supervisan, los cursores del flujo de cambios se cerrarán al llegar a ese punto en el registro de operaciones. Después de estos eventos de invalidación, debe usar `startAfter` (no `resumeAfter`) con un token de reanudación válido si desea reiniciar el flujo.

Monitoreo y controles de salud

Las implementaciones de producción deben incluir una monitorización exhaustiva de varias métricas clave. Para los flujos de cambios, puede empezar por monitorizar el número de cursores activos mediante el comando

```
db.serverStatus().metrics.changeStream.currentOpen .
```

Los eventos de cambio podrían llegar bastante tiempo después de que se realice el cambio en la base de datos. Además, monitoree la latencia y el rendimiento del procesamiento de eventos para garantizar un rendimiento óptimo. Las métricas de persistencia del token de reanudación proporcionan información sobre su capacidad de recuperación, mientras que el tamaño de la ventana del registro de operaciones (medible mediante `rs.printReplicationInfo()`) indica la antigüedad de sus operaciones históricas.

Consideraciones sobre conjuntos de réplicas: Al usar MongoDB autogestionado y para conjuntos de réplicas con miembros árbitros, tenga en cuenta que los flujos de cambio pueden permanecer inactivos si no hay suficientes miembros con datos disponibles para lograr una confirmación mayoritaria. Por ejemplo, en un conjunto de réplicas de tres miembros con dos nodos con datos y un árbitro, si el secundario falla, las escrituras no pueden confirmarse mayoritariamente y el flujo de cambio permanece abierto, pero no envía eventos.

Si anticipa un tiempo de inactividad significativo de un servicio que utiliza flujos de cambio, por mantenimiento o recuperación ante desastres, aumente el tamaño del registro de operaciones para conservar las operaciones durante más tiempo que el período de inactividad estimado. Esto garantiza que su aplicación pueda ponerse al día con todos los cambios una vez que se restablezca la conectividad.

Resumen del ajuste del rendimiento. Al optimizar los flujos de cambio para producción, se deben implementar varias estrategias clave. El filtrado del lado del servidor con canales de agregación reduce significativamente el volumen de eventos, minimizando el tráfico de red y la sobrecarga de procesamiento. Seleccione una opción.

estrategia de búsqueda de documentos adecuada en función de los requisitos específicos de su aplicación, eligiendo cuidadosamente entre las distintas opciones de `fullDocument` para equilibrar la integridad con el rendimiento.

Dimensione su registro de operaciones adecuadamente para los escenarios de tiempo de inactividad máximo previsto, garantizando que pueda retener las operaciones el tiempo suficiente para cubrir las ventanas de mantenimiento o los períodos de recuperación. Configure siempre el tamaño de su grupo de conexiones para que supere el número de flujos de cambio abiertos, evitando así el agotamiento de recursos.

Limite el número de flujos de cambio específicos, ya que no pueden aprovechar las optimizaciones de indexación, lo que podría aumentar la carga del servidor. Para aplicaciones con un volumen de eventos particularmente alto, considere una infraestructura dedicada para aislar el procesamiento de los flujos de cambio de la carga de trabajo de su base de datos principal, evitando así el impacto en el rendimiento de las operaciones críticas.

Resumen

Los flujos de cambio transforman MongoDB de una base de datos tradicional a una plataforma de datos reactiva, lo que permite respuestas en tiempo real a los cambios de datos.

Cuando se implementan cuidadosamente, teniendo en cuenta el rendimiento, proporcionan una base sólida para crear aplicaciones responsivas y basadas en eventos.

El siguiente capítulo se centra en las transacciones, un aspecto importante para crear aplicaciones fiables y consistentes. Profundizaremos en la mecánica de las transacciones ACID multidocumento y analizaremos las consideraciones clave para optimizar su rendimiento en diversas arquitecturas de MongoDB.